

CMSC320TextBook

June 16, 2026

1 CMSC320 Textbook

Dr. Fardina Alam and Gavin Hung

Generated from the Jupyter Book table of contents (`_toc.yml`).

2 About the Book

3 Preface

Since joining UMD, CMSC320 has quickly become one of my favorite courses to teach. Every semester I am reminded how important and how exciting it is for students to learn data science through real-world datasets, exploratory analysis, storytelling with visualizations, and modern machine learning techniques. But as the course grew, it became clear that students needed a more centralized, cohesive resource that brings together the core ideas, examples, and workflows we kept revisiting in class.

This book is my attempt to create that resource. It is designed to be practical, interactive, and visual, reflecting how data science and machine learning are actually practiced. Rather than treating data analysis as a collection of isolated techniques, the goal here is to present the data science lifecycle how data is acquired, cleaned, transformed, modeled, interpreted, and communicated. Machine learning fits naturally in this lifecycle: it becomes a tool for prediction, pattern recognition, and decision-making rather than an algorithmic checklist.

I would also like to thank my former CMSC320 student, Gavin Hung (Currently, Software Engineer at NVIDIA), whose work with me on interactive visualizations inspired the design direction of this book. His contributions reinforced the idea that students not only learn data science by reading and coding, but also by exploring; poking at the data, adjusting parameters, and seeing patterns emerge in real time. That level of interaction helps data science feel less abstract and more intuitive. His personal project at ml-visualized.com further demonstrated how rich and approachable machine learning concepts can become when students are able to experiment visually rather than passively consume static material.

My hope is that this book makes data science and machine learning more accessible for students encountering them for the first time, while still offering enough depth for those who want to explore further. If it helps learners ask better questions, build intuition, and maybe enjoy the process along the way, then it has done its job.

Dr. Fardina F. Alam

4 Chapter 1 - Intro to Data Science

5 Introduction to Data Science

Not too long ago, conversations about careers rarely mentioned terms like **AI, Machine Learning, or Data Science**. Today, those words are everywhere, on campus, from news headlines to LinkedIn posts and even in casual conversations with friends. But surprisingly *few people really know* what they mean. Some assume data science is just cleaning and analyzing data, and others think data science and machine learning are the same thing. While these ideas aren't completely wrong, they leave out most of what data science actually involves.

[Image omitted in PDF — see the online textbook]

Figure 1: Example LinkedIn job trends in AI/Data Science. Source: LinkedIn via Ana Rivas Cano.

In reality, data science involves far more than collecting a dataset and training a model. It includes gathering data, organizing and cleaning it, exploring it to understand patterns and limitations, and transforming it into something that can support analysis or machine learning. And even after modeling, the work continues; data scientists must interpret the results, decide what matters, and communicate insights to the final stakeholder, whether that's a client, a manager, or a team making decisions.

[Image omitted in PDF — see the online textbook]

Figure 2. An illustrative example of a Data Science Life Cycle. Source: Springboard (2022).

Because of all these steps, data science follows a structured workflow often called the **Data Science Life Cycle** (DSLCL).

Throughout this chapter and throughout the rest of the book, we'll walk through that process in detail. But before we do that, it's worth taking a quick look at how data science started and how the field developed into what we know today.

5.1 What is Data Science?

Before we explore the history of the field, it is useful to pause and ask a seemingly simple question: **What is data science?** Although the term is widely used, its meaning depends on who is speaking and in what context. Over the last two decades, researchers, practitioners, and industry leaders have offered formal definitions that capture different facets of the discipline.

Long before the term “data science” became widespread, researchers in the **KDD** community were already thinking about how to extract knowledge from data stored in large databases. **>KDD**, which stands for Knowledge Discovery in Databases, refers to both a field of study and a process that emerged in the **1990s** around the problem of discovering useful patterns and information from data.

From this perspective, Fayyad et al. (1996) defined knowledge discovery as:

“The non-trivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data.” — **Fayyad, Piatetsky-Shapiro & Smyth (1996)**

A few years later, William S. Cleveland (2001) used the term data science to describe a broader vision for the field. His work offered **one of the earliest modern definitions**, framing data

science as an extension of statistics into the era of computing and large-scale data. He described it as:

“A multidisciplinary field that uses scientific methods, data, and computation to generate insight and guide decision-making.” — **William S. Cleveland (2001)**

As the field moved into industry and technology companies, the definition broadened further. Mike Loukides from O'Reilly Media remarked that data science combines statistics, computing, and domain context with a practical orientation:

“Data science is about looking at data, building models, and using those models to understand the world and make decisions.” — **Mike Loukides (2010)**

While industry often emphasizes applications and job roles, academia has also attempted to clarify what data science fundamentally is as a discipline. A particularly influential academic treatment comes from David Donoho (Stanford University), who argued that data science is:

“The science of learning from data, with an emphasis on prediction, inference, and the extraction of knowledge.” — **David Donoho, “50 Years of Data Science,” 2017**

Donoho emphasized that data science sits at the intersection of statistics, computing, and domain expertise, and that its primary goal is insight; whether that insight takes the form of a prediction, a pattern, a decision, or a scientific conclusion.

[Image omitted in PDF — see the online textbook]

Figure 3. A common Venn diagram illustrating the interdisciplinary nature of data science. Original source: Drew Conway (2010), “The Data Science Venn Diagram.”

Similarly, machine learning researcher Zico Kolter from Carnegie Mellon University (CMU) underscored the computational perspective, describing data science as:

“The application of computational and statistical techniques to gain managerial or scientific insight into real-world problems.” — **Zico Kolter, Carnegie Mellon University (CMU)**

Across these definitions, a consistent theme emerges: data science is not just about algorithms, programming, or statistics alone. It integrates mathematics, statistics, computation, and domain knowledge to make sense of data and support decisions in the world. Together, these perspectives show how the field bridges scientific inquiry, computational tools, and practical applications.

A simple working summary

For the purposes of this book, we will adopt a practical working summary:

Data science is an interdisciplinary field focused on discovering patterns and describing relationships in data, and on turning raw information into insight, decisions, and value.

5.2 Why Data Science Matters?

Data science matters because it turns raw data into understanding, and understanding into action. In a world filled with uncertainty and information overload, it gives us a way to navigate, reason, and make better choices.

Imagine trying to make decisions in a world overflowing with information. Every time we click, swipe, buy something, post something, or run an experiment, we leave behind tiny traces of data.

On its own, this data is just noise. What makes it useful is our ability to turn it into something meaningful.

Data science helps us do exactly that. It allows scientists to explore complex questions, businesses to make better decisions, governments to plan and respond, and organizations to understand the world around them. Sometimes the goal is to predict what might happen in the future, such as estimating customer demand or forecasting the weather. Other times, the goal is to explain why something happened or to describe patterns that were not obvious before.

In a broader sense, data science matters because it connects **three** important things: **data, computation, and human judgment**. With these tools, people and organizations can make more informed choices in a world that is increasingly shaped by data.

5.2.1 How Data Science Relates to AI, ML, Deep Learning, and Statistics

Although the terms *AI*, *machine learning*, *deep learning*, *statistics*, and *data science* are often used interchangeably, they refer to different but overlapping ideas. Broadly:

- **AI** focuses on techniques that enable computers to mimic or approximate human intelligence. **Example:** a chatbot that carries on a conversation or an agent that plays chess.
- **Machine Learning (ML)** is a subset of AI focused on learning patterns from data. **Example:** predicting house prices based on past sales.
- **Deep Learning (DL)** is a subset of ML that uses neural networks with many layers. **Example:** recognizing objects in images using convolutional neural networks.
- **Statistics** provides mathematical tools for describing data and making inferences. **Example:** estimating the average test score of a population from a sample.
- **Data Science** connects statistics and computing to manage, analyze, visualize, and extract insight from data. **Example:** analyzing customer data to recommend products or forecast demand.

These fields overlap in practice, especially in modern applications where statistical reasoning, computational tools, and learning algorithms work together.

The diagrams below show two common ways these fields are represented visually. Although simplified, they highlight the core relationships and boundaries.

[Table omitted in PDF — see the online textbook]

6 Brief History of Data Science

Data science is a fairly new field. The term “data science” became popular in the early 2000s to describe a modern profession focused on using math, statistics, and computing to make sense of large and complex datasets, often called “big data.” Although data science feels like a modern invention, its roots stretch back much further than most people realize. At its core, the field grew out of two enduring human goals: to **collect data** and to **make sense of it**.

[Image omitted in PDF — see the online textbook]

Figure 5: A high-level timeline of ideas that shaped the modern field of data science. Source: Adapted from Florian Huber, images of Bayes and Gauss from Wikipedia.

Early Foundations: Statistics Meets Computation (1900s–1950s) In the early 20th century, statistics emerged as a formal discipline, giving scientists tools for summarizing and analyzing data. The arrival of digital computers in the mid-1900s transformed these ideas: statistical methods could now be implemented at speed and at scale, laying the groundwork for computational approaches to data.

Data Analysis as a Scientific Activity (1960s) In the 1960s, the idea of treating **data analysis itself as a scientific activity** began to take shape. The statistician **John Tukey** argued that data analysis deserved a role alongside theory and mathematics as a way of extracting meaning from raw information.

Knowledge Discovery in Databases (1970s–1980s) As more data began to be stored electronically in the 1970s and 1980s, research communities focused on how to organize and learn from large datasets. This gave rise to **KDD** (Knowledge Discovery in Databases), which emphasized finding patterns and insights from stored data; an early precursor to modern data mining and machine learning.

The Phrase “Data Science” Appears (1974) Around the same time, in 1974, **Peter Naur** used the phrase *data science* to describe the emerging combination of computing and data analysis; notably before the term was widely adopted in industry or academia.

Data Mining Becomes a Discipline (1990s) By the 1990s, advances in databases, algorithms, and computing infrastructure pushed data analysis further, helping establish data mining as a research discipline with conferences, communities, and standardized practices.

The Modern Term Takes Shape (Early 2000s) In the early 2000s, the term **data science** began to spread more broadly. In 2001, **William S. Cleveland** proposed redefining statistics for the era of computing and large-scale data. Meanwhile, the internet, search engines, and digital storage were producing unprecedented quantities of information — early signals of what would soon be called **big data**.

Mainstream Adoption and Industry Growth (2010s) In the 2010s, data science entered mainstream awareness. Massive data streams from the web, mobile devices, sensors, and business systems demanded new methods to store, process, analyze, and model information. Organizations began adopting data science to improve decision-making, automate tasks, and gain competitive advantage.

6.0.1 The Big Data Era Changes Everything

A major turning point came when technology companies such as **Google**, **Amazon**, **Facebook**, **Netflix**, and **LinkedIn** began collecting enormous amounts of behavioral data: clicks, searches, purchases, ratings, and social interactions. Traditional tools could not handle data at this scale, leading to new distributed systems such as **Hadoop**, **MapReduce**, and later **Spark**.

6.0.2 What Is Big Data?

“Big data” refers to data that is so large, fast, or complex that traditional tools cannot handle it easily. When people first started using the term, big data was explained with **three V’s**:

- **Volume** — a very large amount of data
- **Velocity** — how quickly data is created and needs to be processed
- **Variety** — data in many different forms (text, images, logs, videos, etc.)

As more companies and researchers began working with big data, they noticed that size alone did not explain the full challenge. Later, two more V’s were added:

- **Veracity** — refers to the quality of data that is being analyzed, as data may be messy, uncertain, or low quality.
- **Value** — making sure the data is useful and worth analyzing .

These V’s help show not only what big data looks like, but also why it matters. In some discussions, additional V’s such as variability, visualization, and validity are also mentioned, although the five V’s remain the most commonly used definition. <!--

[Image omitted in PDF — see the online textbook]

Figure 6. A visual depiction of the 5 V’s of Big Data: Volume, Velocity, Variety, Veracity, and Value. Source: miro.medium.com ->

[Table omitted in PDF — see the online textbook]

What made this period transformative was not just the size of the data, but how it was used. Companies learned to use data to recommend, rank, predict, and personalize services automatically. Data became a competitive advantage. As the value became clear, other industries, from finance and healthcare to transportation and retail, adopted similar strategies. By the late 2010s, data science had shifted from a niche specialty to a mainstream discipline practiced across many sectors.

Today, data science sits at the intersection of **statistics**, **computational methods**, and **domain knowledge**, supported by machine learning and deep learning. It continues to evolve as new tools reshape how we ask questions, what problems we can solve, and how data informs society.

6.0.3 Quick Timeline Summary

Early 1900s — Formal statistical methods emerge
1960s — Tukey emphasizes data analysis as a scientific activity
1970s–1980s — Electronic storage + computing → large datasets
1990s — KDD and data mining research communities develop
Early 2000s — “Data science” becomes a formal term (Cleveland 2001)
2010s — Big data, industry adoption, and machine learning growth
Today — Data science integrates computation, ML/DL, and domain context

7 The Data Science Life Cycle

Now that we have a sense of where data science came from, it is helpful to look at how it actually works in practice. Different organizations may have their own variations, but most data science projects follow a similar life cycle. The goal of this process is to move from raw data to insight, decisions, and value.

[Image omitted in PDF — see the online textbook]

Figure 7: Data Science Life Cycle (DSLC). Source: Dr. John Dickerson

7.0.1 Step I: Data Collection

The life cycle typically begins with **data collection** — gathering information from available sources. Data may come from sensors, surveys, transaction systems, public datasets, or internal databases. Data can be:

- **Primary data**, collected directly for the project (e.g., surveys, experiments, field observations, sensors).
- **Secondary data**, which already exists and was collected for another purpose (e.g., public datasets, social media, administrative records).

[Image omitted in PDF — see the online textbook]

Figure 8: Common data collection methods. Source: EDUCBA

Why it matters: Primary data is highly relevant but often costly and time-consuming. Secondary data is faster and cheaper to obtain, but may not perfectly match the analysis goals or may require additional cleaning and assumptions.

In practice, analysts often blend both types. For example, a public health researcher might conduct new surveys (**primary**) while using census data for context (**secondary**).

7.0.2 Step II: Data Processing

Once collected, data rarely arrives in clean form. **Data processing** involves organizing, cleaning, and transforming raw data so that it can support analysis. This may include fixing errors, handling missing values, resolving inconsistencies, or merging multiple datasets. Feature engineering often occurs here to create variables that better capture relevant relationships.

Why it matters: Clean, well-structured data increases the chance of extracting meaningful insights and prevents misleading results. Processing often consumes a large portion of a project.

Analysts frequently iterate between processing and earlier steps as data gaps or issues emerge.

7.0.3 Step III: Exploratory Data Analysis (EDA)

With processed data, **EDA** helps analysts explore patterns, distributions, correlations, and anomalies. Techniques include descriptive statistics and visualizations. For example, an analyst might examine the relationship between study hours and test scores by calculating average scores, looking at score ranges, and noticing that some students performed exceptionally well.

[Image omitted in PDF — see the online textbook]

Figure 9: Illustrative example of Exploratory Data Analysis (EDA). Source: GeeksforGeeks.

Visualization can make these patterns clearer. Plotting study hours against average scores may reveal that students who study more tend to achieve higher scores—a useful pattern that might inform later modeling.

Why it matters: EDA guides the rest of the project. It shapes hypotheses, informs modeling choices, and reveals data limitations.

EDA often sends analysts back to processing or even data collection to refine inputs.

7.0.4 Step IV: Analysis and Modeling

Next, the project moves into **analysis and modeling**, which may involve statistical inference or machine learning. The goal is to either explain current patterns or predict future outcomes, depending on the context.

Why it matters: Modeling provides structure and rigor. It allows relationships to be quantified and predictions to be made—insights that may be difficult to detect through raw inspection alone.

7.0.5 Step V: Insight and Decision-Making

Finally, results must be translated into decisions, recommendations, or actions. This may involve policy suggestions, business strategies, scientific findings, or product improvements. Clear communication is essential, as stakeholders often are not technical experts.

Why it matters: The value of data science is realized only when insights lead to decisions. A sophisticated model has little impact if it does not shape understanding or drive action.

Decision-making often generates new questions, sending the project back to earlier steps.

7.0.6 A Simple Example

Consider a city trying to improve traffic flow. It might collect data from sensors (**collection**), clean and merge logs (**processing**), explore patterns (**EDA**), build a model to predict congestion (**analysis and modeling**), and adjust traffic policies (**decision-making**). After implementation, new data arrives and the cycle repeats.

7.0.7 A Key Reminder

Data science is not a strictly one-way, linear process; it is dynamic, iterative, and adaptive.

In real projects, analysts often loop backward — re-cleaning data, adjusting models, re-thinking assumptions, or requesting new data. The process behaves more like a feedback loop than an assembly line.

7.1 BONUS SECTION: Career Paths in Data Science and What You Need to Learn

As data science has grown, it has expanded into a collection of roles rather than a single job title. Different organizations emphasize different parts of the data workflow, resulting in specialized paths

such as:

- **Data Scientists:** Blend programming, statistical modeling, and domain knowledge to generate insights and build predictive systems.
- **Data Analysts:** Extract patterns, summarize data, and produce reports; often work with SQL, spreadsheets, and BI tools.
- **Machine Learning Engineers:** Design, optimize, and deploy machine learning models at scale, working closely with software engineering teams.
- **Data Engineers:** Build and maintain data infrastructure; pipelines, databases, and large-scale storage, that make analysis possible.
- **Business Intelligence (BI) Developers:** Create dashboards and reporting systems that support operational and strategic decisions.
- **Decision Scientists:** Bridge analytics and strategy, translating analytical findings into business actions and priorities.

Early models of the field, such as **Drew Conway’s Venn diagram**, highlighted the intersection of hacking skills, statistics, and domain expertise.

[Table omitted in PDF — see the online textbook]

A more recent extension by **Stephan Kolassa** shifts the perspective to focus on **principles and goals** rather than just skill sets. In this framing:

- **Statistics** seeks to **explain and understand** relationships.
- **Machine Learning** focuses on **prediction and generalization**.
- **Artificial Intelligence** aims for **autonomous action or decision-making**.
- **Data Science** integrates these goals with computation, visualization, communication, and context.

This evolution reflects how different data roles prioritize different outcomes: explanatory, predictive, or operational.

7.1.1 What do you need to learn to become a good Data Scientist/Analyst?

There is no single recipe, but most data professionals draw from a few core pillars:

- **Programming & Tools:** Python, R, SQL, data wrangling libraries, and automation.
- **Statistics & Math:** Describing data, testing hypotheses, measuring uncertainty, and evaluating models.
- **Domain Knowledge:** Understanding the context of the data and how decisions are made in that domain.
- **Communication:** Turning findings into stories, dashboards, recommendations, or decisions.
- **Business Thinking:** Knowing what questions matter, how tradeoffs work, and how organizations operate.

Different career paths emphasize different combinations: analysts may lean toward communication and business reasoning, ML engineers toward coding and modeling, and data scientists toward modeling and domain integration.

7.2 Important Tools to Know

Category	Tools / Libraries
Programming	Python, R
Data Manipulation	Pandas, NumPy, dplyr
Databases	SQL (PostgreSQL, MySQL), NoSQL
Machine Learning	scikit-learn, XGBoost, LightGBM
Deep Learning	TensorFlow, Keras, PyTorch
Visualization	Matplotlib, Seaborn, Plotly, ggplot
Big Data	Spark, Hadoop
Deployment / APIs	Flask, FastAPI, Docker
Business Intelligence	Tableau, Power BI

The key idea is that **data science is not a single skill**, but a synthesis. Students enter the field from computer science, statistics, economics, biology, psychology, and many other areas because the field rewards curiosity, adaptability, and the ability to connect data to decisions.

Finally, it is important to note that **job titles in data science are not standardized**. The responsibilities of a “Data Scientist” at a research lab can look very different from the same title at a startup, a healthcare company, a financial firm, or a government agency. Likewise, some companies call roles “Machine Learning Engineer” or “Decision Scientist” while others bundle those tasks into “Data Scientist” or “BI Analyst.” Understanding the actual workflow and expectations of a role is often more important than the title itself.

8 Summary of the Chapter

Data science begins with a simple idea: the world produces traces, and those traces contain patterns. This chapter showed how the field grew out of older traditions—statistics, computing, and scientific experimentation, and how the explosion of digital data pushed those traditions into new territory. The goal is not just to collect information, but to ask questions that matter and to use data to answer them.

We introduced the notion of a **data science lifecycle**: defining a question, gathering data, cleaning and organizing it, analyzing patterns, building models when useful, and finally communicating what was learned. The lifecycle gives data science its shape. It is iterative rather than linear, driven as much by curiosity as by technique.

Along the way, we positioned data science within its broader ecosystem. Machine learning lends predictive tools; AI pushes toward perception and decision-making; databases and engineering keep the pipelines flowing; visualization turns results into stories people can act on. Big data added new challenges of scale and speed, and with it came new tools: Python, SQL, notebooks, and cloud systems, that make the work possible in practice.

This chapter was meant to set the stage. The chapters ahead will look closer at the materials of data science: what data looks like, how it is structured, how we measure and clean it, how we explore it, and how we eventually build models. Before we learn how to reason from data, we must become familiar with the data itself.

8.1 Knowledge Check

[Interactive content omitted in PDF]

9 Chapter 2 - Understand Data Types

10 The Many Shapes of Data

Data rarely shows up in the neat spreadsheet we hope for. It arrives **messy, uneven, unexpected**, a mix of numbers, text, images, timestamps, social connections, and whatever the world records. Before we can analyze anything, we must learn how to recognize the shape that data takes.

Engineers sometimes describe data as “raw.” That sounds intimidating, but it really means the information hasn’t been processed yet. It is potential, waiting to become evidence, insight, or sometimes, just noise.

10.0.1 What Counts as Data?

If you sit in a coffee shop for ten minutes and simply observe, you’ll see data everywhere: prices on a menu, Wi-Fi signal strength, the time between notifications on your phone, the music playlist, the number of people wearing headphones, and the language of their conversations. All of it is data, even if nobody wrote it down.

In computing and data science, we usually store data as numbers, text, images, audio, or combinations of these. But the real distinction isn’t the material; it’s the structure.

What is Data? Data is raw information: facts, measurements, observations, or descriptions that can be stored, transmitted, and analyzed. In data science, data refers to representations of information that can be digitized and manipulated by computational systems. It can take many forms, including numerical, categorical, text, image, audio, temporal, or relational.

Depending on how it is organized, data may be structured, semi-structured, or unstructured.

Scientists, analysts, and businesses collect data because the patterns hidden within it help them understand behavior, make decisions, and predict what comes next.

Remember: data becomes valuable only after it is interpreted and transformed into knowledge or used to answer meaningful questions.

At its simplest, data is recorded information about the world. It might be a number, a word, a photograph, a sound, or a sensor reading. On its own, data does not explain or prove anything; it simply captures what exists or what has happened.

Data becomes useful when we organize it, analyze it, and interpret it in context. A single heart rate reading means very little; a series of readings across time can reveal stress, fitness, or disease. Data does not guarantee truth; it offers evidence.

11 How Do We Describe or Measure Data?

Before we think about how data is stored or structured, it is useful to ask a simpler question: *what kind of information are we dealing with?* Not all data behaves the same way. Some can be counted, some can be measured with precision, and some can only be described or categorized.

Broadly, based on what the data represents, we can divide it into two classical types:

- **Quantitative data**, which expresses numerical quantities (how much, how many, how fast)
- **Qualitative data**, which expresses categories, labels, or descriptive attributes (what kind, which type)

11.0.1 Quantitative Data

Quantitative data expresses **quantities**. It tells us *how much*, *how many*, or *how fast*. This kind of data can be measured numerically and analyzed mathematically. We can add it, average it, compare it, and track how it changes over time.

Quantitative data often comes in two forms:

- **Discrete:** countable values (e.g., number of laptops, number of cars, number of students)
- **Continuous:** measurable values that vary smoothly (e.g., height, weight, temperature, time)

Discrete values make jumps (from 1 to 2 to 3), while continuous values can take any value within a range.

11.0.2 Qualitative Data

Qualitative data describes **qualities, categories, or labels**. Instead of telling us *how much* of something exists, it tells us *what kind* it is.

Examples include hair color, species, survey responses, movie genres, or customer satisfaction levels. Qualitative data can be grouped into categories, but arithmetic operations like averaging or subtracting do not make sense here.

[Image omitted in PDF — see the online textbook]

Figure 1. Classical categorization of data types. Quantitative data may be discrete or continuous, while qualitative data describes categorical properties. Source: International Journal of Neurolinguistics & Gestalt Psychology.

11.0.3 Why This Matters in Data Science

These distinctions shape how we visualize, summarize, and model data. For example:

- Heights (**continuous**) may be plotted on a histogram
- Survey answers (**qualitative**) may be shown as bar charts
- Counts (**discrete**) may be modeled with Poisson or binomial distributions
- Categorical labels may be encoded for machine learning (e.g., **one-hot encoding**)

Knowing *what kind* of data we have is often the first step in deciding *what tools* to use.

12 How Is Data Organized or Stored?

So far, we have described data based on what it represents (quantitative vs. qualitative). Data science also cares about a different and equally important dimension: **how the data is organized for computation**. The structure of data influences how easily it can be stored, queried, cleaned, and analyzed.

Unlike quantitative/qualitative categories (which describe meaning), structural categories describe the **shape and format** of data.

In data science, the key difference isn't what the data *is* made of (numbers, pixels, words), but how it is *organized*. In practice, we encounter three major structural families:

1. **Structured data**: rigid, organized, predictable
2. **Semi-structured data**: flexible, loosely organized
3. **Unstructured data**: non-tabular data (such as text, images, audio, and video) without a predefined schema that does not fit neatly into tables

[Image omitted in PDF — see the online textbook]

[Image omitted in PDF — see the online textbook]

Figure 2. Structured, Semi-Structured, and Unstructured data formats. Source: Skyvia

These categories apply to text, numbers, images, audio, and almost any modern dataset. Each category demands different tools, techniques, and assumptions.

[Image omitted in PDF — see the online textbook]

Figure 3. Comparison of Structured, Semi-Structured, and Unstructured data. Source: Arya.ai

In modern systems, all three often coexist: structured transactional data, semi-structured event logs, and unstructured user content.

12.1 1. Structured Data: The Orderly World

Structured data is neatly organized, predictable, and follows a predefined schema. Structured data behaves politely. It typically appears in **tables**, with rows and columns, where each field has a specific data type (e.g., integer, string, boolean).

Example structures include:

- Tabular data (spreadsheets, CSV files)
- Relational databases (PostgreSQL, MySQL)
- Time-based or Temporal tables (stock prices, sensor logs)

[Image omitted in PDF — see the online textbook]

Figure 4. Structure Data. Source: Miro.medium

Structured data is easy to query (SQL), easy to validate, and easy to join across tables. It is the backbone of business analytics and scientific measurement.

12.1.1 (a) Tabular Data: The Spreadsheet World

Tabular data is what most people picture first, a **rectangular grid** of rows and columns, familiar from Excel or Pandas. Each row represents an entity, each column a property. A database table of airline flights, for example, might have:

- departure airport
- arrival airport
- time
- delay
- airline

[Image omitted in PDF — see the online textbook]

Figure 5. Conceptual diagram illustrating internal structure or hierarchy. Source: Devopedia

There’s a reason the spreadsheet survived decades of technological change: it works. Its shape matches how humans reason about comparisons (Who spent more? Which day had the highest sales?).

12.1.2 (b) Time-Based Data: The Rhythm of Change

Not everything stays constant. Stock prices drift by the second, weather shapes the day hour by hour, and fitness trackers quietly log your heartbeat as you sleep.

Time-based (or **temporal**) data puts events in order and lets us ask how something changes rather than merely what it is. Analysts in finance, meteorology, and operations live in this world. The key idea is not the value itself, but the **trajectory**.

A single temperature reading is trivia; a year of temperature readings reveals climate.

[Image omitted in PDF — see the online textbook]

Figure 6. Examples of time-series visualizations showing how values evolve over time. Source: ChartExpo

Temporal data allows us to detect trends (upward/downward), cycles (seasonal patterns), anomalies (abrupt changes), and responses to interventions (before/after effects). Many forecasting tasks—from weather to sales to traffic—are built on time-series models.

12.2 2. Semi-Structured Data: The Flexible Middle

Semi-structured data does not fit neatly into tables (doesn’t prescribe fixed column) , but it still contains **markers, tags, or key-value pairs** that impose some level of organization (structure and hierarchy).

Common examples include:

- JSON (web APIs, configuration data)
- XML and HTML (documents and web pages)
- logs (server events, application traces)
- key-value pairs (NoSQL databases)

[Image omitted in PDF — see the online textbook]

Figure 7. Semi-structured data can be represented using JSON, XML etc. Source: Medium (miro.medium.com)

Semi-structured data is more flexible than structured data. It supports nesting and irregular formats, which is why modern web APIs and mobile apps use JSON extensively.

12.3 3. Unstructured Data: The Untamed Majority

Most digital information refuses to sit neatly in tables or hierarchical tags. This is **unstructured data**, and it now dominates the modern world.

Unstructured data lacks a predefined schema and is difficult to store in rows and columns. It includes rich media, human language, and other natural forms of expression.

Examples include:

- images (satellite photos, medical scans, selfies)
- audio (voice commands, music, podcasts)
- video (streaming content, surveillance footage)
- text (tweets, emails, reviews, articles)
- graphs of social or biological networks

For decades, computers couldn't make sense of this category. The recent boom in machine learning, NLP for text, CNNs for images, transformers for everything, is largely about taming unstructured information.

12.3.1 (a) Images: Data You Can See

A picture is not a number, until a model turns it into one.

Images are deceptively simple. What we perceive as a photograph is actually a grid of **pixels**, tiny squares holding values for red, green, and blue. Those values range from 0 to 255 and describe how bright each channel should appear.

Image formats involve trade-offs: JPEG compresses aggressively and loses some detail; PNG preserves every pixel; medical imaging prefers lossless formats because a mistake isn't aesthetic, it's scientific.

Compression, resolution, and color channels are not trivia. They determine how algorithms “see.”

12.4 Special Case of Data: “Graphs” — Data with Relationships

Some data is best understood through **relationships**, not just individual values. A social network, a map of roads, or protein interactions inside a cell are examples.

Graphs introduce their own structure:

- **nodes** represent objects (e.g., people, airports, proteins)
- **edges** represent relationships (e.g., friendships, flights, bindings)

The power lies in the **pattern of connections** rather than the elements themselves. Who follows whom? Which cities connect? Which proteins interact? Influence, connectivity, centrality, and path structure become more important than single values.

[Image omitted in PDF — see the online textbook]

Figure 8. Example graph representation showing nodes connected by edges. Source: freeCodeCamp

12.5 Why Data Structure Matters

Data structure determines:

- how easily data can be queried and indexed
- what tools or languages we use (SQL vs. parsers vs. ML models)
- how expensive or difficult preprocessing becomes
- how storage systems are designed (databases vs. file systems vs. cloud object stores)

For example, structured data works well with SQL and joins. Semi-structured data works well with hierarchical queries and document stores. Unstructured data may require embeddings, tokenization, image processing, or deep learning.

Modern data systems often mix all three. A single application may store user profiles in a relational table, telemetry in a JSON log, and user photos as unstructured images.

13 Data Formats in Data Science

Once we understand how data is described and how it is organized, the next layer to consider is how data is **formatted** for storage, transport, and consumption by tools. Formats determine how information is encoded on disk, how it moves between systems, and how efficiently it can be parsed or processed.

Different formats are optimized for different tasks: tabular data favors simple delimiters, hierarchical data favors nested keys, documents favor markup, and rich media requires specialized encoding. Understanding formats is essential for working with modern data pipelines.

These structural categories map naturally to different file formats used in practice:

- **Structured** → CSV, SQL tables
- **Semi-structured** → JSON, XML, HTML, logs
- **Unstructured** → JPEG, MP3, MP4, PDF, TIFF

The choice of format influences the tools we use, the complexity of data cleaning, and the computational effort required to extract meaningful information.

13.1 (I) CSV and TSV

CSV (Comma-Separated Values) and TSV (Tab-Separated Values) are among the simplest and most widely used data formats. They store **tabular**, structured data in plain text, where each row represents a record and each column corresponds to a field.

A **delimiter** is the character that separates values in a row. Common delimiters include commas (,), tabs (\t), semicolons (;), and colons (:).

CSV uses commas as the delimiter; commonly used for tabular data. It is popular because it is lightweight, human-readable, and compatible with nearly every data analysis tool, from Excel to Python's *pandas*. Example CSV snippet:

[Image omitted in PDF — see the online textbook]

Figure 9. Example workflow showing how spreadsheet formats such as Excel (XLSX) can be converted to CSV for use in data processing. Source: myexcelonline.com

Python makes CSV loading straightforward:

```
import pandas as pd
df = pd.read_csv("data.csv")
```

TSV uses tabs instead of commas, often improving readability when fields contain commas.

These formats are ideal for data exchange and quick inspection, but lack explicit typing, indexing, and schema validation.

13.2 (II) JSON

JSON (JavaScript Object Notation) represents **semi-structured**, hierarchical data. It supports nesting, arrays, and variable-length fields, making it flexible for web APIs, mobile applications, and configuration data.

JSON structures are built from **key–value pairs**, where each key identifies a field and each value holds the associated data.

13.2.1 JSON Syntax Basics

At the smallest level, JSON data is expressed as a **key–value pair**: JSON follows a small set of structural rules:

- Data is presented in key/value pairs.
- Data elements are separated by commas.
- Curly brackets {} determine objects.
 - **Objects** are enclosed in { } and contain key–value pairs

[Image omitted in PDF — see the online textbook]

- Square brackets `[]` designate arrays.
 - **Arrays** are enclosed in `[ ]` and hold ordered lists

[Image omitted in PDF — see the online textbook]

- **Keys** are strings: sequences of characters surrounded by quotation marks.
- **Values** may be strings, numbers, booleans, arrays, objects, or `null`

[Image omitted in PDF — see the online textbook]

Remember, A colon is placed between each key and value, with a comma separating pairs. Both components are held in quotation marks. As a result, JSON object literal syntax looks like this:

```
{“key”:“value”,“key”:“value”,“key”:“value”.}
```

Example JSON snippet:

JSON also supports nesting through objects and arrays. For example:

```
{
  "name": "Alice",
  "age": 24,
  "languages": ["Python", "SQL"],
  "education": {
    "degree": "BS",
    "major": "Computer Science",
    "year": 2025
  }
}

{
  "students": [
    {"firstName": "Tom", "lastName": "Jackson"},
    {"firstName": "Linda", "lastName": "Garner"},
  ]
}
```

JSON is human-readable, self-describing, and machine-friendly. Most modern RESTful APIs return data in JSON.

In **Python**, the `json` module is commonly used to work with JSON data. > We use `json.dumps()` to convert Python objects to JSON format and `json.loads()` to convert JSON data back to Python objects.

[Image omitted in PDF — see the online textbook]

Example:

```
import json

# Python object (dict)
```

```

person = {
    "name": "Alice",
    "age": 24,
    "languages": ["Python", "SQL"]
}

# Convert to JSON string
json_str = json.dumps(person)
print(json_str)

# Convert back to Python object
person_back = json.loads(json_str)
print(person_back["languages"])

```

13.3 (III) HTML and XML

HTML (HyperText Markup Language) and XML (eXtensible Markup Language) are markup languages that describe structure using tags.

HTML (HyperText Markup Language) describes the structure and display of web pages. It defines elements such as headings, paragraphs, links, and tables. HTML focuses on **presentation** and is interpreted by web browsers.

Example HTML snippet:

```

<p>Hello, world!</p>
<a href="https://umd.edu">Visit UMD</a>

```

HTML is not intended to store arbitrary data for computation. Instead, it tells browsers how content should appear and how users interact with it.

XML (eXtensible Markup Language) uses tags to describe **data and relationships** rather than presentation. Unlike HTML, XML does not have predefined tags. Users may *define their own custom tags*, which makes XML flexible for representing custom structures and domain-specific information.

XML is **self-describing**, **hierarchical**, and can represent nested structures similar to JSON.

XML is often used for data storage or exchange, especially in enterprise systems, configuration files, and legacy APIs.

Example XML snippet:

```

<person>
  <name>Alice</name>
  <age>24</age>
  <languages>
    <language>Python</language>
    <language>SQL</language>
  </languages>
</person>

```

XML values are contained inside tags, and fields can repeat or nest, which gives XML flexibility for representing structured documents.

HTML is primarily for presentation, while XML is more general-purpose and can encode hierarchical data for communication between systems.

Where They Appear

- HTML → browsers, webpages, crawlers, scraping
- XML → configuration files, enterprise systems, legacy APIs, document formats (e.g., DOCX uses XML internally)
- JSON → REST APIs, mobile apps, NoSQL databases, log formats

All three formats belong to the broader landscape of semi-structured data, sitting between rigid tables and free-form text.

13.4 (IV) Relational Databases

Relational databases store data in a structured form using **tables** (rows and columns) and a well-defined **schema**. Each table represents an entity (e.g., **students**, **courses**, **orders**), and each row represents a record. Tables can be linked through **keys**, allowing meaningful relationships to be expressed and queried efficiently.

Relational formats enforce consistency through datatypes, primary keys, foreign keys, and constraints. They excel in transactional systems (e.g., banking, e-commerce, logistics).

13.4.1 Why “Relational”?

The term **relational** comes from mathematics: each table is a **relation**, each row is a **tuple**, and each column is an **attribute**. Tables become powerful when they are **related** to one another through shared keys.

These relationships allow us to combine information across tables (e.g., *students courses enrollments*) without duplicating data across the system.

13.4.2 Keys: Linking Information

Relational databases rely on two important types of keys:

- A **primary key** uniquely identifies a row within a table (e.g., **student_id**).
- A **foreign key** references a primary key in another table, creating a relationship.

Example (conceptual): - **students(id)** is the primary key for students. - **enrollments(student_id)** is a foreign key pointing back to **students(id)**.

Keys enforce consistency and prevent invalid references (e.g., enrolling a student that does not exist).

13.4.3 What is a Database Schema?

A **schema** is the blueprint of the database. It defines:

- what tables exist
- what columns each table contains
- the datatype of each column
- how tables relate to one another (via keys)
- rules or constraints that ensure valid data

Example schema (informal):

```
students(  
  id INTEGER PRIMARY KEY,  
  name TEXT,  
  major TEXT,  
  year INTEGER  
)
```

[Image omitted in PDF — see the online textbook]

Figure 10. Relational schema showing tables linked through keys. Here, “customer_id” is a **primary key** in the **customers** table and a **foreign key** in the **orders** table, linking orders to customers. Data is stored once and referenced through keys. Source: PlanetScale

Schemas enforce consistency and make it possible to write queries that join or filter across multiple tables.

13.4.4 DBMS: Database Management Systems

A **DBMS (Database Management System)** is software that stores, manages, and provides controlled access to databases. While a **database** refers to the data itself, the **DBMS** manages how the data is stored, retrieved, updated, indexed, and protected.

Relational DBMSs (also called **RDBMS**) implement the relational model and use SQL for querying. Common examples include:

- PostgreSQL
- MySQL
- SQLite
- SQL Server
- Oracle

In short: **the database stores the data; the DBMS manages the data.**

Key idea of DBMS: applications and users never talk directly to the raw data; the DBMS sits in the middle, handling queries, security, concurrency, and storage.

[Image omitted in PDF — see the online textbook]

[Image omitted in PDF — see the online textbook]

Figure 11. Conceptual views of a DBMS. Both diagrams illustrate how applications interact with a database through a DBMS layer that manages queries, storage, and controlled access. Source: HBS SQL Novice Survey (top) and LinkedIn (bottom).

13.4.5 SQL and Querying

The language used to interact with relational databases is **SQL (Structured Query Language)**. SQL supports powerful operations for filtering, joining, aggregating, and updating structured data.

Relational systems enforce consistency through:

- **data types** (integer, string, date, boolean, etc.)
- **primary keys** (unique identifiers for rows)
- **foreign keys** (references linking tables)
- **constraints** (rules that maintain data integrity)

These properties make relational databases ideal for transactional systems such as banking, logistics, retail, reservations, and scientific measurement. **Example Table: Students**

id	name	major	year
1	Alice	Computer Science	2025
2	Bob	Mathematics	2024

Example SQL Query

```
SELECT name, major
FROM students
WHERE year = 2025;
```

(We will talk about SQL in later chapter)

Relational databases excel when data must be consistent, structured, and queryable, especially when multiple tables relate to each other (e.g., students courses enrollments).

They are widely used in banking systems, logistics, healthcare, retail, reservations, and scientific measurement—any domain where correctness and structure matter.

13.5 BONUS Part: NoSQL Databases

Relational databases shine when data is highly structured and relationships are well-defined. But not all data fits neatly into tables. Modern applications produce logs, events, nested objects, user interactions, sensor streams, and media, data that grows fast and changes shape over time.

NoSQL databases *relax the rigid schema* of relational systems to support higher scalability and more flexible data models. Instead of enforcing a single blueprint (schema), NoSQL systems allow the structure of records to evolve.

Different NoSQL databases optimize for different shapes of data:

- **Document stores** (e.g., MongoDB, Couchbase)
Store JSON-like objects with nested fields. Great for user profiles, product catalogs, or application events.
- **Key-value stores** (e.g., Redis, DynamoDB)
Store simple (key $\rightarrow$ value) mappings. Ideal for caching, session storage, configuration, and real-time lookups.
- **Wide column stores** (e.g., Bigtable, Cassandra)
Organize data by column families. Used in time-series analytics, telemetry, and large distributed workloads.
- **Graph databases** (e.g., Neo4j)
Represent nodes and edges directly. Useful for social networks, recommendation engines, fraud detection, and biological pathways.

[Image omitted in PDF — see the online textbook]

Figure 12. High-level comparison of relational (SQL) and non-relational (NoSQL) databases, highlighting differences in schema, consistency, and scalability. Source: ScyllaDB

NoSQL systems often target large-scale, distributed environments and are designed to **scale horizontally** (add more machines) rather than vertically (buy a bigger machine).

NoSQL gained popularity in the era of cloud and internet-scale platforms, where data volumes are high, latency matters, and structure is fluid. Common scenarios include:

- millions of user profiles with different fields
- real-time click streams and telemetry data
- nested JSON returned from APIs
- social networks with rich relationships
- sensor and IoT data
- rapidly evolving application data

In short, **NoSQL trades strict relational structure for flexibility and scale**. It does not replace relational databases, it complements them.

13.6 (V) Images, Audio, and Video

Not all data is textual or numerical. Rich media formats dominate modern applications, especially in machine learning, robotics, healthcare, and online platforms.

Examples include:

- **Images:** JPEG, PNG, TIFF
- **Audio:** WAV, MP3, FLAC
- **Video:** MP4, AVI, MOV

These formats encode pixels, waveforms, or frames in compressed or uncompressed forms.

13.6.1 How Image Data Works

Images are typically organized as a grid of **pixels**, the tiny building blocks that form the picture. Each pixel holds **color information** through separate **channels**. In a standard color image:

- the channels are **red, green, and blue (RGB)**
- each channel acts like a separate layer of information
- each channel value ranges from **0 to 255**, representing intensity
 - 0 means no intensity (dark)
 - 255 means full intensity (bright)

[Image omitted in PDF — see the online textbook]

Figure 13. Visualization related to image data structures or pixel representation. Source: Filestack

Combining intensities across channels produces the full range of visible colors. Machine learning models (e.g., vision transformers, CNNs) operate directly on these pixel arrays and often convert them into structured **embeddings** for downstream tasks such as classification or detection.

Audio and video follow the same idea at higher dimensionalities: audio stores **waveforms** over time; video stores sequences of **image frames**, sometimes with sound and metadata attached.

Machine learning models for speech, vision, and multimodal data help translate these rich formats into structured forms that can be queried, compared, or used for prediction.

13.7 Choosing the Right Format

There is no single “best” format. CSV is ideal for spreadsheets; JSON is ideal for APIs; SQL is ideal for transactions; NoSQL is ideal for flexible schemas; media formats are essential for perception tasks. Data scientists must often convert between formats depending on the application.

14 Where Does Data Come From?

Historically, datasets arrived through organized channels: government surveys, institutional studies, corporate databases. Today, data spills from everyday systems. Almost every action, transaction, or device leaves a trace.

Common sources include:

- transactional databases
- sensors and wearables
- web APIs

- web scraping
- user interactions
- application logs

Some sources deliver structured data; others arrive messy and require transformation before analysis.

14.1 Scraping vs. APIs

When acquiring data from the web, two patterns are common: **web scraping** and **APIs**.

Web scraping imitates a browser, reading HTML and extracting what a human would see. It is useful when no official data interface exists, but fragile and sensitive to ethical and legal boundaries.

APIs, in contrast, are explicit contracts:

“If you send a structured request, I will send structured information.”

APIs typically return machine-readable data (often JSON), avoiding the need to parse HTML manually.

Before scraping, it is ***always worth checking*** if an API exists.

14.1.1 How Web Scraping Works (Conceptual)

[Image omitted in PDF — see the online textbook]

1. Your script sends an HTTP GET request to a URL.
2. The server returns an HTML page.
3. Your program parses and extracts the relevant parts.

14.1.2 (a) BeautifulSoup and Parsing HTML

When websites lack an API, we fall back to HTML, the layer designed for humans. **Beautiful Soup** converts that visual layer into structured data.

BeautifulSoup is a Python library for parsing and extracting information from HTML; it takes the raw HTML returned by a website and lets you navigate and extract useful parts such as tables, links, or text.

HTML is hierarchical: tags contain tags, attributes provide meaning, and text sits inside. BeautifulSoup lets us locate elements (**div**, **table**, **span**) and extract text, links, or tabular content.

[Image omitted in PDF — see the online textbook]

[Image omitted in PDF — see the online textbook]

[Image omitted in PDF — see the online textbook]

Figure X. HTML-based scraping workflow: identify elements visually, locate tags, extract content, convert to structured data.

Scraping is useful for product data, weather reports, job postings, and research tables — information that exists publicly but without a formal data interface.

Tiny Example: Requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
html = requests.get(url).text
soup = BeautifulSoup(html, "html.parser")

titles = [tag.text for tag in soup.find_all("h2")]
```

Pattern: download → parse → select → extract → clean

Not every page has the same structure, so scraping often involves inspection and adaptation.

Bonus Trick: Scraping Tables with pandas Pandas can parse HTML tables directly:

```
import pandas as pd

df_list = pd.read_html("https://example.com/table")
df_list[0].head()
```

If the page contains “< table > ... < /table >” tags, this often works instantly; no tagging, no loops, no manual parsing.

14.1.3 BONUS Pandas Example: Scraping Population Data from an HTML Table

Suppose you want to collect the latest population estimates for U.S. states. The site has no public API, but shows an HTML table visible in the browser; a perfect case for scraping.

The page might display data like this:

State	2024 Population	Growth Rate	Area (sq mi)	Density (per sq mi)
California	39,074,000	-0.59%	163,695	240
Texas	31,161,000	+1.44%	268,597	116
Florida	23,655,000	+1.37%	65,758	345
...	...	...	...	...

Because this data lives inside HTML <table> tags, we can use `requests` to download the page, `BeautifulSoup` to parse the HTML, and then convert the table into a Pandas `DataFrame`. Here is the conceptual Python snippet that accomplishes this:

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# 1. Fetch the HTML
url = "https://worldpopulationreview.com/states"
response = requests.get(url)
```

```

html = response.text

# 2. Parse with BeautifulSoup
soup = BeautifulSoup(html, "html.parser")

# 3. Find the main table on the page
table = soup.find("table")

# 4.1 Extract rows
rows = table.find_all("tr")

# 4.2 Loop over the rows of the table
data = []
for row in rows:
    # Get all header or data cells in this row
    cells = row.find_all(["td", "th"])
    # Extract the text and strip extra spaces
    texts = [cell.get_text(strip=True) for cell in cells]
    if texts: # skip empty rows
        data.append(texts)

# 5. First row is the header, the rest is the body
header = data[0]
body = data[1:]

# 6. Build a DataFrame from the scraped data
df = pd.DataFrame(body, columns=header)

# Take a quick look
df.head()

```

Once in a DataFrame `df`, the data becomes programmable: you can clean, convert types, sort, filter, and analyze.

Why Scraping Is Fragile? Scrapers often break because HTML is designed for humans, not for machines. Small changes can cause failure: - renaming CSS classes - changing layout - adding ads or pop-ups - switching to JavaScript-rendered tables

Contrast: - APIs are contracts - HTML is performance

APIs are meant for machines; HTML is meant for browsers.

14.1.4 Typical API Workflow

APIs remove the guesswork of scraping by returning structured data directly. The workflow is straightforward:

1. Send a request (often a `GET`) to an API endpoint

2. Receive structured data (usually JSON)
3. Convert the JSON into a DataFrame for analysis

Example (conceptual):

```
import requests
response = requests.get("https://api.example.com/weather")
data = response.json()
```

This is the programmatic equivalent of asking a well-behaved machine for information.

14.1.5 (b) RESTful APIs (Representational State Transfer)

RESTful APIs dominate modern data access: a client requests information, a server processes it, and structured data (often JSON) returns.

RESTful API: A style of API where data is organized as resources (e.g., `/users`, `/repos`) accessed using HTTP verbs such as GET, POST, PUT, and DELETE.

[Image omitted in PDF — see the online textbook]

Figure 14. High-level overview of a RESTful API: structured request → structured response.
Source: Medium

Tiny Example: Requests + RESTful APIs The GitHub API is a public **RESTful API** that returns structured JSON data. Unlike scraping, no HTML parsing is required.

```
import requests
import pandas as pd

# 1. Send a GET request to a RESTful API endpoint
url = "https://api.github.com/repositories"
response = requests.get(url)

# 2. Parse the JSON response
data = response.json()

# 3. Convert JSON into a DataFrame
df = pd.json_normalize(data)

df.head()
```

For data science, the key takeaway is practical: RESTful APIs provide predictable, machine-readable access to fresh data. Modern dashboards, pipelines, and ML systems rely on them.

A Note on Etiquette Data availability is not the same as data permission. Good acquisition respects rate limits, terms of service, and the human effort behind the page or system being scraped.

14.2 Summary of the Chapter: The Point of All This

Data does not come in a single form. It may appear as numbers in a table, sounds from a microphone, pixels in an image, words in a review, or connections in a network. Each shape of data brings its own assumptions and tools, and more importantly, its own way of seeing the world. A spreadsheet invites comparison; a time-series reveals change; a graph exposes relationships; and an image asks to be interpreted.

The first step in data science is simply recognizing **what kind of information we have** and **how it is organized**. We do not analyze a voice recording the way we analyze an invoice, nor do we treat a timestamp like a photograph. Structure matters. It determines what questions we can ask and what answers we can trust.

Understanding data shapes is also practical. It tells us which formats are useful, which tools to reach for, and how much cleaning or transformation will be required. We do not clean data just to make it pretty, we shape it so that it can answer a question.

The next chapter takes us from **recognizing data** to **reasoning with data**. It will focus on asking better questions and on the principles of **experimental design**, where curiosity meets structure and evidence begins to speak.

14.3 Knowledge Check

[Interactive content omitted in PDF]

15 Chapter 3 - Experimental Design

16 Experimental Design

In data science, experiments are essential when we want to understand **cause and effect** rather than just correlation. Experimental design gives us a structured way to plan tests, collect data carefully, and draw conclusions we can trust. Data science is not only about analyzing data; it is also about **generating the right data** to answer meaningful questions. Whether the goal is to test a new website feature, evaluate a marketing campaign, or measure the effect of a medical treatment, well-designed experiments help us learn the most from the data we collect and establish cause-and-effect relationships.

While statistics and machine learning often look for patterns in existing data, experimental design asks a deeper question:

“What would happen if we changed something?”

To answer this question, we need a **systematic process** for comparing outcomes, controlling sources of bias, and making valid inferences from our results.

16.0.1 What Is Experimental Design?

Experimental design is a structured method for planning, conducting, and analyzing experiments so that we can answer specific questions accurately and efficiently. In data science, this often involves:

- Formulating a clear hypothesis

- Selecting variables to manipulate and measure
- Choosing a design that reduces bias
- Collecting data in a way that supports reliable conclusions
- Analyzing outcomes with appropriate statistical tools

At its core, experimental design ensures that the signal we want to detect is not drowned out by noise, randomness, or bias.

16.0.2 Why Experiments Matter

In many real-world settings, observational data can tell us what has happened, but not necessarily **why** it happened. Experiments allow us to:

- Test hypotheses
- Compare alternatives (A/B testing)
- Measure causal effects
- Reduce uncertainty before deployment
- Inform decisions with evidence rather than intuition

[Image omitted in PDF — see the online textbook]

Figure 1. Experimental design concepts showing explanatory, response, and lurking variables. Source: GeeksforGeeks (Experimental design explanation).

Tech companies such as Google, Amazon, and Netflix run thousands of controlled experiments each year to optimize recommendations, pricing strategies, user interfaces, and overall user engagement.

For organizations, good experiments reduce risk. Instead of guessing whether a new policy, product, or model will work, experiments let us **try before we commit**.

Experimental design **provides the structure** for collecting, comparing, and interpreting data in a reliable way. Without it, we risk drawing the wrong conclusions, wasting resources, or shipping products that don't actually help.

16.0.3 Why Data Science Needs Experimental Design

Experiments allow us to answer practical questions such as:

- Which version of a webpage increases sales?
- Will a new pricing strategy increase revenue?
- Will students who study more get higher exam scores?

Many data science problems boil down to choosing the option that maximizes a desired outcome (e.g., click-through rate, revenue, accuracy, retention, or health outcomes). In other words:

Given multiple choices, which option leads to the best outcome?

To answer this effectively, we need a plan for:

- what to measure

- what to manipulate
- how to collect data
- how to avoid bias
- how to deal with confounders

Together, these ideas form the foundation of experimental design and set the stage for how we use experiments in data science practice.

In this chapter, we will introduce the key components of experiments, discuss how to reduce bias, and explore how A/B testing is used widely across modern technology platforms.

17 Key Ideas in Experimental Design

Although experimental design is a broad field, several concepts appear repeatedly in data science applications:

17.1 (I) Hypothesis & Research Question

17.1.1 Asking the Right Question

Every experiment begins with **a question**, and asking the right question is important. This is the big-picture motivation behind why we run an experiment at all. It focuses on decision-making, usefulness, and purpose.

“Data is the new science. Big Data holds the answers. Are you asking the right questions?”

— Patrick P. Gelsinger, Former CEO of Intel

A data science project is only as valuable as the question it answers. In practice, these questions can have **different decision goals**. Some questions are simple and descriptive, while others help us make better decisions.

Different Decision Goals in Data Science

Decision Goal	Core Question	Purpose	Example (same scenario)
Descriptive	What happened?	Summarizes past or current state	“Sales dropped last week.”
Diagnostic	Why did it happen?	Explains patterns and associations (not causal)	“Sales dropped because shipping was slow.”
Predictive	What will happen next?	Forecasts future outcomes	“If shipping stays slow, sales will continue to drop.”

Decision Goal	Core Question	Purpose	Example (same scenario)
Causal	What is the effect of doing X?	Identifies cause–effect relationships under interventions	“Speeding up shipping increases sales.”
Prescriptive / Optimization	What should we do?	Chooses the best action to achieve a desired outcome	“Speeding up shipping is the best way to increase sales.”

[Image omitted in PDF — see the online textbook]

Figure 2: Hierarchy of decision goals in data science and analytics.

As we move from descriptive to diagnostic, predictive, causal, and finally prescriptive/optimization questions, the insights become more useful for real-world decisions, but also harder to answer. Harder questions often require more data, stronger assumptions, better models, and sometimes experiments or interventions to figure out cause and choose the best action.

What is Optimization?

Optimization is about choosing the **best action** to reach a goal. Goals can vary depending on the context of the problem, such as increasing sales, reducing wait times, improving accuracy, or boosting conversions. Formally, we define an objective ($f(x)$) (something we care about) and choose the option that maximizes or minimizes ($f(x)$).

For example:

“Will showing product reviews increase purchase conversions?”

A tech company might test two landing pages to see which one produces a higher click-through rate (CTR). A hospital might test different rehabilitation schedules to see which one speeds recovery. A city might test new traffic signal timings to reduce congestion.

Connecting Back to Experiments

Before we can optimize, we often need to understand **causal effects** (e.g., “If we change X, what happens to Y?”). This is where experiments come in. Experiments allow us to compare the results of different actions and see which one performs better.

At this stage, the goal is not to prove anything mathematically. It is simply to clarify what we want to learn and what outcome matters. These questions are later converted into **testable hypotheses** (null vs. alternative) that allow us to evaluate results.

We will discuss null and alternative hypotheses in a *later chapter*.

17.1.2 Hypotheses as Structured Guesses

A hypothesis is a **formal educated guess** about how changing an input affects an outcome:

If we increase study time, exam scores will increase.

If we reduce friction during checkout, purchases will increase.

Hypotheses **do not guarantee truth**; they merely propose a relationship worth testing. Good hypotheses are directional (predict how things move), falsifiable (can be wrong), and relevant (connected to the decision at hand).

In data science, hypotheses often serve a practical purpose: they tell us what data to collect and what comparisons matter.

17.2 (II) Variables & Outcomes

Once we have a research question, the next step is to identify the key components of the experiment: what we **change**, what we **control**, and what we **measure**. In experimental design, these are captured through different type of variables and outcomes.

(a) Control vs. Treatment/Experimental

An experiment typically compares:

- **Control group:** receives the baseline experience
- **Treatment group:** receives the new feature or intervention

Differences in outcomes indicate whether the treatment has an effect.

(b) Independent vs. Dependent Variables

In experiments, we identify the variables:

- What will be changed or manipulated? (**the independent variable**)
- What will be measured? (**the dependent variable**)

Example: If a streaming service wants to see if thumbnail style affects user engagement, the thumbnail is the **independent** variable and engagement (measured as CTR, watch time, or retention) is the **dependent** variable.

[Image omitted in PDF — see the online textbook]

Figure 3. Example diagram showing a comparative setup (e.g., two groups under different conditions or phases of a study). Source: Khan Academy (cdn.kastatic.org)

This mapping matters because a poorly defined outcome leads to wasted experimentation. If you do not know what “better” means, no experiment can tell you what to do.

(c) Metrics / Evaluation Criteria:

Experiments require measurable outcomes; such as:

- Click-through rate (CTR)
- Revenue
- Time on task
- Error rate
- Completion rate

Good metrics align with the decision being made. And Evaluation Criteria focus on how we quantify success (e.g., revenue per user, conversion rate, latency, retention rate).

In data science, outcomes often relate to **optimization goals**. For example:

- A retailer might measure **conversion rate** to see if discounts increase sales.
- A streaming platform might measure **watch time** to test a new recommendation algorithm.
- A telecom company might measure **churn rate** to evaluate a loyalty program.

Clearly defining variables and outcomes ensures that the experiment answers the original question and that results can be compared fairly across different groups.

17.2.1 Example: A/B Testing: The Data Science Workhorse

In industry, the most widely used form of experimentation is A/B testing, where two variants (A = control, B = treatment) are compared.

Examples:

- Netflix tests thumbnail images
- Amazon tests pricing or layouts
- LinkedIn tests recommendation algorithms
- TikTok tests ranking models
- Meta tests news feed ranking features

What makes A/B testing powerful is its simplicity and scalability, large digital platforms run thousands of concurrent experiments every year.

Practical Example:

Imagine a food delivery app wondering whether offering free delivery for first-time buyers increases conversions.

- **Control:** Users see standard delivery fee.
- **Treatment:** Users see free delivery offer.
- **Outcome metric:** Conversion rate on first purchase.

If the treatment produces a statistically significant increase, the company has evidence to roll it out widely. This logic appears almost everywhere; in UX design, pricing, logistics, healthcare, and public policy.

18 Methods for Collecting Data

There are different ways to collect data.

18.1 1. Observational Study

In observational studies, researchers **do not assign** treatments or interventions. They **simply observe** what already happens in the real world. Because there is no random assignment, it is harder to establish causality and confounding is a major concern.

Observational Data vs. Experimental Data

In practice, much of data science relies on observational data: logs, surveys, transactions, or passively collected information. Observational data can reveal correlations, but often struggles with causal inference due to confounding factors.

Experiments, by contrast, are actively designed to create credible comparisons and reduce confounding.

There are **three** common types of observational studies:

[Image omitted in PDF — see the online textbook]

[Table omitted in PDF — see the online textbook]

Figure 10. Three common types of observational studie. Source: amazonaws.com

18.1.1 (a) Cross-Sectional Studies:

A cross-sectional study collects data at **one single point in time**; like taking a snapshot of a population. It **does not follow people over time** and **does not look backward** in time.

Goal: usually to describe what is happening right now or to look for associations between variables, not to establish cause and effect.

It's like taking a picture of a classroom today and measuring everyone's height and weight once.

18.1.2 (b) Retrospective/Historical (Case-Control) Studies:

Retrospective studies start with an **outcome that has already happened** and look **backward in time** to find possible causes.

Example: Compare people with lung cancer (cases) and people without it (controls) and ask about their past smoking habits.

Retrospective studies are useful for rare outcomes, but people may not remember the past accurately.

18.1.3 (c) Prospective (Longitudinal / Cohort) Studies:

Prospective studies follow a group of people (a **cohort**) **forward in time** to see how different characteristics or exposures lead to different outcomes.

Example: Follow a cohort of smokers and non-smokers for several years to see who develops lung cancer.

Prospective studies provide stronger evidence for cause-and-effect, but they take longer and can be expensive.

18.2 2. Survey

A survey is a common way to collect information by asking people questions. Surveys can be done in many formats, online, by phone, by email, or in person and are often used to learn about opinions, behaviors, attitudes, or characteristics of a population.

- Surveys are usually **observational** because researchers do not assign treatments or change anything; they simply gather responses.
- Surveys can be descriptive (e.g., estimating how many students drink coffee) or can look for associations (e.g., relating coffee consumption to sleep), but they cannot easily establish cause and effect.

[Table omitted in PDF — see the online textbook]

Good surveys depend on well-designed questions. Poorly worded or confusing questions can introduce bias, influence responses, or produce misleading results. **Clear, neutral, and specific** questions help improve the quality and reliability of survey data.

18.3 3. Simulation

Sometimes it is difficult, slow, expensive, or even impossible to run a real experiment in the world. In those cases, we can use a **simulation**.

A simulation is a **computer-based experiment** where we create a simple model of the real world and let the computer generate many possible outcomes. Instead of collecting data from people or running a physical experiment, we let the computer “pretend” the experiment happens many times.

In short: simulation is like a virtual experiment.

Why use simulation? Real experiments may be **too costly**, **too risky** and may be **too slow**.

18.3.1 Example

- A company might simulate **customer arrivals** and **service times** to study how long people wait in line.
- In data science, we might simulate **many random samples** to understand how a statistic (like the mean) behaves; this is often called a **Monte Carlo simulation**.

Simulation does not replace real data, but it is a powerful tool for **exploring behavior**, **testing designs**, and **building intuition** when running real experiments is difficult.

19 The Invisible Enemy: Confounders

Experiments would be simple if the world contained only **two variables**: one we change and one we measure. In reality, almost every outcome is influenced by many factors we do not control.

These extra influences are called **confounders**, hidden variables that affect the outcome but are not part of the intended treatment.

What is a Confounder?

A confounder is an outside factor that makes it harder to tell whether the treatment actually caused the change we observe.

A confounder typically: * affects the independent variable (treatment), and * affects the dependent variable (outcome), and * creates the appearance that X causes Y when something else is influencing both

In short, confounders can create a misleading or “fake” relationship between the variables we care about.

[Image omitted in PDF — see the online textbook]

Figure 4. The confounding variable (hot temperature) affects both the independent variable (ice cream consumption) and the dependent variable (sunburns), which can create a misleading relationship if not controlled for. Source: Scribbr article.

For example:

- Comparing two school tutoring programs **without considering family income** may lead to misleading conclusions.
- Online, testing a new website layout **without considering device type (mobile vs desktop)** may distort click-through rates (CTR).

Confounders are dangerous because they can make ineffective treatments look effective or vice versa. Much of experimental design exists to keep them under control.

19.0.1 Strategies for Controlling Confounders

Good experimental design addresses confounding **before** data is collected, and good analysis techniques help address it **after** data collection.

Broadly, strategies fall into two stages:

19.0.2 Design Stage (Before Data Collection)

This is the planning stage; the experiment hasn’t been run yet. Decisions made here affect how data is collected. The best time to handle confounders is early, during this planning of the experiment.

1. Randomization

Randomization just means we randomly put people (or units) into different groups. **Why random?** Because if we don’t, the groups might be unfair from the start.

Randomly assigning participants to treatment groups helps ensure that confounders (both known and unknown) are evenly distributed across groups. This reduces systematic differences and strengthens causal interpretation. > Now if the two groups behave differently, we can be more confident the treatment caused it, not some hidden factor in the background.

We can apply randomization in different ways. For example:

(a) Random Sampling:

We put individuals into groups **purely at random**, without considering characteristics like age, device type, or prior behavior.

- **Example:** Randomly assigning students to two study methods helps balance motivation, prior knowledge, and study habits across groups.

Key idea: Random Sampling balances confounders **on average**.

[Image omitted in PDF — see the online textbook]

Figure 5A. A randomized experimental design showing how a study population is split into treatment and control groups, followed over time, and then compared. Source: Fiveable exp. design visual diagram.

Sometimes randomization alone is not enough, especially when important subgroups behave differently. In those cases, we can use **stratified randomization**.

(b) Stratified Randomization:

We first **group (stratify)** people based on an important characteristic, then randomly assign within each group. We stratify when certain characteristics matter, such as: age, gender, device type, customer type (new vs returning), performance level, income bracket, etc.

- **Example:** Before randomizing, users are split into “mobile” and “desktop” strata. Within each stratum, users are randomly assigned to version A or B. This ensures the comparison isn’t distorted if mobile users behave differently than desktop users.

Key idea: Stratified Randomization balances confounders **within subgroups**, not just on average.

[Image omitted in PDF — see the online textbook]

Figure 5B. Full population being split into strata first, then randomized within each subgroup. Source: Benchmark Six Sigma forum.

In general, randomization helps make the groups similar on average so that differences in outcomes are more likely due to the treatment and not some hidden factor.

2. Restriction

Restriction means we **limit** who or what we include in the experiment so the groups are more similar to begin with.

Restriction reduces confounding by limiting the experiment to a more comparable subgroup, so it cannot vary. Remember, you **only study one subgroup**, everyone outside that subgroup is excluded. Examples:

Example: If we’re testing a new rehab treatment, we might only include patients aged 40–60 instead of everyone from 10 to 90. This avoids age becoming a distracting factor.

[Image omitted in PDF — see the online textbook]

Figure 6. With restriction, we only include adults ages 40–60 in the experiment. Age cannot vary anymore.

Restriction reduces confounders by removing variation in that characteristic. By restricting who is included, we reduce variation and make it easier to compare outcomes fairly.

3. Matching / Blocking

Matching: reduces confounding by pairing or grouping units that share important characteristics before assigning treatments.

We keep participants with different characteristics, but we **pair or group them** so that the treatment and control groups are similar.

- Units can be matched on age, income, or baseline performance.
- One member of each pair receives the **treatment**, and the other receives the **control**.

This makes comparisons more fair because each treated unit has a similar “partner” in the control group.

Key Idea: Unlike restriction, matching does not remove variation; it balances it across groups.

Example: - Pairing 45-year-old participants with other 45-year-olds before random assignment.
- If two students have similar grades and similar study habits, we can match them, one gets Method A, the other gets Method B. Now the comparison is cleaner, because we controlled for those similarities.

Blocking: When matching is done in **groups rather than pairs**, the approach is often called **blocking**.

Key Idea: Group subjects into blocks based on a potential confounder. Then randomize within each block

Example we might block by age group (e.g., 18–25, 26–35, 36–45) and then randomize treatment vs control within each block so age doesn’t skew the results.

(a) Block Design (Matched Pair):

A matched-pair design is a special case of a randomized block design where each block contains two similar participants. The key idea is to **compare like with like**.

- Participants are first **paired based on** a confounding characteristic such as **prior knowledge**, and then randomly assigned to treatment or control within each pair.
- One member of the pair serves as the **control** (normal study time) and the other as the **treatment** (increased study time).

Because both students start with similar baseline ability, differences in exam scores are more likely due to study time rather than pre-existing differences.

[Image omitted in PDF — see the online textbook]

Figure 7. Matched-pair block design. Participants are matched on a key characteristic (e.g., prior knowledge) and then randomly assigned within pairs. Source: Quantifying Health

The matched-pair design controls the confounder within each pair, making the comparison more fair.

19.0.3 Analysis Stage (After Data Collection)

If confounders were not fully handled during design, they can still be addressed at the analysis stage using statistical techniques. This stage happens after the experiment (or observational study) has produced data.

Replication Replication means repeating the experiment across different samples, time periods, or settings.

- A company could rerun an A/B test in week 2 after seeing results in week 1.
- A researcher could reanalyze the data using different subsets or models.

Replication checks whether a result is reliable or just a fluke. It does not remove confounders by itself, but it reveals whether a finding holds up under repeated testing.

Key Summary:

No single strategy removes confounders completely; but combining approaches strengthens the experiment and increases confidence in the conclusions.

20 Bias in Experiments: Placebo & Blinding

Even with randomization and careful design, participants can change their behavior simply because they know they are being studied. This can introduce bias into an experiment.

Placebo Effect The placebo effect happens when people show improvement even though they received a “fake” or placebo treatment with no active ingredient. The improvement comes from expectations, not the treatment itself.

Example: In medical trials, patients receiving a sugar pill may still report less pain because they believe they are receiving medication.

[Image omitted in PDF — see the online textbook]

Figure 8. Illustration of the placebo effect showing how participants’ expectations can influence outcomes even without an active treatment. Source: sciencenotes.org

The placebo effect is a form of **expectation bias**. To minimize it, experiments use **placebo controls** so that both groups have similar expectations. This helps separate the **true treatment effect** from **psychological or expectation effects**.

How to Minimize Bias? Blinding (Single-Blind / Double-Blind) Blinding means that participants, researchers, or both do not know who received the treatment. This prevents behavior or measurement from being influenced by expectations.

- **Single-blind:** participants do not know which group they are in.
- **Double-blind:** neither participants nor researchers know, until the experiment ends.

[Image omitted in PDF — see the online textbook]

Figure 9. Single-blind vs. double-blind study designs. Source: sciencenotes.org

Blinding, especially double-blinding, is a common method for minimizing **expectation bias** and making experimental results more trustworthy.

20.1 Constraints in Experimental Design

Experiments are powerful but imperfect. Key constraints include:

- **Ethical considerations:** especially in health or policy settings
- **Cost:** running experiments may require incentives or infrastructure
- **Interference:** treatments can spill over between users
- **Limited sample sizes**
- **Time to measure outcomes**

In some scenarios (e.g., earthquakes, economic crises, pandemics), controlled experiments are impossible; forcing analysts to rely on natural experiments or causal inference techniques.

21 Summary of the Chapter

In this chapter, we learned how data science uses **experiments** and **observational studies** to answer questions. Experiments can show **cause and effect** because we change something and compare results. Observational studies help us learn about the world without changing anything, but they cannot easily show what causes what.

We also saw that good **experimental design** must deal with **confounders** and **bias** so that comparisons are fair. When experiments are hard to run, **surveys** and **simulations** give us other ways to collect information and test ideas. Understanding how studies are designed helps us avoid confusing **correlation** with **causation**.

21.1 Knowledge Check

[Interactive content omitted in PDF]

22 Chapter 4 - Pandas

23 Pandas: Turning Raw Data into Usable Data

Most data in the real world is not useful when we first encounter it. It arrives as CSV files from city governments, JSON blobs from APIs, HTML tables scraped from websites, logs from servers, or spreadsheets from someone in accounting who has strong opinions about color coding. The formats differ, but the problem is always the same:

“How do I turn this into something I can think with?”

Data scientists do not stare at raw strings and commas; we need representations that **behave like data**, sortable, filterable, joinable, groupable, and ultimately meaningful. This is where Pandas enters the story.

Pandas is not a database, and it is not a spreadsheet. It borrows the best features from both: the tabular structure of Excel, the indexing and querying habits of SQL, and the vectorized speed of NumPy. For the working data scientist, Pandas becomes a lens, **it shapes raw input into analytical objects**.

The core object in Pandas is the **DataFrame**: a table of columns with names and types, where each column behaves like a mathematical vector and each row behaves like an observation. In a DataFrame, data becomes manipulable; you can filter for a year, compute a rate, sort by price, group by category, or merge two messy tables into a coherent whole.

What makes Pandas powerful is not just the DataFrame itself, but the fluidity of movement between formats. A single line of code can read a CSV from disk, parse JSON from an API, extract a table out of HTML, or connect to SQL. In other words:

Pandas is where formats become data, and where data becomes analysis.

Pandas encourages a workflow that feels almost grammatical. We load data, inspect it, clean it, reshape it, and only then begin to analyze. The early steps are not busywork—they are how we teach uncooperative data to behave. Pandas gives us tools for dealing with the inevitable artifacts of real-world datasets: missing values, awkward formats, inconsistent categories, surprising outliers. Machines complain about these imperfections; Pandas negotiates with them.

There is also a quiet power in how well Pandas talks to other parts of the ecosystem. Read a CSV? One line. Join two tables? One line. Group by a category and summarize? One line. Convert the result to NumPy for modeling or to Matplotlib for plotting? Also one line. Pandas isn't flashy; it just fits between everything else data scientists do.

To make this concrete, imagine a small ritual:

```
import pandas as pd

df = pd.read_csv("flights.csv")
```

Two lines, and suddenly a plain-text file becomes a navigable world. We can ask:

- What does it look like? (using `df.head()`)
- What types do we have? `df.describe()` # How do the numbers behave?

Notes: Pandas **does not do** machine learning, that's scikit-learn's domain, and it does not do visualization, that's Matplotlib and Seaborn. But nothing in data science happens without Pandas first creating the table on which those tools can operate. Data cleaning, merging, reshaping, joining, pivoting, filtering, aggregating: these verbs are not glamorous, but they are the backbone of analysis.

To borrow a phrase from experimental science: Pandas prepares the specimen. Machine learning, statistics, and modeling are merely the microscope.

In this chapter, the goal is not to memorize functions but to cultivate a way of thinking about tabular data.

We will learn:

- how DataFrames are constructed
- how to select, slice, filter, and group data
- how to read from CSV, JSON, etc.
- how to reshape data (wide → long)

- how to merge and join tables

Remember: Pandas is not just a library; it is a bridge between the world where data is stored and the world where data is understood.

24 What is Pandas?

Pandas is a powerful open-source Python library that provides high-performance, easy-to-use data structures and data analysis tools. The name “pandas” comes from “panel data,” a term used in econometrics to describe multi-dimensional data. Developed by Wes McKinney in 2008, pandas has become the cornerstone of data manipulation and analysis in the Python ecosystem.

At its core, Pandas helps you:

- Load, clean, and transform datasets.
- Perform statistical operations efficiently.
- Handle missing or inconsistent data.
- Merge, reshape, and aggregate large datasets.

If you have ever worked with spreadsheets in Excel, Pandas offers similar functionality—but with far greater power, speed, and scalability.

25 Why Use Pandas?

Before pandas, data analysis in Python was cumbersome and required jumping between different libraries. Python users relied heavily on lists, dictionaries, and NumPy arrays for handling structured data. While these tools are powerful, they lack built-in functionality for common tasks like handling missing values, grouping data, or joining tables. Pandas solved this by providing:

- **Intuitive data structures:** DataFrames and Series that feel familiar to users from various backgrounds, useful for working with tabular and one-dimensional data.
- **Seamless integration:** Works beautifully with other Python data science libraries (NumPy, Matplotlib, etc).
- **Powerful data manipulation:** Easy filtering, grouping, and transformation of data
- **Performance:** Built on top of highly optimized C code for speed.
- **Time series functionality:** Excellent support for working with time-based data
- **Ease of Use:** Simplifies complex operations into a few lines of readable code.

[Image omitted in PDF — see the online textbook]

26 Installing Pandas

Before using Pandas, we need to make sure it’s installed. Many data science environments already include it, but not all.

How? If you are using Anaconda Distribution, Pandas comes pre-installed. Otherwise, you can install it with:

pip install pandas

Or if you're using Anaconda: **> conda install pandas**

What is the Anaconda Distribution?

The Anaconda Distribution is a popular Python platform that bundles many data science tools (NumPy, Pandas, Jupyter, SciPy, Matplotlib, etc.) into a single installation. It saves beginners from installing packages one-by-one and provides an environment manager for scientific computing. <https://www.anaconda.com/products/distribution>

Official Pandas website: <https://pandas.pydata.org/>

Confirming Installation:

To confirm the installation, Open Python shell or a notebook and run:

```
import pandas as pd
print(pd.__version__)
```

If you see a version number (e.g. 2.2.1), Pandas is installed correctly.

26.1 Using Pandas in Different Environments

(a) Google Colab

Google Colab ships with Pandas pre-installed. You can simply import it:

```
import pandas as pd
```

(b) Jupyter Notebooks

Jupyter is bundled with Anaconda. If needed, you can install it manually:

```
pip install notebook
```

Once Pandas is installed, it works inside any notebook environment.

26.1.1 Why Versions Matter?

Pandas evolves quickly. Version differences affect:

- available functions
- method behaviors (e.g., `merge()`, `read_csv()`)
- performance improvements
- deprecations

Checking your version makes debugging much easier.

Python Compatibility

As of 2024+, Pandas requires **Python 3.9 or newer**. Older Python versions may not support newer Pandas releases.

Scientific libraries often move faster than base Python installations, so compatibility matters.

27 Loading Data with Pandas

The first step of analysis is getting the data into Pandas.

One of Pandas' biggest strengths is its ability to easily import/export datasets from variety of formats: CSV (most common import format), Excel, JSON (common from APIs), SQL Databases (query + load) etc. Here are some example:

- **CSV:** `pd.read_csv("file.csv")`
- **Excel:** `pd.read_excel("file.xlsx")`
- **SQL Databases:** `pd.read_sql(query, connection)`
- **JSON:** `pd.read_json("file.json")`

	Unnamed: 0	default	student	balance	income
0	1	No	No	729.526495	44361.625074
1	2	No	Yes	817.180407	12106.134700
2	3	No	No	1073.549164	31767.138947
3	4	No	No	529.250605	35704.493935
4	5	No	No	785.655883	38463.495879

28 A Beginner's Working Model

Before we touch syntax, it helps to understand what Pandas adds to Python.

28.0.1 From Python Objects to DataFrames

Python already has ways to represent data:

- lists (`[1, 2, 3]`)
- dictionaries (`{"age": 21, "major": "CS"}`)
- lists of dictionaries (a common "JSON-like" pattern)

The problem is that **none of these act like a table**. For example, consider a list of student records:

```
students = [  
    {"name": "Alice", "year": 2025, "major": "CS"},  
    {"name": "Bob", "year": 2024, "major": "Math"},  
]
```

Python can iterate through this, but asking simple questions ("which majors?", "how many in each year?", "filter by year") requires loops, conditionals, and manual bookkeeping.

Pandas provides a format that behaves like a table:

```
import pandas as pd
df = pd.DataFrame(students)
```

Now the same questions become natural:

```
df["major"].value_counts()
df[df["year"] == 2025]
```

This illustrates the core Pandas philosophy:

Take common data-analysis questions and make them **one line instead of a small program**.

29 Core Data Structures

It's tempting to think Pandas has only one object — the `DataFrame` — because it dominates most workflows. But understanding the smaller building block helps.

The strength of Pandas lies in two core objects: 1. **Series**: is a single column (one-dimensional labeled array) 2. **Dataframe**: a table of columns (two-dimensional labeled data structure)

People often describe the relationship like this:

A `DataFrame` is a collection of `Series` that share the same index.

That index part will matter later.

[Image omitted in PDF — see the online textbook]

You won't use `Series` much explicitly at first, but understanding it later pays off (especially for `groupby`, `apply`, or time series).

29.1 Series: The One-Dimensional Workhorse

A `Series` is a one-dimensional labeled array that can hold any data type. Think of it as a single column in a spreadsheet.

[Image omitted in PDF — see the online textbook]

Unlike some arrays that require all elements to be the same type (homogeneous), a `Series` can store different types of values together, such as numbers, text, or dates. Each value has a label called an index, which can be numbers, words, or timestamps, and you can use it to quickly find or select values. Here are some examples:

29.1.1 How to Create a Series

A `Series` can be created directly from a *Python list*, in which case pandas automatically assigns default numeric indexes (0, 1, 2, ...) to each element.

You can also create a `Series` from a *Python dictionary*, where the dictionary keys become the index labels and the dictionary values become the `Series` values. In Python 3.7 and later, the order of the keys is preserved, so the `Series` keeps the same order as the dictionary

```
Mon    22
Tue    25
```

```

Wed      18
Thu      30
Fri      27
Name: Daily_Temps, dtype: int64
Math      90
English   85
Science   95
dtype: int64

```

Homogeneous Series

```

A      10
B      20
C      30
D      40
dtype: int64
The data type is: int64

```

Heterogeneous Series

```

0      10
1     20.5
2    hello
3     True
dtype: object
The data type is: object

```

30 DataFrame: The Two-Dimensional Powerhouse

A DataFrame is a two-dimensional labeled data structure, similar to a table with rows and columns. It is the most commonly used object in Pandas.

[Image omitted in PDF — see the online textbook]

[Image omitted in PDF — see the online textbook]

Every DataFrame has an **index** on the left side. In CSVs it often starts at 0 by default:

```

      name  year  major
0   Alice  2025    CS
1    Bob   2024   Math

```

Spreadsheets also have row numbers, but Pandas treats the index as **part of the data structure itself**. That gives us speed, alignment, joins, slicing, and time-based operations.

Here are some examples:

```

      Name  Age  City
0   Alice   25 New York
1    Bob   30  London

```

```

2 Charlie 35 Paris
3 Diana 28 Tokyo

```

Our DataFrame:

```

  Product  Price  Stock
0  Apple    1.2    45
1  Banana   0.5   120
2  Cherry   3.0    15
3   Date    2.5    80

```

Data types:

```

Product      object
Price        float64
Stock         int64
dtype: object

```

Basic statistics:

```

      Price      Stock
count  4.00000  4.000000
mean   1.80000  65.000000
std     1.15181  45.276926
min     0.50000  15.000000
25%     1.02500  37.500000
50%     1.85000  62.500000
75%     2.62500  90.000000
max     3.00000 120.000000

```

31 Inspecting Data: First Contact

The moment a DataFrame loads, the first questions are never about modeling; they are always about shape:

- How many rows?
- How many columns?
- What are the datatypes?
- Are there missing values? -Does it look reasonable?

Pandas has a small vocabulary for these basic operations.

31.1 Pandas Basic Operations

After reading tabular data as a DataFrame, you would need to have a glimpse of the data. Pandas makes it easy to explore and manipulate. A good first step is to inspect the dataset by previewing how many rows and columns it has, what the column names are, checking dimensions, or reviewing summary information such as data types and statistics. Pandas provides convenient methods for this.

##Viewing/Exploring Data

Command	Description	Default Behavior
<code>df.head()</code>	Displays the first rows of the DataFrame. Useful for quickly previewing the dataset.	Shows 5 rows
<code>df.tail()</code>	Displays the last rows of the DataFrame. Handy for checking the dataset's ending records.	Shows 5 rows
<code>df.shape</code>	Returns a tuple (rows , columns) representing the dimensions of the DataFrame.	N/A
<code>df.info()</code>	Shows column names, data types , memory usage, and count of non-null values.	N/A
<code>df.dtypes</code>	Returns the data type of each column in the DataFrame.	N/A
<code>df.describe()</code>	Provides summary statistics (mean, std, min, max, quartiles) for numeric columns.	Includes numeric columns by default

[Image omitted in PDF — see the online textbook]

These lines form the “hello world” of any data-driven project.

Even in professional settings, someone uploads a CSV to Slack, someone loads it into Pandas, and the first message back is almost always:

“Wait, why is year stored as string?”

or

“We have 800k rows. Expected 500k. Did we duplicate something?”

Learning to **inspect before analyzing** is a habit that prevents embarrassment later.

head

```

      Name  Age  Salary
0    Alice   24   50000
1     Bob   30   60000
2  Charlie   28   55000

```

shape

```
(3, 3)
```

info

```

<bound method DataFrame.info of      Name  Age  Salary
0    Alice   24   50000
1     Bob   30   60000
2  Charlie   28   55000>

```

31.2 Selecting and Indexing Data

After inspecting the structure of a DataFrame, the next step is often to select specific rows and columns (specific parts of the data). Pandas provides several approaches depending on whether you want to select columns, rows, or filter data based on conditions. It lets you choose columns, rows, or both using labels (loc), integer positions (iloc), or conditions.

31.2.1 1. Selecting Columns

Command	Description
<code>df['col']</code>	Selects a single column as a Series.
<code>df[['col1', 'col2']]</code>	Selects multiple columns as a new DataFrame.

31.2.2 2. Selecting Rows

Command	Description
<code>df.loc[row_label]</code>	Select row(s) by label (index name).
<code>df.iloc[row_index]</code>	Select row(s) by integer position .
<code>df.loc[0, 'col']</code>	Select a specific value by row & column.

31.2.3 3. Conditional Selection (Filtering)

Command	Description
<code>df[df['col'] > 50]</code>	Returns rows where condition is True .
<code>df[(df['A'] > 50) & (df['B'] < 100)]</code>	Combine conditions with & (and), (or).

Follow is an example:

```

      Name  Age      City
0   Alice   24  New York
1    Bob   30 Los Angeles
2  Charlie   28   Chicago

```

```

Empty DataFrame
Columns: [Name, Age, City]
Index: []

```

32 Adding a new column to DataFrame

A new column can be added to a pandas DataFrame by assigning a *value*, *list*, or *Series* to a new column name. If the assigned data is a list or Series, its length must match the number of rows in the DataFrame. You can also assign a single value, which will be applied to all rows.

```

      Name  Math
0   Alice   90
1    Bob   85
2  Charlie   95
      Name  Math  English
0   Alice   90     88
1    Bob   85     92
2  Charlie   95     80
      Name  Math  English  Pass

```

0	Alice	90	88	True
1	Bob	85	92	True
2	Charlie	95	80	True

33 Arithmetic Operations and Functions in Pandas

So far we explored how to inspect and select data in Pandas. Once you have access to the right rows and columns, the next step is to perform calculations and apply functions. Pandas makes this process very intuitive by allowing you to apply arithmetic directly to DataFrames or Series, and by offering tools like `apply()`, `map()`, and `applymap()` for more flexibility.

33.1 Arithmetic Operations on Columns

You can directly apply mathematical operations to Pandas Series or DataFrame columns. Operations are vectorized, meaning they are applied element-wise across the column.

In below example, you can notice how the operations are automatically applied to each row.

	Name	Age	Salary
0	Alice	24	50000
1	Bob	30	60000
2	Charlie	28	55000

	Name	Age	Salary
0	Alice	29	55000.0
1	Bob	35	66000.0
2	Charlie	33	60500.0

33.2 Arithmetic Between Columns

You can also perform arithmetic between two or more columns to create new features.

	Name	Age	Salary	Income_per_Age
0	Alice	29	55000.0	1896.551724
1	Bob	35	66000.0	1885.714286
2	Charlie	33	60500.0	1833.333333

34 Applying Built-in Pandas/NumPy Functions

Pandas integrates with NumPy functions, allowing you to apply common statistics directly.

```
60500.0
3.0550504633038935
0    5.385165
1    5.916080
2    5.744563
Name: Age, dtype: float64
```

34.0.1 Applying Functions with apply()

Sometimes you need custom transformations. The `apply()` method lets you apply a function to an entire column (Series) or to each row/column in a DataFrame.

	Name	Age	Salary	Income_per_Age	Age_squared	Total
0	Alice	29	55000.0	1896.551724	841	55029.0
1	Bob	35	66000.0	1885.714286	1225	66035.0
2	Charlie	33	60500.0	1833.333333	1089	60533.0

Note that, we can also apply a Function Elementwise with `applymap()` and to a Single Column with `map()` but not covering in this course.

35 Filtering Data in Pandas

Once you know how to select columns and rows, the next step is learning how to filter data. Filtering helps you focus on only the relevant part of your dataset, whether that means removing unnecessary columns, isolating rows that meet certain conditions, or preparing features for modeling.

35.1 Filtering Columns

Column filtering is about selecting only the columns you need or dropping the ones you don't. This reduces memory usage and keeps your DataFrame manageable.

	Name	Age
0	Alice	29
1	Bob	35
2	Charlie	33

35.2 Dropping Unused Columns

	Name	Age	Salary	Income_per_Age	Total
0	Alice	29	55000.0	1896.551724	55029.0
1	Bob	35	66000.0	1885.714286	66035.0
2	Charlie	33	60500.0	1833.333333	60533.0

This is especially useful when preparing data for machine learning, where only selected features are required.

35.3 Filtering Rows (using Boolean Indexing)

Row filtering is usually done with Boolean indexing, where you apply a condition and return only the rows where that condition is true.

	Name	Age	Salary	Income_per_Age	Total
1	Bob	35	66000.0	1885.714286	66035.0
2	Charlie	33	60500.0	1833.333333	60533.0

36 Combining Multiple Conditions

You can combine conditions using & (and) or | (or).

	Name	Age	Salary	Income_per_Age	Total
1	Bob	35	66000.0	1885.714286	66035.0
2	Charlie	33	60500.0	1833.333333	60533.0

Remember to wrap each condition in parentheses.

37 Filtering Strings

You can filter rows where a text column contains specific values

	Name	Age	Salary	Income_per_Age	Total
1	Bob	35	66000.0	1885.714286	66035.0

37.1 Unique Values and Counting

Sometimes you want to check how many unique values a column has, or count how often each appears.

```
['Alice' 'Bob' 'Charlie']
Name
Alice      1
Bob        1
Charlie    1
Name: count, dtype: int64
```

38 Applying Aggregation Functions Directly to a DataFrame

One of the strengths of Pandas is that you can apply statistical and aggregation methods directly to a DataFrame or Series. These methods summarize data and provide insights without needing extra loops or manual calculations.

38.0.1 Common Aggregation Methods

Here are some of the most commonly used methods:

Method	Description	Works On
.sum()	Returns the sum of values	DataFrame / Series
.mean()	Returns the average (mean) value	DataFrame / Series
.count()	Counts non-null values	DataFrame / Series
.min()	Returns the minimum value	DataFrame / Series
.max()	Returns the maximum value	DataFrame / Series
.std()	Returns the standard deviation	DataFrame / Series
.var()	Returns the variance	DataFrame / Series

Method	Description	Works On
<code>.describe()</code>	Generates summary statistics (count, mean, std, min, quartiles, max)	DataFrame / Series

Example: Aggregating a Series

```
Sum: 300000
Mean: 60000.0
Max: 70000
Std Dev: 7905.694150420948
```

Each method is applied directly to the Series, returning a single value.

Example: Aggregating a DataFrame

```
Age          82
Salary      165000
dtype: int64
Age          27.333333
Salary      55000.000000
dtype: float64
      Age  Salary
count  3.000000    3.0
mean   27.333333  55000.0
std     3.055050   5000.0
min    24.000000  50000.0
25%    26.000000  52500.0
50%    28.000000  55000.0
75%    29.000000  57500.0
max    30.000000  60000.0
```

Notice how these functions automatically ignore non-numeric columns (like “Name”).

39 More Advanced: Filtering Data & Apply Statistical Functions

We can combine **row filtering** with **aggregation functions** to analyze subsets of a DataFrame.

The general syntax is:

```
df[df['column_name'] value]['target_column'].function()
```

where:

- `df[...]` → filters the rows that meet the condition
- `['target_column']` → selects the column to aggregate
- `.function()` → applies the aggregation function

```
68000.0
```

```
7767.45346515403
```

So the syntax pattern is:

```
df[df['condition']] ['column'].aggregation()
```

Expression	Meaning
<code>df[df['Age'] > 30]['Salary'].mean()</code>	Mean of Salary where Age > 30
<code>df[df['Salary'] > 60000]['Name'].count()</code>	Count of employees with Salary > 60k
<code>df[(df['Age'] >= 25) & (df['Age'] <= 40)]['Salary'].std()</code>	Standard deviation of Salary for 25–40 year olds

This pattern allows you to filter data first, then aggregate only on the rows that meet your condition.

40 Grouping Data with `groupby`

While filtering + aggregation lets us summarize a **subset** of data, the `groupby()` method allows us to compute statistics **across categories**.

This is the classic **split–apply–combine** process:

1. **Split** data into groups based on one or more columns.
2. **Apply** an aggregation function to each group.
3. **Combine** results into a new DataFrame or Series.

40.1 Basic Syntax

```
df.groupby('column_name')['target_column'].aggregation_function()   where: -
groupby('column_name') → splits the data into groups.
- ['target_column'] → selects the column to aggregate.
- .aggregation_function() → applies functions like mean(), sum(), count().
```

	Department	Employee	Salary
0	HR	Alice	50000
1	HR	Bob	52000
2	IT	Charlie	60000
3	IT	David	62000
4	Finance	Eva	58000
5	Finance	Frank	60000

```
Department
Finance    59000.0
HR         51000.0
IT         61000.0
Name: Salary, dtype: float64
```

40.1.1 Grouping by Multiple Columns

```
Department  Region
Finance     East      58000.0
            West      60000.0
HR           East      50000.0
            West      52000.0
IT           East      60000.0
            West      62000.0
Name: Salary, dtype: float64
```

41 Exporting Data in Pandas

After processing your data in Pandas, you can save it to files in various formats:

Format	Function	Key Parameters / Notes	Example Usage
CSV	<code>to_csv()</code>	<code>index=False</code> to skip row numbers, <code>sep='\t'</code> for tab-delimited	<code>df.to_csv('data.csv', index=False)</code>
Excel	<code>to_excel()</code>	<code>sheet_name='Sheet1'</code> , requires <code>openpyxl</code>	<code>df.to_excel('data.xlsx', index=False)</code>
JSON	<code>to_json()</code>	<code>orient='records'</code> , <code>lines=True</code> for line-delimited JSON	<code>df.to_json('data.json', orient='records', lines=True)</code>
Pickle	<code>to_pickle()</code>	Python-specific, fast binary format	<code>df.to_pickle('data.pkl')</code>
HTML	<code>to_html()</code>	Saves as an HTML table	<code>df.to_html('data.html', index=False)</code>
Parquet	<code>to_parquet()</code>	Efficient columnar format, great for big data	<code>df.to_parquet('data.pa', index=False)</code>

Tip: Always choose the format based on your use case:

- CSV → universal, easy sharing
- Excel → spreadsheets
- JSON → web APIs or NoSQL
- Parquet → large datasets, high performance

CSV file 'output.csv' created successfully.

Tab-separated CSV file 'output_tab.csv' created successfully.

Excel file 'output.xlsx' created successfully.

JSON file 'output.json' created successfully.

42 Exporting Pandas Data in Google Colab

In Colab, you can save files directly to Google Drive. First, mount your Drive:

```
“python from google.colab import drive drive.mount('/content/drive') # Follow the link and paste the authorization code
```


42.1 Bonus: Writing Pandas DataFrames to a SQL Database

In addition to exporting data to files (such as CSV), pandas can also write DataFrames directly to a SQL database (a topic explored in the next chapter). This is useful when you want to store cleaned or transformed data for later querying or reuse.

The `DataFrame.to_sql()` method allows a pandas DataFrame to be written directly to a SQL table. For example:

```
df.to_sql(
    name='processed_data',
    con=conn,
    if_exists='replace',
    index=False
)
```

In this example:

- **name** specifies the name of the table to be created in the database.
- **con** provides the database connection.
- **if_exists** controls how pandas behaves if a table with the same name already exists:
 - 'replace' removes the existing table and creates a new one,
 - 'append' adds rows to the existing table,
 - 'fail' raises an error.
- **index=False** prevents the DataFrame index from being stored as an extra column in the database table.

These options make it easy to persist processed data from pandas into a relational database without manually writing SQL INSERT statements.

Remember: `to_sql()` is a pandas convenience function. Internally, it generates SQL INSERT statements, but users do not need to write SQL manually. While `if_exists='replace'` is convenient for assignments and experiments, it should be used with caution in real-world databases because it overwrites existing tables.

43 The Pandas Ecosystem: How It Fits In

Pandas does not exist in a vacuum. It is a central hub in the Python data science stack:

- **NumPy:** Provides the foundational n-dimensional array object. Pandas DataFrames are built on top of NumPy arrays.
- **Matplotlib/Seaborn:** Used for visualization. You can plot data directly from DataFrames and Series.
- **Scikit-learn:** The premier machine learning library. It accepts DataFrames and Series as inputs for model training.
- **Jupyter Notebooks:** The ideal interactive environment for exploratory data analysis with pandas.

44 When to Use Pandas (And When Not To)

44.1 Use pandas when:

- Working with tabular data (like spreadsheets or database tables)
- Data cleaning and preprocessing
- Exploratory data analysis
- Medium-sized datasets (up to a few gigabytes)

44.2 Consider alternatives when:

- Working with very large datasets that don't fit in memory.
- Need extremely high performance for numerical computations (consider NumPy directly)
- Working with unstructured data like images or text

45 Summary of the Chapter

Pandas gives Python a **table-shaped way of thinking**. Instead of looping through lists or juggling dictionaries, we work with **Series** and **DataFrames**, objects designed for filtering, joining, grouping, and summarizing data. This chapter showed how Pandas sits between data acquisition (CSV, SQL, APIs, scraping) and downstream analysis (visualization, statistics, machine learning), and how small operations like `head()`, `describe()`, and column selection reveal structure quickly. More than a library, Pandas encourages us to operate on entire columns at once and to reason about data at the level of tables.

With just a few patterns, we can already do a surprising amount of analysis:

- **Filtering + Aggregation** → summarize specific rows based on conditions.
- **GroupBy + Aggregation** → summarize categories (all groups at once).
- **Grouping** can be done on one or multiple columns.

45.1 Pandas: Key Features at a Glance

- **Data Import/Export:** Read from and write to CSV, Excel, SQL, JSON, and many other formats
- **Data Cleaning:** Handle missing values, remove duplicates, filter outliers
- **Data Transformation:** Reshape, pivot, melt, and transform your data
- **Data Aggregation:** Group by categories and compute summary statistics
- **Time Series Analysis:** Work with dates and times effortlessly
- **Visualization Integration:** Works seamlessly with Matplotlib and Seaborn

In later chapters, we will return to Pandas for exploratory data analysis (EDA), visualization, and richer transformations.

45.2 Knowledge Check

[Interactive content omitted in PDF]

45.3 interactive pandas

46 Interactive Pandas Playground

This notebook provides a hands-on environment for exploring pandas operations with sample employee data.

What you can do: - Write and execute pandas code in real-time using the text area below - Experiment with filtering, sorting, grouping, and other pandas operations

Sample data includes: - 5 employees across Engineering, Sales, Marketing, and HR departments
- Age and salary information for analysis practice

Please be patient while the playground loads

[Interactive content omitted in PDF]

47 Chapter 5 - SQL

48 Chapter 6: Mastering SQL for Data Science with Python

Walk into any online store, hospital, bank, or airline system and behind the scenes something is keeping track of who is who, what belongs to whom, and what has changed. A shopping cart remembers the items you chose. A reservation system checks seat availability. A bank records every transaction down to the cent. These systems run on structured data, and for decades that information has been organized using relational databases.

Relational databases were built for reliability and structure. Structured Query Language (SQL) became the language for interacting with them. Instead of telling the computer how to compute something step by step, SQL lets us say what we want: select certain columns, filter certain rows, or join two tables to complete the picture. It is declarative, expressive, and surprisingly good at describing real-world questions.

For data science, this matters because a large portion of the world's structured data already lives in SQL warehouses. Many analyses begin not with modeling, but with **asking for the right data**: which customers, which time period, which features, and which tables. SQL is how we explore, filter, combine, and summarize before anything else; often before Pandas, visualization, or machine learning ever enter the pipeline.

The chapter introduces **Structured Query Language (SQL)**, a powerful and essential tool for data professionals. we will learn how to use SQL to work with real data and how SQL integrates naturally with Python. By the end, the goal is not just to write SQL queries, but to think in SQL: to express questions clearly and retrieve the information needed to answer them; a crucial skill for any data scientist.

48.1 Introduction to SQL for Data Science

Data has become the foundation of decision-making in modern organizations. From social media platforms storing billions of user interactions to hospitals managing electronic health records, most of this information is stored in **databases**. Among different types of databases, **relational databases** are the most widely used.

Relational databases organize data into tables, which consist of **rows (records)** and **columns (attributes)**. This design reflects the way data naturally relates to entities in the real world. For example:

*A retail store has customers (with IDs, names, and ages). Each customer places orders (with product details, dates, and amounts). * The relationship between customers and orders can be represented through keys.*

To interact with these databases, we use a language called **Structured Query Language (SQL)**.

48.1.1 Structured Query Language (SQL).

SQL provides a standardized way to create, read, update, and delete data (commonly referred to as CRUD operations). Unlike programming languages such as Python or Java, SQL is declarative: you specify what you want, and the database figures out how to get it.

This makes it highly efficient for managing large datasets. For data science, SQL is invaluable for:

- **Data Retrieval:** Extracting specific subsets of data from large databases.
- **Data Cleaning and Transformation:** Handling missing values, standardizing formats, and creating new features.
- **Exploratory Data Analysis (EDA):** Performing quick summaries, aggregations, and data profiling.
- **Feature Engineering:** Creating new variables from existing ones before feeding them into machine learning models.

49 The Evolution of Databases and SQL

- **Early data management:** Before databases, organizations stored information in files. This approach led to redundancy, inconsistency, and inefficiency.
- **Birth of the relational model:** In 1970, Edgar F. Codd introduced the relational model, a mathematical foundation for organizing data in tables with well-defined relationships.
- **Development of SQL:** By the late 1970s, IBM developed SEQUEL (Structured English Query Language), which evolved into SQL. It became the ANSI standard in 1986.
- **SQL today:** Almost every relational database system (MySQL, PostgreSQL, Oracle, SQL Server, SQLite) supports SQL, with minor dialect differences.
- **Takeaway:** SQL is not just a programming tool; it is the backbone of modern data storage and analytics.

49.1 When to Use SQL and When to Use Pandas

Data scientists switch between SQL and Pandas because they excel at different stages of the workflow.

SQL makes sense when the data lives in a database.

It offloads computation to the database engine, which is optimized for filtering, joining, and aggregating large datasets—often far bigger than a laptop can hold in memory. SQL is also ideal when multiple systems need consistent access to the same structured data.

Use **SQL** when: - data is stored in a relational database - the dataset is large (beyond local memory) - the main task is selecting, filtering, joining, or summarizing - consistency and shared access matter

Pandas shines once the data is local.

It enables fast, flexible manipulation: reshaping tables, enriching features, exploring distributions, and creating visualizations. Pandas works especially well in notebooks, where analysis is iterative and interactive.

Use **Pandas** when: - data arrives as CSV/Excel/JSON or via APIs and scraping - the task is cleaning, transforming, or feature engineering - visualization or modeling follows - custom logic is easier to express in Python

In practice, a common pipeline looks like:

Database → SQL → subset → Pandas → exploration / visualization / ML

SQL retrieves and trims; Pandas experiments and interprets.

Rule of thumb: > **SQL** for extracting structured data; **Pandas** for understanding and shaping it.

50 Relational Databases: Core Concepts

Relational databases are the backbone of structured data storage. They organize information in a way that ensures consistency, integrity, and efficient retrieval. The fundamental ideas of tables, keys, and relationships help us understand how real-world data is modeled.

50.0.1 Tables, Rows, and Columns

A table is like a spreadsheet.

- **Rows (records/tuples):** Each row corresponds to a single entity or instance of data. For example, one row might represent a single customer.
- **Columns (fields/attributes):** Each column stores one specific type of information about the entity, such as name, age, or gender.
- **Schema:** The structure of the table, which defines what columns exist and their data types (e.g., integer, string, date).

Example: A Customers table may contain columns such as Customer_ID, Name, Age, Gender, and Email. Each row represents one customer.

50.0.2 Keys

Keys in Relational Databases are crucial for ensuring that data remains **unique** and **consistent** across tables.

50.1 1. Primary Key (PK)

The primary key uniquely identifies each row. It contains UNIQUE values in column, and does not allow NULL values.

[Image omitted in PDF — see the online textbook]

Here, Empid is a Primary Key. Example: **Customers Table**

Customer_ID	Name	Age	Gender
101	Alice	25	F
102	Bob	30	M
103	Charlie	28	M

- **Primary Key:** Customer_ID
 - Ensures each customer is uniquely identifiable.
-

50.2 2. Foreign Key (FK)

A foreign key links one table to another. It creates a relationship between two or more tables, a primary key of one table is referred to as a foreign key in another table. It can also accept multiple null values and duplicate values.

[Image omitted in PDF — see the online textbook]

Orders Table

Order_ID	Customer_ID	Product	Quantity
5001	101	Laptop	1
5002	102	Keyboard	2
5003	101	Mouse	1

- Customer_ID here is a **foreign key** connecting each order to the **Customers** table.
 - Prevents creating an order for a non-existent customer.
-

50.3 3. Composite Key

Sometimes, no single column uniquely identifies a row. Composite Key is a combination of more than one column of a table. It can be a Candidate key and Primary key.

[Image omitted in PDF — see the online textbook]

Enrollments Table

Student_ID	Course_ID	Grade
S001	CSE101	A
S001	MTH201	B+
S002	CSE101	A-
S002	PHY110	B

- Neither **Student_ID** nor **Course_ID** alone is unique.
- Together (**Student_ID**, **Course_ID**) form a **composite key**.
- Ensures a student cannot enroll in the same course twice.

50.4 4. Candidate Key

A candidate key is any column (or set of columns) that could serve as a primary key. Candidate Key(s) can identify a record uniquely in a table and which can be selected as a primary key of the table.

It contains UNIQUE values in column, and does not allow NULL values.

[Image omitted in PDF — see the online textbook]

Here, EmpID, EmpLicence and EmpPassport are candidate keys.

Example: **Employees Table**

Employee_ID	Email	SSN	Name
E001	alice@company.com	123-45-6789	Alice
E002	bob@company.com	987-65-4321	Bob
E003	charlie@company.com	111-22-3333	Charlie

- Possible unique identifiers:
 - **Employee_ID**
 - **Email**
 - **SSN**
- Each is a **candidate key**.
- One (e.g., **Employee_ID**) is chosen as the **primary key**.

Remember, Each table can have only **one Primary** key and **multiple Candidate** keys

51 PK-FK Relationships

Relational databases use **primary keys (PK)** and **foreign keys (FK)** to maintain data integrity and model relationships.

51.1 Types of Relationships

- **One-to-One (1:1):** Each person has one passport; each passport belongs to one person.

[Image omitted in PDF — see the online textbook]

- **One-to-Many (1:N):** A customer can have many orders; each order belongs to one customer.

[Image omitted in PDF — see the online textbook]

In this figure, a customer can have many accounts; each account belongs to one customer.

- **Many-to-Many (M:N):** Students enroll in many courses; courses have many students.

[Image omitted in PDF — see the online textbook]

In this figure, each customer can buy more than one product and a product can be bought by many different customers.

51.2 Customers Table

Customer_ID (PK)	Name	Age	Gender
101	Alice	25	F
102	Bob	30	M
103	Charlie	28	M

51.3 Orders Table

Order_ID (PK)	Customer_ID (FK)	Product_ID (FK)	Quantity
5001	101	P100	1
5002	102	P101	2
5003	101	P102	1

- Customer_ID is a **foreign key** referencing Customers.Customer_ID.
- Product_ID is a **foreign key** referencing Products.Product_ID.

51.4 Products Table

Product_ID (PK)	Product_Name	Price
P100	Laptop	1000
P101	Keyboard	50
P102	Mouse	30

51.5 Passports Table (One-to-One Example)

Passport_ID (PK)	Customer_ID (FK)	Expiration_Date
P001	101	2030-12-31
P002	102	2031-06-30
P003	103	2030-09-15

- `Customer_ID` is a **foreign key** referencing `Customers.Customer_ID`.
 - Each customer has exactly **one passport**.
-

51.6 Enrollments Table (Many-to-Many Example)

Student_ID	Course_ID	Grade
S001	CSE101	A
S001	MTH201	B+
S002	CSE101	A-
S002	PHY110	B

- Neither `Student_ID` nor `Course_ID` alone is unique.
 - The combination (`Student_ID`, `Course_ID`) forms a **composite key**.
 - Students can enroll in **many courses**, and courses can have **many students**.
-

51.7 Relationships Overview

From Table	To Table	Type	Notes
Customers	Orders	1:N	One customer → many orders
Products	Orders	1:N	One product → many orders
Customers	Passports	1:1	One customer → one passport
Students	Courses	M:N	Implemented via Enrollments table

51.8 Key Points

- **Primary Key (PK):** Unique identifier for each record. Cannot be NULL.
 - **Foreign Key (FK):** Links a table to another table's primary key. Maintains referential integrity.
 - **Composite Key:** Combination of columns used when a single column is not unique.
 - **Candidate Key:** Any column or combination of columns that could serve as a primary key.
 - **Constraints:** Rules to maintain data validity (e.g., NOT NULL, UNIQUE, CHECK, FOREIGN KEY).
-

51.9 The Role of SQL in Data Science

Think of SQL as your conversation partner with the data. It's a declarative language, which means you simply state your desired outcome, and the database handles the complex task of finding and organizing the data for you. This makes it incredibly efficient for handling massive datasets. A typical data science workflow using SQL might look like this:

- **Data Extraction:** You use a SELECT query to pull a specific subset of data relevant to your project.
- **Data Wrangling:** You perform initial cleaning, filtering (WHERE), and aggregation (GROUP BY) directly in the database.
- **Analysis:** The prepared data is loaded into Python (often as a Pandas DataFrame) for more sophisticated analysis, modeling, and visualization.

51.10 Core SQL Commands: Your Essential Toolkit

We have already learned that SQL is the standard language for managing and querying **relational databases**.

These core commands allow you to **create tables, insert data, retrieve information, update records, and maintain data integrity**.

Whether you are analyzing sales data, customer information, or product inventories, mastering these commands is essential for data-driven tasks.

Command	Purpose	Example
CREATE TABLE	Create a new table in the database	<code>sql CREATE TABLE Customers (CustomerID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(50), Email VARCHAR(100));</code>
INSERT INTO	Adds new rows of data to a table	<code>sql INSERT INTO Customers (CustomerID, Name, Email) VALUES (1, 'John Doe', 'john.doe@example.com');</code>
SELECT	Retrieves data from one or more tables	<code>sql SELECT Name, Email FROM Customers;</code>
WHERE	Filters records based on a condition	<code>sql SELECT Name, Email FROM Customers WHERE CustomerID = 1;</code>

Command	Purpose	Example
UPDATE	Modifies existing data in a table	<code>sql UPDATE Customers SET Email = 'john.doe@newdomain.com' WHERE CustomerID = 1;</code>
DELETE	Removes rows from a table	<code>sql DELETE FROM Customers WHERE CustomerID = 1;</code>
DROP TABLE	Deletes the entire table and all its data	<code>sql DROP TABLE Customers;</code>

Notes:

- **CustomerID** is the **primary key** and uses **AUTO_INCREMENT** to generate unique IDs automatically.
- Always use **WHERE** in **UPDATE** and **DELETE** to avoid modifying all rows by mistake.
- **DROP TABLE** permanently deletes the table and its data, so use with caution.

52 The “Big 6” Elements of a SQL Select Statement

When querying data in SQL, the **SELECT** statement is the foundation. It allows you to **specify what data to retrieve, from where, and how to organize it**. The six key elements (plus **LIMIT**) are essential to writing powerful queries.

- **SELECT:** Specifies which columns you want to retrieve.
 - Example: `SELECT customer_id, amount FROM sales;`
- **FROM:** Specifies the table you are querying.
- **WHERE:** Filters rows based on conditions.
 - Example: `SELECT * FROM sales WHERE amount > 100 AND sale_date >= '2023-01-01';`
 - Example: `SELECT product_id, amount FROM sales ORDER BY amount DESC;`
- **GROUP BY:** Aggregates rows with the same values into summary rows. Useful for metrics like total sales per customer.
- **HAVING:** Filters results of a **GROUP BY** clause, similar to **WHERE** but for aggregated data.
- **ORDER BY:** Sorts the result set. Use **DESC** for descending and **ASC** for ascending.
- **** LIMIT:**** Restricts the number of rows returned in the result set. Often used with **ORDER BY** to get top or bottom results.

[Image omitted in PDF — see the online textbook]

Remember that, although these six key elements (**SELECT**, **FROM**, **WHERE**, **GROUP BY**, **HAVING**, **ORDER BY**) plus **LIMIT** are essential to writing powerful queries; however, **WHERE**, **GROUP BY**, **HAVING**, **ORDER BY**, and **LIMIT** are **optional** depending on the query’s purpose.

Example: Let’s create the **Customers**, **Products**, and **Orders** tables and insert sample data into each.

Tables created and sample data inserted successfully!

52.1 1. SELECT

Specifies which **columns** you want to retrieve from a table. *Example:*

```
“sql SELECT customer_id, amount FROM sales;
```

	order_id	customer_id	sale_date	product_id	amount
0	1	101	2023-01-01	1	150.00
1	2	102	2023-01-02	2	200.50
2	3	101	2023-01-03	3	75.25
3	4	103	2023-01-04	1	150.00
4	5	102	2023-01-05	2	200.50

52.2 2. FROM

Specifies the table(s) you are querying.

```
“sql SELECT customer_id, amount FROM sales;
```

	customer_id	amount
0	101	150.00
1	102	200.50
2	101	75.25
3	103	150.00
4	102	200.50

52.3 3. WHERE

Filters rows based on conditions. Only rows that satisfy the condition are returned.

```
“sql SELECT * FROM sales WHERE amount > 100 AND sale_date >= '2023-01-01';
```

	order_id	customer_id	sale_date	product_id	amount
0	1	101	2023-01-01	1	150.0
1	2	102	2023-01-02	2	200.5
2	4	103	2023-01-04	1	150.0
3	5	102	2023-01-05	2	200.5

52.4 4. GROUPBY

Aggregates rows with the same values into summary rows, such as totals, averages, or counts.

```
“sql SELECT customer_id, SUM(amount) AS total_sales FROM sales GROUP BY customer_id;
```

	customer_id	total_sales
0	101	225.25
1	102	401.00
2	103	150.00

52.5 5. HAVING

Filters results after aggregation. Similar to WHERE, but operates on aggregated data.

Remember, while WHERE filters **raw rows before aggregation**, HAVING filters **groups created by GROUP BY**.

Connection to GROUP BY:

- GROUP BY creates aggregated groups (e.g., total sales per customer).
- HAVING applies conditions on these aggregated values. Without GROUP BY, HAVING can still work on aggregate functions applied to the entire table.

```
“sql SELECT customer_id, SUM(amount) AS total_sales FROM sales GROUP BY customer_id
HAVING SUM(amount) > 100;
```

	customer_id	total_sales
0	101	225.25
1	102	401.00
2	103	150.00

52.6 6. ORDERBY

Sorts the result set by one or more columns.

```
“sql SELECT product_id, amount FROM sales ORDER BY amount DESC; – DESC for descend-
ing, ASC for ascending
```

	product_id	amount
0	2	200.50
1	2	200.50
2	1	150.00
3	1	150.00
4	3	75.25

52.7 Some More SQL Essentials

52.7.1 DISTINCT

Returns **unique values** from a column, removing duplicates.

```
“sql – Get unique customer IDs SELECT DISTINCT customer_id FROM sales;
```

Tables created and sample data inserted successfully!

	customer_id
0	101
1	102
2	103

52.7.2 COUNT

Counts the number of rows that satisfy a condition.

```
“sql – Count total sales SELECT COUNT(*) AS total_sales FROM sales;
```

- Count number of unique customers `SELECT COUNT(DISTINCT customer_id) AS unique_customers FROM sales;`

	unique_customers
0	3

52.7.3 LIMIT

Restricts the number of rows returned, useful for sampling or previewing data.

-- Get the 10 most recent sales

```
SELECT *
FROM sales
ORDER BY sale_date DESC
LIMIT 10;
```

-- Count unique customers but only show the first 5 results

```
SELECT customer_id, COUNT(*) AS num_sales
FROM sales
GROUP BY customer_id
ORDER BY num_sales DESC
LIMIT 5;
```

	customer_id	num_sales
0	102	2
1	101	2
2	103	1

Aggregations and Filtering - Use aggregation functions like `SUM()`, `COUNT()`, `AVG()`, `MIN()`, `MAX()` to summarize data.

- `GROUP BY` allows you to compute metrics per category (e.g., total sales per customer).
- `HAVING` filters aggregated results (useful when you want to filter groups, unlike `WHERE` which filters raw rows).

“sql – Find total sales per customer `SELECT customer_id, SUM(amount) AS total_sales FROM sales GROUP BY customer_id;`

- Find customers with total sales greater than 200 `SELECT customer_id, SUM(amount) AS total_sales FROM sales GROUP BY customer_id HAVING SUM(amount) > 200;`

	customer_id	total_sales
0	101	225.25
1	102	401.00

53 SQL JOINS: Combining Data from Multiple Tables

In relational databases, data is often split across multiple tables. **JOINS** allow you to combine rows from two or more tables based on related columns (usually keys).

53.1 1. INNER JOIN

Returns only the rows where there is a **match in both tables**.

[Image omitted in PDF — see the online textbook]

```
SELECT o.Order_ID, c.Name, p.Product_Name
FROM Orders o
INNER JOIN Customers c
    ON o.Customer_ID = c.CustomerID
INNER JOIN Products p
    ON o.Product_ID = p.ProductID;
```

53.2 2. LEFT JOIN (or LEFT OUTER JOIN)

Returns **all rows from the left table**, and **matched rows from the right table**. If there is no match, the right table columns return NULL.

[Image omitted in PDF — see the online textbook]

```
SELECT o.Order_ID, c.Name, p.Product_Name
FROM Orders o
INNER JOIN Customers c
    ON o.Customer_ID = c.CustomerID
INNER JOIN Products p
    ON o.Product_ID = p.ProductID;
```

Here:

- All customers are shown, even if they haven't placed any orders.
 - Orders columns for customers with no orders will be NULL.
-

53.3 3. RIGHT JOIN (or RIGHT OUTER JOIN)

Returns **all rows from the right table**, and **matched rows from the left table**. If there is no match, the left table columns return NULL.

[Image omitted in PDF — see the online textbook]

```
SELECT o.Order_ID, c.CustomerID, c.Name
FROM Orders o
RIGHT JOIN Customers c
    ON o.Customer_ID = c.CustomerID;
```

Here, All customers appear, even if they have no orders (similar to LEFT JOIN but reversed table order).

53.4 4. FULL OUTER JOIN

Returns **all rows from both tables**, with NULL for missing matches on either side.

[Image omitted in PDF — see the online textbook]

```
“sql SELECT c.CustomerID, c.Name, o.Order_ID FROM Customers c FULL OUTER JOIN
Orders o ON c.CustomerID = o.Customer_ID;
```

Here, we combines the effect of LEFT and RIGHT JOIN. Customers without orders and orders without customers are included with NULL in the missing columns.

Below is a full example of creating and manipulating a table in SQL.

53.4.1 EXAMPLE:

Database populated successfully!

```
--- Sales with Amount > $150 ---
  order_id  customer_id  product_id   sale_date  amount
0         2         102          2  2023-01-02   200.5
1         5         102          2  2023-01-05   200.5
```

```
--- Sales with Product Names ---
  order_id  sale_date  amount  product_name
0         1  2023-01-01  150.00      Laptop
1         2  2023-01-02  200.50      Monitor
2         3  2023-01-03   75.25        Mouse
3         4  2023-01-04  150.00      Laptop
4         5  2023-01-05  200.50      Monitor
```

```
--- Total Sales per Customer ---
  customer_id  total_amount
0         102         401.00
1         101         225.25
2         103         150.00
```

```
--- Customers with Total Sales > $200 ---
  customer_id  total_amount
0         102         401.00
1         101         225.25
```

Connection to database closed.

54 Summary of the Chapter

Relational databases were created to bring order to information. Before them, data lived in scattered files and improvised formats that were hard to search or connect. The relational model solved that problem by putting information into tables and linking them through keys, so a customer's name lived in one place and their orders could point to it. This idea proved durable, and

it still supports systems we use every day, from shopping carts and flight reservations to medical records and bank transactions.

SQL became the language for asking questions of those systems. Instead of explaining how to compute something step by step, SQL lets us say what we want: choose these columns, filter those rows, or join two tables to complete the picture. It is a natural way to reason about structured data.

For data science, relational databases continue to matter. Most structured data still lives in SQL systems, and many analyses begin with a query. SQL works well alongside **Pandas in Python**: SQL retrieves and shapes the data at the source, while Pandas helps us explore, visualize, and analyze it once it is in memory.

This chapter introduced that way of thinking. In later chapters, we will use SQL and Pandas together to explore real datasets and make sense of the stories they contain.

54.1 Key Takeaways: SQL

1. Relational Databases Structure

- Data is organized into **tables**, consisting of **rows** (records) and **columns** (attributes).
- **Primary Keys (PK)** uniquely identify rows, and **Foreign Keys (FK)** link tables to maintain **referential integrity**.

2. Core SQL Commands

- **CREATE TABLE** – define a new table.
- **INSERT INTO** – add rows of data.
- **SELECT** – retrieve data.
- **WHERE** – filter rows.
- **UPDATE / DELETE** – modify or remove data.
- **DROP TABLE** – remove a table permanently.

3. The “Big 6” Elements of a **SELECT** Statement

- **SELECT**: Choose columns.
- **FROM**: Specify tables.
- **WHERE**: Filter rows.
- **GROUP BY**: Aggregate rows.
- **HAVING**: Filter aggregated results.
- **ORDER BY**: Sort results.
- **LIMIT**: Restrict the number of rows returned.

4. JOINS for Combining Tables

- **INNER JOIN:** Only matching rows.
- **LEFT JOIN:** All left table rows, matched right table rows.
- **RIGHT JOIN:** All right table rows, matched left table rows.
- **FULL OUTER JOIN:** All rows from both tables, NULL for missing matches.

5. Aggregations and Filtering

- Use `SUM()`, `COUNT()`, `AVG()`, `MIN()`, `MAX()` for aggregation.
- Use `GROUP BY` to summarize data per category.
- Use `HAVING` to filter **after aggregation** (cannot use `WHERE` for aggregated results).

6. SQL in Python with Pandas

- `sqlite3` allows creating a **lightweight database** in Colab.
- Use `pd.read_sql_query()` to load **SQL query results directly into a DataFrame** for analysis.
- Combining SQL + Pandas enables **powerful data workflows** in Python.

7. Best Practices

- Always use **WHERE** when updating or deleting rows.
- Use **table aliases** for readability in JOINS.
- Use **LIMIT** when exploring large datasets to preview data efficiently.
- Test queries on **sample data** before running on full datasets.

Conclusion:

By mastering table creation, data insertion, `SELECT` statements, JOINS, aggregation, and integration with Pandas, you can perform **complex data analysis** efficiently in SQL and Python. This chapter lays the foundation for building **real-world data pipelines and analytical workflows**.

54.2 Knowledge Check

[Interactive content omitted in PDF]

[Interactive content omitted in PDF]

54.3 interactive sql

55 Interactive SQL Playground

This notebook creates a live SQL environment where you can practice and experiment with SQL queries on a sample employee database.

What you can do: - Write SQL queries in the text area below (or modify the default query) - Execute standard SQL operations: SELECT, WHERE, ORDER BY, GROUP BY, JOIN, etc.

Sample data includes: - 8 sample employees across Engineering, Sales, Marketing, and HR departments - Employee attributes: ID, name, department, salary, hire date, and age

Please be patient while the playground loads

[Interactive content omitted in PDF]

56 Chapter 6 - Probability and Distributions

57 Why Probability Matters in Data Science

Probability is more than mathematics — it is a way of thinking.

It teaches us how to **reason under uncertainty**, how to **update beliefs when new data arrives**, and how to **separate signal from noise**. Every experiment you design, every model you train, and every prediction you report relies on these ideas, even when the formulas stay behind the scenes.

Probability does not promise certainty.

It promises **clarity in uncertain situations**, and that is one of the most honest promises data science can make.

58 A Gentle Walk Through Probability: Making Sense of Uncertainty

Imagine you are standing outside, checking the sky before leaving home. The forecast says there is a **30% chance of rain**. You hesitate. Should you take an umbrella?

That small pause before a decision is where **probability theory** lives.

Probability is not about predicting exactly what will happen. It is about describing what *might* happen, how likely it is, and how our beliefs should change when we learn something new. In data science, probability quietly sits underneath almost everything: experiments, models, predictions, and decisions.

Let's build this story from the ground up.

58.1 Sample Space and Events: Defining the World

Every probability problem starts by defining **what could happen**.

If you flip a coin once, there are only two possible outcomes: **heads** or **tails**. This complete set of possible outcomes is called the **sample space**.

If you flip a coin once, the possible outcomes are {H, T}.

That complete list of outcomes is the **sample space**.

[Image omitted in PDF — see the online textbook]

Sample Space illustration. Source: Online Math Learning.

An **event** is any subset of the sample space, something we care about.

For example: - the coin lands on heads - it rains tomorrow - a customer makes a purchase

Events can be simple or complex, but they always live inside the sample space. Probability asks a single guiding question:

How likely is this event to occur if the situation were repeated many times?

Mathematically

Let Ω be the **sample space**, the set of all possible outcomes.

An **event** A is a set such that $A \subseteq \Omega$.

Probability is a function $P(\cdot)$ that assigns a number to events.

[Image omitted in PDF — see the online textbook]

The relationship between a sample space, events, and probabilities is often visualized using simple diagrams.

58.2 Assigning Probabilities to Events

Once an event is defined, we assign it a number between 0 and 1.

- $P(A)$ represents the probability that event A occurs.
- $1 - P(A)$ represents the probability that event A **does not** occur (the complement of A).

For example, if A is the event “the coin lands on heads”: - $P(A) = 0.5$ - $1 - P(A) = 0.5$

Together, an event and its complement account for all possible outcomes.

Before we start computing probabilities, we need a few basic rules that probabilities must obey.

58.3 The Rules of the Game: Probability Axioms

Probability is not guesswork; it follows strict rules.

- **Probabilities are never negative**
You cannot have a “−20% chance” of rain.
- **Something must happen**
The probabilities of all possible outcomes add up to 1.
- **Mutually exclusive events add cleanly**
If two events cannot happen at the same time, the probability of either happening is the sum of their probabilities.

These ideas are often called the **axioms of probability**.

They are assumed by all statistical and machine learning methods.

These rules ensure that probability behaves consistently and logically. Every model you build and every experiment you analyze rests on these foundations, even when the formulas are hidden.

Probability does not eliminate uncertainty; it helps us **reason clearly in its presence**.

59 Conditional Probability: Updating Beliefs With Context

Now consider the weather again.

You hear there is a 30 percent chance of rain today. Later, you look outside and see dark clouds gathering.

Your **belief changes**.

This adjustment is captured by **conditional probability**, which answers questions of the form:

What is the probability of an event, given that something else has already happened?

In data science, conditional probability appears constantly: - the probability of disease **given** a positive test - the probability a user clicks **given** that they saw an ad - the probability of fraud **given** unusual account activity

We rarely reason in isolation. We reason with context.

Mathematically: Conditional Probability

If $P(B) > 0$, the conditional probability of A given B is

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

This formula says:

among all outcomes where B happens, what fraction also satisfy A ?

[Image omitted in PDF — see the online textbook]

An event (A) and its complement (A^c) together cover the entire sample space (). Source: Stack Overflow

59.0.1 Small Python Simulation

Example: rolling a die

A : the roll is even

B : the roll is greater than 3

In data science, we often observe evidence first and then ask what it tells us about an underlying cause.

Examples: - A medical test comes back positive — how likely is the disease? - An email looks suspicious — how likely is it spam? - A model flags an anomaly — how likely is it fraud?

Bayes' theorem formalizes how **beliefs should change when new evidence appears**.

It tells us how to **update our beliefs** when **new information arrives**.

Bayes' theorem connects four key ideas: - what we believed before seeing data - how likely the evidence is - how common the event is overall - what we should believe after seeing the evidence

Mathematically: Bayes' Theorem

If $P(B) > 0$, then

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

where: - $P(A)$ is the **prior** (what we believed before seeing data) - $P(B | A)$ is the **likelihood** - $P(A | B)$ is the **posterior** (updated belief)

[Image omitted in PDF — see the online textbook]

Bayes' theorem reveals an important lesson. *Evidence alone is not enough.* A positive test result does not automatically mean something is likely. The background rate matters just as much as the test accuracy.

This is why Bayes' theorem is both powerful and humbling. It forces us to confront our assumptions and reminds us that context shapes meaning.

Crucially, Bayes reminds us that evidence must be interpreted in context. A strong signal does not guarantee a likely cause if the cause itself is rare.

Data Science Connection: Bayes' theorem underlies:

- Naive Bayes classifiers
- Probabilistic reasoning in ML
- Model calibration and uncertainty interpretation

59.0.2 Small Python Simulation

Example: medical testing and base rates

- 1% of the population has a condition
- The test detects it 95% of the time
- The test gives false positives 5% of the time

`P(Disease) = 0.01`

`P(Positive) = 0.059000000000000004`

`P(Disease | Positive) = 0.16101694915254236`

60 Law of Total Probability

The **Law of Total Probability** allows us to compute the probability of an event based on a set of **mutually exclusive and exhaustive** scenarios.

If $B_1, B_2, \dots, B_n$ are mutually exclusive and exhaustive events (they cover all possibilities), then for any event A :

$$P(A) = \sum_{i=1}^n P(A | B_i) P(B_i)$$

[Image omitted in PDF — see the online textbook]

Figure: Law of Total Probability; illustrating how an event A can occur under different scenarios B_1, B_2, B_3 and how their contributions combine to form the total probability of A . Source: WallStreetMojo

60.0.1 Explanation

1. **Mutually exclusive:** No two B_i events happen at the same time.
2. **Exhaustive:** One of the B_i events must happen.
3. **Conditional probability:** $P(A | B_i)$ is the probability of A given B_i occurs.
4. The law sums over all possible scenarios to get the total probability of A . > **Mutually exclusive and exhaustive event:** Events that cannot happen at the same time (*mutually exclusive*) and cover all possible outcomes (*exhaustive*).

60.0.2 Example

Suppose there are two factories producing light bulbs:

- Factory 1 produces 60% of all bulbs, with a 1% defect rate.
- Factory 2 produces 40% of all bulbs, with a 2% defect rate.

Question: What is the probability that a randomly chosen bulb is defective?

Solution using the Law of Total Probability:

$$P(\text{Defective}) = P(\text{Defective} | F_1)P(F_1) + P(\text{Defective} | F_2)P(F_2)$$

$$P(\text{Defective}) = 0.01 \times 0.6 + 0.02 \times 0.4 = 0.014$$

So, there is a **1.4% chance** that a randomly chosen bulb is defective.

Probability of a defective bulb: 0.014

60.1 Conditional Independence

In data science, variables often appear related.

However, **once we account for the right context**, some relationships disappear.

This idea is called **conditional independence**, and it is important for: - probabilistic reasoning
- Naive Bayes classifiers - Bayesian networks - causal thinking

Mathematically: Conditional Independence

Two variables X and Y are **conditionally independent given Z** if:

$$P(X | Y, Z) = P(X | Z)$$

Knowing Y provides no additional information about X once Z is known.

[Image omitted in PDF — see the online textbook]

Conditional Independence illustration. Source: Nagwa

60.1.1 Intuition

- X and Y may look related at first
- After conditioning on Z , the relationship disappears
- In simple terms, Z **explains the relationship**

60.1.2 Example

Let: - X = whether a student gets a high exam score
- Y = whether the student attends review sessions
- Z = how much the student studied

Exam scores and review attendance may appear related.

But **once we know how much a student studied**, attending review sessions may not add new information about the exam score.

In this case, X and Y are conditionally independent given Z .

```
(np.float64(0.6200700962816742),  
 np.float64(0.7999884386380716),  
 np.float64(0.8002969264104004))
```

Notice:

- If $P(X | Y) \neq P(X)$, then X and Y are dependent.
- If $P(X | Y, Z) = P(X | Z)$, then X and Y are conditionally independent given Z .

Once Z is known, Y adds no extra information about X .

Data Science Insight

Conditional independence simplifies probability models and is a key assumption in Naive Bayes and Bayesian networks.

61 Expected Value: Summarizes Average Behavior

In data science, we often care less about a single outcome and more about the **long-run average behavior** of a process. This idea is captured by expected value.

[Image omitted in PDF — see the online textbook]

Mathematically: Expected Value > > For a discrete random variable X : > > $\mathbb{E}[X] = \sum_x x P(X = x)$ > > For a continuous random variable X with density $f(x)$: > > $\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x) dx$

Intuition: - Expected value is **not a guaranteed outcome** - It represents the **average result over many repetitions** - It can be **positive, zero, or negative**

A negative expected value means a process loses value *on average* (e.g., average loss, cost, or error).

In data science, expected value connects directly to: -average error -average loss -average reward
This is why it plays a central role in optimization, decision-making, and model evaluation.

62 Probability Distribution: From Single Events to Patterns

So far, we have focused on individual events. In practice, data science is rarely about a single outcome.

Instead, we care about **patterns across many observations**.

A probability distribution describes how probability is spread across the possible values of a variable.

Some variables take **countable values**, such as the number of messages received today or the number of heads in ten coin flips. These follow **discrete distributions**.

Other variables vary **smoothly**, such as height, time, or temperature. These follow **continuous distributions**.

Mathematically: Probability Distributions

A **random variable** X assigns numerical values to outcomes in the sample space.

Discrete random variables take countable values.

Their probabilities are given by $P(X = x)$ and satisfy
$$\sum_x P(X = x) = 1.$$

Continuous random variables take values on a continuum.

They are described by a probability density function $f(x)$ such that
$$\int_{-\infty}^{\infty} f(x) dx = 1.$$

For continuous variables, $P(X = c) = 0$ for any single value c .

[Image omitted in PDF — see the online textbook]

Difference Between Discrete and Continuous Variables. Source: GeeksforGeeks.

Distributions allow us to reason about **averages, variability, typical behavior, and rare extremes**.

They turn uncertainty into structure, which is why they are central to data science.

Small Python Simulation

Discrete vs continuous samples.

Notice: Discrete distributions have separate bars and Continuous distributions form smooth shapes. Both describe uncertainty, but in different ways.

62.1 Common Distributions You Will See Everywhere

In data science, we do not just ask *what happened*.

We ask **how values behave across many observations**.

A probability distribution describes the data-generating process behind what we observe.

Probability distributions are broadly divided into **discrete** and **continuous** distributions.

-
- (1) **Discrete distributions** model outcomes that take **countable values**: - Bernoulli Distribution
- Binomial Distribution
- Poisson Distribution
- Zero-Inflated Poisson Distribution
- (2) **Continuous distributions** model outcomes that vary **smoothly over an interval**: - Uniform Distribution
- Normal (Gaussian) Distribution
- Many more
-

62.1.1 Why This Matters

Different datasets come from different processes: - clicks vs. no-clicks
- event counts per hour
- measurements with noise

Understanding **how your data is distributed** tells you a lot about **how the data was generated**.

Distribution Choice and Analysis

The nature of a distribution affects: - which statistical assumptions are reasonable
- which models are appropriate
- which evaluation metrics make sense

Choosing the wrong distribution can lead to **incorrect conclusions**, even when the computations are correct.

63 Types of Probability Distributions and how it connects to Data Science

63.0.1 (1A). Bernoulli distribution → one yes/no outcome

A **Bernoulli distribution** models a single binary decision: yes/no, success/failure, or 1/0.

A random variable X follows a Bernoulli distribution if:

- $P(X = 1) = p$
- $P(X = 0) = 1 - p$

Mean (Expected Value):

$$\mathbb{E}[X] = p$$

[Table omitted in PDF — see the online textbook]

```
import numpy as np
```

```
np.random.seed(0)
```

```
samples = np.random.binomial(n=1, p=0.3, size=20_000)
samples.mean()
```

63.0.2 (1B) Binomial Distribution → Many Bernoulli Trials

A **Binomial distribution** models the **number of successes** across repeated, independent **Bernoulli trials**.

Each trial: - has two outcomes (success / failure) - uses the same success probability p

Mathematically

If $X \sim \text{Binomial}(n, p)$, then

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Mean: $\mathbb{E}[X] = np$

63.0.3 Quick Intuition (T/F Quiz)

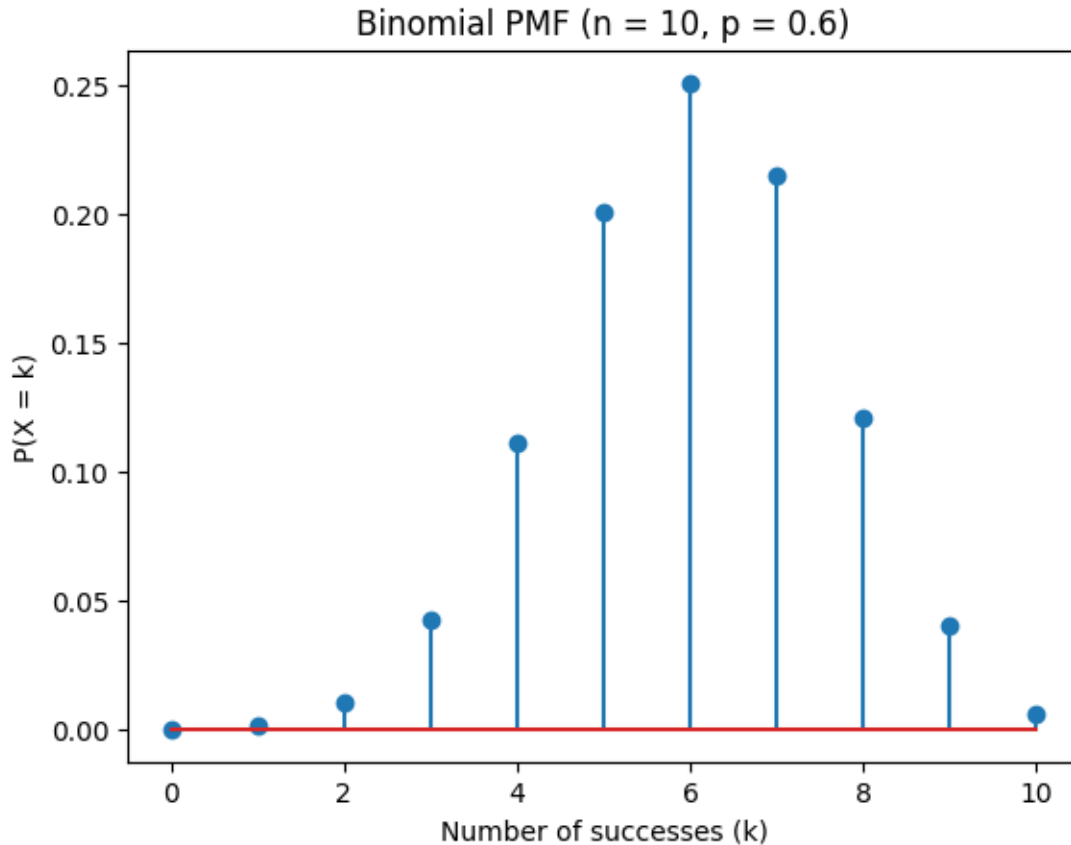
A quiz has **10 True/False questions**.

Each question is a **Bernoulli trial**.

The total number of correct answers follows: [$X \sim \text{Binomial}(10, p)$]

In Data Science, it helps answer questions like how many users will click on an ad or how many tests will pass out of a fixed number of trials.

```
np.random.binomial(n=100, p=0.08, size=10_000).mean()
```



63.0.4 (1C) Poisson Distribution → Event Counts Over Time or Space

A **Poisson distribution** models **how many times an event occurs in a fixed interval** (of time, space, area, etc.). Examples include: - number of emails received per hour
 - number of website requests per minute
 - number of errors in a system per day

Poisson = counting random events in a fixed interval at a constant rate.

Mathematically: Poisson Distribution > > If $X \sim \text{Poisson}(\lambda)$, then > > $P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$

> > **Mean (Expected Value):**

> $\mathbb{E}[X] = \lambda$

[Image omitted in PDF — see the online textbook]

Poisson probability mass function illustrating how the distribution depends on the average rate λ . The x-axis shows the number of events k , and the height of each bar represents $P(X = k)$. When λ is small, most probability mass is concentrated near $k = 0$ or 1. As λ increases, the distribution shifts to the right and becomes more spread out, reflecting higher and more variable event counts.
 Source: Wikimedia Commons.

What Does λ Mean? λ (lambda) is the **average rate** of events per interval. - Example: $\lambda = 3$

means *on average* 3 events per interval - In a Poisson distribution, the **mean equals the variance**

What Is e ? $e \approx 2.718$ is a mathematical constant (Euler's number). It naturally appears in models involving **random arrivals** and **decay**. You do **not** need to compute it manually, software handles it

63.0.5 Key Assumptions (Very Important)

The Poisson model assumes: - **Independent events**

One event does not affect another - **Constant average rate** (λ)

The rate does not change over the interval - **Events occur randomly**

Not in clusters or bursts

If these assumptions fail, Poisson may not be appropriate.

It is useful for modeling arrivals, failures, errors, or requests when events happen independently at a roughly constant rate.

```
np.random.poisson(lam=3, size=10_000).mean()
```

63.0.6 (1D) Zero-Inflated Poisson → Excess Zeros in Count Data

Some real-world count datasets contain **many more zeros** than a standard Poisson model can explain.

Examples: - many users make **zero purchases** - many customers file **no insurance claims** - many days have **no system errors**

[Image omitted in PDF — see the online textbook]

A standard Poisson model assumes zeros occur naturally from random variation. When zeros appear **too frequently**, this assumption breaks.

Idea: Zero-Inflated Poisson (ZIP) > > A Zero-Inflated Poisson model assumes **two underlying processes**: > > **(1) Inflation component (Bernoulli-like)**:

> Determines whether an observation is a *structural zero*

> (e.g., an inactive user with no chance of events) > > **(2) Poisson component**:

> Models the number of events *when activity is possible*

This separation distinguishes: - “**cannot happen**” zeros (structural zeros) - “**could happen but didn't**” zeros (random zeros)

Why This Matters? If excess zeros are ignored: - Poisson underestimates zeros - model fit degrades - conclusions become misleading

63.0.7 When to Use Zero-Inflated Poisson

Use a Zero-Inflated Poisson model when count data has far more zeros than a standard Poisson can explain.

63.0.8 (2A). Uniform → equally likely values

A Uniform distribution assumes all values **in a range are equally likely**.

Mathematically: Uniform Distribution

If $X \sim \text{Uniform}(a, b)$, then

$$f(x) = \frac{1}{b-a} \text{ for } a \leq x \leq b$$

[Image omitted in PDF — see the online textbook]

Source: Geeksforgeeks.

It is often used as a baseline model, for random sampling, simulations, and sanity checks when no additional structure is assumed.

```
np.random.uniform(0, 1, size=10_000).mean()
```

63.0.9 (2B) Normal (Gaussian) Distribution → Noise, Error, and Aggregated Behavior

The **Normal distribution** describes data that clusters around a central value, with fewer observations as you move farther away.

It is often called the **bell-shaped curve**.

Mathematically

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

where: - **Mean** (μ): the center of the distribution — the average or typical value

- **Standard deviation** (σ): how spread out the values are — larger σ means more variability

For a Normal distribution:

$$\text{Mean} = \text{Median} = \text{Mode} = \mu$$

(the average, middle, and most frequent value coincide)

63.0.10 Intuition

The Normal distribution appears when: - many **small, independent effects** add together - we observe **averages** or **measurement noise**

This is why it commonly appears in: - sensor and measurement noise

- model residuals (errors)

- test scores and biological traits

[Image omitted in PDF — see the online textbook]

Bell-shaped Normal distribution showing center and spread. Source: GeeksforGeeks.

63.0.11 The 68–95–99.7 Rule

The Normal distribution has a predictable spread:

- ~68% of values lie within ± 1 of the mean
- ~95% lie within ± 2
- ~99.7% lie within ± 3

This rule helps quickly estimate where most data values fall.

63.0.12 Why the Bell Shape Appears

- Most values are **close to the mean**
- Extreme values are **rare**
- Data spreads out symmetrically on both sides

Averages are common, extremes are rare, and **variability matters**.

```
“python np.random.normal(loc=0, scale=1, size=10_000).mean()
```

```
##Why Distribution Awareness Comes First
```

Understanding how data is distributed helps determine:

- which statistical tests are valid
- which machine learning models are appropriate
- which evaluation metrics are meaningful

Using the wrong distributional assumptions can lead to confident but incorrect conclusions, even when the computations are correct.

In data science, modeling starts with understanding the data-generating process, and probability distributions are the language we use to describe it.

Data Science Connection

- Bernoulli and Binomial → classification outcomes, A/B testing
- Poisson and Zero-Inflated Poisson → event counts, sparse data
- Uniform → random baselines and simulations
- Normal → noise, error, averages, and model residuals

In practice, **distribution awareness comes before modeling**. ***

63.1 interactive distribution

64 Statistical Distribution Explorer

This interactive notebook lets you visualize and compare different probability distributions commonly used in statistics and data science.

Available Distributions: - **Normal Distribution:** The classic bell curve ($\mu=0$, $\sigma=1$) - fundamental in statistics and natural phenomena - **Poisson Distribution:** Models count data and rare events ($\lambda=5$) - useful for event frequency analysis

- **Bimodal Distribution:** A mixture of two normal distributions - demonstrates multi-modal data patterns

Please be patient while the playground loads

[Interactive content omitted in PDF]

65 Central Limit Theorem (Why Normal Appears Everywhere)

In data science, we repeatedly see the Normal distribution, even when the original data is not normal. The reason is the **Central Limit Theorem (CLT)**. > **CLT Theorem:** Regardless of the original distribution, the distribution of sample means approaches a Normal distribution as sample size increases.

Mathematically: Central Limit Theorem (Informal) > > Let $X_1, X_2, \dots, X_n$ be independent random variables with the same mean μ and finite variance. > > As n becomes large, the distribution of the **sample mean** > > $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ > > approaches a **Normal distribution**, regardless of the original distribution of the data.

Intuition: - Individual data points can be messy, skewed, or discrete - Averages of many observations become predictable - Noise cancels out, and structure emerges

This is why:

- averages of clicks
- average model errors
- average measurements

often behave normally, even if the raw data does not.

65.0.1 Central Limit Theorem Visual Overview

[Table omitted in PDF — see the online textbook]

These figures illustrate why sample means tend to follow a **Normal distribution**, even when the original data is not Normal.

Python Simulation: CLT in Action

We will start with data that is not normal (Uniform), then look at averages.

Notice: Even when the raw data is not bell-shaped, the **averages form a bell-shaped curve**. This happens **without assuming normal data**; this is the **Central Limit Theorem** at work.

65.0.2 Why CLT Matters in Data Science

The Central Limit Theorem explains why we can: - use Normal-based confidence intervals - apply z-tests and t-tests - model average error with Gaussian assumptions - trust metrics based on means

Data Science Insight

Many statistical tools assume normality **of averages, not raw data**.

This assumption is justified by the **Central Limit Theorem**.

66 Summary of the Chapter

In this chapter, we introduced probability as a way to reason about **uncertainty** and showed how **probability distributions describe how data is generated** in real-world settings.

We began with the **laws of probability**, which define how probabilities behave and ensure consistency: - probabilities are always non-negative - the total probability of all possible outcomes is 1 - probabilities of mutually exclusive events add together

We then introduced **expected value**, which represents the long-run average outcome of a random process and is a key concept for reasoning about outcomes in data science.

We distinguished between **discrete** and **continuous** distributions and covered the most common ones used in data science:

- **Bernoulli**: models a single yes/no outcome (foundation of binary classification)
- **Binomial**: models repeated Bernoulli trials (used in A/B testing and conversions)
- **Poisson**: models event counts over time or space (arrivals, errors, failures)
- **Zero-Inflated Poisson**: handles count data with many zeros (sparse user activity)
- **Uniform**: assumes all values in a range are equally likely (simulations, baselines)
- **Normal (Gaussian)**: models noise, error, and averages

We also introduced the **Central Limit Theorem**, which explains why **averages often follow a Normal distribution**, even when the original data does not. This idea supports many statistical tools used in data science.

A key takeaway is that **distribution awareness comes before modeling**.

The choice of distribution affects: - which assumptions are valid - which models are appropriate - which evaluation metrics make sense

In data science, effective modeling begins with understanding **how data is generated**.

Probability distributions provide the language to describe that process clearly and correctly.

67 Knowledge Check

68 Mathematical Examples (with Solutions)

68.0.1 Question 1: Law of Total Probability and Conditional Probability

A company runs an email campaign to two groups of users:

- 70% of users are **existing customers**
- 30% of users are **new users**

The probability that a user opens the email is: - 40% for existing customers - 10% for new users

(a) What is the overall probability that a randomly selected user opens the email?

(b) If a user opened the email, what is the probability that the user was an existing customer?

68.0.2 Solution

Let: - E = existing customer

- N = new user

- O = email is opened

Given: - $P(E) = 0.7$, $P(N) = 0.3$

- $P(O | E) = 0.4$

- $P(O | N) = 0.1$

(a) **Law of Total Probability** $P(O) = P(O | E)P(E) + P(O | N)P(N)$

$$P(O) = (0.4)(0.7) + (0.1)(0.3) = 0.28 + 0.03 = 0.31$$

Answer: The probability a user opens the email is **31%**.

(b) **Conditional Probability** $P(E | O) = \frac{P(O | E)P(E)}{P(O)}$

$$P(E | O) = \frac{0.4 \times 0.7}{0.31} \approx 0.903$$

Answer: About **90.3%** of opened emails come from existing customers.

68.0.3 Question 2: Expected Value with Negative Cost (Profit/Loss)

A company launches an online ad with the following outcomes:

- With probability 0.2, the user clicks and the company earns **+\$5**
- With probability 0.8, the user does not click and the company loses **−\$1** (ad cost)

Let X be the profit from showing one ad.

(a) Compute the expected value of X .

(b) Interpret the result.

68.0.4 Solution

Possible outcomes:

- $X = 5$ with probability 0.2
- $X = -1$ with probability 0.8

Expected value:

$$\mathbb{E}[X] = (5)(0.2) + (-1)(0.8)$$

$$\mathbb{E}[X] = 1 - 0.8 = 0.2$$

Answer (a): The expected profit per ad is **\$0.20**.

Answer (b):

Even though most users do not click, the campaign is profitable *on average* over many impressions.

This is why expected value is critical for decision-making in data science and business.

68.0.5 Question 3: Expected Value + Binomial Interpretation

Each user independently clicks an ad with probability $p = 0.05$.

The ad is shown to **200 users**.

- (a) What distribution models the number of clicks?
 - (b) What is the expected number of clicks?
-

68.0.6 Solution

Each user click is a Bernoulli trial. The total number of clicks is the sum of many Bernoulli trials. $X \sim \text{Binomial}(n = 200, p = 0.05)$ Expected value: $E[X] = np = 200 \times 0.05 = 10$ **Answer:** On average, we expect **10 clicks** per campaign.

69 Practice Problems (No Solutions, Try Yourself)

69.0.1 Problem 1: Law of Total Probability

A video platform categorizes users into two groups:

- 65% of users are **subscribers**
- 35% of users are **non-subscribers**

The probability that a user watches a recommended video is: - 50% for subscribers - 20% for non-subscribers

- (a) What is the overall probability that a randomly selected user watches the video?
 - (b) Which law of probability is used to compute this value?
-

69.0.2 Problem 2: Conditional Probability

Using the same setting as Problem 1:

- (a) If a user watched the video, what is the probability that the user was a subscriber?
 - (b) Explain in words what this probability represents.
-

69.0.3 Problem 3: Expected Value with Costs

A delivery service faces the following outcomes per order:

- With probability 0.9, the order is delivered on time with **\$0 cost**
- With probability 0.1, the order is late and incurs a **—\$20 penalty**

Let X be the cost per order.

- (a) Write down the possible values of X and their probabilities.
- (b) Compute the expected cost per order.
- (c) Interpret the expected value in practical terms.

69.0.4 Problem 4: Distribution Choice

For each scenario below, identify an appropriate distribution and briefly justify your choice:

1. Number of emails received per hour
2. Whether a user clicks on an ad
3. Average error across many model predictions
4. Number of insurance claims made by a customer in a month with many zero-claim customers

70 Chapter 7 - Descriptive Statistics

71 Descriptive Statistics: Understanding Data Before Modeling

In data science, the temptation is strong to jump straight into modeling such as regression, classification, neural networks. But before we predict anything, we **must understand what we already have**. Descriptive statistics is the foundation of that understanding.

It answers a deceptively simple question:

What does this dataset look like?

Not through thousands of **raw values**, but through carefully designed summaries that **capture structure, variability, and relationships**. This chapter introduces the fundamental tools used to summarize data responsibly and intelligently.

71.1 Why Summary Statistics Exist?

Imagine you are given a dataset with 50,000 salary records. You cannot stare at every number. You need structure. Descriptive statistics allow us to:

- **Summarize** large datasets concisely
- **Identify** patterns
- **Detect** unusual behavior
- **Prepare** for modeling

But here is the warning:

A summary statistic is *not* the dataset.

It is a **compressed representation** of the dataset. And compression always loses information. If you compute a mean without understanding the distribution, you risk misunderstanding the data entirely.

Same mean same distribution.

Four completely different datasets can have identical mean and variance, yet look nothing alike. That is why summary statistics come after visual and structural understanding, not before.

71.1.1 Case Study: The Fitness App Illusion

You are working for a fitness startup. Your job is to summarize user daily step counts. You compute: Average daily steps = 9,800.

Marketing is thrilled. “Almost at 10,000! Users are active!”

But then you look deeper.

Here are the steps for 8 users (one of them is a marathon runner):

[Image omitted in PDF — see the online textbook]

The mean says: “Users are very active.” The distribution says: “One extreme user is pulling the average.”

This is your **first lesson**:

A single number can hide structure.

72 The Four Pillars of Describing Data

The four questions every Data Scientist should ask when we summarize a dataset (usually describe it along four dimensions):

- Where is the **center** (location)?
- How **spread** out is it?
- What **shape** does it have?
- How do variables **move together**?

These four ideas form the **backbone** of descriptive or summary statistics.

73 (I). Measures of Location: Where Is the Center?

If you plotted your data on a number line, where would the “center” be?

At first, this seems like a simple question, but mathematically, the answer depends on how we define **what it means to be “close” to the data**.

There are two main ways to define closeness:

- **Absolute distance** → leads to the **median**
- **Squared distance** → leads to the **mean**

Because these definitions are different, they produce different notions of “center.”

73.1 (I.a) The Arithmetic Mean

The arithmetic mean is what most people call the “average.”

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The mean minimizes the **sum of squared distances**:

$$\sum_{i=1}^n (x_i - \bar{x})^2$$

That squared term is important. Squaring makes **large distances dramatically more influential**.

The mean is very sensitive to extreme values (outliers).

73.1.1 Example:

[Image omitted in PDF — see the online textbook]

Figure: Replacing a single value (17 → 17000) drastically increases the mean, showing why the mean is not robust to extreme values.

The mean works well when:

- Data is roughly symmetric
- Outliers are meaningful (not errors)
- Total magnitude matters

Example: - Average temperature - Average revenue - Average exam score in a balanced class

73.2 (I.b) The Median

The median is the **middle value** after sorting the data.

- If the dataset has an **odd number** of values → pick the middle one
- If **even** → take the average of the two middle values

The median minimizes the **sum of absolute distances**:

$$\sum_{i=1}^n |x_i - m|$$

Unlike squaring, absolute distance does not amplify extremes. So the median is **robust to outliers**.

73.2.1 Example (Income Data)

Income distributions are usually **right-skewed**:

[Image omitted in PDF — see the online textbook]

Figure: Most people earn moderate incomes. A few people earn extremely high incomes

Because of this:

- Mean income becomes much larger
- Median income stays closer to what most people earn

So the median gives a more realistic “typical value.”

73.2.2 When Mean and Median Differ?

The difference between them tells us about **skewness**:

- **Symmetric distribution** $\rightarrow$ Mean = Median
- **Right-skewed distribution** $\rightarrow$ Mean $>$ Median
- **Left-skewed distribution** $\rightarrow$ Mean $<$ Median

Remember: The mean reacts to the extreme values and the median resists it.

(We will revisit this idea later in the chapter.)

73.2.3 Weighted Mean: When All Data Points Are Not Equal

Sometimes, observations should not contribute equally.

Example: Averaging housing prices across states.

If you ignore population size: - A small state influences the average as much as a large state. - This produces misleading conclusions.

In such cases, we use a **weighted mean**:

$$\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

Where: - w_i = weight (population, sample size, frequency) - x_i = observation

Failing to weight appropriately can significantly distort interpretation.

74 Pythagorean Means: Three Ways to Average

The arithmetic mean is not the only way to compute an average. There are three classical types of averages, known as the **Pythagorean means**:

1. Arithmetic Mean
2. Geometric Mean
3. Harmonic Mean

Each is appropriate for a different type of data. Choosing the wrong mean can lead to incorrect conclusions.

74.1 1. Geometric Mean

$$GM = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

The geometric mean multiplies all values together and then takes the n -th root. It is used when values change **multiplicatively** rather than additively.

Common uses: - Growth rates - Investment returns - Population growth - Ratios

74.1.1 Why Use Geometric Mean (NOT Arithmetic Mean) for Growth?

Growth **compounds over time**, so we must multiply changes instead of adding them.

Example (Investment Growth): Suppose an investment changes by:

- +10%
- +20%
- -5%

We must convert percentages into multipliers (**growth factors**):

- +10% $\rightarrow$ 1.10
- +20% $\rightarrow$ 1.20
- -5% $\rightarrow$ 0.95

What Happens if We Use Arithmetic Mean?

$$\text{Arithmetic Mean} = \frac{10 + 20 - 5}{3} = 8.3\%$$

This is incorrect because it ignores compounding.

Apply Geometric Mean (Correct Method)

Multiply growth factors:

$$1.10 \times 1.20 \times 0.95 = 1.254$$

Now, take the cube root (since there are 3 periods):

$$GM = (1.254)^{1/3}$$

$$GM \approx 1.078$$

Convert back to percentage growth:

$$\text{Average Growth Rate} = 1.078 - 1 = 0.078 = 7.8\%$$

The geometric mean is smaller because it correctly accounts for **compounding effects**.

Key Idea: When values **multiply over time**, always use the geometric mean.

- Remember:**
- It is less sensitive to extreme large values than the arithmetic mean
 - All values must be positive
 - It is appropriate when data represents **relative change** (like growth rates or ratios)

74.2 Harmonic Mean

The **harmonic mean** is another measure of central tendency, but it is designed specifically for averaging **rates, ratios, and “per-unit” quantities**.. Formula:

$$HM = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

Where: - n = number of observations

- x_i = each value

When Should We Use the Harmonic Mean?

Use the harmonic mean when averaging:

- Speeds
- Rates (e.g., km/hour, requests/second)
- Ratios
- “Per unit” quantities

Why? Because when dealing with rates, **time or denominator matters more than magnitude**. Slower rates dominate total time, so they should influence the average more strongly.

Example 1: Average Speed: Imagine you drive:

- First half of the distance at **60 km/h**
- Second half of the distance at **40 km/h**

What is your average speed? Most students instinctively compute:

$$\text{Arithmetic Mean} = \frac{60 + 40}{2} = 50$$

But that’s incorrect. **Why?** Because speed is distance divided by time (speed = distance $\div$ time). You spend **more time traveling at 40 km/h (the slower speed)** than at 60 km/h. Slower speeds should affect the average more (dominate total time). The harmonic mean correctly accounts for this:

$$HM = \frac{2}{\frac{1}{60} + \frac{1}{40}}$$

This gives:

$$HM = 48 \text{ km/h}$$

Remember: When averaging speeds over equal distances:

- Slower speeds dominate total time

- The harmonic mean gives **more weight to smaller values**

Example 2: Server Response Time Suppose two servers process requests at:

- Server A: 100 requests/sec
- Server B: 10 requests/sec

Even if workload is evenly distributed, the slower server becomes the bottleneck. The harmonic mean reflects this by penalizing smaller rates more heavily.

74.2.1 Why Not Arithmetic Mean?

The arithmetic mean assumes “Quantities combine additively”. But rates combine through **time accumulation**, not addition. That’s the difference.

Important Properties:

- Gives more weight to smaller values
- Always less than or equal to the arithmetic mean
- Undefined if any value equals zero
- Only valid for positive numbers

Remember: Use the harmonic mean when averaging anything that is measured **per unit** (per hour, per second, per distance, etc.).

74.3 Big Picture Comparison

Mean Type	Best For	Sensitive To
Arithmetic Mean	Additive quantities	Large values
Geometric Mean	Multiplicative growth processes	Relative scale
Harmonic Mean	Rates and ratios	Small values

75 (II) Measures of Shape: Skewness, Modality, and Distribution Behavior

Understanding the **center** is not enough.

Two datasets can have: - The same mean - The same median - The same variance

And still look completely different. That’s why we must understand **shape**.

76 (II.a) Skewness: Which Direction Is the Tail?

Skewness describes the **asymmetry** of a distribution. It tells us whether one tail is longer than the other.

76.0.1 Three Main Cases

1. Symmetric (Zero-Skewed) Distribution

- Mean = Median
- Tails are balanced
- Example: Exam scores in a well-calibrated course

2. Right-Skewed (Positively Skewed)

- Long tail to the right
- Mean > Median
- Extreme large values pull the mean upward

3. Left-Skewed (Negatively Skewed)

- Long tail to the left
- Mean < Median
- Extreme small values pull the mean downward

[Image omitted in PDF — see the online textbook]

Figure: Comparison of symmetric, right-skewed, and left-skewed distributions, showing the relative positions of mean, median, and mode. Source: Miro.Medium.

Example: Skewness in Real Life

- **Startup salaries:** A few extremely high values create a long right tail → **Right skew (Mean > Median)**
- **Easy exam scores:** A few very low values create a long left tail → **Left skew (Mean < Median)**

Rule: Skewness always points toward the long tail.

76.0.2 Why Skewness Matters?

Skewness helps us understand how data is distributed. When data is highly skewed, a few extreme values can pull the mean toward the long tail, making it less representative of most observations. In these cases, the median often gives a better idea of a typical value, and knowing the skewness helps us interpret averages more correctly.

77 (II.b) Modality: How Many Peaks?

Modality describes the **number of peaks** in a distribution.

It helps reveal whether the data comes from **one group or multiple distinct groups**.

77.0.1 Types of Modality

1. Unimodal

- One peak
- One dominant group

2. Bimodal

- Two peaks
- Two distinct subgroups

3. Multimodal

- More than two peaks
- Multiple underlying processes

[Image omitted in PDF — see the online textbook]

Figure: Examples of distribution modality: unimodal, bimodal, uniform, and multimodal; showing how the number of peaks reveals the underlying structure of the data. Source:Stack Overflow.

Modality helps us understand whether the data comes from **one group or multiple distinct groups**.

77.0.2 Why Modality Matters

Modality helps us determine whether data represents:

- One homogeneous group
- Multiple distinct groups
- Different underlying behaviors

Ignoring modality can hide important patterns within the data.

Quick Rule: Each peak often represents a different subgroup in

77.0.3 Example: Remote vs In-Office Work Hours

You measure daily working hours:

- **Remote workers:** 7–8 hours
- **In-office workers:** 9–10 hours

The histogram shows **two peaks**, one for each group.

Although the overall mean may be around 8.5 hours, there is no single “typical” worker because the data contains two distinct populations.

This is an example of a **bimodal distribution**.

[Image omitted in PDF — see the online textbook]

Figure: A bimodal distribution showing two distinct peaks, representing two underlying groups.

Beyond skewness and modality, we also look at:

77.1 Outliers

Outliers are data points that are far from most other values.

They can:

- Indicate data entry errors
- Represent rare but important events

- Suggest a different underlying data-generating process

Important: Not all outliers should be removed; they may contain valuable information.

78 (III) Measures of Variability: How Spread Out Is the Data?

Knowing the center is not enough. Two datasets can have the same mean, but behave completely differently.

Variability measures how tightly or loosely data points cluster around the center.

78.1 (III.a) Variance

Variance measures the **average squared deviation from the mean**.

78.1.1 Population Variance

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

78.1.2 Sample Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Why divide by $n - 1$ instead of n ? Because when estimating from a sample, we correct for bias. This is called **Bessel's correction**.

78.2 Why Squared Deviations?

We square differences because:

- Positive and negative deviations would cancel out otherwise
- Squaring gives more weight to larger deviations
- It aligns with optimization methods used in statistical modeling

But variance has a **problem**: Its units are squared (example: if data is in dollars, variance is in dollars²). This makes it difficult to interpret directly.

78.3 (III.b) Standard Deviation

The **standard deviation** solves this problem by taking the square root of variance.

$$s = \sqrt{s^2}$$

Now the units match the original data. This makes it easier to interpret.

78.3.1 Interpretation

- **Small standard deviation** → Data points are tightly clustered around the mean
- **Large standard deviation** → Data points are widely spread out

Standard deviation tells us how much variability exists in the dataset.

78.3.2 Example: Same Mean, Different Spread

Dataset A: 50, 51, 49, 50, 50

Dataset B: 10, 90, 30, 70, 50

Both datasets have the same mean (50), but their variability is very different.

[Image omitted in PDF — see the online textbook]

Figure: Dataset A is tightly clustered (small standard deviation), while Dataset B is widely spread out (large standard deviation).

Same average does not mean the data behaves the same — spread matters.

79 (IV) Correlation and Relationships Between Variables

So far we've described **one variable at a time**.

Now we ask:

How do two variables move together?

79.1 (IV.a) Correlation

Correlation measures the **strength and direction of a linear relationship** between two variables. The most common measure is **Pearson's correlation coefficient (r)**.

$$-1 \leq r \leq 1$$

79.2 Interpretation of r

r Value	Meaning
r 1	Strong positive linear relationship
r -1	Strong negative linear relationship
r 0	No linear relationship

[Image omitted in PDF — see the online textbook]

Figure: Scatter plot examples showing positive, negative, and no correlation between two variables.

- **Positive:** As x increases, y increases (e.g., study time and exam score)
- **Negative:** As x increases, y decreases (e.g., price and demand)
- **None:** No clear relationship (e.g., shoe size and exam score)

80 Summary of the Chapter

Different means are suited for different types of data:

- **Arithmetic mean** balances distances
- **Geometric mean** captures proportional growth
- **Harmonic mean** reflects rates and time dominance

Choosing the wrong mean can lead to misleading conclusions.

Key Questions in Descriptive Statistics: Descriptive statistics answer three fundamental questions:

1. **Where is the data located?** → Mean, Median
2. **How spread out is it?** → Variance, Standard Deviation
3. **How do variables move together?** → Correlation

Understanding Distribution Structure: Before analyzing data, consider:

- Is the distribution symmetric or skewed?
- Are there long tails or extreme values?
- Are there multiple peaks indicating subgroups?

Understanding data shape prevents misleading interpretations.

Important Insights About Correlation:

- Correlation measures **linear relationships only**
- Correlation does **not imply causation**
- Outliers can strongly distort correlation

80.1 Final Takeaway

Good modeling begins with good descriptive understanding.

Always examine:

- Center
- Spread
- Shape

- Relationships

We will explore how **data visualization** reveals these patterns more clearly in the next chapter.

80.2 Knowledge Check

[Interactive content omitted in PDF]

81 Chapter 8 - Hypothesis Testing

82 Hypothesis Testing in Data Science

A Story About Decisions, Evidence, and Uncertainty

82.1 Why Hypothesis Testing Exists

Imagine you are a data scientist at a food delivery company. Your team launches a **new recommendation algorithm** to increase the average order value.

After one week:

- Old system average order: **\$24.10**
- New system average order: **\$25.30**

Your manager asks:

“Did the new system actually work?”

The number increased, but that does not prove the algorithm caused the change.

Maybe customers simply ordered more that week.

Maybe there was a holiday.

Maybe the difference is just random fluctuation.

Hypothesis testing helps us answer this question.

It provides a structured way to decide whether an observed change reflects a real effect or ordinary variation.

82.2 The Core Idea

Hypothesis testing is a **decision-making framework under uncertainty**.

We begin with a default assumption: that nothing has changed, and evaluate whether the observed data are unlikely enough to challenge that assumption.

This is similar to a courtroom:

- A defendant is presumed innocent
- Evidence must be strong to establish guilt

In statistics:

- Default assumption → no effect
- Strong evidence → reject that assumption

82.3 Formal Definition

Hypothesis testing is a statistical method used to make inferences about a population based on sample data by evaluating competing claims (hypotheses) and determining whether the observed sample evidence is sufficiently unlikely if the null hypothesis is true.

The overall logic of hypothesis testing can be visualized as a structured decision process:

[Table omitted in PDF — see the online textbook]

Figure: Visual overview of the hypothesis testing decision process.

Both diagrams illustrate the structured sequence of statistical inference: defining hypotheses, selecting a significance level (), computing a test statistic from sample data, and making a decision using either the critical value method or the p-value method to determine whether to reject or fail to reject the null hypothesis Sources: GeeksforGeeks and Vitalflux.

82.4 Steps in Hypothesis Testing

The full workflow follows a structured sequence:

1. State the null and alternative hypotheses
2. Choose a significance level ()
3. Select an appropriate statistical test and compute a test statistic
4. Apply a decision rule using either the **critical value method** or the **p-value method**.
5. Draw a conclusion and interpret the result in context

Both the critical value approach and the p-value approach lead to the **same** statistical decision.

83 Step 1: Define the Hypotheses

Every hypothesis test begins with two competing statements.

83.0.1 Null Hypothesis (H)

The “nothing changed” claim.

It represents the default world — no effect, no difference, no improvement.

Example:

The new recommendation system does NOT change average order value.

Mathematically:

$$H_0 : \mu_{\text{new}} = \mu_{\text{old}}$$

83.0.2 Alternative Hypothesis (H_1 or H_a)

The claim we are testing for.

Example:

The new system increases average order value.

Mathematically:

$$H_1 : \mu_{\text{new}} > \mu_{\text{old}}$$

83.1 Why Assume the Null Is True?

Because science is cautious.

We don't claim something works just because numbers look different.

We require **strong evidence**.

Think of it like this:

- If data fits normal random variation $\rightarrow$ keep the null
- If data looks extremely unlikely $\rightarrow$ reject the null

83.2 Step 2: Understand Random Variation

Even if nothing changes, data still fluctuates.

Example:

Daily average order values might look like: 24.1, 23.8, 24.6, 24.0, 25.0, 23.9 ...

Variation is natural.

So when we see \$25.30, we must ask:

- Is this within normal variation?
- Or unusually large?

To answer this, we calculate a probability.

84 Step 3: Choose a Significance Level ()

Before we look at the data too deeply, we decide:

How much risk of a false alarm are we willing to accept?

This risk is the **significance level**, written as α . Common choice: $\alpha = 0.05$

Meaning:

We accept a 5% chance of rejecting the null hypothesis even when it is actually true (this is called a **Type I error**).

You can think of α like a strictness setting:

- Smaller (like 0.01) $\rightarrow$ harder to reject H_0
- Larger (like 0.10) $\rightarrow$ easier to reject H_0

84.1 Step 4: Two Ways to Make the Same Decision

There are two equivalent ways to decide whether to reject the null hypothesis:

1. The **critical value (rejection region) approach**
2. The **p-value approach**

Both use the same elements:

- significance level (α)
- sampling distribution
- observed test statistic

They differ only in how the decision is evaluated.

[Image omitted in PDF — see the online textbook]

84.2 Step 4A: Critical Value and Rejection Region

The critical value approach defines a **decision boundary**.

84.2.1 Critical Value

A **critical value** is the cutoff separating likely from unlikely outcomes under the null hypothesis. If the test statistic crosses this boundary, the result is statistically significant. It depends on:

- significance level (α)
- sampling distribution (z, t, etc.)
- one-tailed vs two-tailed test

84.2.2 Rejection Region

The **rejection region** contains test statistic values considered too unlikely if H_0 is true. These lie in the extreme tails of the sampling distribution.

Decision rule:

- inside rejection region $\rightarrow$ reject H_0
- outside rejection region $\rightarrow$ fail to reject H_0

Total probability of rejection region = α .

[Image omitted in PDF — see the online textbook]

Figure: Sampling distribution showing rejection regions in a two-tailed test. The shaded tails represent values extreme enough to reject H_0 .

84.2.3 One-Tailed vs Two-Tailed Tests

The rejection region depends on the research question.

One-tailed test: interest in one direction only $\rightarrow$ all in one tail.

Two-tailed test: interest in any difference $\rightarrow$ split across both tails.

Example ($\alpha = 0.05$):

- one-tailed $\rightarrow 0.05$ in one tail
- two-tailed $\rightarrow 0.025$ in each tail

[Image omitted in PDF — see the online textbook]

Figure: Rejection regions for left-tailed, two-tailed, and right-tailed hypothesis tests. Source: Elsevier.

84.3 Step 4B: p-Value Approach

The p-value measures how extreme the observed result is under the null hypothesis.

Decision rule:

- $p\text{-value} \leq \alpha \rightarrow \text{reject } H_0$
- $p\text{-value} > \alpha \rightarrow \text{fail to reject } H_0$

84.4 How the Two Methods Are Connected

Both approaches compare probability to α :

- critical value method $\rightarrow$ compare statistic to boundary
- p-value method $\rightarrow$ compare probability to α

They always lead to the same decision.

84.4.1 Example

Two-tailed test

$\alpha = 0.05$

critical values = ± 1.96

observed statistic = 2.30

Critical value method $\rightarrow 2.30 > 1.96 \rightarrow \text{reject } H_0$

p-value method $\rightarrow p = 0.021 < 0.05 \rightarrow \text{reject } H_0$

Hypothesis testing asks whether the observed result is too unlikely under the null hypothesis to be explained by chance alone.

84.5 Step 5: Compute the Test Statistic

After defining the decision rule, the next step is to measure how extreme the observed data actually is.

We do this by computing a **test statistic**, a standardized numerical summary that compares the sample result to what we would expect if the null hypothesis were true. In simple terms, the test statistic answers:

How far is the observed result from what we would expect under H_0 ?

The choice of test statistic depends on the data and research question. Common examples include:

- **z-statistic** → when population variability is known
- **t-statistic** → when population variability is unknown (most common)
- **χ^2 statistic** → categorical data
- **F statistic** → comparing multiple group means (ANOVA)

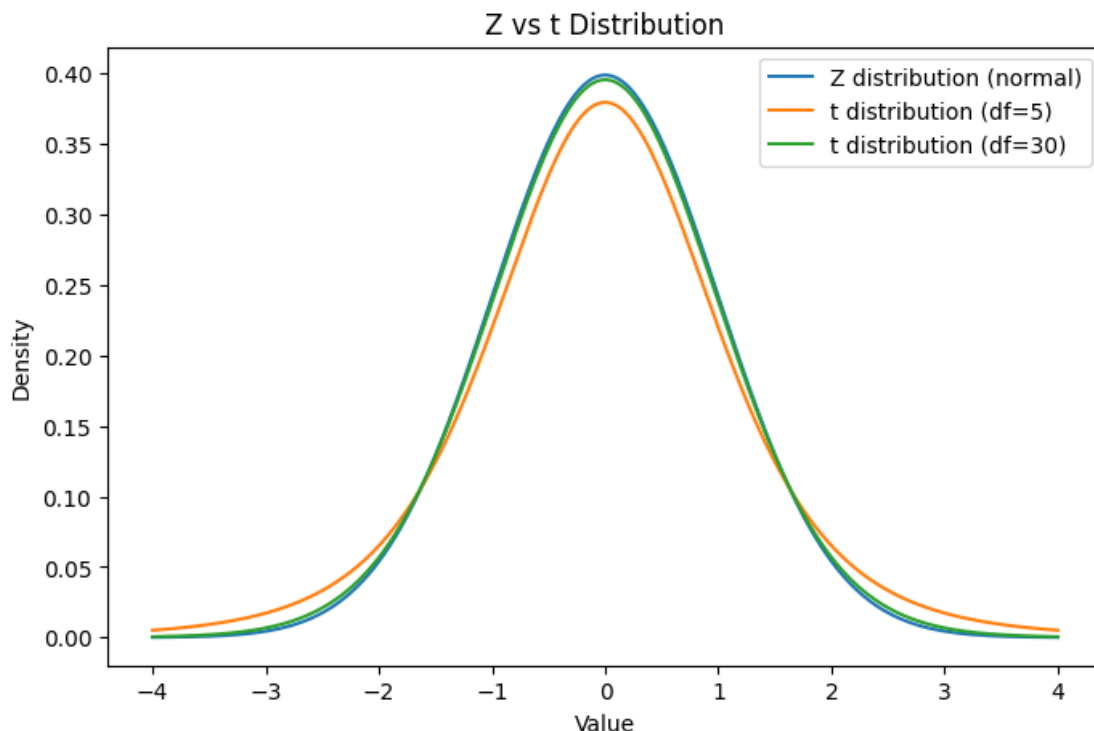
The test statistic places the observed result on the sampling distribution. Once computed, it is compared to the decision rule defined in Step 4 (critical value or p-value). This comparison determines whether the result is statistically significant.

To measure how extreme the observed data is under the null hypothesis, we compute a **test statistic**.

The type of statistic depends on the test being used.

- **z-test** → population variance known
- **t-test** → population variance unknown (most common)
- **chi-square** → categorical data
- **ANOVA** → comparing multiple groups

For comparing means in data science, we most often use a **t-test**.



Notice: - Small sample $\rightarrow$ thicker tails $\rightarrow$ more uncertainty
 - Large sample $\rightarrow$ t looks like normal

As sample size increases, t-test becomes similar to z-test.

84.6 Example Walkthrough

Let's return to the food delivery company.

Data:

- Old system mean = 24.10
- New system mean = 25.30
- Sample size = 200 users
- Sample standard deviation = 5

We run a t-test. Suppose we get: p-value = 0.02

Interpretation: If the new system had no effect, there is only a **2% chance** of seeing this difference.

That is small. So we reject the null hypothesis.

Conclusion: Evidence suggests the new recommendation system increases order value.

84.7 What Hypothesis Testing Does NOT Tell You

Very important. Rejecting the null does NOT prove something is true with certainty. It only means:

The data is unlikely under the null assumption.

Also:

- Small p-value large impact
- Statistical significance practical importance

A tiny improvement can still be statistically significant if sample size is huge.

Always check **effect size** too.

84.8 Two Common Mistakes (Errors)

Because decisions are made under uncertainty, mistakes can happen.

84.8.1 Type I Error: False Alarm

Rejecting a true null.

Example: We think the new system works... but it doesn't.

Probability = (e.g., 5%)

84.8.2 Type II Error: Missed Detection

Failing to reject a false null. Example: The system really works... but we fail to detect it.

Visualizing Type I and Type II Errors Understanding statistical errors becomes much easier when we see them visually. The graph below shows two distributions:

- The **null hypothesis distribution**
- The **alternative hypothesis distribution**

Because these two distributions overlap, mistakes are possible.

- **Type I error** () — rejecting a true null hypothesis (**false positive**)
- **Type II error** () — failing to reject a false null hypothesis (**false negative**)

The shaded regions in the figure show where these errors occur. Everything else represents correct decisions.

[Image omitted in PDF — see the online textbook]

Figure: Graphical representation of hypothesis testing errors. The blue curve shows the distribution when the null hypothesis (H_0) is true, and the green curve shows the distribution when the alternative hypothesis (H_a) is true.

The vertical line indicates the critical value. The shaded area under H_0 beyond this boundary represents the Type I error (), while the shaded area under H_a on the non-rejection side represents the Type II error ().

Source: Wingify (VWO).

84.9 Why This Matters in Data Science

Hypothesis testing appears everywhere:

- A/B testing products
- Evaluating ML model improvements
- Medical research
- Marketing campaigns
- Feature impact analysis
- Policy evaluation

Any time we ask:

Did this change actually cause an effect?

We use hypothesis testing.

Final Business Interpretation: We answer three questions:

1. Is the difference statistically real?
2. How large is the difference?
3. Is it worth implementing?

Data science is not just testing: it is decision making.

In the next section, we will implement a **t-test in Python** and run a real A/B experiment.

85 Running a Real Hypothesis Test in Python

Now we move from concept to practice.

You are still the data scientist at the food delivery company.

We ran an A/B test:

- **Group A** → Old recommendation system
- **Group B** → New recommendation system

Each user places one order. We record the order value.

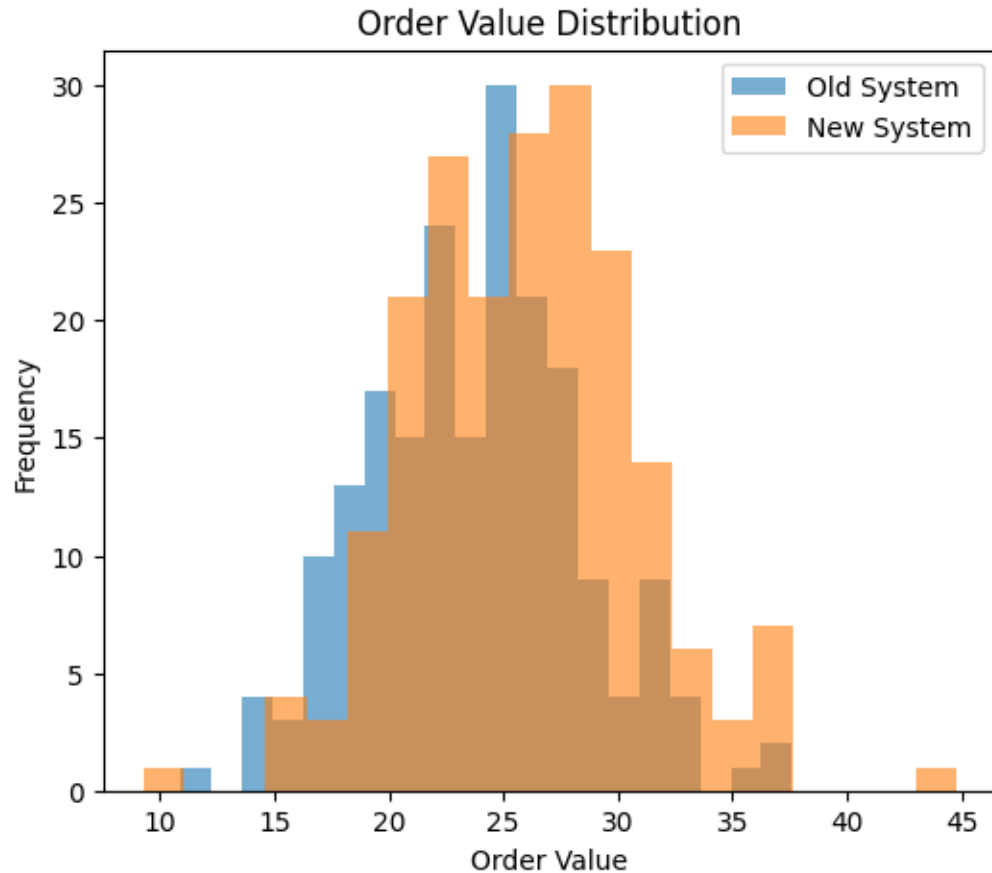
Our question:

Does the new system increase average order value?

We will answer this using a **two-sample t-test**.

Old mean: 23.8

New mean: 25.93



t-statistic: 4.4469
p-value: 1.1e-05
Reject the null hypothesis
Evidence suggests the new system changes order value

86 Different Types of Statistical Tests

Meet Maya, a data scientist who studies patterns in real-world data, from student performance to customer behavior. As she analyzes data, she realizes that different research questions require different statistical tools.

Sometimes she compares averages.

Sometimes she studies relationships between variables.

Sometimes she examines patterns in categories.

Because research questions vary, **statistical tests are designed for different purposes.**

Broadly, statistical tests fall into two main categories:

1. Parametric tests

2. Non-parametric tests

Understanding this distinction helps researchers choose the right method for their data.

86.1 Parametric vs Non-Parametric Tests

86.1.1 Parametric Tests

Parametric tests make assumptions about the data's distribution; most commonly that it follows a **normal distribution**. They typically require:

- numerical (continuous) data
- approximate normality
- independent observations
- sufficient sample size

When these assumptions are satisfied, parametric tests are powerful and precise. They are most often used to **compare means**.

Common parametric test statistics:

- $z \rightarrow$ comparing means with known population variability
- $t \rightarrow$ comparing means with estimated variability
- $F \rightarrow$ comparing multiple group means (ANOVA)

86.1.2 Non-Parametric Tests

Non-parametric tests make fewer assumptions about the data. They are useful when:

- data are not normally distributed
- sample sizes are small
- variables are categorical or ordinal
- parametric assumptions are violated

Instead of comparing means, they often compare **medians, ranks, or frequencies**.

86.2 Tests for Comparing Means

After learning how hypothesis testing works, Maya — our data scientist — faces a practical question:

Which statistical test should I use for my data?

The answer depends on **what kind of averages she wants to compare** and **what assumptions the data satisfy**.

Many real-world questions involve comparing numerical averages; test scores, income, blood pressure, user engagement, or sales. When the data are continuous and follow certain assumptions, we use **parametric tests**.

86.3 Parametric Tests for Means

Parametric tests compare numerical averages under specific assumptions.

They generally require:

- data that are approximately normally distributed
- independent observations
- a sufficient sample size

When these conditions hold, parametric tests are powerful and precise tools for detecting real differences.

Let's follow Maya as she chooses the right test for different situations.

86.3.1 (1) Z-Test

Maya rarely uses this test, but it is important conceptually.

A **z-test** compares a sample mean to a known population mean when:

- the population standard deviation is known
- the sample size is large

Example:

Is the average height of a sample different from the known national average?

In practice, population variability is rarely known, so z-tests are less common.

86.3.2 (2) T-Test

Most of Maya's work relies on **t-tests**, because population variability is usually unknown.

Different research questions require different types of t-tests.

One-Sample T-Test Compare one group's mean to a known value.

Independent Two-Sample T-Test Compare means of two separate groups.

Paired Sample T-Test Compare two measurements from the same individuals.

T-tests compare means when the population standard deviation is unknown, which is almost always the case.

86.3.3 (3) ANOVA (Analysis of Variance)

Eventually Maya studies **more than two groups**. She tests three different teaching methods and wants to compare all their average scores. Running many t-tests would increase the chance of false conclusions. Instead, she uses **ANOVA**, which tests whether at least one group mean differs from the others.

ANOVA tests whether at least one group mean differs from the others using an **F-statistic**, which compares variability between groups to variability within groups.

However, ANOVA **does not identify** which specific groups differ; only that a difference exists somewhere.

86.4 Post-Hoc Analysis

If ANOVA finds a significant difference, Maya performs **post-hoc tests** to determine which specific groups differ.

She performs **post-hoc tests**, which compare group pairs while controlling for increased error from multiple comparisons. These tests reveal exactly where the differences occur.

Some common types of Post Hoc Test: -Tukey's Honest Significant Difference (HSD) -Bonferroni Correction -Duncan's New Multiple Range Test

All of these tests identify which group(s) differ significantly from others

86.5 Non-Parametric Test: When Parametric Assumptions Do Not Hold

Sometimes Maya's data do not meet parametric assumptions.

Maybe the data are not normally distributed.

Maybe the sample is small.

Maybe the variables are categorical instead of numerical.

In these cases, she uses non-parametric tests that rely on **ranks or frequencies instead of raw numerical values**, often focusing on medians rather than means.

Instead of using actual values, these tests **convert data into ranks (ordered from smallest to largest)** and compare those ranks across groups.

86.6 Rank-Based Non-Parametric Tests

When comparing groups without normality assumptions, Maya uses rank-based methods.

- **Mann-Whitney U test** → alternative to independent t-test (compares **medians/ranks**)
- **Wilcoxon signed-rank test** → alternative to paired t-test (compares **median differences**)
- **Kruskal-Wallis test** → alternative to ANOVA (compares **group medians via ranks**)
- **Spearman rank correlation** → association between ranked variables

These tests compare **ordered data (ranks)** and are less sensitive to outliers, often focusing on median-based comparisons instead of means.

86.7 Chi-Square Test (Categorical Data)

The **chi-square test** analyzes categorical data by comparing **observed counts** to **expected counts**. It tests whether distributions differ from what we would expect by chance.

Example:

Is transportation choice associated with income level?

Hypotheses:

- H_0 : variables are independent
- H_a : variables are related

The chi-square statistic measures how far observed frequencies deviate from expected frequencies.

86.8 Choosing the Right Test

To select a statistical test, Maya asks:

1. Is the data numerical or categorical?
2. How many groups are being compared?
3. Are observations independent or paired?
4. Are parametric assumptions satisfied?
5. Are we comparing means, ranks, or frequencies?

87 Summary of the Chapter

Hypothesis testing helps us decide whether a result is real or just due to chance.

We state hypotheses, choose a significance level (α), compute a test statistic, and use a critical value or p-value to make a decision.

Different statistical tests are used for different types of data.

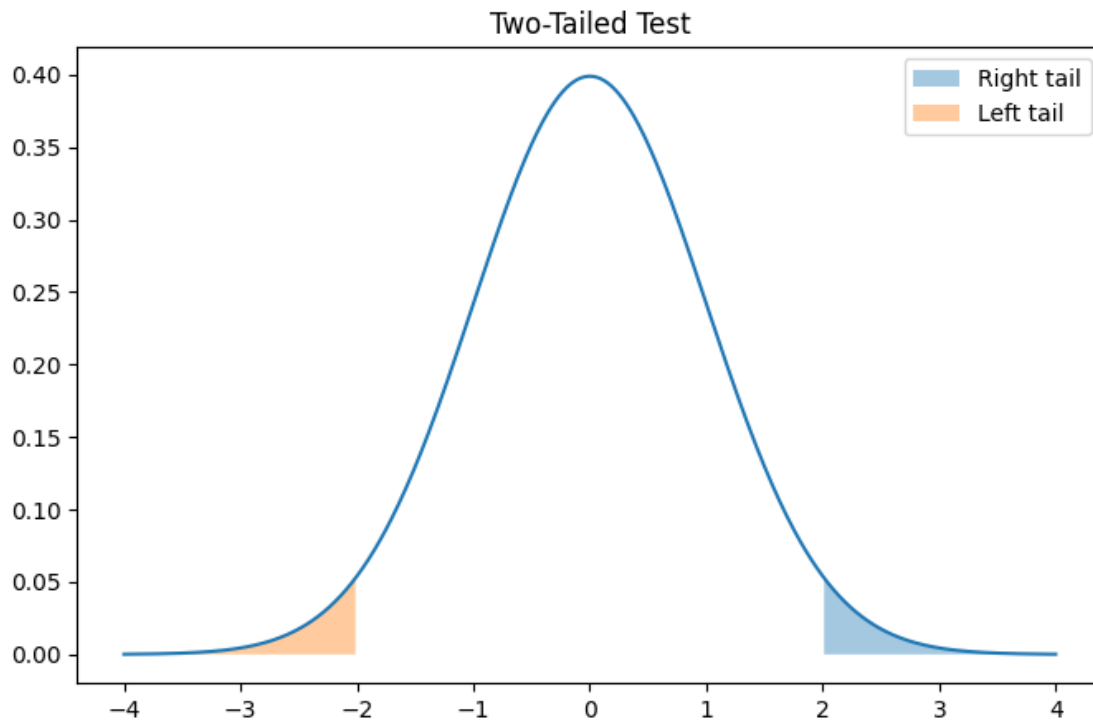
- Parametric tests compare means, while non-parametric tests work when assumptions are not met.

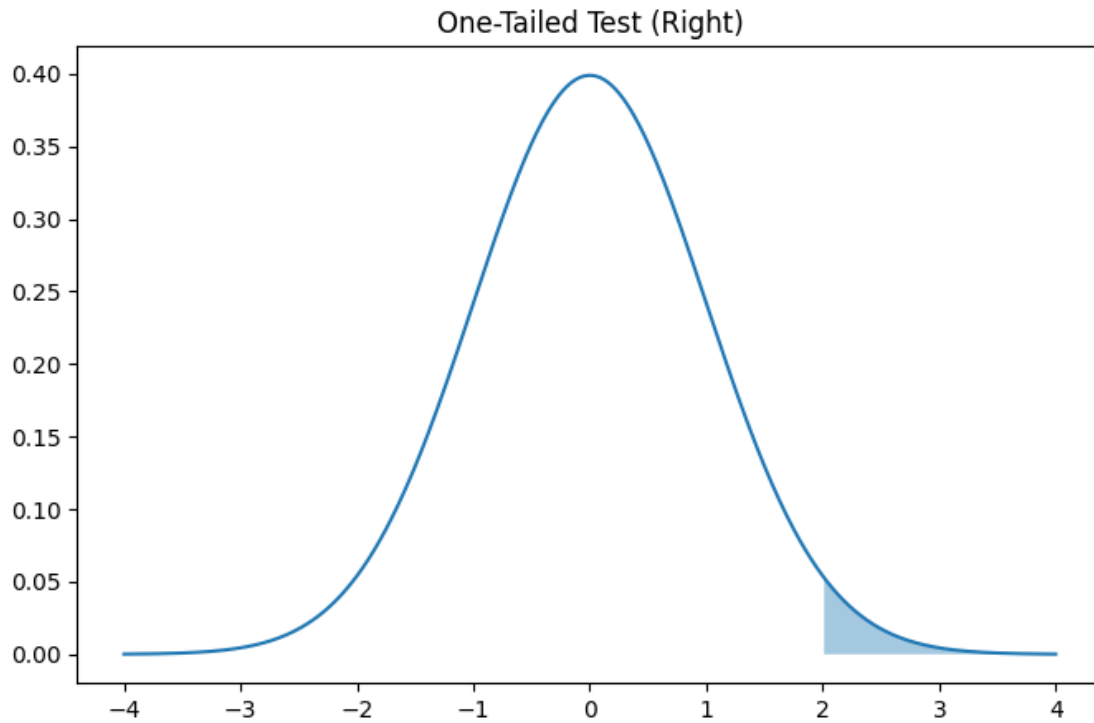
Common tools include t-tests, ANOVA, and chi-square tests.

All aim to answer one question: is the observed difference meaningful?

88 Knowledge Check

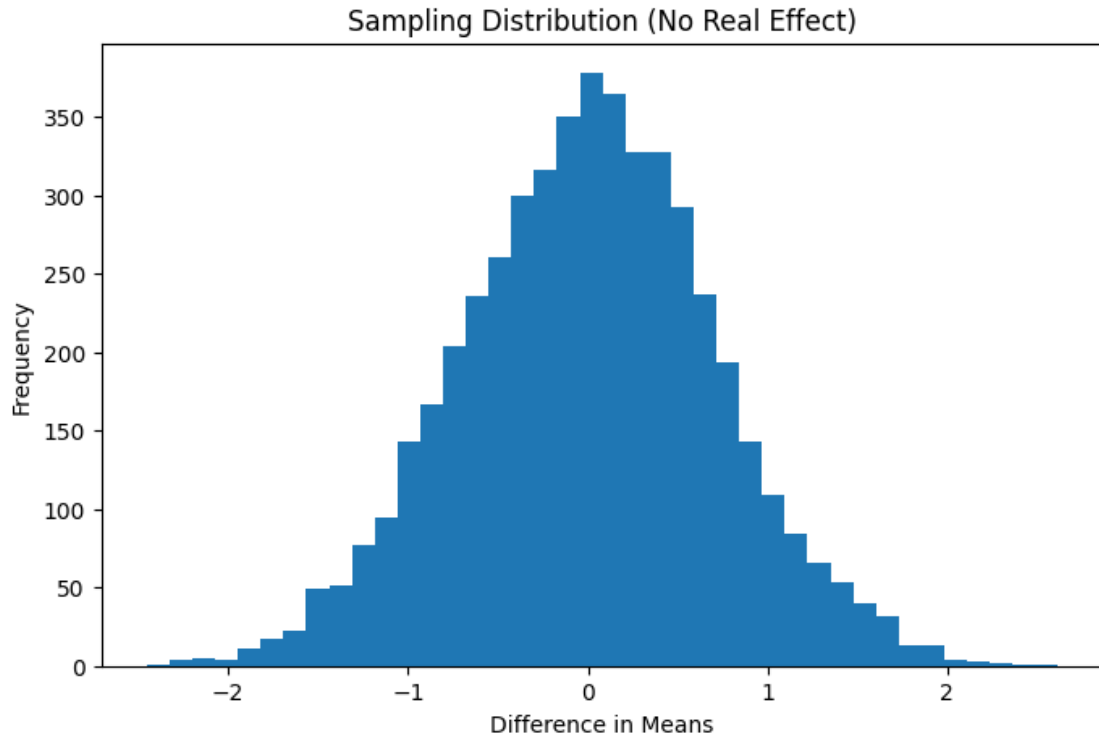
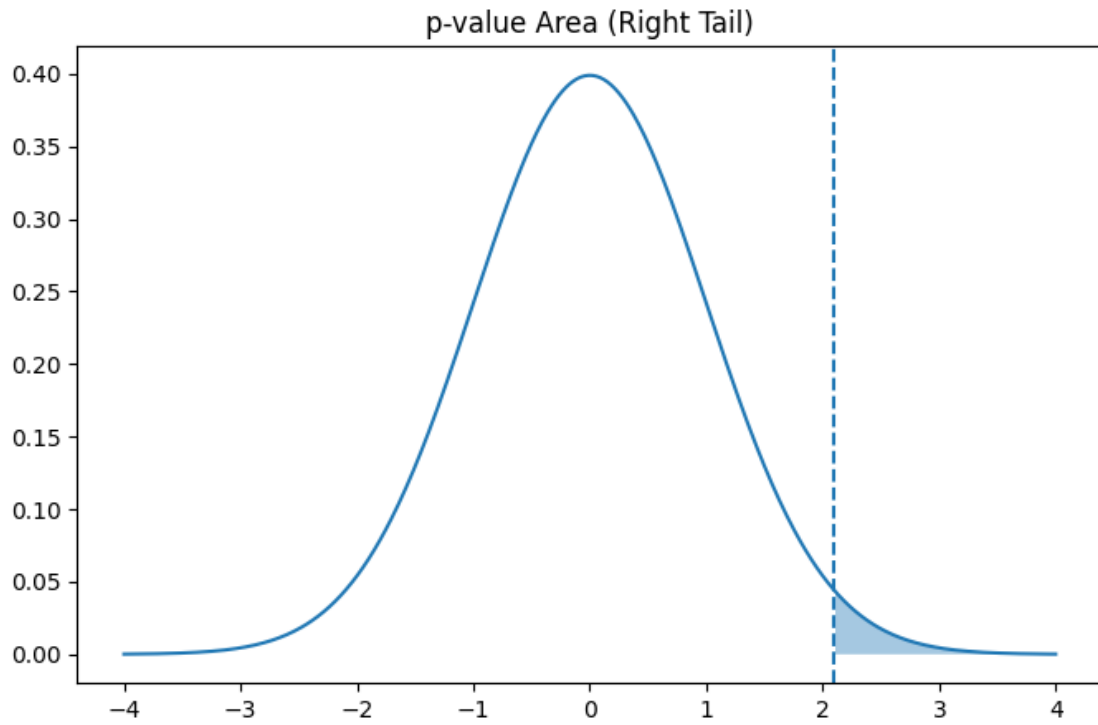
89 BONUS: Python Visualization





Notice: - Two-tailed → split probability across both ends
- One-tailed → all probability in one direction

One-tailed tests are more powerful but must be justified BEFORE analysis.



90 Bonus: Animated Sampling Distribution

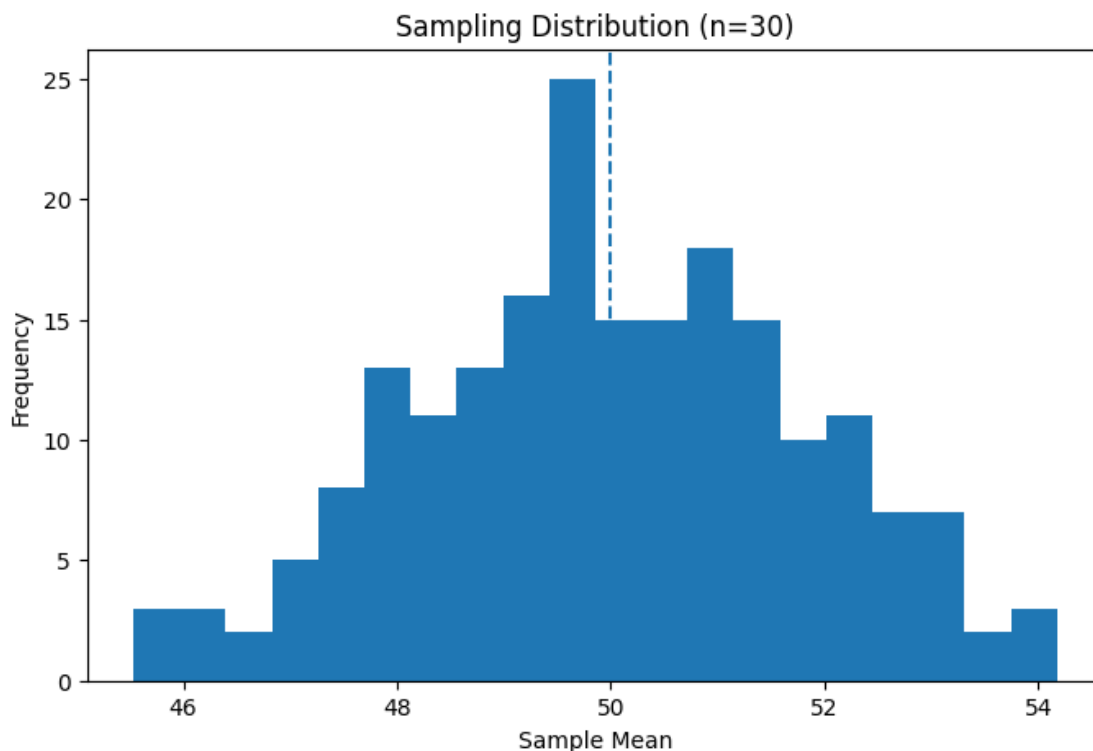
What happens if we repeatedly take samples from a population? Each sample has a different mean. When we collect many sample means, they form a distribution. This animation shows:

1. Draw a sample
2. Compute sample mean
3. Add it to histogram
4. Repeat

Watch how randomness becomes structure.

GIF saved!

<IPython.core.display.Image object>



Notice:

- At first → sample means are scattered
- Over time → they form a bell-shaped curve

This happens even if the population is random.

This is the foundation of hypothesis testing. Remember: - Small samples → wide distribution
- Large samples → narrow distribution

Task: You can compare Sample Sizes Animation

```
sample_size = 5    # try 5, 30, 100
```

91 Chapter 9 - Exploratory Data Analysis

92 The Story of Exploration

Before building predictive models, before testing hypotheses, and before drawing conclusions, a data scientist must first learn to **observe**.

This stage of careful observation is known as **Exploratory Data Analysis (EDA)** ; the process of examining data in its raw form to understand its characteristics, behavior, and limitations before formal modeling begins.

Real-world data rarely arrives clean, structured, or fully explained. Often, we begin with nothing more than rows and columns without context. EDA provides a systematic way to make sense of this uncertainty. It transforms unfamiliar data into interpretable information.

Exploration is not about proving something to be true. It is about becoming familiar with what the data contains and what it might reveal.

92.1 What is Exploratory Data Analysis (EDA)?

When you first open a dataset, all you see is a table of numbers and labels. There is no guide, no explanation; just rows and columns. Exploratory Data Analysis (EDA) is the structured process of making sense of that raw data.

Exploratory Data Analysis (EDA) is the process of understanding data; its structure, quality, patterns, and surprises.

EDA helps us:

- Discover patterns
- Detect anomalies
- Understand structure
- Test assumptions
- Generate hypotheses

Before building models or drawing conclusions, we **must first listen to the data**.

EDA is not a single step; it is an iterative cycle:

- Ask questions
- Explore the data
- Refine the questions
- Repeat

92.2 The EDA Checklist: A Structured Exploration Framework

While EDA encourages open investigation, it should not be random. A practical checklist ensures that exploration remains systematic, thorough, and reproducible. A structured EDA process typically includes:

1. Define the objective
2. Understand data structure and types
3. Assess data quality
4. Explore distributions and variation
5. Examine relationships between variables
6. Transform data when needed
7. Summarize insights and limitations

92.2.1 The EDA Workflow

The EDA workflow illustrates how analysis moves from initial inspection to deeper understanding through

EDA is not random inspection. It is a **structured investigation** that moves from data inspection to insight generation.

[Image omitted in PDF — see the online textbook]

Figure: Overview of the Exploratory Data Analysis process, from inspection and cleaning to visualization and insight.

Understanding data is a process, not a single action. Each step builds on the previous one, gradually refining understanding. The process is iterative rather than strictly linear.

92.3 Defining the Objective and the Key Variable

Every analysis begins with a question. Without a clear objective, exploration becomes unfocused.

Most datasets contain one variable representing the primary outcome of interest, often called the **target variable** or **key variable**. Examples include survival, revenue, disease status, or performance. EDA seeks to understand:

- how this variable behaves
- how other variables relate to it
- what factors may influence it

92.4 Asking the Right Questions

Good analysis begins with good questions.

Two types of questions are especially powerful:

1. **Variation within variables** How does a single variable behave? What is its distribution? Range? Spread?

2. **Co-variation between variables** How do variables move together? Is there a relationship? A dependency?

Often one variable is central, and we examine how others relate to it.

Understanding both variation and co-variation is essential for meaningful analysis.

93 Understanding Data Structure

Before analyzing anything, look at how the data is organized.

In any dataset, **rows represent observations and columns represent attributes** describing those observations. But sometimes the same individual appears multiple times, measured at different points in time. When data tracks changes across repeated measurements, it is called **longitudinal data**.

Consider a dataset tracking patients:

```
import pandas as pd

data = {
    "patient_id": [101, 101, 102, 102, 103],
    "visit_date": ["2024-01-01", "2024-02-01", "2024-01-05", "2024-02-05", "2024-01-10"],
    "age": [55, 55, 62, 62, 48],
    "weight": [180, 178, 165, 164, 150]
}

df = pd.DataFrame(data)
df
```

Notice that, **Patient 101 appears multiple times** because measurements were taken at different visits. Age stays the same, while weight changes over time. This shows that each row is an observation at a specific time point and each column is an attribute. Some variables are constant, others vary across visits.

Recognizing what is fixed and what changes is the foundation of exploratory data analysis.

Once we understand how the data is structured, the next step is to identify what kind of data each variable contains.

94 Understanding Variable Types

Once we understand the structure of a dataset, the next question is:

What kind of data is each variable?

In Python, we can inspect technical data types using:

```
df.dtypes
```

94.0.1 Example Output

```
patient_id    int64
visit_date    object
age           int64
weight        int64
```

However, a programming data type (such as int64 or object) is not the same as a statistical variable type. For analysis, we classify variables conceptually.

Continuous Variables (Quantitative): represent numeric measurements on a scale. Arithmetic operations are meaningful, and values can vary across a range.

Examples: age, weight, blood pressure

Typical summaries: mean, range, standard deviation, histograms, box plots.

Categorical Variables (Qualitative): represent group labels rather than numeric measurements.

Examples: gender, treatment group, cancer stage

Typical summaries: frequency counts, proportions, bar charts

94.0.2 Different Roles of Variables:

In addition to variable type (continuous or categorical), variables can also play different **roles** in a dataset.

Some variables uniquely identify individuals, some describe their starting characteristics, and others record measurements that change over time.

Identifier → uniquely labels an individual

Baseline variable → measured once at study start

Time-varying variable → measured repeatedly over time

Understanding both the *type* and the *role* of a variable helps determine how it should be analyzed.

94.0.3 Example from Our Dataset

Variable	Type	Reason
patient_id	Identifier	Labels a person
visit_date	Time variable	Measurement moment
age	Baseline numeric	Fixed characteristic
weight	Continuous	Changes over time

95 Exploratory Data Analysis (EDA): A Practical Workflow

Once we understand why exploration matters, the next question is practical:

How do we explore a dataset?

Although every dataset is different, analysts follow a consistent workflow that moves from basic inspection to deeper patterns and relationships.

95.1 Step-by-Step EDA Workflow

95.1.1 1. Inspect structure

Examine dataset size, variables, and data types.

95.1.2 2. Check data quality

Identify missing values, duplicates, and inconsistencies.

95.1.3 3. Examine individual variables

Understand distributions and typical values.

95.1.4 4. Explore variation

Assess spread, skewness, and extreme observations.

95.1.5 5. Investigate relationships

Examine how variables interact.

95.1.6 6. Consider time and context

Identify trends, cycles, or temporal patterns.

95.1.7 7. Generate insights

Interpret patterns and refine questions.

95.1.8 8. Prepare for modeling

Ensure data is understood, cleaned, and structured.

95.2 Example: Data Manipulation During EDA (with Python)

Exploration often requires transforming data to improve interpretability.

95.2.1 Step 1: Load Data

Load the dataset into our environment so you can inspect and analyze it.

```
import pandas as pd
df = pd.read_csv("your_dataset.csv")
df.head()
```

Preview confirms successful loading and column structure.

95.2.2 Step 2: Understand Structure

Examine the size, columns, and data types of the dataset.

```
df.shape
df.info()
```

Check: - Number of rows and columns
- Data types of each variable
- Presence of missing values

Understanding structure prevents misinterpretation later.

95.2.3 Step 3: Summary Statistics

Generate numerical summaries to understand the typical values and overall spread of numeric variables. These statistics provide a quick snapshot of central tendency and variability.

```
df.describe()
```

Look for:

- Mean vs median (possible skewness)
- Range (min and max)
- Spread (standard deviation)

This helps identify unusual values or large variation.

95.2.4 Step 4: Missing Values

Identify where data is incomplete. Missing values are common in real-world datasets and must be handled carefully because they can bias analysis or break models.

```
df.isnull().sum()
```

This step helps you decide whether to:

- Remove rows
- Fill values (imputation)
- Investigate why data is missing

Missing data must be handled before modeling.

95.2.5 Step 5: Distribution of Variables

Visualize how values are spread across a variable. Distributions reveal patterns such as symmetry, skewness, clustering, or extreme values.

```
import matplotlib.pyplot as plt
```

```
df["column_name"].hist()  
plt.show()
```

Look for: - Skewed or symmetric shape
- Concentration of values
- Unusual gaps or spikes

Understanding distribution helps choose appropriate statistical methods.

95.2.6 Step 6: Outlier Detection

Detect values that differ substantially from the rest of the data.

Outliers may represent data errors, rare events, or important observations.

```
import seaborn as sns
sns.boxplot(x=df["column_name"])
```

Outlier detection helps determine whether unusual values should be removed, corrected, or studied further.

95.2.7 Step 7: Relationships

Examine how two variables move together. Understanding relationships helps identify patterns, trends, or potential predictive connections.

```
sns.scatterplot(data=df, x="feature1", y="feature2")
```

Look for:

- Positive or negative trends
- Clusters or groups
- Nonlinear patterns

This step is essential for modeling and hypothesis generation.

95.2.8 Step 8: Correlation

Measure the strength of linear relationships between numeric variables. Correlation helps identify which variables move together and which are independent.

```
df.corr(numeric_only=True)
```

Strong correlations may indicate:

- Predictive relationships
- Redundant variables
- Multicollinearity risks in models

EDA Goal Understand structure, quality, distribution, and relationships BEFORE modeling.

Now that we understand the overall EDA workflow, let's look more closely at one of its core components: summarizing individual variables.

96 Summarizing Variables: Categorical vs Numerical

EDA becomes much easier when we combine *interpretation* with *quick, repeatable code*. In this section, we will summarize two kinds of variables:

- **Categorical variables** (group labels)
- **Numerical variables** (measured quantities)

Throughout, assume your dataset is in a DataFrame called `df`.

96.0.1 A quick setup

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

96.1 Summarizing Categorical Variables

Categorical variables describe membership in groups (e.g., sex, class, department). A good first question is: **Which categories are most common?**

96.1.1 Frequency table (counts)

```
col = "your_categorical_column"
df[col].value_counts(dropna=False)
```

96.1.2 Relative frequency (proportions)

```
df[col].value_counts(normalize=True, dropna=False)
```

96.1.3 Bar chart

```
col = "your_categorical_column"

counts = df[col].value_counts(dropna=False)
counts.plot(kind="bar")
plt.xlabel(col)
plt.ylabel("Count")
plt.title(f"Counts of {col}")
plt.show()
```

Interpretation tip:

A dominant category may indicate imbalance (important for modeling and fairness).

96.2 Summarizing Numerical Variables

Numerical variables represent quantities (e.g., age, income, fare).

A good first question is: **What is typical, and how much do values vary?**

96.2.1 Summary statistics

```
col = "your_numeric_column"
df[col].describe()
```

96.2.2 Quick quantiles (often more informative than the mean)

```
df[col].quantile([0.0, 0.25, 0.5, 0.75, 1.0])
```

Interpretation tip:

If the mean and median are far apart, the distribution may be skewed.

96.3 Visualizing Distributions and Detecting Outliers

Numbers summarize. Visuals reveal *shape*.

96.3.1 Histogram (distribution shape)

Histograms reveal the overall shape of the data; something summary statistics alone cannot show.

```
col = "your_numeric_column"

df[col].hist(bins=30)
plt.xlabel(col)
plt.ylabel("Frequency")
plt.title(f"Histogram of {col}")
plt.show()
```

What to look for: - Skewness (long tail on one side) - Multiple peaks (possible subgroups) - Unexpected spikes (data entry issues)

96.3.2 Box plot (outliers)

```
col = "your_numeric_column"

sns.boxplot(x=df[col])
plt.title(f"Boxplot of {col}")
plt.show()
```

If a distribution is strongly skewed, consider **log transformation** before modeling.

Reminder:

Outliers are not always “bad.” They may be errors, or they may be rare but meaningful cases.

96.4 A useful helper: identify categorical vs numerical columns

When datasets are large, it helps to quickly separate column types.

```
numeric_cols = df.select_dtypes(include="number").columns.tolist()
categorical_cols = df.select_dtypes(exclude="number").columns.tolist()

numeric_cols, categorical_cols
```

96.5 BONUS: Quick EDA summary loop (fast and practical)

This prints a short summary for each column and makes simple plots.

```
for col in df.columns:
    print("\n" + "="*60)
    print("Column:", col)
    print("Type:", df[col].dtype)
    print("Missing:", df[col].isna().sum())

    if pd.api.types.is_numeric_dtype(df[col]):
        print(df[col].describe())
        df[col].hist(bins=30)
        plt.title(f"Histogram of {col}")
        plt.show()

        sns.boxplot(x=df[col])
        plt.title(f"Boxplot of {col}")
        plt.show()
    else:
        print(df[col].value_counts(dropna=False).head(10))
        df[col].value_counts(dropna=False).head(10).plot(kind="bar")
        plt.title(f"Top categories of {col}")
        plt.show()
```

96.5.1 Mini “Try It” Questions (for students)

1. Pick one categorical column. Which category is most common? Is it heavily imbalanced?
2. Pick one numeric column. Is it skewed? How can you tell from the histogram?
3. Does the boxplot show outliers? What might explain them?

96.6 Exploring Relationships Between Variables

So far, we have examined variables individually; their distributions, typical values, and unusual observations. The next step in exploratory analysis is to understand **how variables interact**.

Relationships between variables reveal structure in the data. They help us identify patterns, associations, and possible predictive signals.

However, relationships must be interpreted carefully. Association does not imply causation.

96.6.1 Visualizing Relationships

The most common way to examine relationships between two numerical variables is a **scatter plot**. Each point represents an observation, positioned according to its values on two variables.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.scatterplot(data=df, x="feature1", y="feature2")
plt.title("Relationship between feature1 and feature2")
plt.show()
```

What to look for:

- upward trend → positive association
- downward trend → negative association
- clusters → subgroups
- curved pattern → nonlinear relationship
- no pattern → weak or no relationship

96.6.2 Comparing Groups

When one variable is categorical and the other numeric, we compare distributions across groups.

```
sns.boxplot(data=df, x="category", y="numeric_value")
plt.title("Distribution across categories")
plt.show()
```

This reveals whether different groups behave differently.

96.6.3 Measuring Correlation

Correlation quantifies the strength and direction of a linear relationship between numeric variables.

```
df[["feature1", "feature2"]].corr()
```

Correlation values range from:

- $+1$ → perfect positive relationship
- 0 → no linear relationship
- -1 → perfect negative relationship

Remember: correlation measures association, not cause.

96.7 Multivariate Exploration

Real-world phenomena rarely depend on only two variables. Multivariate exploration examines **multiple variables simultaneously**.

This helps reveal interactions, hidden groupings, and complex structure that pairwise analysis may miss.

96.7.1 Pairwise Relationships

A **pairplot** visualizes relationships among many numeric variables at once.

```
sns.pairplot(df.select_dtypes(include="number"))
```

This shows:

- distributions on the diagonal
- pairwise scatter plots elsewhere

Patterns across multiple variables become visible.

96.7.2 Correlation Heatmap

A heatmap provides a compact visual summary of correlations across many variables.

```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

This helps identify:

- strongly related variables
- redundant features
- potential predictors

96.7.3 Why Multivariate Exploration Matters

Many patterns only appear when several variables are considered together.

Multivariate analysis helps uncover interactions and structure that single-variable or pairwise analysis cannot reveal.

97 Handling Missing Data and Data Quality Issues

Exploratory analysis is not only about discovering patterns. It is also about identifying problems. Real-world data is rarely perfect. Missing values, duplicates, inconsistent formats, and unexpected values are common. If these issues are ignored, they can distort conclusions and weaken models.

Before modeling, we must understand the quality of our data.

97.1 Identifying Missing Values

The first step is to measure how much data is missing.

```
df.isna().sum()
```

This shows the number of missing values in each column.

To view proportions:

```
df.isna().mean()
```

High missingness may require:

- Removing rows
- Removing columns
- Imputation (Later Topic)
- Investigating the source

Missing data is not always random. Understanding why values are missing is often as important as the missingness itself.

97.2 Handling Missing Values

Common approaches include:

1. Dropping rows

```
df.dropna()
```

2. Filling with summary statistics

```
df["col"].fillna(df["col"].mean(), inplace=True)
```

3. Filling categorical values

```
df["category"].fillna("Unknown", inplace=True)
```

Choice depends on:

- proportion missing
- importance of variable
- modeling goals

97.3 Detecting Duplicates

```
df.duplicated().sum()
```

Remove duplicates if necessary:

```
df = df.drop_duplicates()
```

Duplicates can inflate sample size artificially.

97.4 Checking Data Types and Convert if necessary

Sometimes numeric data is stored as text. Convert if necessary:

```
df["col"] = pd.to_numeric(df["col"], errors="coerce")
```

Sanity Checks: Look for impossible values: - Negative ages - Future dates - Extreme outliers

EDA includes validating whether values make logical sense.

97.5 Data Transformation During EDA

Sometimes variables need reshaping to reveal patterns.

```
#Log Transformation (for skewed data)
import numpy as np
df["log_income"] = np.log(df["income"])
```

Creating New Features

```
df["bmi"] = df["weight"] / (df["height"]**2)
```

Converting Dates

```
df["date"] = pd.to_datetime(df["date"])
```

```
df["year"] = df["date"].dt.year
```

Feature engineering (later topic) often begins during exploration.

Why Data Quality Matters: Models assume structure in the data. If the data is flawed, the model will learn flawed patterns.

Exploratory Data Analysis is not only about discovering insight, it is about building trust in the data.

98 Advanced EDA Techniques

Once basic exploration is complete, analysts often use more advanced visualization tools to reveal complex patterns quickly.

98.1 Multivariate Visualization Tools

98.1.1 Correlation Heatmap

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
```

```
plt.show()
```

98.1.2 Pairplot

```
sns.pairplot(df.select_dtypes(include="number"))
```

98.1.3 Missingness Map

```
!pip install missingno
```

```
import missingno as msno
```

```
msno.matrix(df)
```

Think Like a Data Scientist Always ask WHY a pattern exists.

Common Mistake Correlation Causation.

Visualization Rule If you cannot explain the plot clearly, you do not understand the data yet.

99 Quick EDA Template

```
df.head()
```

```
df.info()
```

```

df.describe()
df.isnull().sum()

df.hist(figsize=(10,8))

sns.boxplot(data=df)

sns.pairplot(df.select_dtypes(include="number"))

sns.heatmap(df.corr(numeric_only=True), annot=True)

```

99.1 Special Case: Exploring Event Patterns in Temporal Data

When datasets contain time information, additional patterns may appear, trends, clustering, and seasonality. Temporal data often records events along a continuous time axis, requiring specialized visualization.

99.2 Exploring Event Patterns in Temporal Data

Temporal data often records *events* (logins, purchases, symptoms, alarms) along a continuous variable such as time. Beyond basic distributions, three practical visual checks help reveal **frequency**, **clustering**, and **seasonality**.

99.2.1 1) Event Frequency Over Time (Counts)

This view answers: *When do events happen more often?*
It highlights peaks, dips, and trends by counting events per date.

```

import pandas as pd
import matplotlib.pyplot as plt

df["date"] = pd.to_datetime(df["date"])

# Count events per date and plot in time order
df["date"].value_counts().sort_index().plot()
plt.xlabel("Time")
plt.ylabel("Event Count")
plt.title("Events Over Time")
plt.show()

```

99.2.2 2) Event Timeline (Scatter Plot)

Sometimes you want to see exact timing rather than aggregated counts. A simple trick is to plot every event on the same horizontal line.

```

# If you have numeric time (e.g., days since start)
df["zeroes"] = 0
df.plot.scatter("days_since_start", "zeroes", alpha=0.4)
plt.xlabel("Time (days since start)")

```



```
plt.title("Event Occurrence Timeline")
plt.show()
```

Tip: Lower alpha helps when many events overlap.

99.2.3 3) Seasonal Pattern Check (Cyclic Time)

To check for yearly seasonality, map time into a repeating cycle (0–364) and look for repeating structure.

```
# Map time into a yearly cycle (0-364)
df["day_of_year"] = df["days_since_start"] % 365

# Scatter: if patterns repeat at similar day-of-year values, seasonality is likely
df.plot.scatter("day_of_year", "x", alpha=0.5)
plt.xlabel("Day of Year")
plt.title("Seasonal Pattern Check")
plt.show()
```

If you want to color by day-of-year (optional):

```
df["color"] = df["day_of_year"].apply(lambda d: (d/365, 0, 0))
df.plot.scatter("day_of_year", "x", c=df["color"], alpha=0.6)
plt.xlabel("Day of Year")
plt.title("Seasonal Pattern Check (Colored)")
plt.show()
```

Remember: - Counts over time reveal trends and bursts. - Timeline scatter reveals clustering and gaps. - Cyclic plots reveal seasonality (weekly/monthly/yearly patterns).

100 Case Study: Exploratory Data Analysis of the Titanic Dataset

To see how these ideas come together, we now explore a real dataset. [The Titanic dataset](#) contains information about passengers and whether they survived the disaster.

100.1 Load Dataset

```
import seaborn as sns
df = sns.load_dataset("titanic")
df.head()
```

100.2 Structure

```
df.shape
df.info()
```

Key variables include:

- survival status

- passenger class
- age
- fare
- gender

100.3 Missing Values

```
df.isnull().sum()
```

100.4 Survival Count

```
sns.countplot(data=df, x="survived")
```

100.5 Survival by Gender

```
sns.countplot(data=df, x="sex", hue="survived")
```

Observation: Survival differs strongly between males and females.

100.5.1 Survival by Passenger Class

```
sns.countplot(data=df, x="class", hue="survived")
plt.title("Survival by Class")
plt.show()
```

Observation: Higher-class passengers appear more likely to survive.

100.6 Age Distribution

```
df["age"].hist(bins=30)
```

Observation: Ages vary widely with some missing values.

100.7 Fare vs Survival

```
sns.boxplot(data=df, x="survived", y="fare")
```

Observation: Survivors tend to have paid higher fares.

100.7.1 Multivariate View

```
sns.pairplot(df[["age", "fare", "survived"]].dropna(), hue="survived")
```

This allows simultaneous comparison of multiple relationships.

100.8 What We Learned from the Case Study

From simple visual exploration, we discovered:

- survival differs by gender
- survival differs by passenger class
- fare relates to survival
- age varies widely

These patterns suggest hypotheses, but they do not prove causation.

Further statistical analysis would be required to confirm them.

Interpretation EDA reveals patterns but does NOT prove causation.

101 Mini Exercises: Try yourself:

1. Which passenger class had highest survival rate?
2. Did higher fare increase survival?
3. Plot age distribution by survival.
4. Check correlation between fare and age.
5. Identify one unexpected pattern.

102 BONUS: Visual Cheat Sheet

Goal	Visualization
Distribution	Histogram
Outliers	Boxplot
Counts	Bar chart
Relationship	Scatter plot
Correlation	Heatmap
Multivariable	Pairplot

103 Summary of the Chapter

Exploratory Data Analysis (EDA) is the first step in understanding any dataset. Before modeling or hypothesis testing, analysts examine data structure, quality, distributions, and relationships to transform raw data into meaningful insight.

In this chapter, we learned how to understand data structure and variable types, summarize categorical and numerical variables, visualize distributions and detect outliers, explore relationships, assess data quality, and apply EDA to real data.

EDA is systematic and iterative; each discovery leads to new questions and deeper understanding.

104 Student EDA Checklist

Use this checklist whenever you explore a new dataset.

104.0.1 Dataset Understanding

What does each variable mean?
Units?
Numeric or categorical?

104.0.2 Data Quality

Missing values
Duplicates
Impossible values

104.0.3 Distribution

Histograms
Outliers

104.0.4 Relationships

Scatter plots
Correlations

104.0.5 Insight

Surprising findings
Questions for modeling

104.1 Knowledge Check

[Interactive content omitted in PDF]

105 BONUS: EDA Example for Common Tasks

This bonus example demonstrates several common operations performed during Exploratory Data Analysis (EDA). We begin by inspecting the dataset using `df.dtypes`, `count()`, and `isna().sum()` to understand data types and identify missing values. We also loop through columns and use `pd.isna()` for row-level checks.

Next, we convert and format dates using `pd.to_datetime()` and `.strftime()`. We clean and analyze text with string methods such as `.str.contains()`, `.split()`, `.replace()`, and `.capitalize()`, and detect numeric strings using `.isdigit()`.

We further demonstrate how to access specific rows with `.iloc[]`, check whether data is empty using `.empty`, and combine datasets with `pd.merge()`.

Together, these operations illustrate essential inspection, cleaning, and transformation steps that prepare data for meaningful analysis and modeling.

`dtypes:`

<code>name</code>	<code>object</code>
<code>age</code>	<code>float64</code>
<code>city</code>	<code>object</code>

```
date_str      object
comment       object
dtype: object
```

```
count (non-missing):
  name      2
  age       2
  city      3
  date_str  2
  comment   3
dtype: int64
```

```
missing per column:
  name      1
  age       1
  city      0
  date_str  1
  comment   0
dtype: int64
```

```
name      | dtype=object | missing=1
age       | dtype=float64 | missing=1
city      | dtype=object | missing=0
date_str  | dtype=object | missing=1
comment   | dtype=object | missing=0
Row 1: age is missing
```

Formatted dates:

	date_str	date_fmt
0	03/05/2026	2026-03-05
1	03/06/2026	2026-03-06
2	None	NaN

```
age.empty: False
```

Rows where city contains 'New':

	name	age	city	date_str	comment	date	date_fmt
0	alice	22.0	New York	03/05/2026	good job 100	2026-03-05	2026-03-05
2	None	30.0	New York	None	ok 7	NaT	NaN

Tokenized comments:

	comment	words
0	good job 100	[good, job, 100]
1	needs improvement	[needs, improvement]
2	ok 7	[ok, 7]

Example tokens: ['good', 'job', '100', 'extra']

Index of 'job': 1

```
Digit-only tokens: ['100']
```

```
Clean names:
```

```
      name name_clean
0  alice      Alice
1   bob       Bob
2   None    Unknown
```

```
Merged:
```

```
      city state
0  New York   NY
1   Boston   MA
2  New York   NY
```

```
First row (iloc[0]):
```

```
name                alice
age                 22.0
city               New York
date_str            03/05/2026
comment            good job 100
date               2026-03-05 00:00:00
date_fmt            2026-03-05
words              [good, job, 100, extra]
name_clean          Alice
state              NY
```

```
Name: 0, dtype: object
```

106 Chapter 10 - Intro to ML

107 Learning From Experience: The Story of Machine Learning

Imagine you were tasked with building a perfect **Spam Filter** in the 1990s. As a programmer, you would write thousands of manual rules:

IF email contains “FREE MONEY” *OR* “CLAIM PRIZE” *THEN* classify as SPAM.

This works for a day, but then spammers change “FREE MONEY” to “FR€€ M0N€Y.” You add new rules, but the spammers always win. The problem is **too complex** and **changes too fast** for fixed, explicit rules.

The Machine Learning Solution: *Machine Learning (ML) solves this by abandoning fixed rules. Instead of telling the computer how* to classify spam, we show it a mountain of data—millions of emails already labeled as “Spam” or “Not Spam.”*

The ML algorithm then **learns the hidden patterns and correlations** on its own: not just individual words, but the *combination* of words, the structure of the text, and even the sender’s metadata.

This is the core idea:

Instead of programming explicit rules, we let the computer learn patterns from data.

107.1 The Human Analogy

Before we define anything formally, let's think like humans.

How do we learn?

A child sees many dogs. Some are big. Some are small. Some bark loudly. Over time, the child builds an internal concept:

“If it has four legs, fur, and barks; it's probably a dog.”

No one writes down a perfect rule. The child learns from examples.

Machine Learning works the same way. Instead of explicitly writing rules like:

IF a fruit is red, round, and grows on a tree → apple

We show the computer thousands of fruit images labeled apple, orange, banana, and so on. Over time, it learns the patterns that distinguish one fruit from another — on its own.

“Learning from examples” - that is the core idea.

107.2 Formal Definition: What is Machine Learning?

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that focuses on developing systems that can learn from data and improve their performance on a specific task over time, **without being explicitly programmed** for that task.

At its core, Machine Learning shifts the paradigm of computing:

Instead of writing explicit rules, we allow computers to discover patterns from data and use those patterns to make predictions or decisions. A widely accepted formal definition is:

A machine learns from **experience (E)** with respect to some **task (T)**, and its **performance (P)** in the task improves.

This definition highlights three essential components:

- **Task (T):** What the system is trying to do (e.g., classify images, predict prices, detect fraud)
- **Experience (E):** The data the system learns from
- **Performance (P):** How well the system performs the task

Learning occurs when performance improves as experience increases.

Think of it this way: instead of a traditional programmer writing a specific rule like “*IF the email contains ‘prize’ and ‘claim now’ THEN classify as spam,*” an ML algorithm is fed thousands of examples of spam and not-spam emails. It then learns the hidden patterns and rules on its own to make the classification.

The core idea can be summarized as:

Experience (E) → Task (T) → Performance (P)

[Image omitted in PDF — see the online textbook]

107.2.1 Why Machine Learning?

In traditional programming, we give the computer **explicit rules** to follow. However, some problems are too complex to define with simple rules; for example, identifying a cat in an image or predicting stock prices. Machine Learning (ML) helps solve such problems by **learning patterns from data** rather than following fixed instructions.

107.2.2 Traditional Programming vs Machine Learning

Traditional Programming	Machine Learning
Rules + Data $\rightarrow$ Output	Data + Output $\rightarrow$ Rules (Model)

Example: Instead of writing rules to detect spam manually, we show the computer many examples of **spam** and **not spam** emails, and it learns the pattern itself.

107.2.3 Real-World Applications

- Recommendation Systems: Netflix suggestions, Amazon ‘Customers Also Bought...’
- Natural Language Processing (NLP): Translation (Google Translate), Voice Assistants (Siri, Alexa).
- Computer Vision: Facial recognition, self-driving cars.
- Prediction: Stock market forecasting, predicting equipment failure.
- Banking $\rightarrow$ fraud detection

107.3 Key Terminology You Must Know

Before we look at the process, let’s establish the fundamental vocabulary.

Term	Analogy	Technical Description
Feature (or Attribute)	The ingredients of a recipe.	An input variable used to make a prediction. <i>Example: Square footage, number of bedrooms, and location for a house.</i>
Label (or Target)	The final dish price.	The output variable that we are trying to predict or classify. <i>Example: The actual selling price of the house or whether a photo contains a dog.</i>
Model	The trained chef.	The algorithm (the mathematical function) that has been trained on the data to find the relationship between the features and the label.

Term	Analogy	Technical Description
Training	Practicing the recipe.	The process where the algorithm learns the optimal parameters by being fed the training data (Features + Labels).
Prediction (or Inference)	Guessing the cost of a new recipe.	The output generated by the trained model when given new, unseen data (Features only).

107.4 How Machines Learn

107.4.1 The Core Idea

ML algorithms use **data** to find **patterns**, and then use those patterns to make **predictions**.

For example: | Input (Feature) | Output (Target) | | Hours studied = 1 | Score = 50 | | Hours studied = 2 | Score = 60 | | Hours studied = 3 | Score = 70 |

From this, the model learns that *more hours studied* $\rightarrow$ *higher score*, and can predict the score for 4 hours studied.

107.4.2 For this problem, the ML Process Flow will be

1. **Collect Data** (Gathering the Hours Studied vs. Score pairs).
2. **Prepare Data** (Cleaning errors, normalizing features).
3. **Train Model** (The algorithm learns the $\text{Score} \approx 10 \times \text{Hours}$ pattern).
4. **Evaluate Model** (Testing the model's predictions on new, unseen data).
5. **Deploy** (Using the model to predict a score for the next student)

[Image omitted in PDF — see the online textbook]

We will discuss about this more later.

108 Types of Machine Learning

ML problems are categorized based on the data they use and the goal they pursue. The Three Main Types of Machine Learning are - | Type | Data Type | Goal | | Supervised | Labeled | Predict output | | Unsupervised | Unlabeled | Discover structure | | Reinforcement | Sequential | Maximize reward |

[Image omitted in PDF — see the online textbook]

Figure: Overview of machine learning paradigms and representative tasks. Adapted from Frontiers in Pharmacology (2021).

108.0.1 1. Supervised Learning

In supervised learning, the algorithm learns from **labeled data**, meaning we already know the correct answers.

The machine is “**supervised**” because all the training data is **labeled**—it includes the correct answers.

- **Goal:** Learn a mapping from Features (**X**) to Labels (**y**).
- **Data:** Input data **with** corresponding correct output labels.
- **Tasks: Examples:**
 - **Classification:** Predicting a **category** (discrete value).
 - * *Examples:* Spam/Not Spam, Cat/Dog, Good Loan/Bad Loan.
 - **Regression:** Predicting a **continuous value** (a number).
 - * *Examples:* House Price, Temperature, Stock Price.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

predictions = clf.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy: {accuracy:.2f}")
```

108.0.2 2. Unsupervised Learning

Unsupervised learning deals with **unlabeled data**, where the goal is to find hidden patterns or groupings.

The machine is “**unsupervised**” because the data is **unlabeled**—it has no correct answers.

- **Goal:** Discover hidden patterns, structures, and relationships within the data on its own.
- **Data:** Input data **without** corresponding output labels.
- **Tasks:**
 - **Clustering:** Grouping similar data points together.
 - * *Example:* Segmenting customers into different marketing groups (e.g., ‘High Spenders’ vs. ‘Occasional Buyers’).
 - * *Example:* Grouping customers by purchasing habits
 - **Dimensionality Reduction:** Simplifying data by reducing the number of features.

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
```

```
X = load_iris().data
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
print("Cluster Centers:\n", kmeans.cluster_centers_)
```

108.0.3 3. Reinforcement Learning

Reinforcement Learning is about learning through **rewards and penalties** — the model learns from its own actions to maximize total reward.

This type of ML involves an **Agent** interacting with an **Environment**.

- Key Concepts:**
- **Agent:** The learner or decision-maker (e.g., a robot or game AI).
 - **Environment:** Everything the agent interacts with (e.g., game board, robot arena).
 - **Action:** Any move the agent can take.
 - **Reward:** Feedback received after an action (positive or negative).
 - **Goal:** Learn an optimal sequence of actions to maximize total reward.

Mechanism: The agent performs an action, receives a **reward** for a good action or a **penalty** for a bad one, and adjusts its behavior (policy) accordingly to improve future decisions. Repeat until the agent achieves an optimal strategy.

Examples of Reinforcement Learning:

- **Game AI:** Training AI to play complex games like Chess, Go, or Atari.

- **Robotics:** Controlling autonomous robots.
- **Robot Learning:** Teaching a robot to walk or balance.
- **Continuous Learning:** A game-playing AI improving over time as it gains experience.

```
import random
# The agent learns which action is best by repeatedly trying actions
# and adjusting its behavior based on rewards.

# initial preference for two actions
value_A = 0
value_B = 0

learning_rate = 0.1

for step in range(20):

    # choose action (random at first, later prefers higher value)
    if value_A > value_B:
        action = "A"
    else:
        action = random.choice(["A", "B"])

    # environment gives reward
    if action == "A":
        reward = 1
```

```

else:
    reward = 0

# update estimated value
if action == "A":
    value_A = value_A + learning_rate * (reward - value_A)
else:
    value_B = value_B + learning_rate * (reward - value_B)

print(f"Step {step+1}: chose {action}, reward={reward}")

print("\nLearned values:")
print("A =", round(value_A, 2))
print("B =", round(value_B, 2))

```

108.1 The Machine Learning Workflow (Focused on Supervised)

This is the standard process for building a predictive model.

108.1.1 Steps in the Workflow

1. **Collect Data:** Gather a relevant dataset (Features and Labels) from sensors, databases, or APIs.
2. **Data Cleaning:** Handle missing values, duplicates, and outliers.
3. **Feature Engineering:** Select or create the most important variables.
4. **Split Data:** Separate the data into two parts:
 - **Training Set:** Used to teach the model. (~70-80% of data)
 - **Test Set:** Used to evaluate the model on unseen data. (~20-30% of data)
5. **Choose Model:** Select an appropriate algorithm (e.g., Linear Regression, Decision Tree, Neural Network).
6. **Train Model:** Feed the **Training Set** to the algorithm using the `.fit()` method. The model learns the pattern.
7. **Evaluate Model:** Use the trained model to make predictions on the **Test Set** and calculate its **accuracy** or error (the performance P).
8. **Deploy:** Once the model is satisfactory, integrate it into a real-world application or production system to make live predictions.

[Image omitted in PDF — see the online textbook]

Figure: Machine Learning Workflow.

108.2 Your First Algorithm: k-Nearest Neighbors (KNN)

To kick off your journey, we'll start with the **k-Nearest Neighbors (KNN)** algorithm. It's often called a **"lazy learner"** because it doesn't really "train" a model in the traditional sense; it just memorizes the training data.

108.2.1 How a Standard KNN Works: The Social Rule

Imagine you want to classify a new student (the new data point) into one of several social groups (the classes) based on their hobbies (the features).

1. **k is the “Number of Friends”:** You first choose a value for k (a common choice is $k = 3$ or $k = 5$). This is the number of neighbors the algorithm will look at.
2. **Find the Nearest Neighbors:** When a new data point comes in, the algorithm calculates the **distance** (usually Euclidean distance, like measuring a straight line) between the new point and **every** point in the training data.
3. **Vote:** It identifies the k points closest to the new point.
4. **Classify:** The new point is assigned the **class (label)** that is most common among its k nearest neighbors. It’s a majority vote!

[Image omitted in PDF — see the online textbook]

Concept	KNN Analogy	Description
New Sample	The new student.	The data point to be classified.
k Value	Number of closest friends to ask.	The number of nearest data points the algorithm considers.
Distance Metric	How similar their hobbies are.	The mathematical function used to measure the similarity (distance) between points.
Prediction	The most popular social group among those k friends.	The result of the majority vote.

108.3 Steps of standard KNN Algorithm:

- Step-1: Select the number K of the neighbors
- Step-2: Calculate the Euclidean distance of K number of neighbors
- Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.
- Step-4: Among these k neighbors, count the number of the data points in each category.
- Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.
- Step-6: Our model is ready.

[Image omitted in PDF — see the online textbook]

Key takeaway: KNN is simple, powerful, and easy to interpret, but choosing the right k and calculating all those distances can make it slow with very large datasets.

108.3.1 Next Steps:

In the code cell below, we will perform the full **5-step workflow** using a simple **Classification** algorithm to predict the species of a flower based on its measurements!

```
--- STEP 1: Loading Data ---
```

```
First 5 data samples:
```

```
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                    5.1                3.5                1.4                0.2
```

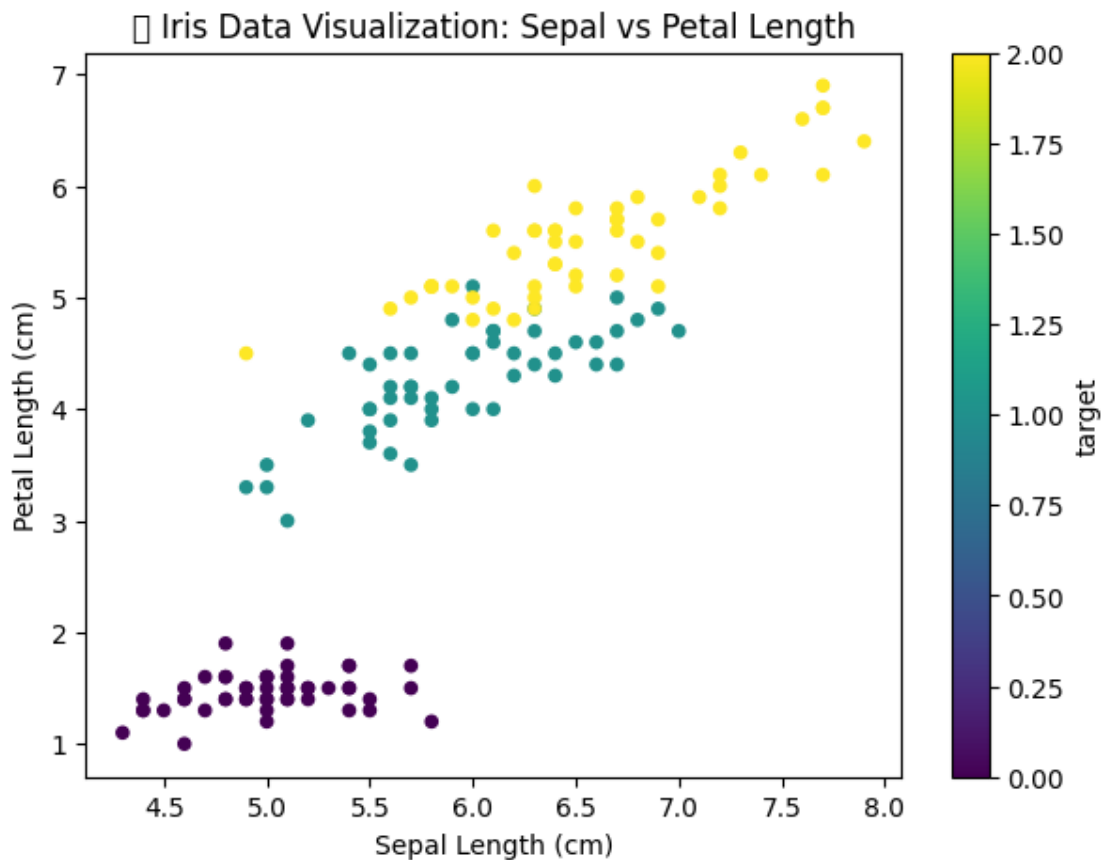
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

	target
0	0
1	0
2	0
3	0
4	0

```

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151:
UserWarning: Glyph 127800 (\N{CHERRY BLOSSOM}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)

```



Features (X) shape: (150, 4)

Label (y) shape: (150,)

Training Samples (for E - Experience): 105

```

Testing Samples (for P - Performance check): 45
-----
Training the K-Nearest Neighbors model on the Training Set...
Training complete! Model has learned patterns.
-----
First 5 Actual Labels (y_test):    [1 0 2 1 1]
First 5 Model Predictions (y_pred): [1 0 2 1 1]
-----
Overall Model Accuracy on Test Data (Performance P): 1.00

Training Decision Tree Model...
Decision Tree Model Accuracy: 1.00
-----
Model Comparison:
KNN (k=5) Performance: 1.00
Decision Tree Performance: 1.00

```

109 Evaluation, Boundaries, and Generalization (Some more ML Concepts)

Machine learning models do not just make predictions, they **draw boundaries** in data. Understanding these boundaries helps us understand why models succeed, and why they fail.

109.1 Understanding the Decision Boundary

The **Decision Boundary** is the abstract line (in 2D), curve, or surface (in higher dimensions) that separates different classes in the feature space.

The **feature space** is the geometric representation of data using input variables (features). Each data point becomes a point in this space.

The decision boundary represents the region where the model switches its prediction from one class to another

[Image omitted in PDF — see the online textbook]

Figure: Example of a decision boundary separating two classes in feature space.

Wherever the model's prediction changes, the boundary lies.

109.1.1 K-Nearest Neighbors (KNN) and Its Boundary

To make this idea concrete, let us examine how decision boundaries appear in a specific model: **K-Nearest Neighbors (KNN)**.

KNN classifies a new point based on the majority label of its k closest neighbors.

For KNN, the decision boundary is inherently complex and non-linear. The decision boundary (its shape) in KNN depends entirely on:

- The distribution of training data points
- The value of k

[Image omitted in PDF — see the online textbook]

Figure: Decision boundaries for different values of k in KNN.

The boundary exists where the majority vote of the k closest neighbors changes. * A **small** k results in a very jagged, flexible boundary. * A **large** k results in a very smooth, simple boundary.

For example, here for **Small K (i.e. K=1)**: - Ignores broader patterns, focusing only on the closest point. - Decision Boundary: Irregular and jagged—highly sensitive to individual data points. - Behavior: **Overfits**, memorizes training data and lacks generalization (captures noise rather than general structure).

For **Large K (i.e. K=N (All Data Points as Neighbors))** : - Decision Boundary: Smooth and overly simplistic. - Behavior: **Underfits**, ignores local structure and treats the entire dataset as a single group. - The model treats most of the dataset similarly and may predict the majority class for nearly all inputs. - Predicts the mode (classification) or average (regression) for all new points.

This example illustrates a core idea: *Model complexity directly influences the shape of the decision boundary.*

109.2 The Core Challenge: Generalization, Overfitting, and Underfitting

109.2.1 Generalization

The ultimate goal of Machine Learning is **generalization**: the model's ability to perform well on new, previously unseen data.

A model that performs well on training data but poorly on new data has **failed to generalize**.

To measure generalization, we use a **test set**, which contains data not seen during training. The test set provides the most reliable estimate of real-world performance. Poor generalization results from either Underfitting or Overfitting.

[Image omitted in PDF — see the online textbook]

Figure: Underfitting vs. overfitting illustrated through model complexity.

109.2.2 The Bias-Variance Trade-off

Overfitting and underfitting arise from a fundamental trade-off known as the **bias–variance trade-off**.

Model performance is a balance between **two types of errors**:

1. **Bias (Underfitting)**: The error introduced by the model being **too simple** to capture the true patterns. i.e in KNN, high bias occurs when k is very large, causing excessive averaging.
2. **Variance (Overfitting)**: The error introduced by the model being **too complex** and capturing noise/outliers specific to the training data. i.e in KNN, high variance occurs when k is very small (e.g., $k = 1$).

109.2.3 Underfitting (High Bias)

- **Definition**: The model is **too simple** and fails to learn the basic patterns.
- **Indicator**: **Low** accuracy on **both** Training and Test data.

- **In KNN:** Caused by a **very large** k . The model is too constrained by averaging too many points.

109.2.4 Overfitting (High Variance)

- **Definition:** The model learns the training data *too well*, memorizing noise instead of general rules.
- **Indicator:** **Very High** Training Accuracy, but **Significantly Lower** Test Accuracy.
- **In KNN:** Caused by a **very small** k (e.g., $k = 1$). The boundary is too complex and sensitive to noise.

The key is to adjust the model's complexity (like changing k) to maximize **Generalization** (Test Accuracy) without sacrificing too much Training Accuracy.

[Image omitted in PDF — see the online textbook]

109.2.5 Underfitting vs Overfitting

Type	Description	Visual Intuition
Underfitting	Model too simple: fails to capture patterns	High train & test error
Overfitting	Model too complex: memorizes training data	Low train, high test error

109.2.6 Back to KNN: Understanding the KNN Decision Boundary

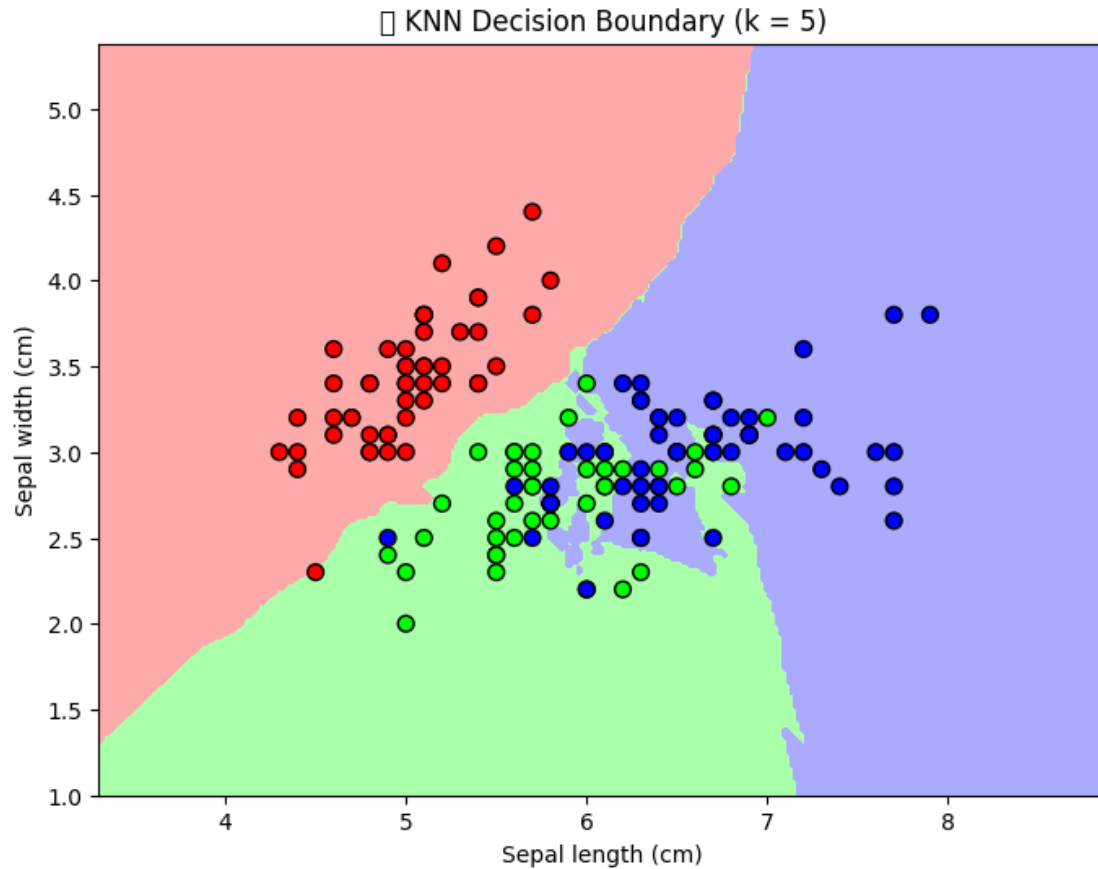
K-Nearest Neighbors (KNN) is an **instance-based** learning algorithm.

It doesn't build an explicit mathematical model; instead, it **classifies new points** based on the majority vote of their k nearest neighbors.

- Each point is classified by **looking at its neighbors** in the feature space.
- **Decision boundaries** are formed where the majority class changes, often resulting in **non-linear, curved boundaries**.
- As k increases, the boundary becomes **smoother** (less sensitive to noise).

Let's visualize how KNN creates these boundaries using two features of the Iris dataset.

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151:
UserWarning: Glyph 127800 (\N{CHERRY BLOSSOM}) missing from font(s) DejaVu Sans.
fig.canvas.print_figure(bytes_io, **kw)
```



Discussion: - Notice how the colored regions represent **KNN’s decision boundaries**; areas where the algorithm assigns new samples to a specific class. - The **boundaries are flexible and follow the data**, unlike linear models. - Use our Interactive Visualization, try changing `n_neighbors` (e.g., 1, 3, 10) to see how it affects the smoothness of boundaries.

109.3 Distance Metrics and Variants in KNN

The **choice of distance metric** greatly influences how KNN defines “neighborhoods” and classifies points.

It depends on the **nature of the data** and the **problem context**.

109.3.1 Common Distance Metrics

109.4 1. Euclidean Distance (L2 Norm)

Detail	Description
Description	Measures the straight-line distance between two points in space.

Detail	Description
Use When	Features are continuous and measured on the same scale . Differences in magnitude are meaningful (e.g., height, weight).
Caution	Highly sensitive to outliers and varying feature scales — always normalize or standardize data before use.
Formula	$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

109.5 2. Manhattan (City Block) Distance (L1 Norm)

Detail	Description
Description	Measures the distance by summing absolute differences along each dimension (like moving along a grid).
Use When	Movement is restricted to orthogonal directions (e.g., city blocks). Features contribute independently to distance. Works better than Euclidean for high-dimensional data as it reduces the impact of large single-feature deviations.
Caution	Less sensitive to outliers than Euclidean, but still requires feature scaling for optimal performance.
Formula	$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n x_i - y_i $

[Image omitted in PDF — see the online textbook]

109.6 3. Hamming Distance

Detail	Description
Description	Counts the number of features where two samples differ (mismatches between attribute values).
Use When	Data is categorical or binary (e.g., yes/no, text encodings, DNA sequences).
Caution	Only works for discrete or categorical features. It gives a binary measure (different or not different) for each feature.
Formula	$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n \mathbb{I}(x_i \neq y_i)$ (Where $\mathbb{I}(\cdot)$ is the indicator function.)

109.7 4. Cosine Similarity

Detail	Description
Description	Measures the cosine of the angle between two vectors — focuses purely on orientation , not magnitude or size.
Use When	Direction matters more than size (e.g., text analysis, where document length shouldn't bias similarity). Common in recommender systems .
Caution	Returns similarity (1 is close, 0 is far) rather than distance. It is often converted to a distance metric as: $d = 1 - \text{similarity}$.
Formula	$\text{Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\ \mathbf{x}\ \ \mathbf{y}\ }$

109.8 Weighted KNN

So far, we have talked about **standard KNN**, where every one of the k neighbors has an equal vote. Now, we will introduce **Weighted K-Nearest Neighbors** changes this by giving more weight to the neighbors that are closer to the new data point.

Why? According to **Weighted KNN**, closer neighbors have a stronger influence on the prediction.

Mathematically, each neighbor is weighted by:

$$w_i = \frac{1}{d_i + \varepsilon}$$

Where:

d_i = distance between the query point and the i^{th} neighbor

ε = a small constant to prevent division by zero

This means points closer to the query point contribute **more** to the prediction than distant ones.

In `scikit-learn`, you can enable this using:

```
KNeighborsClassifier(n_neighbors=5, weights='distance')
```

Key Points to remember:

- Standard KNN treats all neighbors equally (`weights='uniform'`).
- Weighted KNN emphasizes closer neighbors, making the classifier more robust to noise.
- Works best when your features are continuous numeric values (like Iris measurements).-

110 Ethics and Responsible ML

110.0.1 Why Ethics Matter?

Machine Learning systems influence real-world decisions; from hiring and loan approvals to medical diagnoses and criminal justice assessments. These decisions can significantly affect people's lives. Therefore, fairness, accountability, and transparency are not optional; they are essential.

Key ethical principles include:

- **Bias:** When training data reflects historical inequalities or social prejudice, models may learn and amplify those patterns.
- **Fairness:** Models should avoid systematically disadvantaging particular groups.
- **Transparency:** Decisions made by models should be explainable and interpretable.
- **Privacy:** Personal data must be handled responsibly and protected from misuse.
- **Accountability:** Organizations deploying ML systems must take responsibility for their outcomes.
- **Robustness:** Models should remain reliable under changing or unexpected conditions.
- **Human Oversight:** High-stakes decisions should include meaningful human review.

Example: A hiring model should not unfairly prefer one gender or racial group due to biased historical hiring data.

Machine Learning systems reflect the data and values embedded in them. Responsible ML requires not only technical skill but also ethical awareness and critical thinking.

111 Summary of the Chapter

In this chapter, we walked through the **complete Machine Learning workflow** — from data to deployment-ready model (conceptually).

- We covered:**
1. **Understanding the dataset** – explored real data (Iris flower dataset).
 2. **Defining features and labels** – identified what to predict and what to use for prediction.
 3. **Training a model** – taught a KNN to learn from data.
 4. **Evaluating performance** – measured accuracy on unseen test data.
 5. **Visualizing data** – saw how features relate and form clusters.

Key takeaway:

Machine Learning is about teaching computers to **learn from data and improve over time** — not just follow fixed rules. - ML enables systems to learn from data automatically. - Three main types: Supervised, Unsupervised, and Reinforcement. - Workflow: Data → Model → Evaluation → Deployment. - Ethical ML ensures fairness, transparency, and accountability.

111.1 Hands-on Practice

111.1.1 Task: Identify the ML Type

1. Predicting house prices $\rightarrow$?
2. Grouping customers $\rightarrow$?
3. Teaching a robot to play chess $\rightarrow$?

111.1.2 Reflection Questions

1. What distinguishes Machine Learning from traditional programming?
2. Why is testing important after training a model?
3. How can bias in ML models be reduced?

111.2 Knowledge Check

[Interactive content omitted in PDF]

111.3 interactive knn

112 Interactive K-Nearest Neighbors

This tool lets you explore how the K-Nearest Neighbors (KNN) algorithm works by finding the k closest points to a target node.

How to Use:

- Adjust the k slider to change the number of nearest neighbors (1-15).
- Observe how changing k affects which points are considered “neighbors”
- View statistics including individual distances and the average distance to all k neighbors

[Interactive content omitted in PDF]

113 Chapter 11 - Feature Engineering

114 Introduction to Feature Engineering

In many machine learning tasks, the **quality of the features** often matters more than the complexity of the model. Feature engineering refers to the process of **transforming raw data into meaningful input variables (features)** that improve the performance of machine learning models.

Raw data collected from real-world systems is often:

- messy
- incomplete
- poorly formatted
- not directly useful for modeling

Feature engineering helps convert raw data into **structured, informative inputs** that models can learn from.

A simple model with well-engineered features can often perform **better than a complex model trained on poorly prepared data**.

114.1 What is a Feature?

A **feature** is an individual measurable property **used as input** to a machine learning model. In a dataset:

- **Rows** represent observations
- **Columns** represent features

Example dataset:

Student_ID	Study_Hours	Attendance	Final_Grade
1	5	90	85
2	2	60	65
3	8	95	92

Here:

- **Study_Hours** → feature
- **Attendance** → feature
- **Final_Grade** → target variable

114.2 Why Feature Engineering Matters

Feature engineering helps models:

- capture meaningful patterns
- reduce noise
- improve prediction accuracy
- generalize better to unseen data

Example:

- Raw feature: Date = 2026-03-11
- Engineered features:
 - Day_of_week = Wednesday
 - Month = March
 - Is_weekend = False

These engineered features may better capture behavioral patterns.

114.3 Feature Engineering in the Data Science Pipeline

Feature engineering occurs **after data cleaning but before model training**. Typical workflow:

Raw Data	→	Data Cleaning	→	Feature Engineering	→	Model Training	→	Model Evaluation
----------	---	---------------	---	---------------------	---	----------------	---	------------------

[Image omitted in PDF — see the online textbook]

Figure: Overview of the Feature Engineering Process. Source: GeeksforGeeks

Feature engineering is often **iterative**, meaning data scientists repeatedly refine features to improve model performance.

115 Types of Feature Engineering

Feature engineering generally involves **three** major types of operations:

- **Feature Creation**
- **Feature Transformation**
- **Feature Selection**

Each type plays a different role in improving the quality of input data for machine learning models.

115.1 1. Feature Creation

Feature creation involves **generating new variables from existing data**. These new variables may capture relationships that the original features do not directly represent.

Examples:

Original Features	Engineered Feature
height, weight	$BMI = \text{weight} / \text{height}^2$
purchase_count, visits	average_purchase_value
date	day_of_week, month, is_weekend

115.1.1 Example: Housing Dataset

Original features:

Size_sqft	Bedrooms	Price
1500	3	450000

Engineered features:

- price_per_sqft
- bedrooms_per_sqft
- house_age

These engineered features may better capture housing patterns.

	size_sqft	price	price_per_sqft
0	1500	450000	300.000000
1	1800	520000	288.888889
2	1200	350000	291.666667

Remember, Feature creation often uses domain knowledge to design useful variables.

115.2 Types of Feature Creation:

Common creation techniques include:

- Polynomial Features
- Interaction Features ### 1a. Polynomial Features

Polynomial features are created by **raising existing features to a power**. They allow models to capture **nonlinear relationships** between variables.

For example, suppose we are predicting house prices using the size of a house.

Size (sqft)	Price
1000	200000
1500	300000
2000	450000

The relationship between size and price may **not be perfectly linear**. To capture nonlinear patterns, we can create polynomial features:

$$x^2, x^3, \dots$$

115.2.1 Example:

Size	Size ²
1000	1,000,000
1500	2,250,000

These new variables allow models to learn **curved relationships**.

115.2.2 Example Python Code

```
from sklearn.preprocessing import PolynomialFeatures
import pandas as pd

df = pd.DataFrame({
    "size": [1000, 1500, 2000]
})
```

```
poly = PolynomialFeatures(degree=2, include_bias=False)

poly_features = poly.fit_transform(df)

pd.DataFrame(poly_features, columns=["size", "size_squared"])
```

Polynomial features are commonly used with models such as:

- Linear Regression
- Logistic Regression

They allow simple models to capture complex patterns.

115.2.3 1b. Interaction Features

Interaction features are created by **multiplying two or more features together**. They capture relationships where the **combined effect of multiple variables matters**.

115.2.4 Example

Suppose we want to predict **house prices** using:

- house size
- neighborhood quality score

Size	Neighborhood Score
1500	8
1500	4

Two houses may have the **same size**, but if one is in a better neighborhood, the price may be higher. An interaction feature can capture this relationship.

$$Interaction = Size \times NeighborhoodScore$$

115.2.5 Example Table

Size	Neighborhood	Size $\times$ Neighborhood
1500	8	12000
1500	4	6000

This feature helps the model understand that **price depends on both variables together**.

115.2.6 Example Python Code

```
df["size_neighborhood_interaction"] = df["size"] * df["neighborhood_score"]
```

115.2.7 Why These Features Matter?

Polynomial and interaction features help models capture complex relationships in data.

- Polynomial Features Capture nonlinear patterns
- Interaction Features Capture relationships between multiple variables

These techniques are widely used in: - regression models - recommendation systems - predictive analytics

115.3 2. Feature Transformation

Feature transformation refers to modifying existing features to improve their **scale, distribution, or representation** so that machine learning models can learn patterns more effectively.

Common transformation techniques include:

- Scaling and Normalization
- Standardization
- Log Transformation
- Encoding Categorical Variables
- Discretization (Binning)
- Text Feature Extraction (**Later Chapter**)

115.3.1 2a. Scaling and Normalization

Scaling adjusts the **range of numerical values** so that features are comparable.

Example:

Feature	Range
Age	18 – 60
Income	20,000 – 200,000

Because income has a much larger range, some models may give it **more influence**.

115.3.2 Min-Max Normalization

This method rescales values to the **0–1 range**.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Example python code:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df[["income"]])
```

115.3.3 Standardization

Standardization transforms values so that they have mean = 0 and standard deviation = 1

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Where: μ = mean σ = standard deviation

Example python code:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_standardized = scaler.fit_transform(df[["income"]])
```

Standardization is commonly used in algorithms such as:

- Logistic Regression
- Support Vector Machines
- K-Means Clustering

115.3.4 2b. Log Transformation

Log transformation is used when data is **highly skewed** or contains **very large values**. It helps reduce the influence of extreme values and makes the distribution more balanced.

Example Data

Income
30000
40000
50000
900000

Applying a log transformation compresses large values.

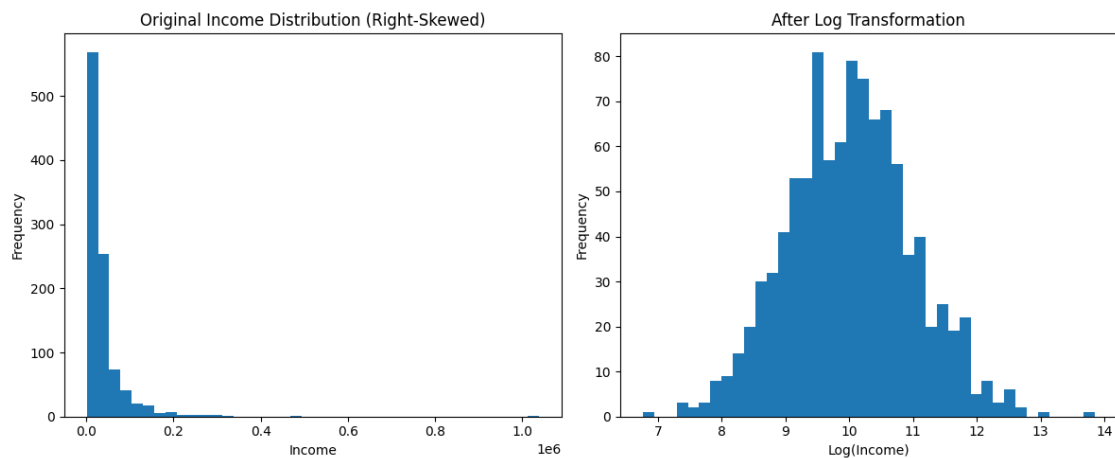
$$x' = \log(x)$$

This transformation is commonly used for variables such as:

- income
- population

- sales
- transaction amounts

	income	log_income
0	30000	10.308953
1	40000	10.596635
2	50000	10.819778
3	900000	13.710150



Remember, Log transformation is especially useful when a feature has a right-skewed distribution, meaning a small number of observations have extremely large values.

115.3.5 2c. Feature Encoding (Categorical Variables)

Machine learning models require **numerical input**, so categorical variables must be converted into numbers before they can be used in most algorithms. Example categorical feature:

Color
Red
Blue
Green

Several encoding techniques can be used depending on the **type of categorical data** and the **number of categories**.

115.3.6 (I) One-Hot Encoding

One-hot encoding creates a **binary column for each category**.

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0

[Image omitted in PDF — see the online textbook]

Figure: One-Hot Encoding. Source: Medium

Suitable for: Nominal data (categories with no natural order)

Example code:

```
pd.get_dummies(df["color"])
```

115.3.7 (II) Label Encoding

Each category is assigned a **numeric label**.

Category	Encoded Value
Low	1
Medium	2
High	3

[Image omitted in PDF — see the online textbook]

Figure: Label Encoding. Source: Daily Dose of Data Science

Suitable for: Ordinal data (categories with natural order)

Example code:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df["category_encoded"] = encoder.fit_transform(df["category"])
```

Note: Label encoding may introduce **artificial ordering** when categories do not have a natural order.

115.3.8 (III) Frequency Encoding

Frequency encoding replaces categories with the **frequency of their occurrence in the dataset**.

Example:

Category	Frequency
A	0.40
B	0.35
C	0.25

Suitable for: High-cardinality categorical variables.

High-cardinality categorical variables are categorical features that contain a **large number of unique categories** (for example, hundreds or thousands of values such as city names, product IDs, or user IDs).

Example:

City
New York
Chicago
Los Angeles
Houston
Phoenix
...

If a feature contains many unique categories, using **One-Hot Encoding** would create a large number of new columns. In such cases, **frequency encoding** is often a more efficient approach.

Example Python Code:

```
freq = df["category"].value_counts(normalize=True)
df["category_freq"] = df["category"].map(freq)
```

115.3.9 (IV) Target (Mean) Encoding

Target encoding replaces each category with the **average value of the target variable for that category**.

Example:

City	Average House Price
City A	350000
City B	420000

In this approach, each category is replaced by the **mean value of the target variable** for that category.

For example, if houses in **City A** have an average price of **350,000**, then all instances of **City A** may be encoded as **350000**.

Suitable for: Categorical variables with **many categories** where the category has a relationship with the target variable.

Warning: Target encoding must be applied carefully to **avoid data leakage**, especially when the target values from the test data influence the encoding.

115.4 —————

115.4.1 BONUS: What is Data Leakage?

Data leakage occurs when a model accidentally uses **information that would not be available at prediction time**. This can cause the model to appear **very accurate during training**, but perform poorly on new data.

115.4.2 Example of Leakage with Target Encoding

Suppose we want to predict **house prices** using the city as a feature.

City	Price
A	300000
A	400000
B	420000

If we compute the **average price using the entire dataset**, including the test data, the model indirectly gains information about the values it is supposed to predict. This creates **data leakage**.

115.4.3 Correct Approach

To avoid leakage:

1. Split the dataset into **training and test sets first**.
2. Compute the **target encoding using only the training data**.
3. Apply the learned encoding to the test data.

115.5 —————

115.5.1 Example Python Code

```
import pandas as pd

# Example dataset
df = pd.DataFrame({
    "city": ["A", "A", "B", "B", "B"],
    "price": [300000, 400000, 420000, 410000, 430000]
})

# Compute mean target value for each category
target_mean = df.groupby("city")["price"].mean()

# Map the mean values back to the dataset
df["city_target_encoded"] = df["city"].map(target_mean)

df
```

Note: In practice, target encoding should be computed **using training data only**

115.5.2 (V) Probability Encoding

Probability encoding replaces categories with the **conditional probability of the target variable given the category**.

Example:

Category	P(Target = 1 Category)
A	0.70
B	0.45

In this case, category **A** has a 70% probability of belonging to the target class, while category **B** has a 45% probability.

This approach is useful in **classification problems** where categories have different probabilities of belonging to the target class.

115.5.3 2d. Discretization (Binning)

Discretization, also called **binning**, is the process of converting **continuous numerical variables into categorical groups (bins)**. Instead of using the exact numeric value, the variable is grouped into ranges.

This can help:

- simplify patterns in the data
- reduce noise
- improve interpretability
- make some models easier to train

115.5.4 Example

Age	Age_Group
22	Young
35	Adult
65	Senior

Instead of using exact ages, we categorize them into meaningful groups.

115.5.5 Example Code

```
import pandas as pd
```

```
df = pd.DataFrame({
```

```

    "age": [22, 35, 65, 18, 45]
})

df["age_group"] = pd.cut(df["age"], bins=[0, 18, 35, 60, 100])

df

```

115.6 Types of Binning:

- **Equal Width Binning:** Each bin has the same numerical range. Example:
 - 0–20
 - 20–40
 - 40–60
 - 60–80
- **Equal Frequency Binning:** Each bin contains approximately the same number of observations.

Example Python code:

```
df["age_bin"] = pd.qcut(df["age"], q=3)
```

Binning is helpful when:

- the exact numeric value is not necessary
- the variable has high variability
- interpretability is important

Example: - Age → Age group - Income → Income bracket - Temperature → Cold / Moderate / Hot

115.7 3. Feature Selection

Feature selection is the process of **choosing the most relevant features for a machine learning model and removing irrelevant or redundant variables**.

Reducing unnecessary features can help:

- improve model performance
- reduce overfitting
- decrease training time
- make models easier to interpret

115.7.1 Example

Original features:

Size	Bedrooms	Zip Code	Price
------	----------	----------	-------

If **Zip Code** does not contribute to predicting house price, it may be removed.

Selected features:

Size | Bedrooms | Price |

115.7.2 Common Approaches

Method	Description
Filter Methods	Use statistical measures such as correlation to select features
Wrapper Methods	Evaluate different feature combinations using a model
Embedded Methods	Feature selection occurs during model training (e.g., Lasso, decision trees)

116 Summary of the Chapter

Feature engineering is the process of transforming raw data into meaningful inputs that improve the performance of machine learning models.

In this chapter, we introduced several important feature engineering techniques:

- **Feature Creation:** Generating new features from existing variables (e.g., polynomial features, interaction features).
- **Feature Transformation:** Modifying features to improve scale, distribution, or representation (e.g., scaling, log transformation, encoding).
 - **Encoding Categorical Variables:** Converting categorical data into numerical form using methods such as one-hot encoding, label encoding, frequency encoding, and target encoding.
 - **Discretization (Binning):** Converting continuous numerical variables into categorical groups.
- **Feature Selection:** Identifying and keeping the most informative features while removing irrelevant or redundant ones.

Effective feature engineering can significantly improve model performance and interpretability. In practice, it is often an **iterative process**, where features are refined and evaluated repeatedly during model development.

116.1 Knowledge Check

[Interactive content omitted in PDF]

117 Chapter 14 - Decision Tree

118 Introduction: The Power of Simple Questions

Imagine you are a doctor diagnosing a patient. You don't run every test simultaneously. Instead, you ask a sequence of questions: "Does the patient have a fever?" If yes, you ask, "Is there a rash?" If no, you might ask, "Is the pain localized?" This hierarchical process of asking questions to narrow down possibilities is the very essence of a Decision Tree.

A Decision Tree is a **non-parametric** (no assumptions about data distribution or functional form, allowing them to learn patterns directly from the data), **supervised learning** algorithm used for both classification and regression tasks. Its model forms a tree-like structure, mimicking human decision-making processes.

Parameteric vs Non-Parametric: A Decision Tree (DT) is non-parametric, meaning it makes no assumptions about data distribution or functional form and has no fixed number of parameters—its complexity (splits, depth, etc.) adapts to the data; unlike parametric models (e.g., linear or logistic regression) that assume a specific functional form and have fixed parameters.

They **mimic human decision-making** by breaking down complex decisions into a series of simpler questions. A Decision Tree represents decisions in a flowchart-like, hierarchical structure that models possible consequences—including outcomes, costs, and probabilities and is highly interpretable.

[Image omitted in PDF — see the online textbook]

They are especially popular because they require minimal data preprocessing, can handle both numerical and categorical data, and provide models that are easily interpretable.

It has many strengths:

- **Interpretability:** The resulting model is a white box. The rules for making a prediction can be easily understood and explained, even to non-experts.
- **Few Data Preprocessing Requirements:** They require little data preparation (e.g., no need for feature scaling or normalization).
- **Handles Mixed Data Types:** They can work with both numerical and categorical data.
- **Non-Linearity:** They can capture complex non-linear relationships between features and the target variable.

However, this simplicity can also be a weakness because it often leads to **overfitting**. When a tree grows too deep, it starts to learn the noise in the training data, not just the underlying pattern, which causes poor performance on new data.

In this chapter, we will break down how the **Decision Tree algorithm works**, understand how to create **good splitting questions**, learn how to **prune** an overly complex tree, and see how Decision Trees serve as the foundation for advanced ensemble methods like Random Forests and Gradient Boosting Machines

119 Section 1: Basic Concepts of Decision Tree

Next, we will learn some basic concepts related to decision tree.

119.1 Structure of a Decision Tree and Terminology

To understand how a tree is built, we must first understand its components. A Decision Tree is a graph composed of:

[Image omitted in PDF — see the online textbook]

- **Root Node:** The topmost node, representing the entire dataset. It is the starting point for the splitting process.
- **Internal Nodes (Decision Nodes):** Nodes that represent a decision point or a test on a specific feature. Each internal node splits the data into two or more branches.
- **Branches (Edges):** The outcome of a decision node test, leading to the next node.
- **Leaf Nodes (Terminal Nodes):** The final nodes in the tree that do not split further. Each leaf node represents a class label (in classification) or a continuous value (in regression) which is the final prediction.

The path from the root to a leaf represents a series of conjunctions (AND operations) of the conditions at the internal nodes, forming a classification rule.

119.1.1 More Terminology

- **Sub-Tree:** A portion of the main tree.
- **Splitting:** The process of dividing a node into two or more sub-nodes.
- **Pruning:** The process of removing branches to reduce the complexity of the tree and prevent overfitting (we will talk about it later in this chapter).

119.1.2 Example

Consider the classic “Play Tennis” dataset, where we want to predict whether to play tennis based on weather conditions (Outlook, Temperature, Humidity, Windy).

A simplified decision tree might look like:

[Image omitted in PDF — see the online textbook]

120 Section 2: The Tree-Building Algorithm: A Greedy, Recursive Process

The construction of a Decision Tree is a **greedy**, **recursive**, and **top-down** process, known as Recursive Binary Splitting (or partitioning).

Key Idea: The core idea behind building a decision tree is **recursive partitioning**; repeatedly splitting the dataset based on an attribute that maximizes separation between classes (or minimizes error in regression).

120.1 Detailed Concept of Decision Tree Learning Algorithm:

- **Greedy:** At each node, the algorithm selects the locally optimal split without considering the overall tree structure or future splits. It makes the best decision it can at that moment.
- **Recursive:** The process is repeated for each subset of the data created by previous splits.
- **Top-Down:** It begins at the root and successively splits the dataset into smaller groups.

120.1.1 The Core Algorithm (ID3 and its successors C4.5, CART):

The **Iterative Dichotomiser 3 (ID3)** algorithm is a decision tree induction algorithm that constructs decision trees using a top-down approach. The main goal of ID3 is to find the most informative attributes at each step to create partitions that yield the **highest information gain** or **decrease in impurity**.

Let's break down the ID3 algorithm into a step-by-step process:

- 1. **Start** at the root node with the entire dataset.
- 2. **For each feature in the dataset**, calculate a “goodness” metric (e.g., Information Gain, Gini Impurity) for all possible split points.

In this chapter, we will focus on “Entropy” and “Information Gain”.

- 3. Select the feature and split point that **maximizes this “goodness” metric**. This becomes the splitting rule for the node.
- 4. **Split the dataset at the current node** into child nodes based on the selected rule.

The process recursively **repeat steps 2-4** for each child node until a **stopping criterion** is met.

Common stopping criteria include:

- The node is “**pure**” (homogeneous: all samples belong to the same class).
- No more attributes remain for further splitting.
- A stopping condition (like a predefined maximum depth) is reached.
- The node contains fewer samples than the predefined minimum threshold.
- A split does not produce a significant improvement (minimum gain threshold).

121 Splitting Criteria (Attribute Selection Measures):

The algorithm needs a metric to determine the “**best**” split. For classification trees, the goal is to **maximize the homogeneity** of the resulting subsets. The main metrics are **Entropy** and **Information Gain** (Used in ID3, C4.5).

121.1 Entropy

Entropy measures the randomness or **impurity** in the dataset.

In information theory, the entropy of a random variable is the average level of “information”, “surprise”, or “uncertainty” inherent to the variable’s possible outcomes.

In a decision tree, a node that contains a mix of different outcomes (for example, 2 Pass and 2 Fail) has **higher entropy**, meaning more disorder or uncertainty. If a node has only one type of outcome (for example, all Pass or all Fail), it is **pure and has low entropy**.

- **Maximum entropy = 1** → occurs when the outcomes are equally mixed (e.g., 50% Pass, 50% Fail).
- **Minimum entropy = 0** → occurs when all outcomes are the same (e.g., 100% Pass or 100% Fail).

In short: Mixed outcomes $\rightarrow$ high entropy (uncertain) and Single outcome $\rightarrow$ low entropy (certain)

[Image omitted in PDF — see the online textbook]

The first thing to understand about Decision Trees is that they split the data **based on the input features into smaller groups** that are **more homogeneous (or “pure”)** with respect to the target variable.

For example, if the target variable has two possible outcomes — say 1 and 0 (represented by green and red dots) — the Decision Tree looks for splits in the features that create groups mostly containing either 1’s or 0’s, rather than a mix of both.

In simple terms, the tree keeps splitting the data using feature values so that each group becomes as pure as possible in terms of the target outcome.

[Image omitted in PDF — see the online textbook]

121.1.1 Definition

Entropy is defined (or calculated) as:

$$Entropy(S) = - \sum_{k=1}^c p_k \log_2(p_k)$$

where:

- S = the current node (or dataset)
- c = number of classes
- p_k = proportion of data points in class k within node S

121.1.2 Properties of Entropy

- **Entropy = 0:**
The node is **pure** — all samples belong to a single class (no uncertainty).
- **Entropy = 1:**
The node is **maximally impure** — classes are **equally mixed** (e.g., 50% Pass, 50% Fail).
- **Range:**
 $0 \leq Entropy(S) \leq 1$ for binary classification.
(For multi-class problems, entropy can be higher, depending on the number of classes.)

121.1.3 Example

If a node contains 4 samples:

- 3 are **Pass**,
- 1 is **Fail**,

then:

$$Entropy(S) = -\left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4}\right)$$

$$Entropy(S) = -(0.75 \times -0.415) - (0.25 \times -2)$$

$$Entropy(S) \approx 0.81$$

So, this node has some impurity but is not completely mixed.

121.1.4 Try it Yourself

If a node contains **5 examples**, where **4 are Pass** and **1 is Fail**, calculate the entropy.

You can use the Python function below to verify your answer!

121.2 Information Gain (IG)

Once we know how to calculate **Entropy**, we can measure how much **uncertainty** (or impurity) is reduced when we split a dataset using a feature. This reduction is called **Information Gain**.

Information Gain measures the **reduction in entropy** achieved by a split. It tells us **how much uncertainty (impurity) is removed** by a particular split.

121.2.1 Definition

Formally, suppose a split divides the parent node S into V subsets:

$$S_1, S_2, \dots, S_V$$

Let:

- $|S|$ = total number of samples in the parent node
- $|S_v|$ = number of samples in child node S_v

Information Gain for a feature **A** is defined as:

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where:

- S = parent node (original dataset)
- A = feature used for splitting
- S_v = subset of S corresponding to value v of feature A
- $|S|$, $|S_v|$ = number of samples in S and S_v , respectively

The algorithm selects the **feature with the highest Information Gain** for splitting.

121.2.2 Intuition

Information Gain measures how much a split **reduces impurity**: - A **high IG** means the feature created **purier subsets** (good split).

- A **low IG** means the feature did not reduce impurity much (poor split).

Decision Trees choose the feature with the **highest Information Gain** at each step.

121.2.3 Example

Suppose a dataset has 10 samples:

- 6 **Pass**, 4 **Fail**

Then the initial (parent) entropy is:

$$Entropy(S) = - \left(\frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10} \right) \approx 0.971$$

Now we split based on **Feature A** (e.g., “Study Hours”) which has two possible values:

- **A = High**: 5 samples → 4 Pass, 1 Fail

- **A = Low**: 5 samples → 2 Pass, 3 Fail

Compute entropy for each group:

$$Entropy(S_{High}) = - \left(\frac{4}{5} \log_2 \frac{4}{5} + \frac{1}{5} \log_2 \frac{1}{5} \right) \approx 0.722$$

$$Entropy(S_{Low}) = - \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right) \approx 0.971$$

Then the weighted average entropy after splitting is:

$$Entropy_{after} = \frac{5}{10}(0.722) + \frac{5}{10}(0.971) = 0.847$$

Finally, the **Information Gain** is:

[Image omitted in PDF — see the online textbook]

$$IG(S, A) = 0.971 - 0.847 = 0.124$$

So, splitting on **Feature A** reduces uncertainty by **0.124 bits**.

121.2.4 Additional Example:

Example illustrating the calculation of information gain. Source: Hendler 2018, slide 46.

[Image omitted in PDF — see the online textbook]

121.2.5 Try it Yourself

If you have another feature **B** that splits the same data into: - **B = Yes**: 6 samples (5 Pass, 1 Fail)

- **B = No**: 4 samples (1 Pass, 3 Fail)

Try calculating:

$$Entropy(S_{Yes}), Entropy(S_{No}), \text{ and } IG(S, B)$$

Which feature gives a higher Information Gain — **A** or **B**?

122 How Decision Trees Decide Which Feature to Split (Using Entropy and Information Gain)

Decision Trees use **Entropy** and **Information Gain** to decide **how to split the data at each node**. The goal is to create **subgroups (child nodes) that are as pure as possible** with respect to the target variable.

The core idea of a Decision Tree is **recursive splitting**: at each node, we pick the **feature that best separates the data** into purer subgroups.

122.0.1 Step-by-Step Process

1. Start at the root node

- The root node contains the entire dataset.
- Calculate the **Entropy** of the root node to measure the overall uncertainty.

2. Evaluate each feature

- For each feature, split the dataset according to its possible values.
- Compute the **entropy of each child node** and the **weighted average entropy**:

$$Entropy_{after_split} = \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- Calculate **Information Gain**:

$$IG(S, A) = Entropy(S) - Entropy_{after_split}$$

- A **higher IG** means the feature produces **purer child nodes**.
- #### 3. Choose the best feature to split
- Compare the Information Gain of all features.
 - Select the **feature with the highest IG** for splitting.

- This ensures each split reduces the most uncertainty.
 - Creates child nodes that are **more homogeneous** than the parent
4. **Repeat recursively for each child node**
- Treat each child node as a new node and repeat Steps 1–3.
 - Continue until:
 - Nodes are **pure** (entropy = 0), or
 - A **stopping condition** is reached (e.g., max depth, min samples per node).
-

122.0.2 Key Idea

- **Entropy** tells us how **mixed** the node is.
 - **Information Gain** tells us how **good a feature is at reducing uncertainty**.
 - Decision Trees **always pick the feature that maximizes Information Gain** to make the next split.
-

122.1 Detailed Example 01: How Decision Trees Decide Which Feature to Split

Consider an example where we are building a decision tree to predict whether a loan given to a person would result in a write-off or not. Our entire population consists of 30 instances. 16 belong to the write-off class and the other 14 belong to the non-write-off class. We have two features, namely “Balance” that can take on two values -> “< 50K” or “>50K” and “Residence” that can take on three values -> “OWN”, “RENT” or “OTHER”.

123 Overfitting and Pruning

123.1 Overfitting in Decision Tree

Decision trees are prone to overfitting, meaning they become too complex and capture noise in the training data (**perfectly fitting the training data**), but performing poorly on unseen data (**poor generalization**). This often happens with very deep trees with many nodes that achieve perfect classification on the training set (pure leaf nodes).

123.2 Pruning Methods

Pruning is the technique used to combat overfitting by reducing the size of the tree. There are two main strategies:

1. **Pre-pruning (Early Stopping)** : Stop growing the tree early before it becomes too complex. How? by setting parameters like:
 - Maximum depth of the tree.
 - Minimum number of samples required to split a node.

- Minimum number of samples required to be in a leaf node.
- 2. **Post-pruning:** Grow a full tree and then remove branches that do not improve accuracy on validation data. The most common technique is Reduced Error Pruning.

123.3 interactive entropy

124 Interactive Entropy and Information Gain

Information Entropy measures how unpredictable/mixed a dataset is:

- **Low entropy** (close to 0): Dataset is pure (all one class)
- **High entropy** (close to $\log(\# \text{ of classes})$): Dataset is well-mixed

Information Gain measures how much we reduce entropy with a split:

- **Higher gain** = Better split (we removed more uncertainty)
- **Lower gain** = Worse split (didn't help much)

Decision Trees choose splits that **maximize Information Gain** to build effective classifiers!

124.1 Try These Experiments

1. Set all points to one color
2. Split with all points on one side - notice information gain = 0
3. Create a perfect split where each side has only one color - maximum gain!

[Interactive content omitted in PDF]

124.2 Results

1. Notice that you can't split pure data to gain anything - there's no uncertainty left to remove!
 - entropy before = 0
 - entropy after = 0
 - information gain = 0
2. Useless Split. If the split doesn't actually separate the classes, you've gained nothing. The entropy stays exactly the same because all points remain mixed together on one side. This teaches you that Decision Trees must be strategic about WHERE they split
 - entropy before = 1.0
 - entropy after = 1.0 (unchanged!)
 - information gain = 0
3. This is the goal! You went from maximum confusion (mixed colors) to zero confusion (pure branches)
 - entropy before = 1.0 (maximum confusion for 2 classes)
 - entropy after = 0 (both branches pure!)
 - information gain = 1.0 (maximum possible!)

125 Chapter 16 - Regression

126 Introduction to “Regression”

Regression is one of the most fundamental techniques in data analysis and machine learning. At its core, regression is about understanding relationships between variables and using those relationships to make predictions.

In many real-world problems, we are not just interested in classification (yes/no), but in predicting a continuous value. For example:

- What will be the price of a house given its size?
- How will sales change with advertising budget?
- What temperature can we expect tomorrow?

These types of problems naturally lead to regression models.

In regression, we typically define: - **Dependent variable (target)**: the value we want to predict
- **Independent variables (features)**: the inputs used to make predictions

126.1 A More Intuitive Way to Think About Regression

Before jumping into formulas, it helps to think about regression in a more natural way.

Imagine you are looking at data points plotted on a graph; perhaps house sizes on the **x-axis** and prices on the **y-axis**. The points are scattered, but you can visually sense a trend: larger houses tend to cost more.

[Image omitted in PDF — see the online textbook]

Regression is essentially the process of drawing a line (or curve) that best captures this trend. It is not about perfectly matching every single point. Instead, it is about capturing the **overall pattern** so that we can:

- Understand how variables are related
- Make reasonable predictions for new, unseen data

In that sense, regression is both: - A **predictive tool**, and
- An **explanatory tool**

126.2 Another Perspective: Best Fit Line

Another way to think about regression is through the idea of a **best fit line**.

[Image omitted in PDF — see the online textbook]

Illustration of How Linear Regression Models Relationships Using a Straight Line.
Source: Dede, Medium.com

Here, the goal is to draw a line that **minimizes the overall error** between actual data points and predicted values. This line may not pass through every point, but it balances all of them in the best possible way.

126.3 Goal of Regression: Best-Fit Line

The goal is to find the **best-fit line**.

The central goal of regression is to find a function that **best** represents the **relationship between input variables (features) and the target variable**. In linear regression, this function takes the form of a straight line.

But what does “**best**” actually mean?

The **best-fit line** is the line that most closely follows the overall pattern in the data. It does not try to pass through every data point. Instead, it balances all points by minimizing the overall prediction error.

A good regression line should:

- Fit the data well without overreacting to noise
- Minimize prediction errors across all data points
- Capture the underlying relationship between variables

This is an important shift in thinking:

> We are not trying to eliminate error completely, we are trying to **minimize it in a principled way**.

126.3.1 Visual Intuition

If you imagine multiple possible lines passing through the same dataset: - Some lines will be too steep

- Some will be too flat

- Some will miss most points

[Table omitted in PDF — see the online textbook]

Left: Best-fit regression line capturing the overall trend.

Right: Residuals showing the difference between actual and predicted values.

The best-fit line is the one that **balances all deviations (errors)** in the most optimal way.

126.3.2 Comparing Different Fits

Not all lines fit the data equally well. Some lines capture the trend better, while others produce large errors.

[Image omitted in PDF — see the online textbook]

Different lines produce different total errors.

The best-fit line is the one that minimizes the overall residual error.

From above diagram notice that, each line produces a different total error:

- **Blue line** → smallest error → best fit

- **Green line** → moderate error
- **Red line** → very large error → poor fit

This motivates why we need a **formal way (cost function)** to measure error. This is exactly what leads us to concepts of Residuals, Loss function and Cost function (MSE) (we will cover later in this chapter).

126.4 Why Regression Matters

Regression appears everywhere, often in ways we do not immediately notice:

- In economics, to understand how income affects spending
- In healthcare, to study how dosage impacts recovery
- In business, to forecast future demand
- In social sciences, to analyze relationships between variables

What makes regression powerful is that it provides **quantitative insight**. It does not just say “these are related” — it tells us **how strongly** they are related and in what direction.

127 Types of Regression

As we move from simple problems to more complex ones, different types of regression models are used depending on the nature of the data and the relationship.

127.1 1. Simple Linear Regression

The simplest form of regression is **linear regression with one variable**, where we assume a straight-line relationship between input and output.

we model the relationship between **one feature** and **the target** using a straight line.

Example: Predicting salary based on years of experience

- One input variable
- Assumes a linear relationship
- Easy to interpret

The model is written as:

$$y = mx + b$$

Where: - y is the predicted value

- x is the input feature

- m is the slope (how much y changes with x)

- b is the intercept (value of y when $x = 0$)

[Image omitted in PDF — see the online textbook]

A Simple View of Linear Regression. Source:Medium.com

127.1.1 Intuition

Think of this as fitting the “best possible straight line” through your data points. The model tries to capture the overall trend in the data.

For example: - If house size increases, price usually increases → positive slope - If study hours increase, exam score increases → positive slope

Key Insight: The model does **not** try to pass through every point. Instead, it minimizes the overall error across all points.

127.2 2. Multiple Linear Regression

In reality, most outcomes depend on multiple factors, a single feature is rarely sufficient. Multiple linear regression extends the idea of a straight line to multiple dimensions.

- Multiple input variables
- Each feature contributes to the prediction
- Helps capture more realistic scenarios

Example: Predicting house price depends on:

- Size
- Number of bedrooms
- Location
- Age of the property

This leads to **multiple linear regression**, where we use several features:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where: - n is the degree of polynomial - b_0 is the intercept

- $b_1, b_2, \dots, b_n$ are coefficients

- Each x_i is a feature

Each feature contributes independently to the prediction.

127.2.1 Interpretation

One of the most powerful aspects of multiple regression is interpretation:

- b_1 : effect of x_1 while keeping all other variables constant
- b_2 : effect of x_2 , and so on

This allows us to answer questions like: - “How much does price increase per extra bedroom, *holding size constant?*”

127.2.2 Important Note

Features should not be highly correlated with each other (**multicollinearity**), as it can make interpretation unstable.

127.3 3. Non-Linear Relationships and Polynomial Regression

Not all relationships are linear. Sometimes the data curves Polynomial regression allows the model to fit curves instead of straight lines.

- Captures non-linear patterns
- Still based on linear modeling techniques
- Risk of overfitting if degree is too high

Example: - Growth may accelerate over time - Costs may increase at an increasing rate

To handle this, we use **polynomial regression**:

$$y = a + bx + cx^2 + dx^3 + \dots$$

[Image omitted in PDF — see the online textbook]

A Simple View of Polynomial Regression. Source: Statisticalaid.com

Even though the curve looks non-linear, the model is still **linear in parameters**, which allows us to train it using the same techniques as linear regression.

127.3.1 Tradeoff

- Low degree → underfitting
- High degree → overfitting

Choosing the right degree is critical.

127.4 Python Implementation

In practice, regression is often implemented using libraries such as:

```
### Basic Linear Regression
```

```
```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

train model
```

```

model = LinearRegression()
model.fit(X_train, y_train)

predictions
predictions = model.predict(X_test)

Basic Polynomial Regression

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)

```

## 127.5 How the Regression Model Learns

The key idea behind regression is to find parameters (like  $m$  and  $b$ ) that minimize prediction error.

## 127.6 Residuals, Loss, and Cost Function

To formalize the idea of “best fit,” we need **a way to measure error**.

### 127.6.1 Residual (Error)

The residual represents how far off our prediction is from the true value (Difference between actual and predicted values).

$$\text{error} = y - \hat{y}$$

- $y \rightarrow$  actual value
- $\hat{y} \rightarrow$  predicted value

*[Table omitted in PDF — see the online textbook]*

Left: Residuals shown as vertical distances between actual data points and the regression line.

Right: Residuals ideally scattered randomly around zero, indicating a good model fit. Sources: StatisticsFromAtoZ.com | Statology.org

From the above visualization, you can see that, a residual can be visualized as the **vertical distance** between:

- the actual data point
- the regression line

The goal of regression is to make these vertical distances as small as possible. Here:

- Positive residual  $\rightarrow$  prediction is too low
- Negative residual  $\rightarrow$  prediction is too high

### 127.6.2 Loss Function (per data point)

$$\text{Loss} = (y - \hat{y})^2$$

The loss function measures how wrong a **single prediction** is.

### 127.6.3 Cost Function (overall)

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Why squared error? - Avoid negative values

- Penalize large errors more
- Smooth optimization

### 127.6.4 Cost Function (Overall Error)

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

The cost function aggregates error across all data points.

- Provides a single number representing model performance
  - This is what we minimize during training
- 

### 127.6.5 Important Distinction

- **Residual** → error for one data point
- **Loss** → squared error for one data point
- **Cost** → average loss across all data points

Understanding this hierarchy is critical for grasping how regression models are trained.

---

### 127.6.6 Error Measurement

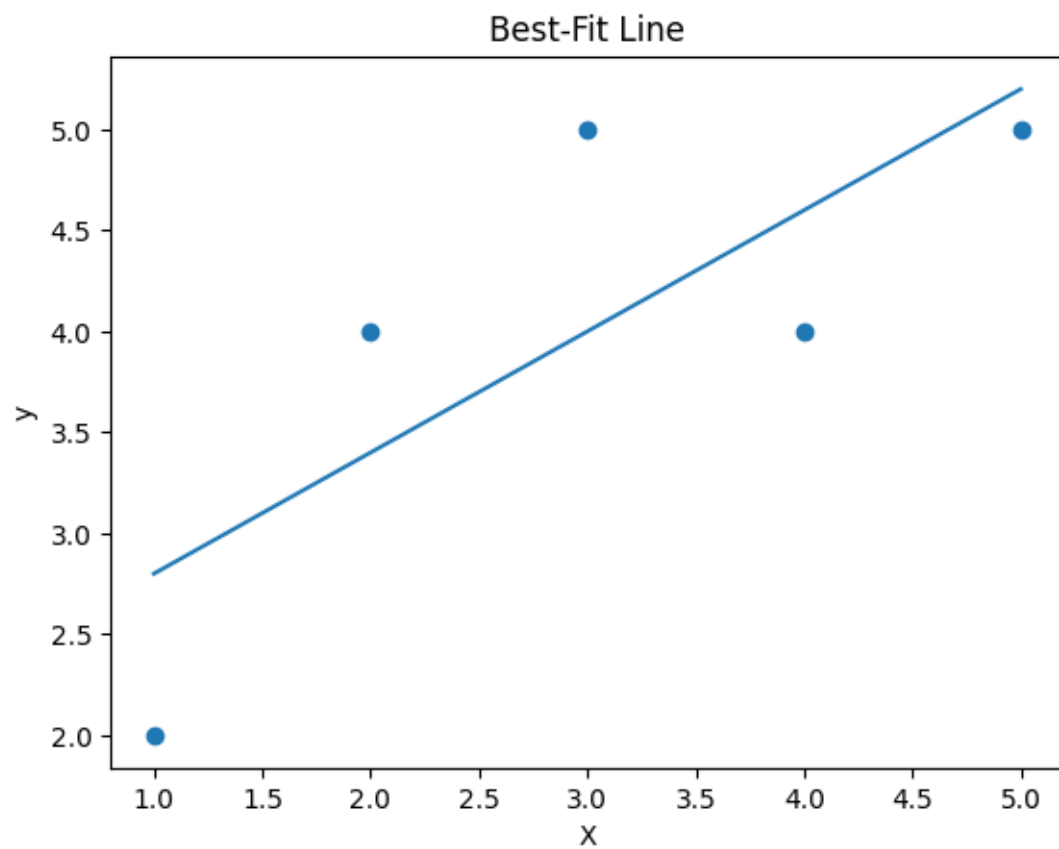
For each data point:

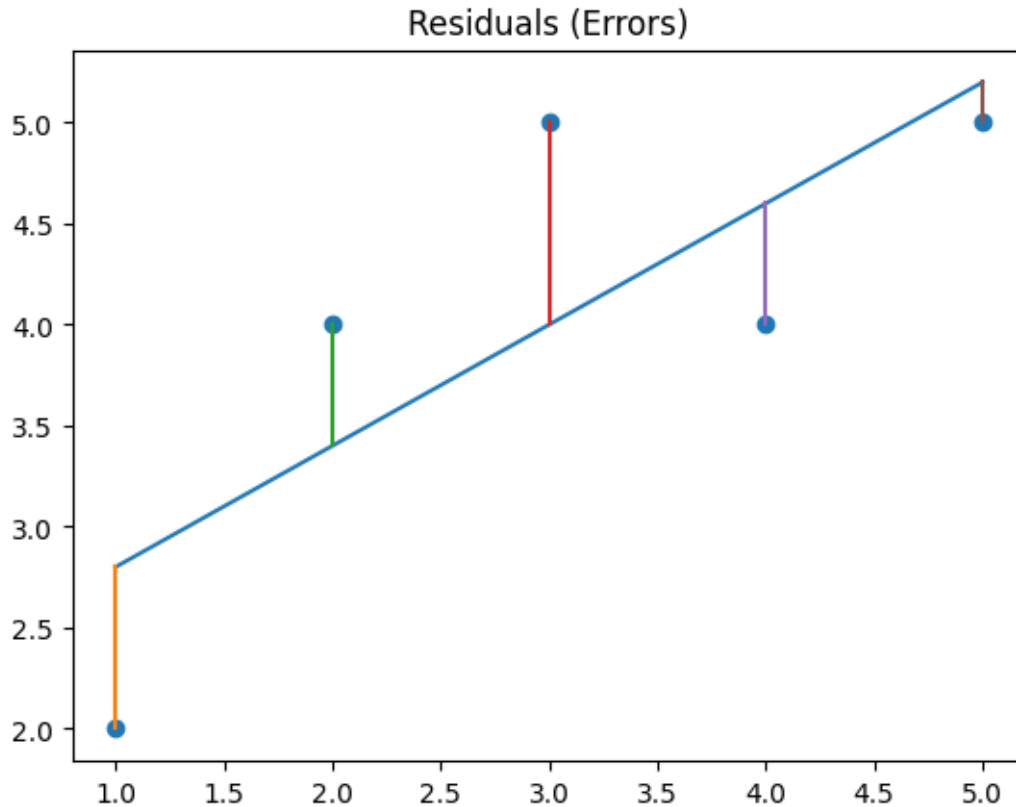
$$\text{error} = y_i - \hat{y}_i$$

To combine all errors, we use **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Why square the error? - Ensures all values are positive - Penalizes larger errors more heavily





```
Residuals: [-0.8 0.6 1. -0.6 -0.2]
Squared Errors: [0.64 0.36 1. 0.36 0.04]
MSE: 0.47999999999999987
```

At this stage, regression becomes more than just drawing a line.

It becomes an **optimization problem**:

Instead of manually choosing a line, we now aim to **find the parameters of the model (e.g., slope and intercept)** that minimize the overall prediction error.

In other words, we are solving:

Find the parameter values that **minimize the cost function**.

This shift from visualization to optimization is fundamental. It connects regression to a much broader class of machine learning algorithms, where learning is framed as minimizing a well-defined objective function.

### 127.6.7 Optimization in Linear Regression

To minimize the cost function (typically Mean Squared Error), we use two main approaches:

The model adjusts parameters to minimize MSE using: - Gradient Descent (iterative approach) - Normal Equation (closed-form solution for linear regression)

## Gradient Descent (Iterative Method)

- Starts with initial guesses for parameters
- Updates parameters step-by-step
- Moves in the direction that reduces error
- Continues until convergence

Key idea: Move **opposite to the gradient** to reach the minimum.

### 127.6.8 Gradient Descent Visualization

*[Image omitted in PDF — see the online textbook]*

Gradient descent iteratively updates parameters by moving in the direction of steepest decrease in the cost function.

---

## 127.7 Gradient Descent: Pseudocode

The following pseudocode describes how gradient descent iteratively updates model parameters to minimize the cost function.

### 127.7.1 General Gradient Descent Algorithm

**Input:** - Training data  $(X, y)$

- Learning rate  $\alpha$

- Number of iterations (or convergence condition):  $T$

**Initialize:** - Set initial values for parameters (e.g., slope  $m$  and intercept  $b$ ):  $m = 0, b = 0$

**Repeat until convergence:** (For  $t = 1$  to  $T$ .)

1. Compute predictions using current parameters  
 $\hat{y} = f(X)$
2. Compute the error (residuals)  
 $error = y - \hat{y}$
3. Compute gradients of the cost function with respect to parameters
  - Gradient w.r.t slope

$$dm = -\frac{2}{n} \sum X \cdot error$$

- Gradient w.r.t intercept

$$db = -\frac{2}{n} \sum error$$

here:  $dm$  is gradient with respect to  $m$  and  $db$  is gradient with respect to  $b$  4. Update parameters using the gradients

$$m = m - \alpha * (dm)$$

$$b = b - \alpha * (db)$$

5. Optionally compute cost (MSE) to monitor progress

**Output:** - Final optimized parameters ( $m$ ,  $b$ ): return final values of  $m$  and  $b$

At each step: - The gradient tells us **which direction increases error** - We move in the **opposite direction** to reduce error

**\*\* Stopping Criteria: \*\***

The algorithm stops when: - Change in cost becomes very small, or  
- A fixed number of iterations is reached

---

**Why Optimization Matters?** This formulation allows us to: - Systematically improve predictions

- Scale to complex models
- Apply the same idea across many ML algorithms

## 127.8 Learning Rate ( $\alpha$ )

A key component of gradient descent is the **learning rate**, usually denoted by  $\alpha$ .

The learning rate controls **how large each update step is** when adjusting the model parameters.

**Why Learning Rate Matters** At each step, gradient descent updates parameters as:

$$\text{New parameter} = \text{Old parameter} - \alpha \times (\text{gradient})$$

The value of  $\alpha$  determines how quickly (and how well) the model learns.

### 127.8.1 Effects of Different Learning Rates

- **Too small ( $\alpha$  is very low)**
  - Updates are tiny
  - Convergence is very slow
  - Training may take many iterations
- **Too large ( $\alpha$  is very high)**
  - Steps may overshoot the minimum
  - The model may oscillate or diverge
  - May never converge
- **Well-chosen learning rate**

- Efficient convergence
- Stable updates
- Faster training

*[Image omitted in PDF — see the online textbook]*

Different learning rates lead to very different behaviors: slow convergence, optimal convergence, or divergence.

**Intuition** You can think of gradient descent like walking downhill:

- Learning rate = **step size**
- Small steps → slow but safe
- Large steps → fast but risky

The goal is to take steps that are **large enough to move quickly**, but **small enough to avoid missing the minimum**.

In practice, common learning rates include: 0.001, 0.01, 0.1, 0.05 etc. Often, the best value is found through experimentation or validation.

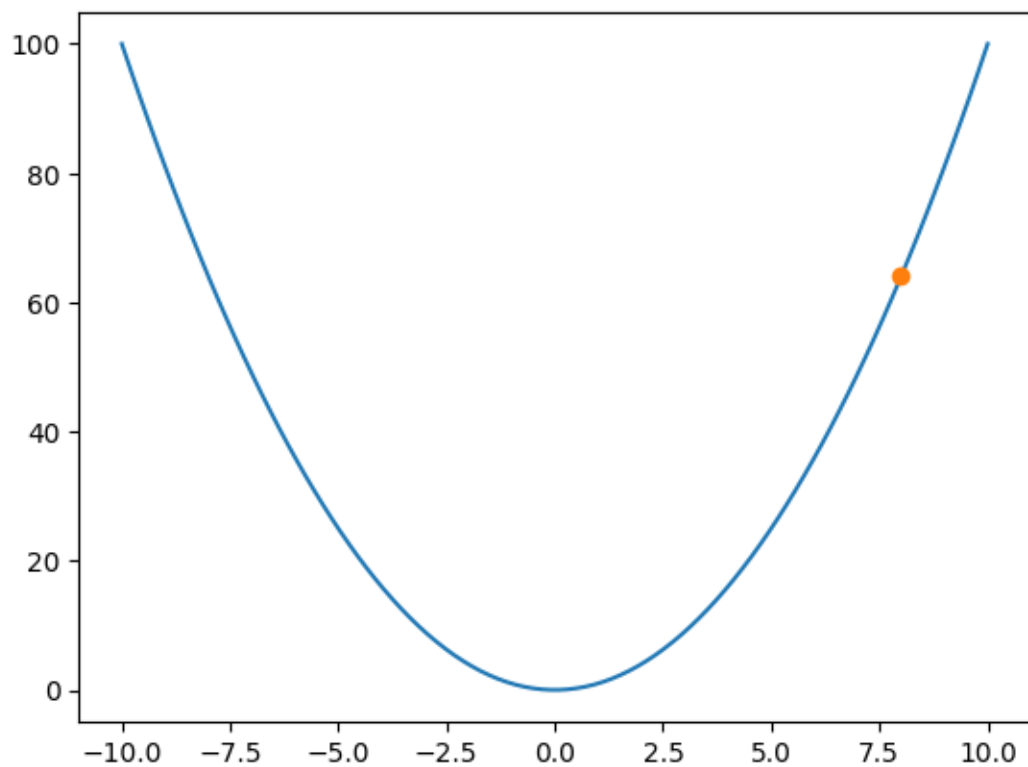
## 127.8.2 Summary:

- Gradient tells us **which direction to move**
- Learning rate tells us **how far to move**

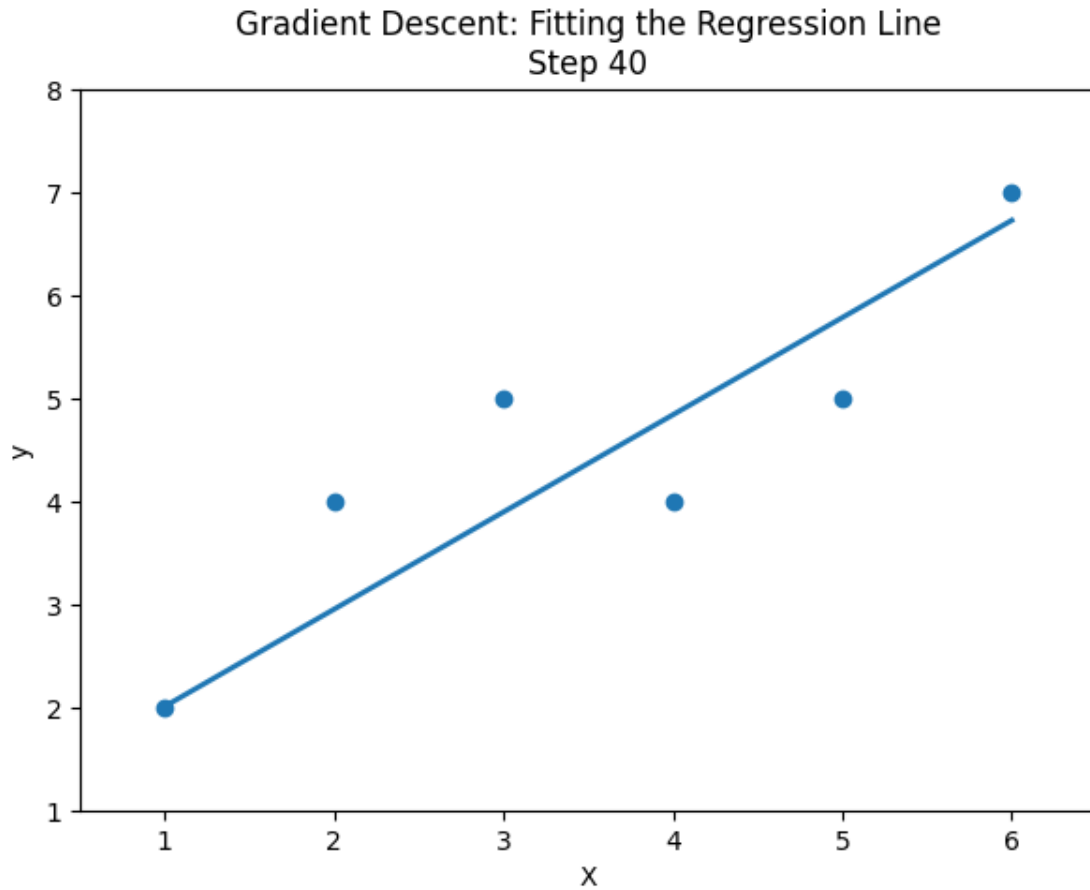
Both are essential for successfully minimizing the cost function.

<IPython.core.display.HTML object>





<IPython.core.display.HTML object>



'\nThis comparison shows why the learning rate matters:\n- A very small learning rate moves slowly and takes many steps.\n- A well-chosen learning rate reaches the minimum efficiently.\n- A very large learning rate overshoots and may fail to converge.\n'



### 127.8.3 Connecting Back to Regression

In linear regression:

- The parameters are slope ( $m$ ) and intercept ( $b$ )
- Gradient descent updates these parameters
- Each update reduces prediction error

This is how the model “learns” from data.

## 128 Assumptions of Linear Regression (L.I.N.E)

For linear regression to produce reliable and interpretable results, certain assumptions must hold.

A helpful way to remember them is **L.I.N.E**:

- **L — Linearity**
- **I — Independence of errors**
- **N — Normality of errors**
- **E — Equal variance (homoscedasticity)**

### 128.0.1 1. Linearity

The relationship between the input variables and the target should be linear.

If the true relationship is curved, a linear model will underfit the data.

### 128.0.2 2. Independence of Errors

Residuals (errors) should not be correlated with each other.

This is especially important in time-series data, where errors can depend on previous observations.

### 128.0.3 3. Normality of Errors

Residuals should be approximately normally distributed.

This assumption is particularly important for:

- statistical inference

- confidence intervals

- hypothesis testing

### 128.0.4 4. Equal Variance (Homoscedasticity)

The spread of residuals should remain constant across all values of the input.

If variance changes (heteroscedasticity), predictions may become unreliable.

### 128.0.5 No Multicollinearity (for Multiple Regression)

Input features should not be highly correlated with each other.

High correlation between features can: - distort coefficient estimates  
- make interpretation unstable

**Why Assumptions Matter** When these assumptions are violated:

- The model may still produce predictions
- But interpretations (coefficients, significance) become unreliable
- Confidence in results decreases

In practice: - Always check residual plots

- Look for patterns, trends, or changing variance

---

**Summary:** For linear regression to work well, certain assumptions are made:

1. **Linearity**

The relationship between input and output is linear.

2. **Independence of errors**

Errors are not correlated with each other.

3. **Homoscedasticity**

The variance of errors is constant across all values of  $x$ .

4. **Normality of errors**

Errors are normally distributed.

5. **No multicollinearity (important for multiple regression)**

Features should not be highly correlated with each other.

Violating these assumptions may lead to unreliable predictions.

---

## 129 Evaluation Metrics

To evaluate regression models, we use several metrics:

### 129.1 Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- Penalizes large errors more
- Commonly used in optimization

### 129.1.1 Why Mean Squared Error (MSE)?

MSE is widely used because:

- It strongly penalizes large mistakes
- It leads to a smooth optimization surface
- It has a closed-form solution (OLS)
- It works well with gradient-based optimization

### 129.1.2 Alternative Error Measures (Brief Insight)

Although MSE is standard, other options exist:

- **MAE (Mean Absolute Error)** → treats all errors equally
- **RMSE** → same unit as target variable

Each metric emphasizes different aspects of model performance.

## 129.2 Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- Easy to interpret
- Treats all errors equally

## 129.3 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

- Same unit as the target variable
- Easier to interpret than MSE

## 129.4 R-squared ( $R^2$ )

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- Measures how well the model explains variance
- Value ranges from 0 to 1
  - 1 → perfect fit
  - 0 → no explanatory power (Model explains nothing (same as predicting the mean))

### Understanding the Components Total Sum of Squares ( $SS_{tot}$ )

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

- Captures the **total variation** in the data

- Measures how far actual values are from the mean
- Represents the baseline (no model)

### Residual Sum of Squares ( $SS_{res}$ )

$$SS_{res} = \sum (y_i - \hat{y}_i)^2$$

- Captures the **unexplained variation**
- Measures how far predictions are from actual values
- Represents model error

#### 129.4.1 Intuition

- ( $SS_{tot}$ ): Total variability in data
- ( $SS_{res}$ ): Remaining error after modeling

$$R^2 = \text{Explained Variation} = 1 - \frac{\text{Unexplained}}{\text{Total}}$$

Remember:

- Smaller  $SS_{res} \rightarrow$  better fit  $\rightarrow$  higher  $R^2$
- If  $SS_{res} = 0$ , then  $R^2 = 1$  (perfect model)

#### 129.4.2 Simple Takeaway

$R^2$  tells us what fraction of the data's variability (spread) is explained by the model.

*# Evaluating the Model*

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
mse = mean_squared_error(y_test, predictions)
```

```
r2 = r2_score(y_test, predictions)
```

```
print("MSE:", mse)
```

```
print("R2 Score:", r2)
```

## 130 Bias-Variance Tradeoff

A key concept in regression (and ML in general):

- **High Bias**  $\rightarrow$  Model is too simple  $\rightarrow$  Underfitting
- **High Variance**  $\rightarrow$  Model is too complex  $\rightarrow$  Overfitting

### 130.0.1 Goal:

Find a balance where the model generalizes well to unseen data.

## 130.1 Overfitting and Regularization

### 130.1.1 Overfitting

Occurs when the model: - Fits training data extremely well - Fails on new (test) data

### 130.1.2 Solutions

1. Reduce model complexity
2. Use fewer features
3. Apply **regularization**

## 131 Regularization Techniques

### 131.1 4. Regularized Regression (Controlling Overfitting)

As models become more complex (especially with many features), they may start overfitting.

Regularization helps control this by penalizing large coefficients.

#### 131.1.1 Ridge Regression (L2)

- Adds penalty on squared coefficients

$$\lambda \sum w_i^2$$

- Shrinks coefficients; Keeps all features
  - Shrinks weights but does not remove features

Helps when: - Many features exist

- Features are slightly correlated

#### 131.2 Lasso Regression (L1)

Adds penalty on absolute coefficients:

$$\lambda \sum |w_i|$$

- Can shrink some coefficients to exactly zero

This makes it useful for: - Feature selection

- Simplifying models

### 131.2.1 Elastic Net

- Combines Ridge and Lasso
- Useful when features are highly correlated

```
Implementation of Ridge and Lasso
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

Ridge and Lasso
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```

### 131.3 Worked Example (Conceptual)

Suppose we have:

Size (sq ft)	Price (\$)
1000	200,000
1500	300,000
2000	400,000

We try to fit a line:

$$Price = m \cdot Size + b$$

The model finds: - A slope  $m$  that captures how price changes with size - An intercept  $b$  that anchors the line

Once trained, we can predict: - Price of a 1800 sq ft house - Price of unseen data points

### 131.4 Applications of Regression

Regression is widely used across domains:

- **Finance:** stock price prediction, risk modeling



- **Marketing:** sales forecasting
- **Healthcare:** predicting disease progression
- **Economics:** demand estimation
- **Engineering:** system modeling

## 132 Summary of the Chapter

Regression provides a simple yet powerful way to model relationships between variables.

Key takeaways: - It predicts continuous outcomes - Linear regression is the foundation - Model quality depends on assumptions and evaluation - Regularization helps prevent overfitting - Understanding interpretation is as important as prediction

Remember that, Regression is not just about fitting models, it is about understanding relationships in data. A good regression model should:

- Capture the true trend
- Generalize to new data
- Remain interpretable

As you work with real datasets, you will realize that choosing the right type of regression is as important as training the model itself.

### 132.1 Knowledge Check

*[Interactive content omitted in PDF]*

## 133 Chapter 17 - Clustering and PCA

## 134 Unsupervised Learning: Discovering Patterns Without Answers

### 134.1 A Different Kind of Learning

So far, most of our machine learning tasks follow a familiar pattern:

- Inputs (features)
- Outputs (labels)

The model learns a mapping from input  $\rightarrow$  output. This is called **supervised learning**. But now consider a different situation.

You are given a dataset with many data points and features—but **no labels**. No correct answers. No categories. Just raw data. This raises an important question:

**Can we learn something meaningful just from the structure of the data itself?**

*[Image omitted in PDF — see the online textbook]*

Figure: Unsupervised learning groups unlabeled data into patterns based on similarity.

Source: MathWorks

This is where **unsupervised learning** begins.

## 134.2 2. The Problem of Too Many Features

Consider analyzing customer behavior with features like: - Age

- Income

- Purchase frequency

- Website activity

Now scale this to: - 50, 100, or even 500 features

We quickly face challenges:

- Visualization becomes impossible
- Computation slows down
- Many features are redundant or noisy
- Relationships become hard to interpret

The data exists in a **high-dimensional space**, beyond our intuitive understanding. So we ask:

**Can we simplify the data while preserving its important information?**

## 135 Part A: Dimensionality Reduction

Dimensionality reduction addresses this challenge.

- It reduces the number of features while preserving the most important structure.

Think of it like: - Summarizing a long story

- Compressing an image

Less data, but same essential meaning

One of the most powerful techniques for this is: **Principal Component Analysis (PCA)**

### 135.1 The Curse of Dimensionality

Before PCA, we must understand why high dimensions are problematic.

#### 135.1.1 The Curse of Dimensionality

*[Image omitted in PDF — see the online textbook]*

Figure: Hughes Phenomenon: Adding features helps initially, but too many features with limited data reduces performance.

Source: Medium

As dimensions increase:

### 135.1.2 1. Data Becomes Sparse

Points spread far apart, making the space mostly empty.

### 135.1.3 2. Distance Becomes Less Meaningful

- Nearest and farthest points become similar
- Hard to distinguish similarity

*[Image omitted in PDF — see the online textbook]*

Figure: Distances lose meaning as dimensionality increases.

Source: Medium

### 135.1.4 3. More Data is Required

High dimensions require exponentially more data to learn effectively.

### 135.1.5 4. Noise and Redundancy Increase

- Irrelevant features
- Correlated features
- Added noise

## 135.2 Intuition

- 2D → easy to understand
- 3D → harder
- 100D → nearly impossible to reason about

High-dimensional space behaves very differently from what we expect.

## 135.3 Why This Matters

Because of these effects: - Model performance can degrade  
- Computation becomes inefficient  
- Interpretation becomes difficult

This is why we use **dimensionality reduction**

Reduce complexity while preserving structure

And one of the most important tools for this is: **PCA**

## 136 What is Principal Component Analysis (PCA)?

Principal Component Analysis (PCA) is an **unsupervised feature extraction technique used for dimensionality reduction**, where new features are created to preserve the most important information in the data. PCA: - Reduces the number of features

- Preserves most of the variance
- Transforms correlated variables into a smaller set of **uncorrelated principal components (PCs)**

Each principal component is: - A linear combination of original features

- Uncorrelated with others
- Ordered by importance (variance captured)

### 136.1 PCA: Looking at Data from a Better Angle

**How can we represent data in fewer dimensions while losing the least information?**

Imagine data as a cloud of points. By rotating our view, we can find a direction where the data spreads the most.

This direction is **PC1** (captures maximum variance)

Next, PCA finds another direction that: - Is **perpendicular (orthogonal)** to PC1

- Captures the next highest variance

This becomes **PC2**

*[Image omitted in PDF — see the online textbook]*

Figure: PCA rotates axes to align with directions of maximum variance.

Source: Machine Learning Plus

Each additional component captures the next most important variation while remaining orthogonal to previous ones.

### 136.2 Why This Idea Works

The key intuition behind PCA is:

**Variance represents information**

- Low variance → little information
- High variance → meaningful patterns

PCA keeps high-variance directions and removes low-variance noise.

### 136.3 Removing Redundancy

Real-world features are often correlated (e.g., income & spending).

PCA: - Combines correlated features

- Produces **uncorrelated components**
- Removes redundancy

## 136.4 The Underlying Mechanism

PCA is based on three key ideas:

- **Covariance matrix** → captures relationships between features
- **Eigenvectors** → directions of maximum variance
- **Eigenvalues** → importance of each direction

These define a new coordinate system aligned with the most informative directions.

PCA **rotates the data to align with its most informative directions**, allowing us to reduce dimensions while preserving structure. →

## 136.5 Mathematical Idea (Core of PCA)

$$Z = XW$$

Where: - (X) = original data matrix  
- (W) = matrix of eigenvectors (principal directions)  
- (Z) = transformed data (principal components)

PCA projects data onto directions where **variance is maximized**.

## 136.6 Principal Components, Eigenvectors, and Eigenvalues

Each principal component is a **linear combination** of original features.

Example:

$$PC_1 = v_{11}X_1 + v_{12}X_2 + v_{13}X_3$$

$$PC_2 = v_{21}X_1 + v_{22}X_2 + v_{23}X_3$$

Where: - v = weights (from eigenvectors)  
- X = original features

These weights tell us how much each original feature contributes to a given principal component.

**Eigenvectors** define the directions (principal components)

**Eigenvalues** measure how much variance each component captures

- Larger eigenvalue = more important component

*[Image omitted in PDF — see the online textbook]*

Figure: PCA projects data onto new axes that capture maximum variance.

Source: Medium

## 136.7 Key Properties of Principal Components

- Principal components are **orthogonal**, meaning they are uncorrelated
- **PC1** captures the maximum variance
- **PC2** captures the next highest variance
- Each additional component captures less variance than the previous one

## 137 Step-by-Step: How PCA Works

Conceptually, PCA follows these steps:

1. **Standardize the data**  
so that all features contribute equally
2. **Compute the covariance matrix**  
to understand how features vary together
3. **Find eigenvectors and eigenvalues**
  - Eigenvectors give the directions
  - Eigenvalues give the importance of those directions
4. **Sort the components by importance**  
so the directions with the highest variance come first
5. **Select the top (k) components**  
based on how much variance you want to preserve
6. **Project the data into the new space**  
to obtain a lower-dimensional representation

*[Image omitted in PDF — see the online textbook]*

Figure: PCA identifies new axes (principal components) that align with directions of maximum variance in the data. Source: Devopedia.

In this chapter, we will use a Irish dataset throughout having 150 samples and 4 features; small enough to inspect, structured enough to be interesting. Later we will also see how to implement PCA simply using python sklearn library.

```
Dataset loaded: (150, 4) → {np.str_('setosa'): np.int64(50),
np.str_('versicolor'): np.int64(50), np.str_('virginica'): np.int64(50)}
```

### 137.1 Step 1: Standardize the Data

PCA is sensitive to scale. A feature measured in kilometres will dominate one measured in grams purely because of its larger numbers, not because it is more informative.

### 137.1.1 Mean-centring

Subtract the mean of each feature so the data cloud sits at the origin:

$$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$$

### 137.1.2 Standardisation (z-score)

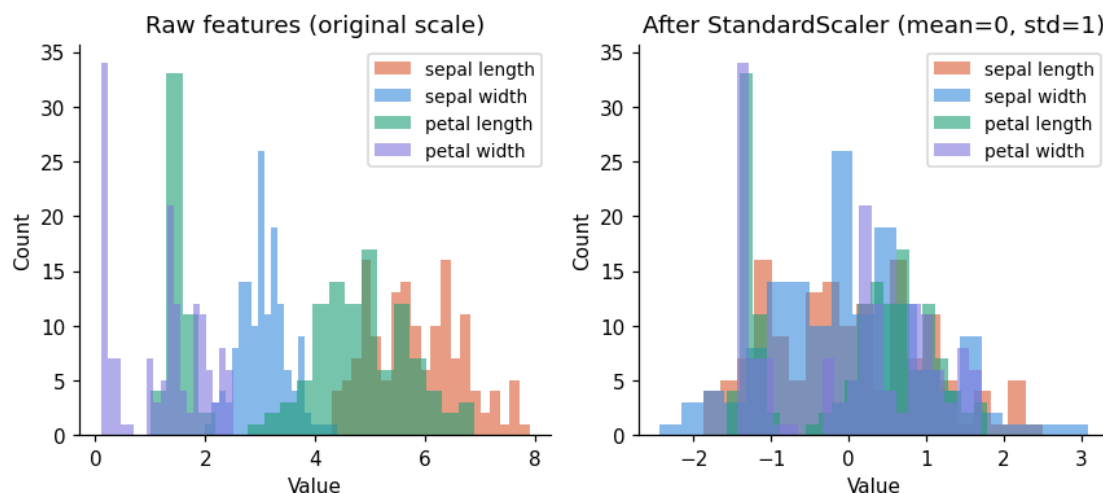
Divide by the standard deviation so all features have unit variance:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$$

Remember: **Always standardise before PCA** unless your features are already on the same scale and you explicitly want to weight by raw variance.

Once we standardize, **mean becomes 0** and **standard deviation becomes 1**; ensures all features contribute equally.

#### Step 1 — Effect of standardisation



Means (should be ~0): [-0. -0. -0. -0.]

StdDevs (should be ~1): [1. 1. 1. 1.]

### 137.2 Step 2: Compute Covariance Matrix

Before PCA can find anything, it needs to understand how your features relate to each other. This is the job of the **covariance matrix C**.

**The covariance matrix** tells us **how features vary together**.

Sign	Meaning
<b>Positive</b>	Features increase together

Sign	Meaning
<b>Negative</b>	One rises as the other falls
<b>Zero</b>	No linear relationship

This helps PCA understand relationships between features.

### 137.2.1 Formula

For a dataset  $X$  with  $n$  samples and  $p$  features (already **mean-centred**), the covariance matrix is:

$$C = \frac{1}{n-1} X^T X$$

It is a  $p \times p$  **symmetric** matrix where:

- **Diagonal entries**  $C_{ii}$  = variance of feature  $i$
- **Off-diagonal entries**  $C_{ij}$  = covariance between feature  $i$  and feature  $j$
- **Symmetry**:  $C_{ij} = C_{ji}$  always

For a 3-feature dataset it looks like:

$$C = \begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \text{Cov}(x_1, x_3) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \text{Cov}(x_2, x_3) \\ \text{Cov}(x_3, x_1) & \text{Cov}(x_3, x_2) & \text{Var}(x_3) \end{bmatrix}$$

Entry	Meaning
Diagonal $C_{ii}$	Variance of feature $i$
Off-diagonal $C_{ij} > 0$	Features move together
Off-diagonal $C_{ij} < 0$	Features move oppositely
Off-diagonal $C_{ij} = 0$	No linear relationship

**Key insight:** If features are correlated, there are directions in the data space that capture more variance than any single original axis. PCA finds exactly those directions via **eigendecomposition of C**.

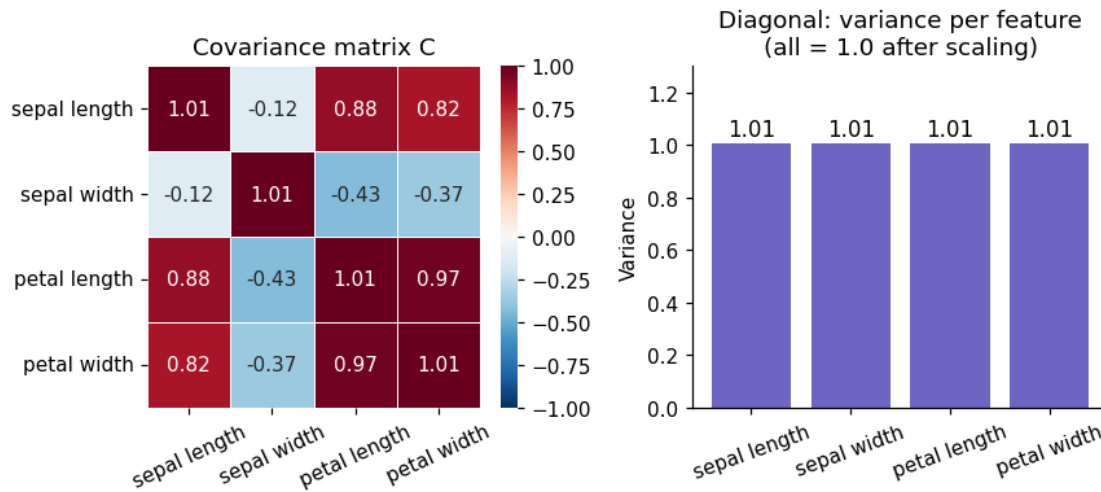
Covariance matrix C (4×4):

```
[[1.007 -0.118 0.878 0.823]
 [-0.118 1.007 -0.431 -0.369]
 [0.878 -0.431 1.007 0.969]
 [0.823 -0.369 0.969 1.007]]
```

Max difference vs np.cov: 4.440892098500626e-16



## Step 2 — Covariance matrix



### 137.3 Step 3: Eigendecomposition

The core of PCA is solving the **eigenvalue problem**:

$$C\mathbf{v} = \lambda\mathbf{v}$$

Where:

Symbol	Name	Role
$C$	Covariance matrix	Encodes all feature relationships
$\mathbf{v}$	<b>Eigenvector</b> (direction)	A principal component axis in feature space
$\lambda$	<b>Eigenvalue</b> (importance)	Variance captured along that direction

Eigenvectors are the directions the covariance matrix only **stretches**, never rotates. Eigenvalues tell you **how much** it stretches — i.e. how much variance lies along that direction.

This gives us our **principal components**, a new set of axes aligned with the natural structure of the data, ordered by how much they explain.

#### 137.3.1 Properties

- There are exactly  $p$  eigenpairs for a  $p \times p$  matrix
- Eigenvectors are **orthogonal** (perpendicular) to each other; PCs are uncorrelated by construction

- Sort by  $\lambda$  descending: PC1 = maximum variance, PC2 = second most, etc.
- $\sum_i \lambda_i = p$  (total variance equals number of features, after standardisation)

### 137.4 Step 4: Sort Components

Eigendecomposition returns  $p$  eigenpairs in arbitrary order. We sort them so the most informative components come first:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$$

- **Sort eigenvalues** in descending order
- **Reorder eigenvectors** to match (each eigenvector travels with its eigenvalue)
- **Select top  $k$  eigenvectors** — these are your  $k$  principal components

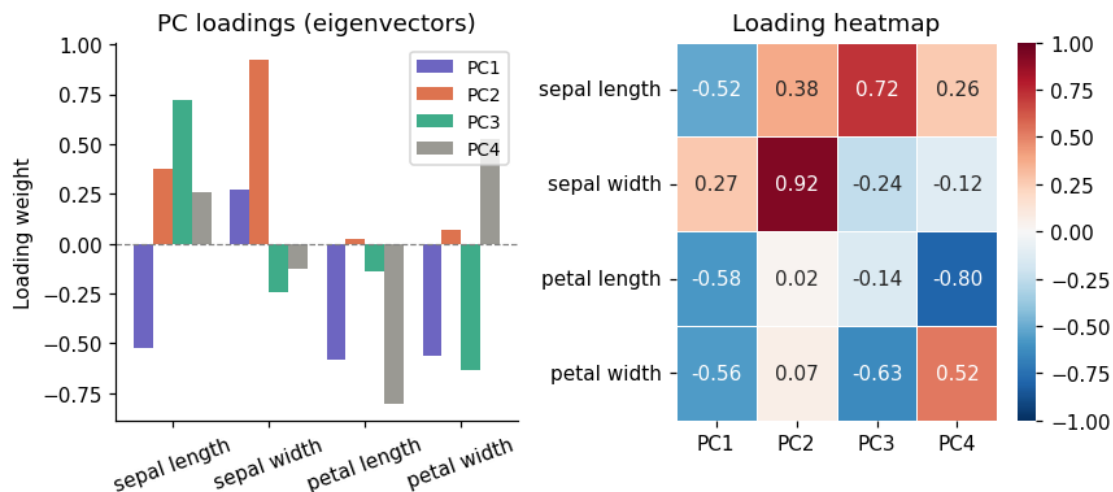
Most important directions first. PC1 always captures the most variance, PC2 the second most, and so on.

Eigenvalues (= variance per PC):

```
PC1: = 2.9381 (73.0% of variance)
 loadings: [-0.521 0.269 -0.58 -0.565]
PC2: = 0.9202 (22.9% of variance)
 loadings: [0.377 0.923 0.024 0.067]
PC3: = 0.1477 (3.7% of variance)
 loadings: [0.72 -0.244 -0.142 -0.634]
PC4: = 0.0209 (0.5% of variance)
 loadings: [0.261 -0.124 -0.801 0.524]
```

Sum of eigenvalues = 4.0 (=  $p = 4$ , checks out)

Step 3 — Eigendecomposition: principal component loadings

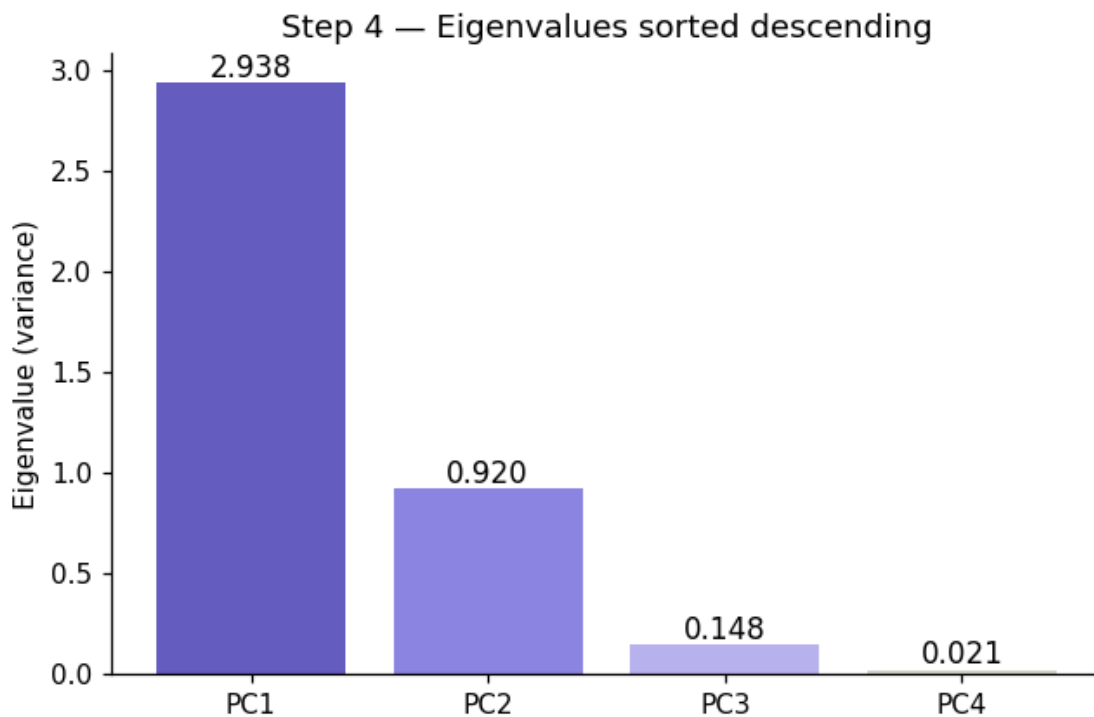


Before sorting (eigh returns ascending order):

pair 1: = 0.0209  
pair 2: = 0.1477  
pair 3: = 0.9202  
pair 4: = 2.9381

After sorting (descending):

PC1: = 2.9381 ← most important  
PC2: = 0.9202 ← next important  
PC3: = 0.1477 ← next important  
PC4: = 0.0209 ← least important



### 137.5 Step 5: Choose Number of Components (k)

We don't always keep all  $p$  components; only those that preserve most of the variance. How many components should you keep? We need to know the **explained variance ratio** of PC  $i$ .

#### 137.5.1 Explained Variance Ratio (EVR)

Measures how much variance **each individual PC** explains:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

Example: - PC1 = 50% - PC2 = 30% - PC3 = 20%

### 137.5.2 Cumulative Explained Variance (CEV)

Total variance retained by the **first  $k$  components combined**:

$$\text{CEV}(k) = \sum_{i=1}^k \text{EVR}_i = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^p \lambda_j}$$

Example: - First 2 PCs = 80%

- If threshold = 80% → **keep 2 PCs**

### 137.5.3 Rules of thumb for choosing $k$

Rule	Description
<b>Scree plot elbow</b>	Pick $k$ where the curve bends — adding more PCs gives diminishing returns
<b>Variance threshold</b>	Keep enough PCs to explain 95% (or 90%) of variance
<b>Kaiser criterion</b>	Keep PCs with $\lambda_i > 1$ (captures more than one average feature's worth of variance)
<b>Downstream task</b>	For K-Means clustering, 2–10 components is usually sufficient

## 137.6 Step 6: Transform the Data (Projection to Lower Dimensions)

Once you have selected the top  $k$  eigenvectors, **project** the data onto them:

$$Z = X \cdot W$$

Where: -  $X$  is the standardised data matrix — shape  $(n \times p)$  -  $W = [\mathbf{v}_1 \mid \mathbf{v}_2 \mid \dots \mid \mathbf{v}_k]$  — the top  $k$  eigenvectors stacked as columns, shape  $(p \times k)$  -  $Z$  is the **reduced data matrix** — shape  $(n \times k)$

### 137.6.1 What this achieves

Before	After
$n$ samples $\times$ $p$ original features	$n$ samples $\times$ $k$ principal components
Features may be correlated	Components are uncorrelated by construction
High-dimensional, noisy	Lower-dimensional, most information preserved

The projection is a **rotation** of the coordinate axes; no information is added, only the least-variant dimensions are discarded.

### 137.7 Benefits and Limitations of PCA

PCA reduces dimensionality, improves computational efficiency, and makes data easier to visualize (especially in 2D or 3D). By removing correlated features, it can also help reduce overfitting.

However, PCA assumes linear relationships, and the resulting components can be difficult to interpret. Since it is unsupervised, it does not consider output labels and may lose some information during compression.

PCA is one of the most important tools for: - Data preprocessing

- Noise reduction

- Feature extraction

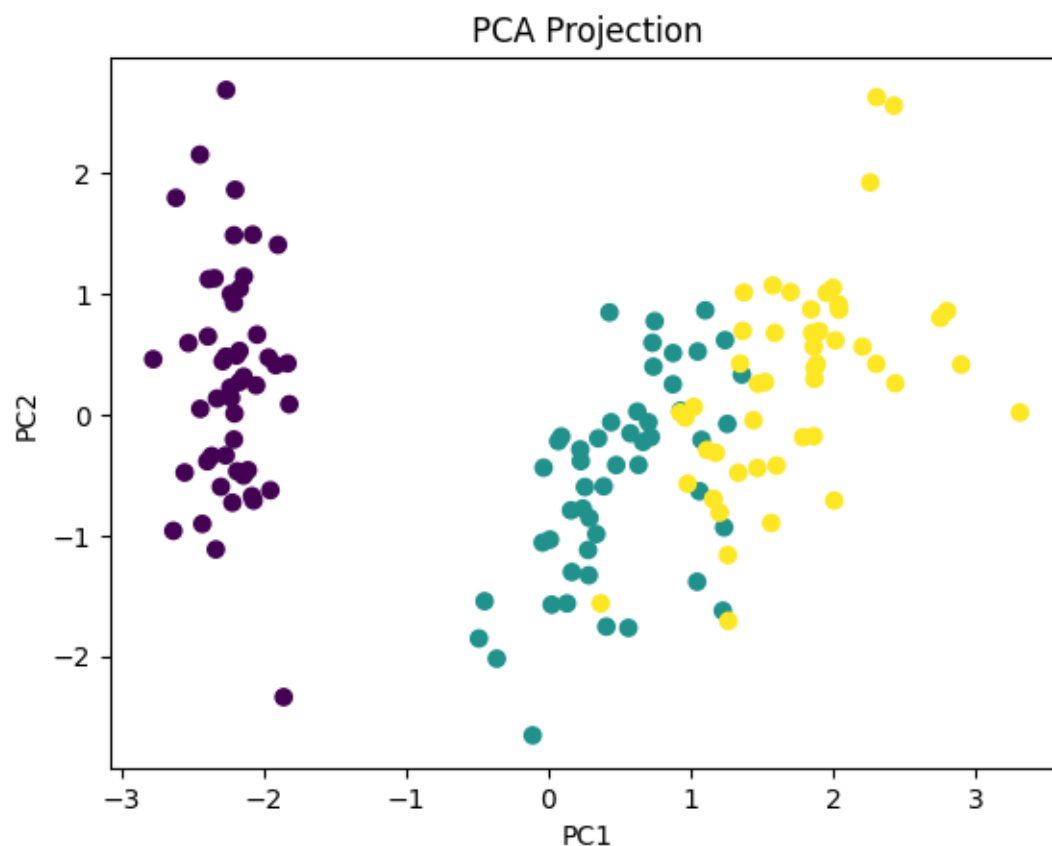
- Visualization

```
<IPython.core.display.HTML object>
```

```
#PCA in Practice (using Sciklearn)
```

```
Explained variance ratio: [0.72962445 0.22850762]
```

```
Total variance retained: 0.9581320720000166
```



What We Achieved We reduced: From 4 dimensions  $\rightarrow$  2 dimensions. While still preserving most of the structure. This is why PCA is so powerful.

## 138 Part B: From Understanding to Grouping

Now that we have simplified the data using dimensionality reduction, we move to the next natural question:

**Can we automatically group similar data points together?**

This leads us to: **Clustering**

### 138.1 Clustering: Letting Data Form Its Own Groups

Clustering is about:

- Finding natural groupings in data
- Without using any labels

Instead of being told what belongs where, the model discovers patterns on its own. It learns: - Which points are similar  
- Which points are different

### 138.2 K-Means: A Simple Clustering Algorithm

One of the most widely used clustering algorithms is: **K-Means**. K-Means divides the dataset into **K clusters**. Each cluster has a center called a **centroid**

$$\min \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Where: - K = number of clusters

-  $C_i$  = set of points in cluster (i)

-  $\mu_i$  = centroid (mean) of cluster (i)

-  $\|x - \mu_i\|^2$  = squared distance between a point and its centroid

K-Means tries to **minimize the total squared distance** between data points and their assigned cluster centers. This quantity is also known as: **Inertia** or **WCSS (Within-Cluster Sum of Squares)**.

*[Image omitted in PDF — see the online textbook]*

Figure: Illustration of K-Means objective showing compact clusters (low intra-cluster distance) and well-separated clusters (high inter-cluster distance). Source: Medium

**WCSS** is the sum of squared distances from each point to its assigned centroid.

- It measures how compact the clusters are
- Lower value  $\rightarrow$  better clustering

### 138.3 Intra-cluster vs Inter-cluster

#### 138.3.1 (a) Intra-cluster Distance (Within Cluster)

- Distance between points and their centroid
- Should be **small**
- Measures how compact each cluster is

K-Means **minimizes this**

#### 138.3.2 (b) Inter-cluster Distance (Between Clusters)

- Distance between different cluster centroids
- Should be **large**
- Ensures clusters are well separated

K-Means does not explicitly maximize this,  
but minimizing intra-cluster distance indirectly improves separation.

---

Remember: Good clustering means: - Points within a cluster are **close together** (low intra-cluster distance)  
- Different clusters are **far apart** (high inter-cluster separation)  
\*\*\*

### 138.4 How K-Means Works (Step-by-Step)

*[Image omitted in PDF — see the online textbook]*

Figure: K-Means clustering process showing iterative steps: initializing centroids, assigning points to nearest centroid, and updating centroids until convergence. Source: Simplilear

Think of it like this:

1. Randomly place **K centroids**
2. Assign each data point to the nearest centroid
3. Recompute centroids (mean of assigned points)
4. Move centroids to new positions
5. Repeat the process

Eventually:

- The centroids stop moving
- Cluster assignments stop changing

This is called **convergence**

### 138.5 Intuition

- Points “pull” the centroid toward them
- Centroids adjust to best represent their cluster
- Over time, stable groups emerge

### 138.6 Key Insight

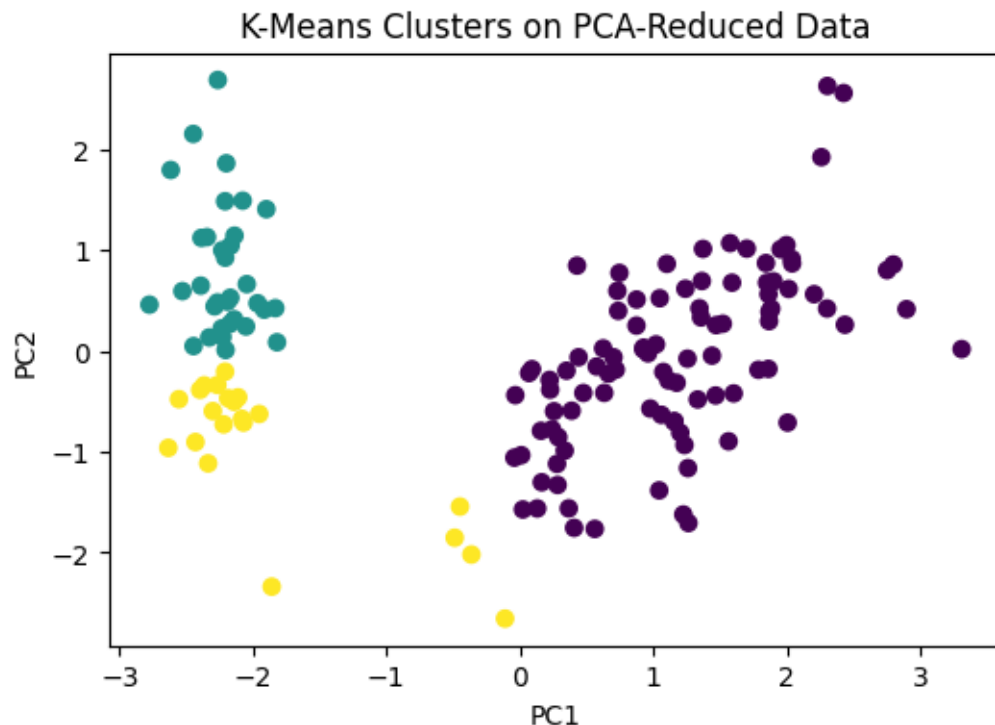
K-Means does not know anything about the data beforehand. It simply uses **distance** to group similar points together.

Now that we understand how clustering works, we can apply it to **PCA-reduced data** to: - Improve efficiency

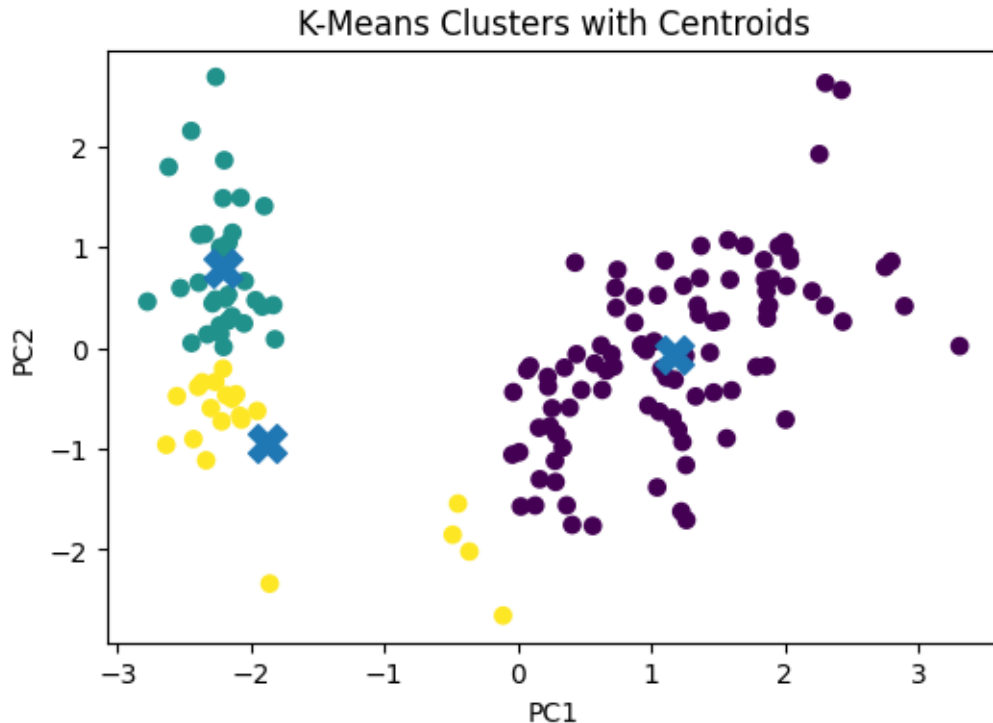
- Visualize clusters easily
- Reduce noise before grouping

## 139 K-Means in Python

Now let us apply K-Means in Python and visualize the clusters on the PCA-reduced data.







Here: Each color represents a different cluster The X markers represent the cluster centroids

## 140 Choosing the Right Value of K

One practical challenge in K-Means is deciding:

- How many clusters should we use?

The algorithm requires us to specify K in advance, but often we do not know the best value beforehand. A common way to choose K is the **Elbow Method**.

### 140.1 Elbow Method

The Elbow Method helps us determine the **optimal number of clusters (K)** in K-Means. It examines how the inertia or WCSS changes as we increase the number of clusters. It works by: -

- Running K-Means for different values of K
- Computing **WCSS (inertia)** for each K
- Plotting K vs WCSS

As **WCSS (inertia)** measures how tightly the data points are grouped around their centroids, lower inertia means points are closer to their centroids.

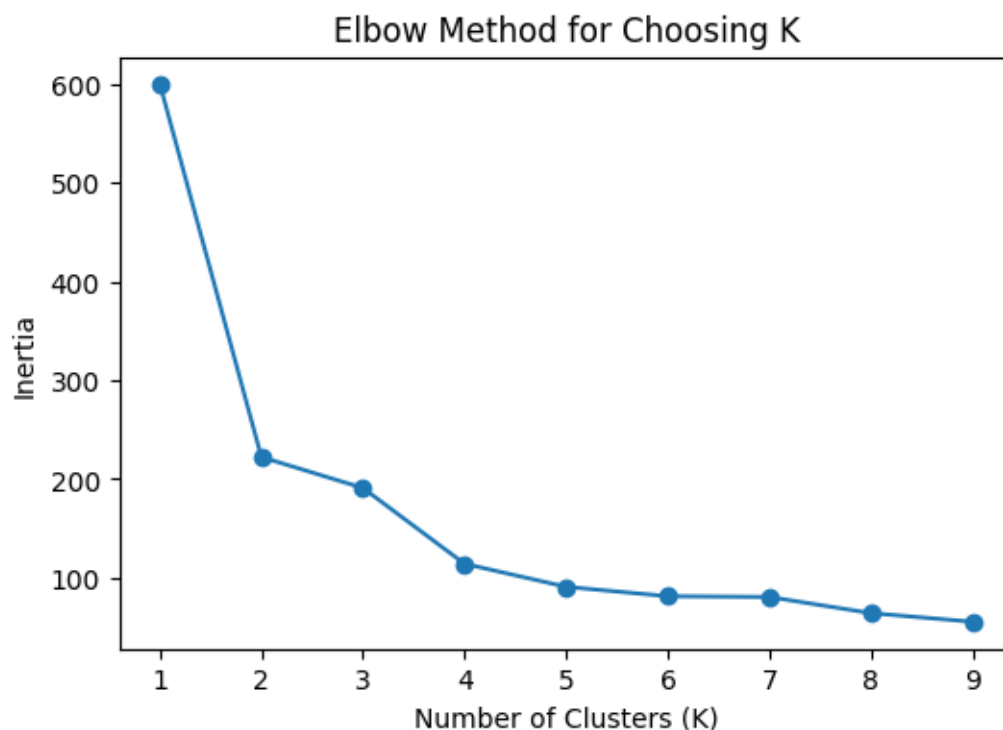
- As we increase K: - WCSS **decreases** (clusters become tighter)
- But the improvement **slows down after a point** (becomes small).

We look for the point where the curve starts to bend like an elbow.

[Image omitted in PDF — see the online textbook]

Figure: Elbow method showing WCSS (inertia) vs number of clusters (K). The “elbow point” indicates the optimal number of clusters. Source: GeeksforGeeks

The point where the curve bends is called the “**elbow**”. This is typically the best choice for K. From the above figure, notice how the curve drops sharply at first, then flattens; the curve bends sharply at  $K = 4$ , then 4 may be a reasonable number of clusters. this change in slope is the “elbow.”



## 141 Bonus Section: Silhouette Score (Evaluating Clustering Quality)

While the Elbow Method helps us choose the number of clusters using WCSS, it does not always give a clear answer. To further evaluate clustering quality, we use: **Silhouette Score**

### 141.1 What is Silhouette Score?

Silhouette Score measures how well each data point fits within its assigned cluster compared to other clusters.

For each point, it considers:

- $a(i)$  = average distance to points in the same cluster (intra-cluster distance)

- **b(i)** = average distance to points in the nearest neighboring cluster (inter-cluster distance)

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

**Interpretation:** The score ranges from **-1 to 1**:

- **Close to +1** → well-clustered (far from other clusters)
- **Close to 0** → overlapping clusters
- **Close to -1** → wrongly assigned cluster

The intuition is that, a good clustering should have: - **Small intra-cluster** distance (points close together)

- **Large inter-cluster** distance (clusters far apart)

Silhouette score combines both ideas into a single metric.

## 141.2 Using Silhouette Score to Choose K

Instead of only relying on the Elbow Method, we can:

- Compute silhouette score for different values of K
- Choose the K with the **highest silhouette score**

*[Image omitted in PDF — see the online textbook]*

*[Image omitted in PDF — see the online textbook]*

*[Image omitted in PDF — see the online textbook]*

Figure: (a) Illustrating intra-cluster compactness and inter-cluster separation, (b) clustering of data points into groups based on similarity, (c) Elbow method and Silhouette score used to evaluate clustering quality.

Sources: Medium; GeeksforGeeks

## 141.3 Python Example

```
from sklearn.metrics import silhouette_score

scores = []

for k in range(2, 10):
 km = KMeans(n_clusters=k, random_state=42)
 labels = km.fit_predict(X_scaled)
 score = silhouette_score(X_scaled, labels)
 scores.append(score)

plt.plot(range(2, 10), scores, marker='o')
plt.xlabel("K")
```

```
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs Number of Clusters")
plt.show()
```

## 141.4 Summary of K-Means

K-Means is simple and powerful, but it comes with some assumptions:

### Strengths:

- Easy to understand and implement
- Works well when clusters are compact and roughly spherical
- Fast for large datasets

**Limitations:** - Must choose K in advance - Sensitive to initialization - Can struggle when clusters have irregular shapes - Distance-based, so scaling matters

## 142 Beyond K-Means

K-Means is a very useful starting point, but it is not the only clustering method. Two important alternatives are:

- Hierarchical Clustering
- DBSCAN

### 142.1 Hierarchical Clustering

Hierarchical clustering builds a **hierarchy of clusters** instead of assigning all points at once.

#### 142.1.1 Key Idea

Instead of fixing the number of clusters beforehand, hierarchical clustering creates a **tree-like structure** of clusters. This structure is called a **dendrogram**

A dendrogram shows: - How clusters are merged or split  
- The distance at which this happens

By “cutting” the dendrogram at a certain height, we choose the number of clusters.

*[Image omitted in PDF — see the online textbook]*

Figure: Dendrogram illustrating hierarchical clustering. The vertical axis shows the distance at which clusters are merged, and cutting the tree at different heights results in different numbers of clusters.

Source: Medium

There are **two common approaches**:

- **(1) Agglomerative:** (Start small → merge upward) Start with each point as its own cluster, then repeatedly merge the closest clusters.
- **(2) Divisive:** (Start big → split downward) Start with one large cluster, then repeatedly split it

Agglomerative merges clusters step-by-step, while divisive splits clusters step-by-step, forming a hierarchy of clusters.

**(1) Agglomerative (Bottom-Up): How it works:** Start with individual points → merge clusters

> Compute all distances → pick the smallest → merge → repeat ##### Algorithm Steps:

1. **Initialize:**

Treat each data point as its own cluster

2. **Compute Distance Matrix:**

Calculate pairwise distances between all clusters

Distance between clusters depends on the **linkage method**:

- **Single Linkage:** minimum distance between any two points
- **Complete Linkage:** maximum distance between any two points
- **Average Linkage:** average distance between all pairs
- **Ward's Method:** minimizes variance within clusters

The **linkage method controls how clusters are merged**, and can significantly affect the final clustering structure.

*[Image omitted in PDF — see the online textbook]*

Figure: Comparison of linkage methods in hierarchical clustering: single, complete, average, and Ward's method.

Single linkage may create long chains, while complete linkage produces tighter and more compact clusters.

Source: Analytics Vidhya

This step gives us a **table of distances between every pair of clusters**

**Example: Distance Matrix**

	A	B	C
A	—	2	5
B	2	—	3
C	5	3	—

3. **Find Closest Clusters:**

From the distance matrix, identify the **two clusters with the smallest distance**

- Example: smallest distance = **2 (A–B)**
- So clusters **A and B** will be merged next

4. **Merge Clusters:**

Combine the selected clusters into a single cluster

5. **Update Distances:**

Recompute distances between the new cluster and all others  
(using the same linkage method)

6. **Repeat Steps 3–5**

Continue until:

- Only one cluster remains OR
- Desired number of clusters is reached

**(2) Divisive (Top-Down): How it works:** Start with one cluster → split into smaller clusters

**Algorithm Steps:**

1. **Initialize:**

Put all data points into one cluster

2. **Select Cluster to Split:**

Choose the cluster that is most heterogeneous  
(e.g., highest variance or largest size)

3. **Split Cluster:**

Divide it into two smaller or sub clusters (often using a method like K-Means).cThe goal is to maximize separation between the two new clusters

4. **Evaluate Split:**

Check if further splitting improves cluster quality (e.g., better separation or reduced variance).

5. **Repeat Steps 2–4**

Continue until:

- Each point is its own cluster OR
- Desired number of clusters is reached

Divisive clustering is less commonly used because splitting is computationally more complex than merging.

*[Image omitted in PDF — see the online textbook]*

Figure: Divisive clustering process (top-down). The algorithm starts with one large cluster and recursively splits it into smaller clusters until the desired structure is obtained.

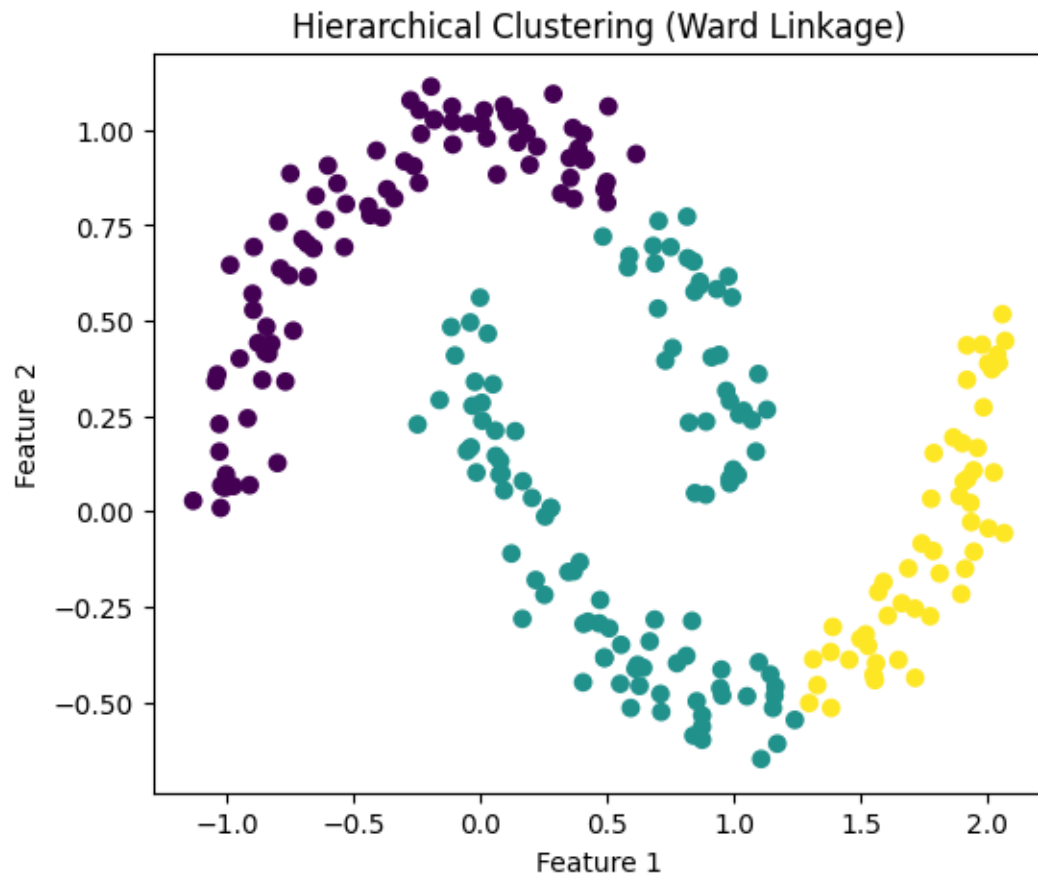
Source: GeeksforGeeks

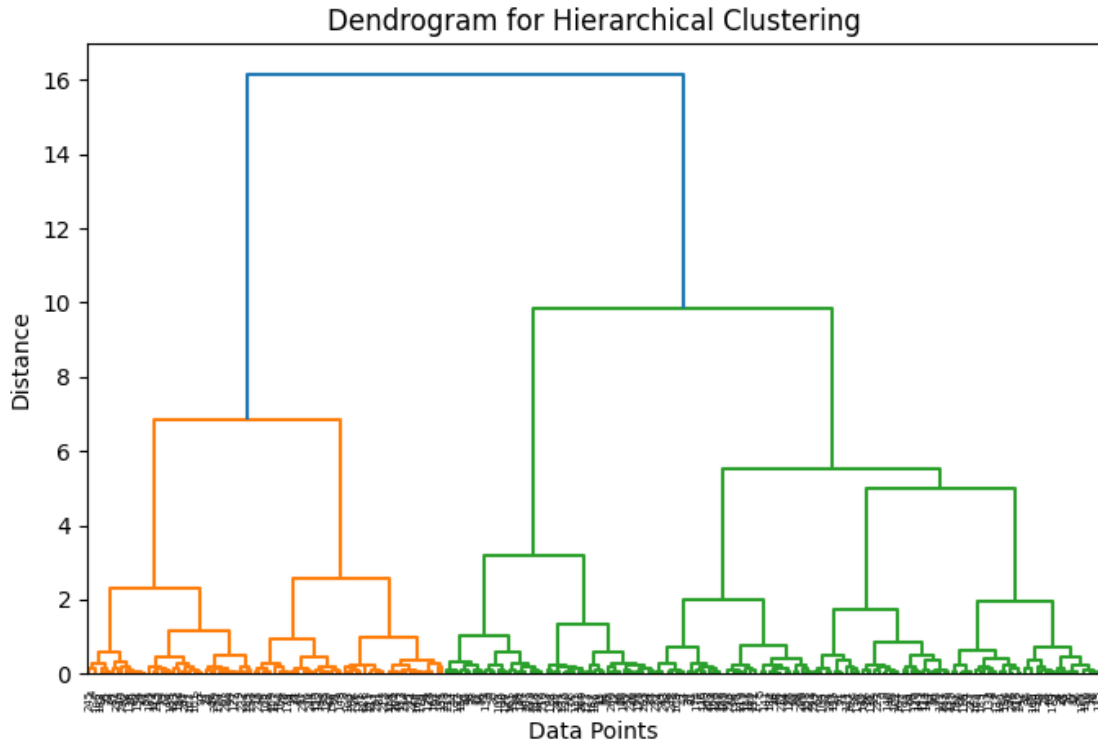
## 142.2 Why Use Hierarchical Clustering?

- No need to specify number of clusters initially
- Provides insight into data structure
- Useful for small to medium datasets

### 142.3 Limitation

- Computationally expensive for large datasets
- Decisions (merge/split) cannot be undone





## 143 DBSCAN

DBSCAN stands for: **Density-Based Spatial Clustering of Applications with Noise**. Unlike K-Means, DBSCAN does not try to create a fixed number of clusters. Instead, it groups together points that are packed closely together and labels isolated points as noise.

### 143.1 Main Idea

- Dense regions form clusters
- Sparse regions may be treated as outliers

This makes DBSCAN especially useful when:

- Clusters have irregular shapes
- The dataset contains noise or outliers
- Number of clusters is **unknown**

#### 143.1.1 Key Parameters:

DBSCAN uses two parameters:  $\epsilon$  (neighborhood radius) and **minPts** (minimum points) to define dense regions and form clusters.

- **(epsilon):** radius defining neighborhood
- **minPts:** minimum number of points required to form a dense region



## 143.2 Types of Points

- **Core Point:** has at least `minPts` neighbors within
- **Border Point:** close to a core point but not dense enough itself
- **Noise Point:** not close to any cluster

## 143.3 How DBSCAN Works

1. Pick an unvisited point
2. Find all points within distance  $\epsilon$  (epsilon)
  - This forms the point's **neighborhood**
3. Check if the neighborhood has at least `minPts` points
  - If YES  $\rightarrow$  it is a **core point**, start a new cluster
  - If NO  $\rightarrow$  mark it as **noise** (may later become a border point)
4. Expand the cluster:
  - Add all points within  $\epsilon$  of the core point
  - For each new point:
    - If it also has `minPts` neighbors  $\rightarrow$  expand furtherThis creates a **chain of density-connected points**
5. Repeat until all reachable points are added
6. Move to next unvisited point and repeat

DBSCAN finds clusters based on density, grouping nearby points together while identifying sparse points as noise.

*[Image omitted in PDF — see the online textbook]*

Figure: DBSCAN clustering showing core points (dense regions), border points (edges of clusters), and noise points (outliers).

Source: GeeksforGeeks

In this figure: - **Core points** form the dense center of clusters  
- **Border points** lie on the edges of clusters  
- **Noise points** are isolated and do not belong to any cluster

DBSCAN builds clusters by expanding from core points and including all **density-connected** points (points reachable through neighboring core points).

<IPython.core.display.HTML object>

## 143.4 Why use DBSCAN?

- Does not require specifying the number of clusters in advance
- Can detect outliers
- Works well for arbitrarily shaped clusters

##Limitation Sensitive to parameter choices such as eps and min\_samples

## 143.5 Comparing the Methods

K-Means - Best for compact, spherical clusters - Requires choosing K

Hierarchical Clustering

- Builds a tree-like clustering structure
- Useful for exploring relationships between clusters

DBSCAN

- Density-based
- Handles noise and irregular cluster shapes better

## 144 Summary of the Chapter

In this chapter, we explored **unsupervised learning**, where models learn patterns from data without labels. We first addressed the challenge of high-dimensional data using **Principal Component Analysis (PCA)**, which reduces the number of features while preserving the most important information by capturing directions of maximum variance. This helps simplify data, remove redundancy, and make patterns easier to understand and visualize.

---

**PCA Workflow:** Data  $\rightarrow$  Covariance  $\rightarrow$  Eigenvectors  $\rightarrow$  Sort  $\rightarrow$  Select  $\rightarrow$  Project  $\rightarrow$  Reduced Data

---

We then focused on **clustering**, which groups similar data points together. We studied **K-Means**, which forms clusters by minimizing intra-cluster distance (WCSS), along with methods like the **Elbow Method** and **Silhouette Score** to choose the number of clusters. We also covered **Hierarchical Clustering**, which builds a tree-like structure of clusters, and **DBSCAN**, a density-based method that can find arbitrarily shaped clusters and detect noise. Together, these techniques show how we can uncover meaningful structure in data even without labels.

## 145 Knowledge Check

### 145.1 interactive k means

## 146 Interactive K Means

This tool lets you explore how different activation functions (like ReLU, Sigmoid, Tanh) transform an input value x.

How to Use:

- Select an activation function from the dropdown.
- Change the input x value using the slider or input box.
- View the output and see how the function transforms x in real-time on the graph.
- Use this to build intuition about how activation functions behave in neural networks!

*[Interactive content omitted in PDF]*

## 146.1 interactive pca

## 147 Interactive PCA

This tool shows the gradient value at each node during the backpropagation process.

How to Use:

- Adjust the value of the input nodes
- View the gradients at each node in red values
- Try to do the gradient calculations by hand and double check your work

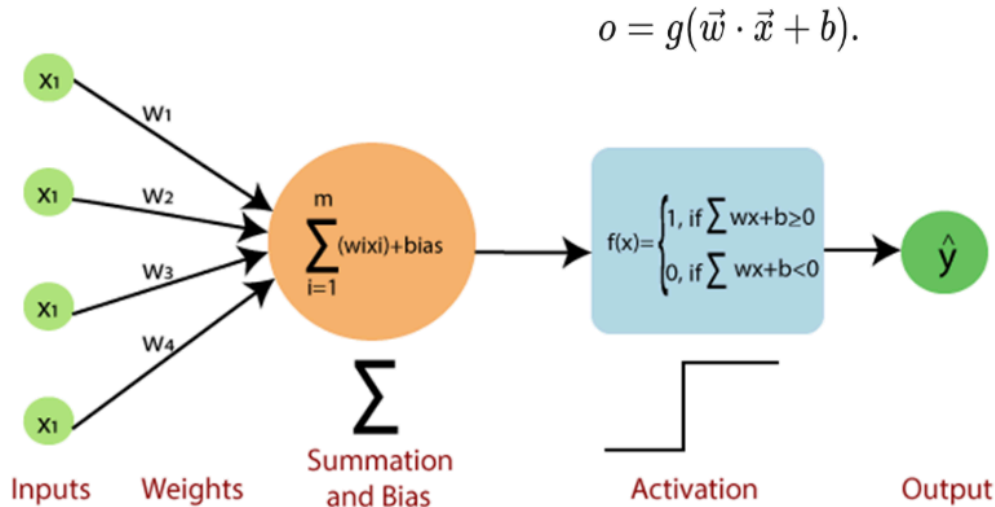
*[Interactive content omitted in PDF]*

## 148 Chapter 18. Neural Network

## 149 Chapter 18: Introduction to the Neural Network

Artificial intelligence has revolutionized the way machines learn from data, enabling them to perform tasks that once seemed impossible. From powering voice assistants to enabling autonomous vehicles, AI systems are now integrated into many aspects of daily life. At the core of this transformation are neural networks—computational models inspired by **the structure and function of the human brain**. These networks consist of interconnected nodes, or “neurons,” that process information in layers, allowing them to recognize patterns, make decisions, and improve over time. From early single-layer perceptrons to today’s deep architectures, neural networks have become a cornerstone of modern AI, powering breakthroughs in fields like healthcare, finance, and autonomous systems.

Neural networks are a **type of supervised learning algorithm** capable of identifying intricate patterns and relationships within data, making them suitable for tackling problems that traditional models often struggle with.



Although neural network algorithms have existed for many years, recent advancements in their architectures have led to significant improvements in performance on large-scale machine learning tasks. These developments form the foundation of what is now referred to as the “deep learning” methodology.

Deep learning, a branch of machine learning and artificial intelligence, leverages neural networks with multiple hidden layers to address highly complex tasks. These tasks range from natural language processing—such as speech recognition and text interpretation—to computer vision applications like object detection and image classification. The rise of deep learning over the past twenty years can be attributed to its remarkable effectiveness, the surge in computational power, and the growing accessibility of vast datasets.

### 149.1 Fundamentals of Neural Networks

Neural networks are a powerful class of supervised learning algorithms capable of modeling complex, nonlinear relationships in data. Unlike traditional machine learning models, which rely on handcrafted features, neural networks automatically learn hierarchical representations from raw input. Key components include:

- **Input Layer:** Receives raw data (e.g., pixels in an image, words in a text).
- **Hidden Layers:** Intermediate layers that transform inputs through weighted connections and activation functions.
- **Output Layer:** Produces the final prediction (e.g., class label, regression value).
- **Weights & Biases:** Adjustable parameters learned during training.
- **Activation Functions:** Introduce nonlinearity (e.g., ReLU, Sigmoid, Tanh).

Training a neural network involves **forward propagation** (passing data through the network) and **backpropagation** (adjusting weights based on error gradients using optimization techniques like gradient descent).

## 149.2 Neurons: The Building Blocks

A neuron (or node) is the fundamental unit of a neural network, mimicking biological neurons. It receives inputs, processes them using weights and an activation function, and produces an output. Mathematically, a neuron's operation can be represented as:

$$y = f \left( \sum_{i=1}^n (w_i x_i) + b \right)$$

Where: -  $x_i$  = input features -  $w_i$  = weights

- $b$  = bias term
- $f$  = activation function

## 149.3 Layers: Input, Hidden, and Output

Layers: Input, Hidden, and Output\*\* Neural networks are organized into layers:

1. **Input Layer:**
  - The first layer that receives raw data (e.g., pixel values in an image, word embeddings in text).
2. **Hidden Layers:**
  - Intermediate layers between input and output where feature extraction and transformation occur.
  - Deep networks have multiple hidden layers
3. **Output Layer:**
  - Produces final predictions
  - Classification: class probabilities
  - Regression: continuous values

## 149.4 Weights and Biases

Component	Role
<b>Weights</b> (( $w_i$ ))	Determine connection strength between neurons. They are adjusted during training to minimize prediction errors.
<b>Bias</b> (( $b$ ))	Allows shifting the activation function to improve model flexibility.

## 149.5 18.1 Activation Functions

Activation functions are essential components of neural networks, introducing non-linearities that enable the model to learn complex patterns. Without them, a neural network would simply be a linear regression model, incapable of handling intricate data relationships. This section explores

four widely used activation functions: Sigmoid, ReLU, Tanh, and Softmax, discussing their properties, advantages, and limitations. Each has unique properties and use cases depending on the architecture and goal of the model.

Function	Formula	Use Case
<b>Sigmoid</b>	$\sigma(x) = \frac{1}{1+e^{-x}}$	Binary classification (outputs between 0 and 1)
<b>ReLU</b>	$f(x) = \max(0, x)$	Default choice for hidden layers (fast computation)
<b>Tanh</b>	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Similar to sigmoid but outputs between -1 and 1
<b>Softmax</b>	$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$	Multi-class classification (outputs probabilities)

**The sigmoid function:** maps any real-valued number to a range between 0 and 1, making it suitable for binary classification tasks where outputs represent probabilities. This function  $\sigma(x) = \frac{1}{1+e^{-x}}$  maps any real-valued input into a probability-like output, making it useful in binary classification and output layers where probabilistic interpretation is needed.

Advantages: \* Useful when outputs need to be interpreted as probabilities. \* Differentiable, allowing gradient-based optimization.

Disadvantages: \* Vanishing Gradients: For very large or small inputs, gradients become nearly zero, slowing down learning. \* Not Zero-Centered: Outputs are always positive, leading to inefficient weight updates \* Computationally Expensive: Involves exponentiation operations.

**ReLU (Rectified Linear Unit) Function:** is one of the most popular activation functions due to its simplicity and effectiveness. It outputs the input directly if positive; otherwise, it outputs zero ( $f(x) = \max(0, x)$ ). It is non-linear but simple; computationally efficient.

Advantages:

- Avoids Vanishing Gradient (for positive inputs): Unlike sigmoid, gradients remain strong for active neurons.
- Fast Computation: No complex exponentials.
- Sparsity: Can deactivate neurons (output zero), making the network more efficient.

Disadvantages:

- Dying ReLU Problem: If many neurons output zero (due to negative inputs), they stop learning entirely.
- Not Zero-Centered: Like sigmoid, can lead to slower convergence.

What is the **Dying ReLU Problem**? If a neuron consistently receives negative inputs, its output becomes zero, and its weights stop updating (since the gradient is also zero). Over time, this can cause some neurons to “die” and never activate again, reducing the model’s capacity to learn (neurons stop contributing to learning).

Solutions to Dying ReLU:

- Leaky ReLU: Allows a small negative slope (e.g., 0.01) for negative inputs.

$$\text{Leaky ReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha x, & \text{if } x < 0 \end{cases}$$

where  $\alpha$  is a small positive constant (e.g., 0.01).

- Parametric ReLU (PReLU): Learns the negative slope during training.
- Exponential Linear Unit (ELU): Smoothly handles negative inputs.

**Tanh (Hyperbolic Tangent) Function:** The tanh function is similar to sigmoid but maps inputs to a range between -1 and 1, making it zero-centered,  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ . \* In neural networks, an activation function is zero-centered if its output values are symmetrically distributed around zero (i.e., they have a mean of zero). This property helps in maintaining stable and efficient training by preventing systematic weight updates in a single direction.

- When activation outputs are not zero-centered (e.g., sigmoid outputs between 0 and 1), gradients during backpropagation tend to be either all positive or all negative, leading to inefficient weight updates: tend to update in the same direction (either always increasing or always decreasing), slowing down convergence.

Advantages:

- Zero-centered output allows for better convergence during gradient descent.
- Stronger gradients than sigmoid for inputs near 0.

Disadvantages:

- Still suffers from the vanishing gradient problem for very large or very small inputs:: Like sigmoid, gradients become very small for extreme values.
- Slightly More Computationally Expensive: Due to exponential operations.

**The Softmax function** is typically used in the output layer of a multi-class classification model. It converts a vector of raw scores (logits) into a probability distribution over predicted output classes: **Softmax** |  $\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ .

Advantages:

- Ensures that the sum of the outputs is 1, making them interpretable as probabilities.
- Highlights the highest-valued input while suppressing the rest, which helps in clear class predictions.

Disadvantages:

- Exponentially sensitive to input scale—can cause numerical instability if logits are too large.
- When classes are not mutually exclusive, Softmax is not ideal (use sigmoid instead for multi-label classification).

[Click Here for Interactive Activation Function Visualization](#)

## 149.6 \*\* Loss Functions\*\*

Loss functions (or cost functions) measure how well a neural network's predictions match the true target values. During training, the goal is to minimize the loss by adjusting the model's parameters. The choice of loss function depends on the type of task:

- **Mean Squared Error (MSE):** is widely used in regression problems, such as predicting house prices or temperature, where the output is continuous and goal is to minimize the average squared difference between predicted and actual values.

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

As it calculates the average squared difference between predicted values and actual values, which means larger errors are penalized more heavily. While MSE is straightforward and differentiable, making it compatible with gradient descent, it has notable drawbacks: it is highly sensitive to outliers due to the squaring operation, and it performs poorly in classification tasks, often leading to slow convergence.

- **Cross-Entropy Loss:** On the other hand, Cross-Entropy Loss is widely used for classification tasks, both binary and multi-class. It measures the difference between the predicted probability distribution and the true label distribution.
- For binary classification:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- - For multi-class classification:

$$L_{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

For binary classification, it penalizes the model when it confidently predicts the wrong class, encouraging outputs closer to the true labels. In multi-class settings, cross-entropy works with softmax outputs to handle multiple classes simultaneously. One disadvantage of cross-entropy loss is that it can become very large when the model assigns near-zero probabilities to the true class, which may cause instability during training if not handled properly.

## 149.7 interactive activation functions

## 150 Interactive Activation Functions

This tool lets you explore how different activation functions (like ReLU, Sigmoid, Tanh) transform an input value x.

How to Use:

- Select an activation function from the dropdown.
- Change the input x value using the slider or input box.
- View the output and see how the function transforms x in real-time on the graph.



- Use this to build intuition about how activation functions behave in neural networks!

*[Interactive content omitted in PDF]*

## 151 Gradient Descent

Gradient Descent is an algorithm that finds the local extrema of a function. This is applicable to machine learning, because we want to find the optimal parameters that minimize our loss function. In machine learning, loss functions quantify the amount of error between the predicted values from a machine learning model and the actual expected values. In this notebook, we will perform linear regression by using gradient descent to find the optimal slope and y-intercept.

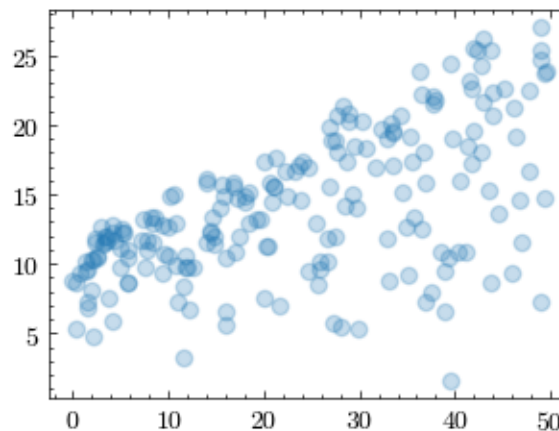
*[Image omitted in PDF — see the online textbook]*

### 151.1 Training Dataset

Importing the libraries

Let's look at the training dataset. We will use columns 2 and 4 of the txt file. The linear regression model will find the optimal slope and y-intercept to fit the data.

```
<matplotlib.collections.PathCollection at 0xffff5020a450>
```



### 151.2 Loss Function: Mean Squared Error

For the linear regression model, the predicted value  $\hat{y}$  is the dot product of the weight and x input vectors plus a bias term

$$\hat{y} = wx + b$$

We will use the mean squared error function as our loss function.

$$\begin{aligned}
MSE &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2 \\
&= \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2
\end{aligned}$$

### 151.3 Loss Function Gradient

In each epoch of gradient descent, a parameter is updated by subtracting the product of the gradient of the function and the learning rate ( $lr$ ). The learning rate controls how much the parameters should change. Small learning rates are precise, but are slow. Large learning rates are fast, but may prevent the model from finding the local extrema.

$$X_{n+1} = X_n - lr * \frac{\partial}{\partial X} f(X_n)$$

Since we are finding the optimal slope ( $w$ ) and y-intercept ( $b$ ) for our linear regression model, we must find the partial derivatives of the loss function with respect to  $w$  and  $b$ .

### 151.4 Loss Function in Terms of W

Loss function with respect to  $w$ :

$$\begin{aligned}
\frac{\partial}{\partial w} (MSE) &= \frac{\partial}{\partial w} \left[ \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2 \right] \\
&= \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial w} [(y_i - (wx_i + b))^2] \\
&= \frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b)) \frac{\partial}{\partial w} [y_i - (wx_i + b)] \\
&= \frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b)) (-x_i) \\
&= -\frac{2}{n} \sum_{i=1}^n x_i (y_i - (wx_i + b))
\end{aligned}$$

### 151.5 Loss Function in Terms of b

Loss function with respect to  $b$ :

$$\begin{aligned}
\frac{\partial}{\partial b} (MSE) &= \frac{\partial}{\partial b} \left[ \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2 \right] \\
&= \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial b} [(y_i - (wx_i + b))^2] \\
&= \frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b)) \frac{\partial}{\partial b} [y_i - (wx_i + b)] \\
&= \frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b)) (-1) \\
&= -\frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b))
\end{aligned}$$

## 151.6 Training the Linear Regression Model

Let's define a function that uses the gradient algorithm to update the parameters of the loss function. The function uses the gradient functions we derived earlier.

General Gradient Descent Equation:

$$X_{n+1} = X_n - lr * \frac{\partial}{\partial X} f(X_n).$$

Bias Gradient Descent:

$$\begin{aligned}
b &= b - \eta \frac{\partial}{\partial b} [L(\vec{w}, b)] \\
&= b - \eta \left[ -\frac{2}{n} \sum_{i=1}^n (y_i - (wx_i + b)) \right]
\end{aligned}$$

Weights Gradient Descent:

$$\begin{aligned}
b &= b - \eta \frac{\partial}{\partial b} [L(\vec{w}, b)] \\
&= b - \eta \left[ -\frac{2}{n} \sum_{i=1}^n x_i (y_i - (wx_i + b)) \right]
\end{aligned}$$

## 151.7 Graphing functions

Let's define helper functions to plot the graphs.

## 151.8 Training the model

The train function will update the parameters in each epoch and update the visualization.

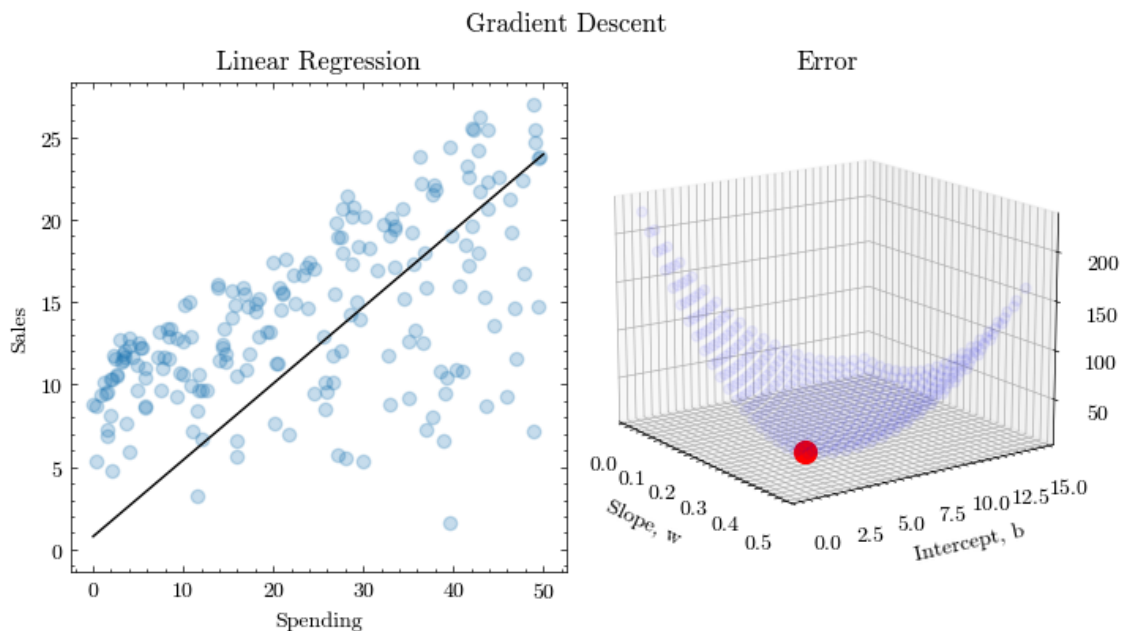
Let's train the linear regression model on a sample dataset.

```
epoch: 0 loss: 197.25274270926414
epoch: 5 loss: 112.88939636458916
epoch: 10 loss: 74.66409989764938
```

```

epoch: 15 loss: 57.34204113744597
epoch: 20 loss: 49.490344392988355
epoch: 25 loss: 45.929244266843334
epoch: 30 loss: 44.31200864892866
epoch: 35 loss: 43.57543700948768
epoch: 40 loss: 43.237843105871576
epoch: 45 loss: 43.08099946508624
epoch: 50 loss: 43.00603965880256
epoch: 55 loss: 42.968173826457125
epoch: 1000 loss: 41.62567353799647
epoch: 2000 loss: 40.307849717949104
epoch: 3000 loss: 39.06382181292731

```



```
(np.float64(0.4560414772297029), np.float64(1.0259430403235812))
```

## 151.9 18.2 Optimizers

Optimizers are algorithms that adjust the weights of a neural network to minimize the loss function during training. Two most basic optimizers are - **SGD (Stochastic Gradient Descent)**: Basic optimization - **Adam**: Adaptive learning rates (most popular)

Stochastic Gradient Descent (SGD) updates the model's parameters by computing the gradient of the loss on a small batch of data and moving in the direction that reduces the loss. While simple and effective, SGD uses a fixed learning rate and can be slow to converge, especially for complex models.

To address these limitations, Adam (Adaptive Moment Estimation) is widely used due to its ability to adapt the learning rate for each parameter individually by combining the benefits of momentum

and RMSProp optimizers. Adam often results in faster convergence and better performance without much tuning, making it the most popular choice for training deep neural networks.

Note: Momentum helps speed up learning by smoothing updates using past gradients, while RMSProp adapts learning rates based on recent gradient magnitudes. Adam combines both techniques for efficient and stable training.

## 151.10 Training Techniques & Regularization

Training a neural network consists of two main steps performed repeatedly over many iterations: the **forward pass** and the **backward pass**.

### 151.10.1 Forward Pass

During the forward pass, input data flows through the network, and the final output layer generates predictions ( $\hat{y}^{(i)}$ ) for each input example (i). The network then computes the loss function (L), which quantifies the difference between predicted outputs and true targets ( $y^{(i)}$ ). For a batch of (m) samples, the average loss is:

$$L = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

This loss guides how well the model is performing.

Here are the details of step by step:

### 151.11 Forward Pass: Step-by-Step

The forward pass is the phase in which input data flows through the neural network to produce predictions.

*[Image omitted in PDF — see the online textbook]*

Figure: Forward Pass in Neural Networks

Here's how it works:

#### Input Layer

Each input example is denoted as  $x_i^{(0)}$  which is fed into the network.

#### Hidden Layers

For each hidden layer (  $l = 1, 2, \dots, L$  ), the network performs the following steps:

##### Linear transformation:

$$z_i^{[l]} = W^{[l]}x_i^{(l-1)} + b^{[l]}$$

##### Non-linear activation:

$$x_i^{(l)} = \sigma(z_i^{[l]})$$

Here, (  $x_i^{(l)}$  ) is the activated output of layer (  $l$  ), used as input for the next layer.

## Output Layer

After the last hidden layer (  $L$  ), the output layer produces the final prediction:

$$\hat{y}_i = f(x_i^{(L)})$$

where (  $f$  ) is an appropriate output function (e.g., identity for regression, sigmoid for binary classification, softmax for multi-class classification).

## Loss Computation

The predicted output  $\hat{y}_i$  is compared to the true label  $y_i$  using a loss function  $L(\hat{y}_i, y_i)$ .

Example (Mean Squared Error):

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

## Average Loss Over the Batch

For a batch of (  $m$  ) samples, the average loss is:

$$L = \frac{1}{m} \sum_{i=1}^m L(\hat{y}_i, y_i)$$

This average loss quantifies how well the model is performing and is used in the backward pass to update model parameters.

[Click Here for Interactive Forward Propagation Visualization](#)

## 151.12 Backward Pass (Backpropagation)

In the backward pass, gradients of the loss with respect to each parameter (weights  $W^{[l]}$  at layer  $l$ ) are computed using the chain rule.

The chain rule lets us break down the gradient of the loss into simpler parts by following the flow of computations backward through the network. It helps calculate how changes in weights affect the final loss by multiplying the derivatives of each intermediate step.

Mathematically:

$$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial a^{[l]}} \cdot \frac{\partial a^{[l]}}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

where:

- $a^{[l]}$  is the activation output of layer  $l$ ,
- $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$  is the input to the activation function.

## Weight Update

The weights are then updated using gradient descent:

$$W^{[l]} := W^{[l]} - \alpha \cdot \frac{\partial L}{\partial W^{[l]}}$$

where:

- $W^{[l]}$  = current weights at layer  $l$ ,
- $\alpha$  = learning rate (controls the step size),
- $\frac{\partial L}{\partial W^{[l]}}$  = gradient of the loss with respect to the weights.

### Repeat

This process of forward pass, backward pass, and weight update is repeated over many epochs (full passes over the training data) until the error is minimized.

*[Image omitted in PDF — see the online textbook]*

Figure: Backpropagation in Neural Networks

[Click Here for Interactive Backpropagation Visualization](#)

## 151.13 Regularization Techniques

Regularization adds penalty terms to the loss function to prevent overfitting, which happens when the model learns the training data too well including noise, resulting in poor performance on new, unseen data. By constraining model complexity, regularization encourages the model to generalize better.

---

### 151.13.1 L2 Regularization (Weight Decay):

$$L_{reg} = L + \lambda \sum \|W\|_2^2 = L + \lambda \sum W^2$$

L2 regularization adds the sum of the squared weights to the loss function. The hyperparameter  $\lambda$  controls the strength of this penalty. This encourages the model to keep weights small but not necessarily zero, which tends to distribute the “importance” across many features and reduces overfitting.

---

### 151.13.2 L1 Regularization:

$$L_{reg} = L + \lambda \sum |W|$$

L1 regularization adds the sum of the absolute values of the weights to the loss. This tends to push some weights exactly to zero, effectively performing feature selection by encouraging sparsity in the model. This can be useful when you want a simpler model with fewer active features.

---

### 151.13.3 Why Use Regularization?

- **Improves Generalization:** By limiting the size or number of weights, the model avoids fitting noise and irrelevant patterns in the training data.

- **Controls Model Complexity:** Prevents weights from growing too large, which can cause unstable predictions.
  - **Feature Selection (L1):** Helps identify and ignore irrelevant features by forcing their weights to zero.
- 

#### 151.13.4 Choosing $\lambda$

The regularization strength  $\lambda$  is a hyperparameter that must be tuned carefully:

- Too small: little effect on overfitting
  - Too large: model may underfit by being too constrained
- 

#### 151.13.5 Example: When to Use L1 vs. L2 Regularization

- **L2 Regularization (Ridge):**  
Use when most features are relevant and you want to keep them all but reduce overfitting by shrinking weights smoothly.  
*Example:* Predicting house prices using many meaningful features.
  - **L1 Regularization (Lasso):**  
Use when you expect only a few important features and want the model to ignore irrelevant ones by setting some weights exactly to zero.  
*Example:* Selecting important genes from thousands of candidates in a biological study.
- 

#### 151.13.6 Other Common Regularization Methods (Brief)

- **Dropout:** Randomly sets some activations to zero during training to prevent co-adaptation of neurons, helping the model generalize better.
- **Early Stopping:** Stops training when performance on a validation set stops improving, preventing overfitting.

### 151.14 Putting It All Together: How Neural Network Training Happens

After understanding how regularization helps prevent overfitting and improve generalization, it's important to see how the entire training process operates in practice. Training a neural network involves repeatedly feeding data through the model in manageable portions, updating parameters, and gradually improving performance.

The next section explains the key concepts of **batch size**, **iteration**, and **epoch**, which organize how training data is processed and how the model learns over time.

---



## 151.15 Neural Network Training: Batch Size, Iteration, and Epoch

When training a neural network, the dataset is usually too large to process all at once, so it is split into smaller parts called **batches**.

---

### 151.15.1 Batch Size

The number of training examples used in one forward and backward pass.  
For example, a batch size of 32 means the model processes 32 samples before updating weights.

---

### 151.15.2 Iteration

One update of the model's parameters (weights and biases).  
Each iteration uses one batch of data.

---

### 151.15.3 Epoch

One full pass over the entire training dataset.  
If the dataset has 1000 samples and batch size is 100, then:

$$1 \text{ epoch} = \frac{1000}{100} = 10 \text{ iterations}$$

*[Image omitted in PDF — see the online textbook]*

Figure: Epoch in Machine Learning

---

### 151.15.4 Training Process Overview

1. **Start Training:** Initialize model parameters.
2. For each epoch (repeat multiple times):
  - Divide data into batches according to batch size.
  - For each batch:
    - Perform forward pass to calculate predictions.
    - Calculate loss based on predictions and true labels.
    - Perform backward pass (backpropagation) to compute gradients.
    - Update weights using gradients (e.g., gradient descent).
3. Repeat until model performance stabilizes or desired accuracy is reached.

*[Image omitted in PDF — see the online textbook]*

Figure: In the Input Training Dataset step, as different kinds of strategies, we can assign a Batch Size to determine how many training datasets are going to be used for one weights updating process. For example, there are 5000 images in total as the training datasets. If we set the Batch Size = 1000, we get 5 batches of training datasets. As a result, the weights updating process will be executed 5 times (Iterations).

Ref: [medium.com/@crazyhatcap](https://medium.com/@crazyhatcap)

## 151.16 interactive forward propagation

## 152 Interactive Forward Propagation

This tool shows the activation values at each node from the forward propagation process.

How to Use:

- Adjust the value of the input nodes
- View the activation at each node change
- Try to do the calculation by hand and double check your work

*[Interactive content omitted in PDF]*

## 152.1 interactive backward propagation

## 153 Interactive Backward Propagation

This tool shows the gradient value at each node during the backpropagation process.

How to Use:

- Adjust the value of the input nodes
- View the gradients at each node in red values
- Try to do the gradient calculations by hand and double check your work

*[Interactive content omitted in PDF]*

## 153.1 Putting It All Together: How Neural Network Training Happens

After understanding how regularization helps prevent overfitting and improve generalization, it's important to see how the entire training process operates in practice. Training a neural network involves repeatedly feeding data through the model in manageable portions, updating parameters, and gradually improving performance.

The next section explains the key concepts of **batch size**, **iteration**, and **epoch**, which organize how training data is processed and how the model learns over time.

---

## 153.2 Neural Network Training: Batch Size, Iteration, and Epoch

When training a neural network, the dataset is usually too large to process all at once, so it is split into smaller parts called **batches**.

---

### 153.2.1 Batch Size

The number of training examples used in one forward and backward pass.  
For example, a batch size of 32 means the model processes 32 samples before updating weights.

---

### 153.2.2 Iteration

One update of the model's parameters (weights and biases).  
Each iteration uses one batch of data.

---

### 153.2.3 Epoch

One full pass over the entire training dataset.  
If the dataset has 1000 samples and batch size is 100, then:

$$1 \text{ epoch} = \frac{1000}{100} = 10 \text{ iterations}$$

*[Image omitted in PDF — see the online textbook]*

Figure: Epoch in Machine Learning

---

### 153.2.4 Training Process Overview

1. **Start Training:** Initialize model parameters.
2. For each epoch (repeat multiple times):
  - Divide data into batches according to batch size.
  - For each batch:
    - Perform forward pass to calculate predictions.
    - Calculate loss based on predictions and true labels.
    - Perform backward pass (backpropagation) to compute gradients.
    - Update weights using gradients (e.g., gradient descent).
3. Repeat until model performance stabilizes or desired accuracy is reached.

*[Image omitted in PDF — see the online textbook]*

Figure: In the Input Training Dataset step, as different kinds of strategies, we can assign a Batch Size to determine how many training datasets are going to be used for one weights updating process. For example, there are 5000 images in total as the training datasets. If we set the Batch Size =

1000, we get 5 batches of training datasets. As a result, the weights updating process will be executed 5 times (Iterations).

Ref: [medium.com/@crazyhatcap](https://medium.com/@crazyhatcap)

### 153.3 Chapter Summary

In this chapter, we explored the key components of training neural networks, including the forward and backward passes, how loss guides learning, and how weights are updated iteratively. We also discussed the role of batch size, iterations, and epochs in organizing the training process for efficient and effective learning.

Understanding these concepts lays the foundation for building, training, and optimizing deep learning models. In the next chapter, we will delve into advanced optimization techniques and strategies to further improve model performance.

### 153.4 Knowledge Check

*[Interactive content omitted in PDF]*

## 154 Chapter 19 - Convolutional Neural Network

### 155 Introduction

In the **previous chapter**, we explored the foundational concepts behind neural networks and saw how they can approximate functions using layers of interconnected neurons. However, standard feedforward networks struggle with high-dimensional data like images, where preserving spatial relationships is essential. We now shift our focus to **Convolutional Neural Networks (CNNs)**, a specialized type of neural network designed to process and interpret image data effectively.

CNNs revolutionized the field of computer vision and are now the backbone of many modern applications, including facial recognition, medical image analysis, and autonomous driving.

Unlike fully connected neural networks, CNNs leverage three key ideas:

- Local Receptive Fields (**Convolution operations**)
- Parameter Sharing (**Reduces the number of trainable weights**)
- Spatial Hierarchies (**Pooling layers** for downsampling)

These properties make CNNs **translation-invariant** and **computationally efficient** compared to dense neural networks when dealing with high-dimensional data like images.

### 156 19.1 Why Not Use Fully Connected Networks for Images?

Consider a grayscale image of size 28×28 pixels. If we treat this as a flat input vector to a dense network, it results in 784 input nodes. For a 100x100 image, the number of inputs jumps to 10,000! Fully connected networks don't scale well and tend to ignore spatial hierarchies like "edges" or "patterns."

CNNs solve this by leveraging local connectivity, weight sharing, and spatial hierarchies to detect meaningful patterns in images.

[Image omitted in PDF — see the online textbook]

Figure: Fully connected networks flatten images into vectors, losing important spatial relationships like shape, texture, and position. This may work for simple images but fails on complex ones with pixel dependencies. CNNs preserve spatial information by using filters, making them more effective for image tasks like recognition and detection.

## 157 19.2 CNN Architecture Overview

A typical CNN consists of the following layers: \* **Convolutional layer(s)**: Extracts features from the input dataset using learnable filters known as kernels applied to input images.

- **Pooling layer(s)**: Reduces the spatial dimensions of the input volume, speeding up computation, reducing memory usage, and helping prevent overfitting.
- **Fully connected layer(s)**: Used for the final classification or regression task.

[Image omitted in PDF — see the online textbook]

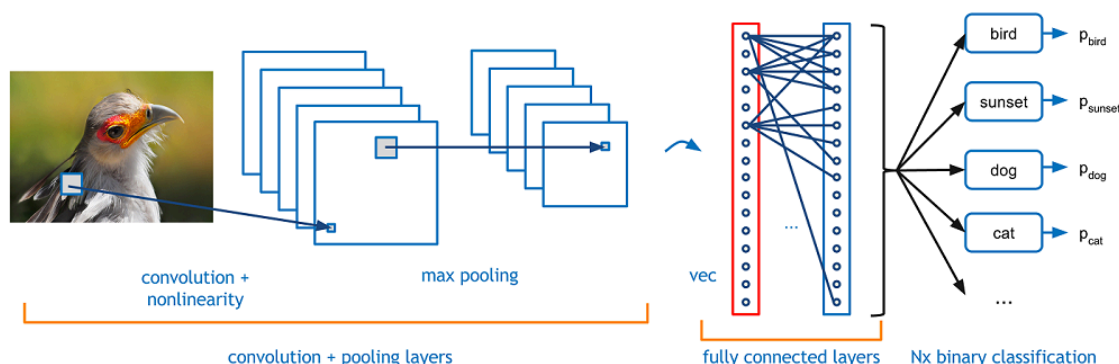


Figure: A simplified scheme of a simple CNN (with an example).

These layers are often supplemented with dropout and batch normalization for improved training stability.

### 157.1 19.2.1 Convolution Layer

The **convolutional layer** is the **core** building block of a CNN. It applies a set of learnable filters (kernels) to the input, each detecting different features such as edges, textures, or patterns. A convolution operation refers to sliding a filter across the input image and computing the dot product between the filter weights and the corresponding input region. This operation produces a **feature map (also known as activation map)** that highlights where the detected feature appears in the input.

[Image omitted in PDF — see the online textbook]

Figure: An example of a convolution operation applied to a  $5 \times 5$  pixel, single-channel image using a  $3 \times 3$  filter.

**A Filter, or Kernel:** in a CNN, is a small matrix used in the convolution operation. It's a set of learnable weights that are applied to the input data to produce the output

feature map. These small matrix of weights slides over the input data (such as an image), performs element-wise multiplication with the part of the input it is currently on, and then sums up all the results into a single output pixel. This process is known as convolution. Kernels are the key elements that allow CNNs to automatically learn spatial hierarchies of features within the input data. In image processing, a kernel might be a small matrix like 3x3 or 5x5.

Remember that, the filter values are trainable parameters, much like weights in a standard neural network. These values of the filters are learned during training through **backpropagation**, similar to weights in a traditional neural network. By sharing weights across the entire image, CNNs drastically reduce the number of parameters while retaining spatial information. The network learns multiple filters at each layer, each specialized to detect different features in the image, such as vertical edges, corners, or more complex textures.

There are some key parameters in a convolutional layer that include:

- **Kernel size** (e.g., 3×3 or 5×5), which determines the receptive field.
- **Stride**, the step size with which the filter moves across the image. A stride of 1 moves pixel-by-pixel, while a larger stride reduces spatial dimensions.
- **Padding**, Controls whether we allow the output size to shrink or keep it the same as the input. It allows to adds extra pixels (usually zeros) around an image before convolution. It helps control the output size and preserves edge information.

Padding are two types: \* **Same padding**: Adds zeros so that the output has the same spatial dimensions (height and width) as the input. This helps preserve the size of the image throughout convolutional layers (same padding keeps the output the same size as the input, making it easier to stack many layers without losing resolution or cutting off edge information). Example: 5×5 input → 5×5 output (3×3 kernel, stride=1).

- **Valid padding**: Adds no padding, so the convolution is only applied where the filter fully overlaps the input. This causes the output to shrink in size compared to the input. Example stride=1: 5×5 input → 3×3 output (3×3 kernel).

**Remember:** Higher strides = faster dimension reduction = less computation but potentially losing information.

## 157.2 Feature Map Size Calculation:

The output spatial size (height or width) of a feature map after a convolution or pooling operation is calculated using:

$$\text{Output Size} = \left\lfloor \frac{\text{Input Size} - \text{Kernel Size} + 2 \times \text{Padding}}{\text{Stride}} \right\rfloor + 1$$

**Where:** - **Input Size (N)**: Height or width of the input (e.g., 32 for 32×32 image). - **Kernel Size (F)**: Size of the filter (e.g., 3 for a 3×3 kernel) - **Padding (P)**: Number of zeros added around the input (e.g., padding is 0 in ‘valid’ padding, and typically 1 for ‘same’ padding with a 3×3 kernel). - **Stride (S)**: Step size of the filter (e.g., 1 = move 1 pixel, 2 = skip every other pixel) -  $\lfloor \cdot \rfloor$ : Floor operation (rounds down to the nearest neighbour)

---

**Example:**

For an input of size 32, a kernel size of 5, stride = 1, and padding = 0:

$$\text{Output Size} = \lfloor \frac{32-5+2 \times 0}{1} \rfloor + 1 = \lfloor 27 \rfloor + 1 = 28$$

Output feature map size: **28 × 28**.

**Some More Common Examples:** 1. 32×32 input, 3×3 kernel: - stride=1, padding=0 → 30×30 output - stride=1, padding=1 → 32×32 output (same padding) - stride=2, padding=0 → 15×15 output - stride=2, padding=1 → 16×16 output

2. 224×224 input (common ImageNet size), 7×7 kernel:

- stride=2, padding=3 → 112×112 output (Used in many CNN architectures)

Key Notes: - ‘Same’ padding maintains size when stride=1 - Stride>1 reduces dimensions (useful for downsampling) - Pooling layers use same calculation (typically stride=2)

### 157.3 Movement of the Kernel:

The kernel starts at the top-left corner of the input and moves right by the stride value, scanning horizontally. Once it reaches the edge, it returns to the left and shifts down by the stride to begin the next row. This top-to-bottom, left-to-right serpentine pattern continues until the entire input is covered.

*[Table omitted in PDF — see the online textbook]*

### 157.4 19.2.2 Activation Function (ReLU, Leaky ReLU, etc.)

After each convolution, an activation function introduces non-linearity, allowing the network to learn complex patterns. **The Rectified Linear Unit (ReLU)** is the most widely used due to its simplicity and effectiveness in mitigating the **vanishing gradient problem**. ReLU outputs the input directly if it is positive; otherwise, it outputs zero.

*[Table omitted in PDF — see the online textbook]*

The **vanishing gradient** problem occurs when gradients (used to update network weights during training) become very small, almost zero, especially in deep networks. This slows or stops learning because weights stop changing effectively. ReLU helps prevent this by allowing gradients to flow well for positive inputs.

In addition to introducing non-linearity, ReLU improves training efficiency by setting all negative values to zero. This creates sparsity in the network, meaning only the most important features (positive activations) are passed forward. This reduces complexity, helps the model focus on strong signals, and overall enables the CNN to learn faster and more effectively.

Other versions of ReLU, like **Leaky ReLU** and **Parametric ReLU (PReLU)**, allow a small, non-zero output for negative input values instead of just zero. This helps avoid the problem of “dead neurons,” where some neurons stop learning completely.

*[Image omitted in PDF — see the online textbook]*

### 157.5 19.2.3 Pooling Layers

After applying convolution and activation (e.g., ReLU), a **pooling layer** (also called subsampling or downsampling) is used on the resulting **feature map** or **activation map**.

**What is Pooling?** A down-sampling operation that reduces the spatial dimensions (width and height) of the feature maps while keeping the most important information. This makes the network faster, uses less memory, and helps avoid overfitting.

*[Image omitted in PDF — see the online textbook]*

Some of the most common types are:

- **Max Pooling:** Selects the maximum value in each window of the feature map. It keeps the most important (strongest) features and helps detect edges or textures strongly.

*[Image omitted in PDF — see the online textbook]*

- **Average Pooling:** Computes the average of all values in each window, providing smoother downsampling. This leads to smoother results, but may lose some important details. It is less commonly used in modern CNNs than max pooling.
- **Sum Pooling:** Adds up all the values in the window. It behaves similarly to average pooling, but keeps the total strength of the signal.

*[Image omitted in PDF — see the online textbook]*

Pooling helps in achieving **translation invariance**, meaning the network becomes less sensitive to the exact position of features in the input.

**Note: Translation invariance** is an important concept in computer vision that helps neural networks recognize objects even if they shift slightly in an image. This means the model becomes less sensitive to small movements or changes in position. For example, if a feature like a cat's ear or an edge moves a little to the left or right, the network can still recognize it. This is made possible by pooling layers, which summarize small regions of the activation maps—using operations like max or average—rather than keeping exact pixel locations. This allows the network to focus more on what feature is present, rather than where exactly it is. As a result, even when the appearance of an object changes slightly or it appears in a different location, the model can still identify it accurately. This makes translation invariance especially valuable for tasks like object detection and image classification.

*[Image omitted in PDF — see the online textbook]*

Figure: Let's consider the images above. We can recognize a woman in both images even though the pixel values were shifted. An image classifier should predict the label "woman" for both images. In fact, the classifier output should not be influenced by the position of the target. Hence, the output of the classifier function is translation invariant. Ref: <https://www.baeldung.com/cs/translation-invariance-equivariance>.

## 157.6 19.2.4 Flattening and Fully Connected (Dense) Layer

After several convolutional and pooling layers, the high-level features are flattened into a 1D vector and passed through one or more fully connected (dense) layers, just like in a traditional neural network. These layers perform the final classification or regression by combining all learned high-level features extracted by the convolutional layers. The last layer typically uses a softmax activation for classification or a linear activation for regression.



After several convolutional and pooling layers have extracted useful features, the output is **flattened into a 1D vector**—turning the multi-dimensional data into a simple list of numbers.

*[Table omitted in PDF — see the online textbook]*

This vector is then passed to one or more fully connected (dense) layers, just like in a traditional neural network. These layers connect every input to every output, allowing the model to learn complex patterns by combining all the features. In the final layer:

- **Classification** tasks often use: Softmax activation for multi-class output Sigmoid for binary output.
- **Regression** tasks usually use: Linear activation to produce continuous values.

*[Image omitted in PDF — see the online textbook]*

This part of the network is responsible for making the final prediction based on the high-level features extracted earlier.

## 158 19.3 Advantages of using CNNs Over Traditional Neural Networks.

Traditional neural networks process input data in a flattened form, losing spatial structure and requiring an excessive number of parameters for high-dimensional data like images. CNNs overcome these limitations through:

- **Local Connectivity:** Neurons in convolutional layers connect only to small regions of the input, reducing parameters while preserving spatial relationships.
- **Weight Sharing:** The same filter is applied across the entire image (instead of having one weight per input feature (as in FFNNs), CNNs use one filter across many locations), drastically cutting down trainable weights and significantly reduces the number of parameters, allowing CNNs to scale to large input sizes like 224x224 images, which would be computationally intractable with fully connected layers.
- **Hierarchical Feature Learning:** Early layers detect simple features (edges, textures), while deeper layers recognize complex patterns (shapes, objects).

This makes CNNs far more efficient for image-related tasks compared to fully connected networks.

## 159 19.4 Training CNNs

The training process for CNNs is similar to that of traditional feedforward neural networks (FFNNs) and involves several key steps:

- **Feedforward pass:** The input data passes through convolutional, activation, pooling, and fully connected layers to produce a prediction.
- **Loss computation:** A loss function (such as cross-entropy for classification) measures the difference between the predicted output and the true label.
- **Backpropagation:** Gradients of the loss with respect to all learnable parameters (filters, biases, weights) are calculated using the chain rule.

- **Parameter update:** Optimizers like Stochastic Gradient Descent (SGD) or Adam adjust the parameters to minimize the loss.

To improve training and prevent overfitting, CNNs often include:

- **Dropout:** Randomly disables a fraction of neurons during training to encourage the network to learn more robust features.
- **Batch Normalization:** Normalizes layer inputs within each mini-batch to speed up training and improve stability.

Through many iterations of this process, the filters in the convolutional layers learn to extract increasingly abstract and useful features from the input data, enabling the model to make accurate predictions.

**A simple CNN code snippets using Pytorch is given below:**

#19.5 Transfer Learning in CNNs Transfer learning is a technique where a pretrained convolutional neural network (CNN), usually trained on a large dataset like ImageNet, is reused as the starting point for a new, related task. Instead of training a CNN from scratch (which requires a lot of data and computing resources), transfer learning leverages the knowledge the network has already learned—such as detecting edges, textures, and shapes.

There are two common ways to use transfer learning in CNNs:

- **Feature Extraction:** The pretrained CNN's convolutional base is used to extract features from new images. The fully connected layers at the end are replaced with new layers suited for the new task. The pretrained layers' weights are usually frozen (not updated during training), and only the new classifier layers are trained.
- **Fine-tuning:** After replacing the final layers, some or all of the pretrained convolutional layers are unfrozen and retrained (with a smaller learning rate). This allows the model to adapt more specifically to the new dataset while still using the valuable low-level features learned from the large original dataset.

*[Image omitted in PDF — see the online textbook]*

Transfer learning is especially useful when you have limited labeled data for your target task, as it helps improve accuracy and speeds up training by starting from a pretrained model.

## 160 Summary of the Chapter

CNNs are a powerful evolution of traditional neural networks tailored to work with spatial data like images. Their architecture—built on convolution, activation, and pooling layers—enables them to learn hierarchical, spatially-aware representations that general neural networks can't.

A CNN consists of several key layers:

- \* Convolutional layers: Extract features using learnable filters
- \* Pooling layers: Reduce spatial dimensions (max pooling, average pooling)
- \* Fully connected layers: Perform classification/Regression.
- \* Activation functions: Introduce non-linearity (ReLU, softmax)

As we move forward, CNNs form the basis for advanced models in vision and even inspire architectures used in other domains such as NLP (e.g., CNN-based text classifiers).

## 161 Knowledge Check

### 161.1 interactive cnn

## 162 Interactive CNN

*[Interactive content omitted in PDF]*

## 163 Chapter 20 - NLPs

## 164 Introduction: What is Natural Language Processing?

Natural Language Processing (NLP) is a subfield of Artificial Intelligence (AI) and Computational Linguistics focused on enabling machines to understand, interpret, and generate human language in a valuable way. As the primary mode of human communication, natural language is rich, nuanced, and often ambiguous. NLP bridges the gap between human language and machine-understandable formats, enabling a wide range of applications, from automated translation systems to intelligent chatbots and sentiment analysis engines.

In simple terms:

NLP teaches machines how to read, listen, understand, and talk like humans, or at least as close as we can get.

With the rapid growth of digital data—emails, social media posts, news articles, product reviews, and more—NLP has become a cornerstone technology in data-driven decision-making. NLP combines techniques from computer science, linguistics, and machine learning to process and analyze large amounts of natural language data. Advances in machine learning and deep learning have pushed NLP from rule-based systems to state-of-the-art models capable of reasoning with context, tone, and semantics. Applications of NLP include:

- Machine Translation (e.g., Google Translate)
- Sentiment Analysis (e.g., analyzing product reviews)
- Chatbots & Virtual Assistants (e.g., Siri, Alexa)
- Text Summarization (e.g., news article summarization)
- Speech Recognition (e.g., voice-to-text transcription)

## 165 20.1 The Importance of NLP

Language is the primary medium of human communication, making NLP essential for bridging the gap between humans and machines. Key reasons for its importance include:

- **Automation of Text Processing:** Reduces manual effort in analyzing documents, emails, and reports.
- **Enhancing Human-Computer Interaction:** Enables voice assistants and chatbots to respond naturally.
- **Business Intelligence:** Helps companies analyze customer feedback, social media trends, and market sentiments.

- **Accessibility:** Powers tools like speech-to-text for the hearing impaired and language translation for global communication.

## 166 20.2 Historical Evolution of NLP

### 166.0.1 20.2.1 Early Rule-Based Systems (1950s–1980s)

NLP began in the 1950s with a focus on symbolic approaches. Early researchers developed rule-based systems where grammar rules were manually coded. For example, machine translation systems like the Georgetown-IBM experiment (1954) translated a few Russian sentences into English using a small set of linguistic rules.

Challenges:

- High maintenance effort
- Poor adaptability to new domains
- Limited understanding of context

### 166.0.2 20.2.2 Statistical NLP (1990s–2010s)

With the availability of large datasets and increased computational power, NLP shifted toward statistical models. Probabilistic approaches, such as n-gram language models, Bayesian classifiers, and Hidden Markov Models (HMMs), became popular. These methods learned patterns from text data instead of relying solely on hand-written rules.

Key advantages:

- Better scalability
- Adaptation to multiple domains
- Limitations remained in handling long-term dependencies and semantic understanding.

### 166.0.3 20.2.3 Neural NLP and Deep Learning Era (2013–Present)

The advent of deep learning revolutionized NLP. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks improved the modeling of sequential data. Later, transformer architectures—introduced in “**Attention Is All You Need**” (2017)—enabled massive improvements in tasks like translation, summarization, and question answering. Pre-trained language models such as BERT, GPT, and T5 have since set new benchmarks across nearly all NLP tasks.

## 167 20.3 The Challenge of Human Language

Despite advancements, NLP faces several challenges due to the complexity of human language. Understanding human language is much harder than processing numbers or images. Why? Because language is messy.

### 167.0.1 Key Challenges in NLP

Despite advancements, NLP faces several challenges due to the complexity of human language:

## 1. Ambiguity

Type	Description	Example
<b>Lexical Ambiguity</b>	Words with multiple meanings.	“Bank” (financial institution vs. river edge)
<b>Syntactic Ambiguity</b>	Sentence structures with multiple interpretations.	“I saw the man with the telescope” (who had the telescope?)
<b>Semantic Ambiguity</b>	Phrases requiring context for disambiguation.	“Time flies like an arrow” (time moves quickly vs. insects resembling arrows?)

## 2. Context Understanding:

- Sentences change meaning based on situation. E.g., cold ice cream vs. cold soup.
- Human language depends on **tone, sarcasm, and cultural references**, which are hard for machines to interpret.
- **Example:**
  - *Sincere*: “That was a **great** movie!” (positive)
  - *Sarcastic*: “That was a **great** movie...” (negative, with ironic tone)

## 3. Slang & Informal Text

- Handling emojis, abbreviations (e.g., “BRB,” “LOL”).

## 4. Data Sparsity & Rare Words

- **Problem:** Many words appear infrequently in training data, making it hard for models to learn their meanings.
- **Solutions:**
  - **Subword Tokenization:** Break words into smaller units (e.g., Byte Pair Encoding)
  - **Pre-trained Embeddings:** Use models like Word2Vec or GloVe

## 5. Multilingual & Low-Resource Languages

- **Problem:** Bias toward high-resource languages (English, Chinese) as most NLP models are trained on these high-resource languages, leaving many languages underrepresented.
- **Solutions:**
  - **Multilingual Models:** mBERT, XLM-R
  - **Transfer Learning:** Adapt pre-trained models

## 6. Morphological Complexity

- Different languages have different grammar rules.

Language	Challenge	Example (Analysis)
<b>Finnish</b>	Agglutinative morphology	“Talossanikin” = “talo-ssa-ni-kin” (house-in-my-too)
<b>Arabic</b>	Root-and-pattern system	” “ (k-t-b) → “kataba” (wrote), “kitāb” (book)
<b>Turkish</b>	Vowel harmony	“evlerimizde” (in our houses) → 4 suffixes

## 168 20.4 Fundamental NLP Tasks: Pipeline of NLP

NLP covers a broad range of tasks. Some are foundational building blocks; others are application-focused. The Natural Language Processing (NLP) pipeline includes: - **Raw Data (documents/text) Collection** - **Text Preprocessing** - **Basic Preprocessing**: Text cleaning, Tokenization, Stemming etc. - **Advanced Preprocessing**: POS tagging etc. - **Feature Extraction/Engineering** - **ML Modeling** - **Evaluation, Deployment and Monitoring**

*[Image omitted in PDF — see the online textbook]*

*[Image omitted in PDF — see the online textbook]*

*[Image omitted in PDF — see the online textbook]*

### 168.0.1 20.4.1 Data acquisition / raw data collection

The first step in NLP is to obtain the raw text data in the form of documents, such as tweets, articles, reviews.

### 168.1 Basic Preprocessing:

#### 168.1.1 20.4.2 Text preprocessing

Before analysis, raw text must be cleaned and structured. That is why, in this step, the raw text is pre-processed by performing various operations such as tokenization, stop-word removal, stemming, and lemmatization, etc. This step is important as it helps to clean and normalize the data, making it easier to work.

*[Image omitted in PDF — see the online textbook]*

*[Image omitted in PDF — see the online textbook]*

- **Text Cleaning**: Removing noise, unwanted symbols, and irrelevant data.
- **Tokenization**: Splitting text into words or subwords (making it easier for machines to process). Example: “I love NLP!” → [“I”, “love”, “NLP”, “!”]

*[Image omitted in PDF — see the online textbook]*

There can be different type of tokenizations:

*[Image omitted in PDF — see the online textbook]*

- **Stopword Removal**: (Filtering out common words (e.g., “the,” “is”)) It involves identifying and removing common words such as “the”, “a”, “an”, “and”, “of”, that are unlikely to carry

significant meaning in the text. These words are commonly referred to as “stop words” because they can be “stopped” or removed without affecting the overall meaning of the text.

*[Image omitted in PDF — see the online textbook]*

By removing stop words, it becomes possible to reduce the size of the text data and focus on the most meaningful words in the text.

- **Stemming & Lemmatization:** Reducing words to their base forms (e.g., “running” → “run”).
  - **Stemming:** Remove affixes from a word to get base form (stem) of a word (stem might not be a lexicographically correct word). Example: “running” → “run” (rough cut). Good for search engines and simple tasks.
  - **Lemmatization:** Find lemma (dictionary form) of a inflected word (a lemma is always a lexicographically correct word). Example: “running” → “run” (dictionary-based, more precise). Best for tasks where meaning and context are important, like text classification and sentiment analysis.
- **Lowercasing and Normalization:** Ensuring consistent text representation.
- **Handling Punctuation & Numbers::** Remove or normalize.
- **Spell Correction:** Fix typos (“NLP is awsome” → “awesome”).
- **Special Character Removal:** Remove punctuation (e.g., “Hello!” → “Hello”), numbers, emojis (or converting emojis into textual descriptions), HTML tags (e.g., “good” → “good”), and other non-alphabetic symbols. Extra spaces, tabs, and formatting characters are also cleaned to ensure uniform text.

## 168.2 Advanced Preprocessing:

- **Part-of-Speech (POS) Tagging:** is the process of assigning grammatical labels (lexical category) to each word in a sentence, such as noun, verb, adjective, or preposition. For example, in the sentence “The quick brown fox jumps over the lazy dog,” the words would be tagged as follows: “The” as a determiner, “quick” and “brown” as adjectives, “fox” and “dog” as nouns, “jumps” as a verb, and “over” as a preposition.

**Some Common POS Tags:** NN (Noun), VB (Verb), JJ (Adjective), RB (Adverb), PRP (Pronoun), IN (Preposition) and DT (Determiner).

*[Image omitted in PDF — see the online textbook]*

Why It Matters: - Helps machines understand sentence structure (grammar). - Essential for lemmatization (reducing words to their base forms). - Improves text analysis in tasks like parsing, translation, and sentiment analysis.

- **Named Entity Recognition (NER):** Finding names of people, organizations, locations, etc.
- **Dependency Parsing:** Understanding grammatical relationships between words.

## 169 20.5 Text Representation and Feature Extraction/Representation (Turning texts into vectors)

Machines cannot directly understand raw text; it must first be transformed into a numerical format. Feature extraction is the process of converting text into numerical representations—such as vectors capturing word frequencies or semantic meanings—so that machine learning models can interpret and use them for tasks such as sentiment analysis, classification, or translation.

There are several common approaches to representing text numerically:

### 169.0.1 20.5.1 Bag-of-Words (BoW)

The **Bag-of-Words (BoW)** model is one of the simplest and most widely used text representation techniques. It represents text by counting the occurrences of each word in the document, completely ignoring grammar, word order, and context.

**Concept:** Treats each document as a “bag” of words—only the words themselves and their counts matter, not the order in which they appear.

*[Image omitted in PDF — see the online textbook]*

**Steps to follow:**

- Tokenize the text (split into words).
- Build a vocabulary of all unique words across the corpus.
- Count the frequency of each word in each document.
  - Why? Each document is transformed into a vector where each position corresponds to a word in the vocabulary and the value is the frequency (count) of that word in the document.

**Example:**

Document: “Hi. This is an example sentence in an Example Document.”

- **Tokenization (lowercased, punctuation removed):**
- [hi, this, is, an, example, sentence, in, an, example, document]
- **Vocabulary:** [hi, this, is, an, example, sentence, in, document]
- Word Frequencies in the document: [1, 1, 1, 2, 2, 1, 1, 1]
- **Word Frequencies in the document:** [1, 1, 1, 2, 2, 1, 1, 1] → corresponds to the order of the vocabulary.

So the BoW vector for this document is: [1, 1, 1, 2, 2, 1, 1, 1]

**More Example(s):**

*[Image omitted in PDF — see the online textbook]*

**Advantages:** Simple and fast, works well for basic text classification.

**Limitations:** Ignores meaning and word order, produces sparse vectors with many zeros for large vocabularies. As vocabulary size can grow rapidly, increasing memory usage.



## 169.0.2 20.5.2 N-grams

BoW consider individual words (unigrams), but language meaning often depends on word combinations. N-grams are a variation of Bag of Words that captures sequences of N adjacent words.

**Types:**

- **Unigram:** single words (“machine”, “learning”)
- **Bigram:** pairs of words (“machine learning”)
- **Trigram:** triples (“natural language processing”)

**Example:**

*[Image omitted in PDF — see the online textbook]*

N-grams help models understand phrases and partial word order, improving performance in many tasks. Still it has some limitation such as larger n can create exponentially larger vocabulary, rare phrases may not generalize well, etc.

## 169.0.3 20.5.3 TF-IDF (Term Frequency-Inverse Document Frequency)

While BoW treats all words equally, TF-IDF weighs words based on how important they are to a document relative to a collection (corpus) of documents.

TF-IDF assigns more weight to **informative words** that are **frequent** in a specific document but **rare** across the corpus. This helps downweight common words like “the” or “is.”

**Formula(s): Term Frequency (TF):** How often a word appears in a document.

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_{t' \in d} \text{count}(t', d)}$$

**Document Frequency (DF):** Number of documents that contain the term.

$$\text{DF}(t) = n_t$$

where ( $n_t$ ) is the number of documents containing term ( $t$ ).

**Inverse Document Frequency (IDF):** How rare a word is across all documents.

$$\text{IDF}(t) = \log \left( \frac{N}{\text{DF}(t)} \right)$$

where ( $N$ ) is the total number of documents.

**TF-IDF:**

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Words that appear frequently in one document but rarely in others get higher weights, highlighting their importance. This helps reduce the impact of common words like “the” or “is” that appear everywhere.

**Example:** Calculating DF, IDF (base-10), TF, and TF-IDF

We have **3 documents**:

1. machine learning is fun
2. machine learning is powerful
3. deep learning powers AI

**Vocabulary:**

[machine, learning, is, fun, powerful, deep, powers, ai]

---

#### 169.0.4 Step 1 — Document Frequency (DF)

DF counts how many documents contain the term:

$$DF(\text{machine}) = 2 \quad (\text{Doc1 Doc2})$$

$$DF(\text{fun}) = 1 \quad (\text{Doc1 only})$$

---

#### 169.0.5 Step 2 — Inverse Document Frequency (IDF)

Using **log base 10** (to match the numeric examples):

$$IDF(t) = \log_{10} \left( \frac{N}{DF(t)} \right), \quad N = 3$$

So:

$$IDF(\text{machine}) = \log_{10} \left( \frac{3}{2} \right) \approx 0.17609$$

$$IDF(\text{fun}) = \log_{10} \left( \frac{3}{1} \right) \approx 0.47712$$

(Interpretation: rarer terms  $\rightarrow$  larger IDF.)

---

### 169.0.6 Step 3 — Term Frequency (TF)

For a term ( $t$ ) in document ( $d$ ):

$$TF(t, d) = \frac{\text{count}(t, d)}{\text{total terms in } d}$$

Example for **Doc 1** (“machine learning is fun”):

Total terms in Doc1 = 4.

$$TF(\text{machine}, \text{Doc}_1) = \frac{1}{4} = 0.25$$

$$TF(\text{fun}, \text{Doc}_1) = \frac{1}{4} = 0.25$$

---

### 169.0.7 Step 4 — TF-IDF

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

For **Doc 1**:

$$TF\text{-}IDF(\text{machine}, \text{Doc}_1) = 0.25 \times 0.17609 \approx 0.04402$$

$$TF\text{-}IDF(\text{fun}, \text{Doc}_1) = 0.25 \times 0.47712 \approx 0.11928$$

(Despite equal TF in Doc1, “fun” scores higher because it is rarer across the corpus.)

#### **Observation:**

Even though “machine” and “fun” occur the same number of times in Doc 1, “fun” has a higher TF-IDF score because it is rarer in the overall corpus.

#### **More Example(s):**

*[Image omitted in PDF — see the online textbook]*

### 169.0.8 20.5.4 Word Embeddings

Traditional frequency-based methods treat each word as independent, but words have semantic relationships. Word embeddings map words into dense vectors in a continuous space, where similar words lie close together.

Popular algorithms include: Word2Vec, GloVe

These embeddings capture relationships like:  $\text{vector}(\text{“king”}) - \text{vector}(\text{“man”}) + \text{vector}(\text{“woman”})$   
 $\text{vector}(\text{“queen”})$

## 169.0.9 20.5.5 Contextual Embeddings

Unlike static embeddings (Word2Vec, GloVe), contextual embeddings generate word representations based on surrounding words, capturing meaning that depends on context. For example, the word “bank” in “river bank” vs. “bank account” will have different embeddings.

Modern NLP models such as BERT and GPT produce contextual embeddings, enabling them to understand nuances and disambiguate meanings dynamically.

## 169.1 interactive ngram

## 170 Interactive N-Gram

*[Interactive content omitted in PDF]*

## 170.1 interactive text preprocessing

## 171 Interactive Text Preprocessing

*[Interactive content omitted in PDF]*

## 171.1 interactive tf

## 172 Interactive Text Frequency

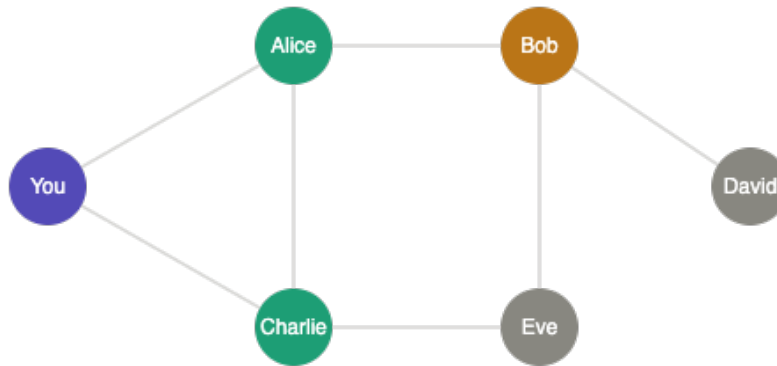
*[Interactive content omitted in PDF]*

## 173 Chapter 21 - Graph Theory

## 174 A Story That Becomes Data

Imagine your first week at university. You meet **Alice** during orientation and **Charlie** in class. Alice later introduces you to **Bob**, who in turn knows **David**. Meanwhile Charlie knows Eve. At first this is just a story, five people, a handful of introductions.

But as a data scientist, you start asking structural questions. Who is most connected? Who acts as a bridge between groups? Can you reach David without going through Alice? If Alice leaves the university, does the network stay connected?



To answer these, we need to convert the story into a formal structure: **a graph**.

### 174.1 What is a Graph?

A **graph** is a structure that represents:

- **Nodes (Vertices):** entities (people, cities, users)
- **Edges:** relationships (friendship, roads, links)

Key Idea: Graph = Data + Relationships

#### 174.1.1 Definition

A **graph** is a mathematical structure:

$$G = (V, E)$$

Symbol	Meaning	Example
V	Set of <b>vertices</b> (nodes)	{You, Alice, Bob, Charlie, David, Eve}
E	Set of <b>edges</b> (connections)	{(You, Alice), (Alice, Bob), ...}

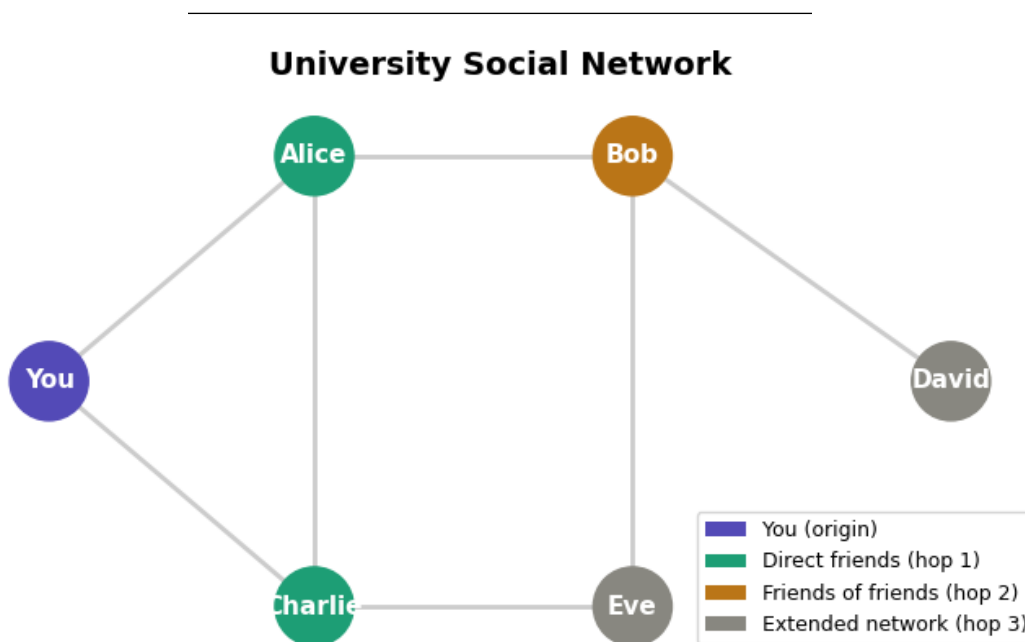
#### 174.1.2 Types of Graphs

Type	Description	Example
<b>Undirected</b>	Edges have no direction; mutual relationship	Friendship network
<b>Directed (Digraph)</b>	Edges have direction (A→B B→A)	Twitter follows, web links

Type	Description	Example
<b>Weighted</b>	Each edge carries a numeric value	Road distances, transaction amounts
<b>Bipartite</b>	Nodes split into two groups; edges only between groups	Users   Movies
<b>Multigraph</b>	Multiple edges allowed between same pair	Multiple flights between two cities

### 174.1.3 Key Terminology

Term	Definition	Example
Degree	Number of edges connected to a node	Alice has degree 3 (connected to You, Bob, Charlie)
Path	A sequence of nodes connected by edges	You → Alice → Bob → David
Shortest Path	The path with fewest hops (or lowest weight)	The path of length 3 above is the only way to reach David
Cycle	A path that starts and ends at the same node	You → Alice → Charlie → You
Connected	Every node can reach every other node	Our university graph is connected
Component	A maximal connected subgraph	Disconnecting Alice would create two components



Nodes : 6  
Edges : 7  
Density: 0.467

## 174.2 Applications of Graphs

Graphs are not just academic; they power real-world systems:

- **Navigation (Google Maps):** Road networks are weighted directed graphs: nodes are intersections, edges are road segments, weights are travel times that update in real time from GPS data. Dijkstra and A\* (which uses geographic distance as a heuristic to guide search) find your route. The challenge is scale: the US road network has ~50 million nodes. > Nodes → locations, Edges → roads (Used for shortest path).
- **Social Networks:** Facebook’s social graph had ~3 billion nodes at last public reporting. It powers friend recommendations (who shares many mutual friends with you?), feed ranking, and ad targeting. The graph is stored in a custom distributed system called TAO. > Nodes → users, Edges → connections (Used for recommendations).
- **Recommendation Systems:** Graphs like “Bipartite” (where every edge connects a vertex from one set to a vertex from the other set.) connect users to items they’ve interacted with. Collaborative filtering (we will discuss this in *later chapter*) asks: which items are connected to users similar to you? More sophisticated approaches use graph neural networks that propagate information through multi-hop neighborhoods. Web search and PageRank  
Nodes → users + products, Edges → interactions  
- **Web Search:** The web is a directed graph with ~60 trillion pages. PageRank assigns each page a score equal to the sum of PageRank scores of pages linking to it, divided by their out-degree — a recursive definition solved as an eigenvector computation. Pages cited by authoritative pages inherit authority. Nodes → webpages, Edges → hyperlinks
- **Biology and medicine:** Protein-protein interaction networks reveal which proteins work together. Metabolic networks model how cells convert nutrients to energy. Drug repurposing finds new uses for existing drugs by identifying proteins structurally close to a disease target in the interaction graph. Graph neural networks now power breakthrough protein structure prediction tools.
- **Fraud detection:** Financial transaction graphs detect anomalous patterns — cycles of transactions that suggest money laundering, or suspicious clustering of accounts. Graph-based features dramatically outperform purely tabular approaches for this problem.

Domain	Nodes	Edges	Algorithm used
<b>Navigation</b> (Google Maps)	Intersections	Roads	Dijkstra / A*
<b>Social Networks</b> (Facebook)	Users	Friendships	Community detection, BFS
<b>Recommendations</b> (Netflix)	Users + Movies	Ratings/views	Collaborative filtering, GNN
<b>Web Search</b> (Google)	Web pages	Hyperlinks	PageRank

Domain	Nodes	Edges	Algorithm used
<b>Biology</b>	Proteins	Interactions	Centrality, clustering
<b>Fraud Detection</b>	Accounts	Transactions	Anomaly detection
<b>Supply Chain</b>	Warehouses/ports	Routes	Min-cost flow

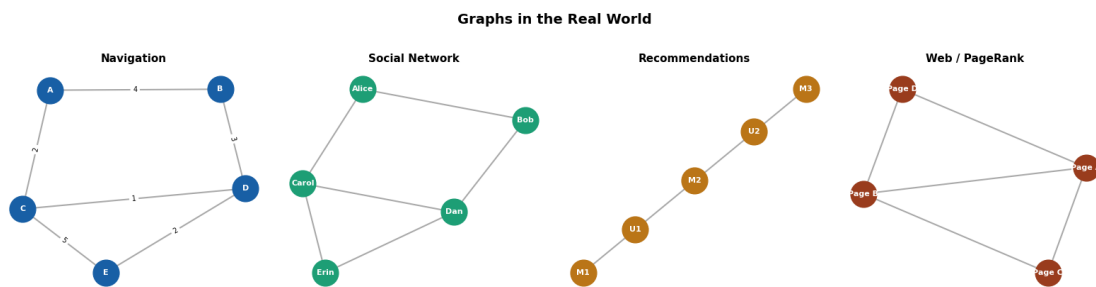
## 174.3 Graph Processing

Once we build a graph, we analyze it.

### 174.3.1 Common Tasks:

- **Traversal (BFS, DFS):** Systematically explore all nodes and edges in a graph.
- **Shortest Path:** Find the minimum-cost route between two nodes.
- **Community Detection:** Identify groups of densely connected nodes.
- **Ranking nodes:** Measure and rank node importance based on structure
- **Link prediction:** Predict likely future or missing connections between nodes.

Insight: Graph processing = turning connections into decisions



## 174.4 Representing Graphs: 3 Ways to Store a Graph

Before running any algorithm, you must choose a data structure (because you need to store the graph). The right choice depends on the graph's **size**, **density**, and what **operations** you need to perform fast.

### 174.4.1 Summary

Property	Adjacency Matrix	Adjacency List	Adjacency Dictionary
Space	$O(V^2)$	$O(V + E)$	$O(V + E)$
Edge lookup	$O(1)$	$O(\text{degree})$	$O(1)$ average
Iterate neighbors	$O(V)$	$O(\text{degree})$	$O(\text{degree})$
Best for	Dense graphs, fast lookups	Sparse graphs (most real-world)	Fast lookups with flexible structure
Worst for	Large sparse graphs	Edge weight matrix	Slight overhead due to hashing



### 174.4.2 (1) Adjacency Matrix

An  $N \times N$  matrix where  $M[i][j] = 1$  (or the edge weight) if an edge exists from node  $i$  to node  $j$ , otherwise 0.

- Edge existence check in  $O(1)$
- Wastes  $O(V^2)$  memory — 1 million nodes =  $10^{12}$  entries

*[Image omitted in PDF — see the online textbook]*

Adjacency Matrix. Source: GeeksforGeeks

```
4 nodes: You=0, Alice=1, Bob=2, Charlie=3
adj_matrix = [
 [0, 1, 0, 1], # You connects to Alice and Charlie
 [1, 0, 1, 1], # Alice connects to You, Bob, Charlie
 [0, 1, 0, 0], # Bob connects to Alice
 [1, 1, 0, 0], # Charlie connects to You and Alice
]
```

### 174.4.3 (2) Adjacency List

Each node stores a **list of its neighbors**. Only edges that exist take up memory.

- Space efficient:  $O(V + E)$  - far more efficient for sparse graphs (most real-world graphs have far fewer edges than the maximum possible).
- Fast neighbor iteration — most graph algorithms use this
- Checking if a specific edge exists requires scanning the neighbor list:  $O(\text{degree})$

*[Image omitted in PDF — see the online textbook]*

Adjacency List. Source: GeeksforGeeks

```
graph = {
 'You': ['Alice', 'Charlie'],
 'Alice': ['You', 'Bob', 'Charlie'],
 'Bob': ['Alice', 'David'],
 'Charlie': ['You', 'Alice', 'Eve'],
 'David': ['Bob'],
 'Eve': ['Bob', 'Charlie'],
}
```

Real-world graphs use libraries like [NetworkX](#) (Python), [igraph](#), or graph databases like [Neo4j](#) for billion-edge scale.

### 174.4.4 (3) Adjacency Dictionary

An **adjacency dictionary** is a dict-of-dicts. It stores each node and its neighbors as **key–value pairs**.

- Outer key  $\rightarrow$  node

- Inner key  $\rightarrow$  neighbor
- Inner value  $\rightarrow$  weight (or 1 if unweighted)

Some key advantages: - Fast edge lookup  $\rightarrow$  average **O(1)**

- Naturally supports **weighted graphs**
- Flexible for storing extra information (distance, cost, frequency)

**Intuition:** *Adjacency List* = simple neighbors ; *Adjacency Dictionary* = neighbors + extra information

```
graph_weighted = {
 'You': {'Alice': 2, 'Charlie': 1},
 'Alice': {'You': 2, 'Bob': 3, 'Charlie': 2},
 'Bob': {'Alice': 3, 'David': 4},
 'Charlie': {'You': 1, 'Alice': 2, 'Eve': 5},
 'David': {'Bob': 4},
 'Eve': {'Bob': 2, 'Charlie': 5},
}
```

Compared to an adjacency list (dict of lists):

Operation	Adj. List	Adj. Dictionary
Space	$O(V + E)$	$O(V + E)$
Check if edge (u,v) exists	$O(\text{degree}(u))$	<b>O(1)</b>
Get weight of edge (u,v)	$O(\text{degree}(u))$	<b>O(1)</b>
Add edge	$O(1)$ append	$O(1)$ assign
Remove edge	$O(\text{degree})$	<b>O(1)</b> del
Iterate neighbors of u	$O(\text{degree})$	$O(\text{degree})$

This is the representation used internally by **NetworkX** (`G._adj`) and by most production graph libraries.

Variants: - **Unweighted** — inner value is 1 (or `{}`) — just tracks existence - **Weighted** — inner value is a number (distance, cost, probability) - **Attributed** — inner value is a dict holding multiple properties per edge

=== Adjacency Matrix ===

	You	Alice	Bob	Charlie	David	Eve
You	0	1	0	1	0	0
Alice	1	0	1	1	0	0
Bob	0	1	0	0	1	1
Charlie	1	1	0	0	0	1
David	0	0	1	0	0	0
Eve	0	0	1	1	0	0

## Adjacency Matrix

You	0	1	0	1	0	0
Alice	1	0	1	1	0	0
Bob	0	1	0	0	1	1
Charlie	1	1	0	0	0	1
David	0	0	1	0	0	0
Eve	0	0	1	1	0	0
	You	Alice	Bob	Charlie	David	Eve

Matrix size :  $6 \times 6 = 36$  cells  
 Non-zero : 14 (edges  $\times 2$  for undirected)  
 Sparsity : 61.1%  $\leftarrow$  most cells are wasted zeros

=== Adjacency List ===

```

You → ['Alice', 'Charlie']
Alice → ['You', 'Bob', 'Charlie']
Bob → ['Alice', 'David', 'Eve']
Charlie → ['You', 'Alice', 'Eve']
David → ['Bob']
Eve → ['Bob', 'Charlie']

```

Total entries stored: 14 ( $= 2 \times |E| = 2 \times 7$ )  
 vs adjacency matrix : 36 entries  
 Memory saving : 61.1% fewer entries

===  $O(1)$  edge operations ===

Edge (You, Alice) exists?    True

```

Edge (You, David) exists? False
Weight of (Alice, Bob): 0.5
Degree of Alice: 3

=== Adding and removing edges ===
After adding David-Eve: ['Bob', 'Eve']
After removing David-Eve: ['Bob']

=== Iterating neighbors ===
Alice → You weight=0.9
Alice → Bob weight=0.5
Alice → Charlie weight=0.8

```

## 175 Graph Featurization

Raw graph structure alone is not always enough for machine learning. **Featurization** converts graph structure and node/edge properties into numeric vectors that models can consume.

### 175.1 Why Featurization?

Graphs are inherently relational, but most ML models expect **fixed-size numeric inputs**. Featurization bridges this gap.

### 175.2 Types of Graph Features

There are four main types:

Type	What it captures	Examples
<b>1. Graph-level features</b>	Properties of the whole graph	Diameter, density, spectral features
<b>2. Edge features</b>	Properties of connections	Weight, transaction amount, timestamp
<b>3. Node features</b>	Properties of individual nodes	Age, degree, label
<b>Structural features</b>	Topological role of a node	Centrality, clustering coefficient, motif count

In Graph Neural Networks, the model learns to aggregate these features across multi-hop neighborhoods automatically.

#### 175.2.1 1. Graph-Level Features (Global View)

Describe the **entire graph** as a whole.

#### 175.2.2 What they capture:

- Overall connectivity
- Global structure

- Network complexity

### 175.2.3 Common Features:

- **Diameter** → longest shortest path in the graph
- **Average path length** → average distance between nodes
- **Density** → how many edges exist vs possible edges
- **Modularity** → strength of community structure

### 175.2.4 Example:

- Social network: **High density** → many people are connected; **Low diameter** → information spreads quickly.
- Used in: Graph classification (e.g., molecule type, network type)

### 175.2.5 2. Edge-Level Features (Relationship View)

Describe **connections between two nodes**.

### 175.2.6 What they capture:

- Strength or nature of relationships
- Likelihood of future connections

### 175.2.7 Common Features:

- **Weight** → distance, cost, interaction frequency
- **Common neighbors** → shared connections
- **Jaccard similarity** → similarity between neighborhoods
- **Edge betweenness** → importance of an edge

### 175.2.8 Example:

- E-commerce:
  - Edge = user → product
  - Weight = number of purchases
- Social network:
  - Two users with many common friends → high similarity

Used in: Link prediction, Recommendation systems, Fraud detection.

### 175.2.9 3. Node-Level Features (Local View — Focus)

Describe **individual nodes**. There are two ways - via “Degree” and via “Centrality”

(a) **Degree:** Number of connections a node has

**Types:** - **In-degree** → incoming edges

- **Out-degree** → outgoing edges

#### 175.2.10 Example:

- Twitter:
  - High in-degree → many followers
  - High out-degree → follows many people

**Interpretation:** High degree → more connected → potentially more influence

(b) **Centrality (Covered Next Section)** Centrality measures capture **node importance** in the network.

- **Degree Centrality:** Measures importance based on the number of direct connections a node has.
- **Closeness Centrality:** Measures how quickly a node can reach all other nodes in the network.
- **Betweenness Centrality:** Measures how often a node lies on the shortest paths between other nodes (bridge nodes).
- **Eigenvector Centrality:** Measures importance by considering not just connections, but how important those neighbors are.

### 175.3 4. Structural Features (Topological Role)

Describe how a node is positioned in the **overall structure**.

#### 175.3.1 What they capture:

- Local neighborhood patterns
- Role of node in graph structure

#### 175.3.2 Common Features:

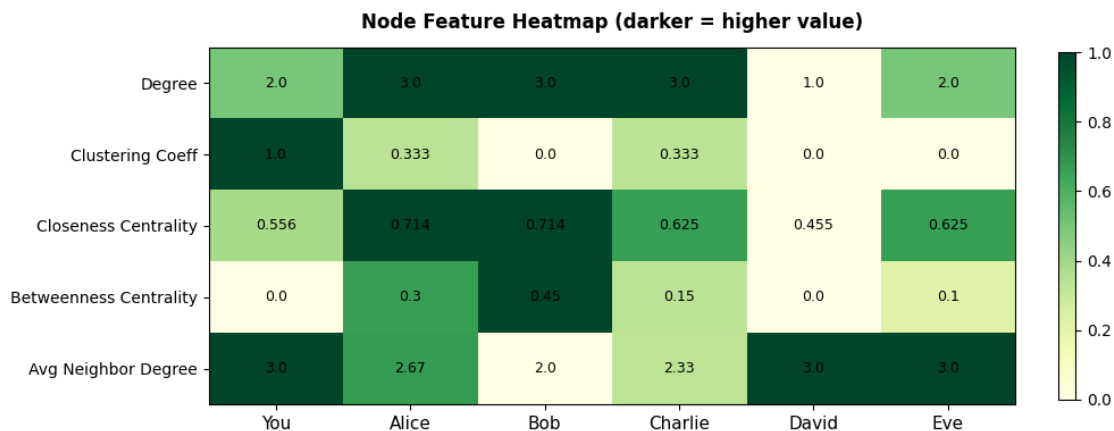
- **Clustering coefficient** → how connected neighbors are
- **Triangle count** → number of triangles a node participates in
- **Motifs** → recurring subgraph patterns

### 175.3.3 Example:

- Social network: High clustering → tight friend group
- Fraud detection: Unusual patterns → suspicious behavior

=== Node Feature Matrix (structural features) ===

	Degree	Clustering Coeff	Closeness Centrality	Betweenness Centrality
Avg Neighbor Degree				
Node				
You	2.0	1.000	0.556	0.00
3.00				
Alice	3.0	0.333	0.714	0.30
2.67				
Bob	3.0	0.000	0.714	0.45
2.00				
Charlie	3.0	0.333	0.625	0.15
2.33				
David	1.0	0.000	0.455	0.00
3.00				
Eve	2.0	0.000	0.625	0.10
3.00				



## 175.4 Centrality Measures ( Node Importance)

Not all nodes are equal. Some nodes connect many others; some nodes act as bridges and some nodes are closer to everyone.

**Centrality measures** quantify **how important or influential a node is** within a graph (or network).

**Key insight:** Different centrality metrics answer different questions about importance.

A node can rank #1 on one metric and #5 on another.

We will cover three foundational measures:

Measure	Core question	Analogy
<b>Degree centrality</b>	How many direct connections?	Popularity
<b>Closeness centrality</b>	How quickly can you reach everyone?	Efficiency
<b>Betweenness centrality</b>	How often do you sit on the shortest path between others?	Broker / gatekeeper

[Image omitted in PDF — see the online textbook]

Illustration of different centrality roles in a network. Source: towardsdatascience.com

#### 175.4.1 1. Degree Centrality

The simplest measure. Measures importance based on **number of direct connections**.

A node's degree centrality is its **number of edges divided by the maximum possible edges**.

**Example:** who has the *most connections*? Alice and Bob tie in our example. In a social network, a person with many friends has high degree centrality

$$C_D(v) = \frac{\deg(v)}{N - 1}$$

where N is the total number of nodes.

- High degree → directly connected to many nodes → “popular” or “well-networked”
- Fast to compute:  $\mathbf{O(V + E)}$
- Limitation: ignores the quality/importance of connections (a node connected to 10 isolated nodes vs. 10 hubs)

#### 175.4.2 2. Closeness Centrality

Measures how **close a node is to all other nodes**. It focus on how **quickly** a node can reach all others (based on shortest path). A node is close-central if its average shortest path to every other node is short.

**Example:** who can reach everyone else *most quickly on average*? A node that can quickly reach everyone (few hops) has high closeness.

$$C_C(v) = \frac{N - 1}{\sum_{u \neq v} d(v, u)}$$

where  $d(v, u)$  is the shortest path length between nodes v and u.

[Image omitted in PDF — see the online textbook]

Nodes with high closeness are closer (shorter paths) to all others. Source: www.ebi.ac.uk

- High closeness → information/influence spreads **fast** from this node
- In a social network: the person who hears gossip first, or spreads news fastest
- In a supply chain: the warehouse that can ship to any destination quickest
- Complexity:  $\mathbf{O(V \times (V + E))}$  using BFS from each node



### 175.4.3 3. Betweenness Centrality

Measures how often a node acts as a **bridge** along the shortest path between two other nodes.

How often a node lies on **shortest paths between other nodes**. Identifies **bridge nodes**

**Example:** who is *connected to other well-connected nodes*? This is the core idea behind Google's PageRank. A person connecting two groups has high betweenness

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where: -  $\sigma_{st}$  = total number of shortest paths from s to t -  $\sigma_{st}(v)$  = number of those paths that pass through v

**Notes:** The normalized version divides by  $(N - 1)(N - 2)/2$  so scores fall in  $[0, 1]$ . But in this course, we are not doing it.

- High betweenness → **gatekeeper / broker** — controls information flow
- Removing a high-betweenness node disrupts the network most
- Used in: fraud detection (unusual intermediaries), infrastructure resilience, influence campaigns
- Complexity: **O(VE)** (Brandes algorithm)

### 175.5 When to Use Which?

- Degree → quick measure of connectivity
- Closeness → efficiency of communication
- Betweenness → control over information flow
- Eigenvector → influence in the network

### 175.6 Connection to Featurization

Centrality measures are **node-level features** used in: - Recommendation systems

- Social network analysis

- Fraud detection

- Graph-based machine learning

These features help models understand **node importance and role**

Degree: {'You': 0.4, 'Alice': 0.4, 'Charlie': 0.4, 'Bob': 0.4, 'David': 0.2, 'Eve': 0.2}

Closeness: {'You': 0.5555555555555556, 'Alice': 0.5555555555555556, 'Charlie': 0.45454545454545453, 'Bob': 0.45454545454545453, 'David': 0.3333333333333333, 'Eve': 0.3333333333333333}

Betweenness: {'You': 0.6000000000000001, 'Alice': 0.6000000000000001, 'Charlie': 0.4, 'Bob': 0.4, 'David': 0.0, 'Eve': 0.0}

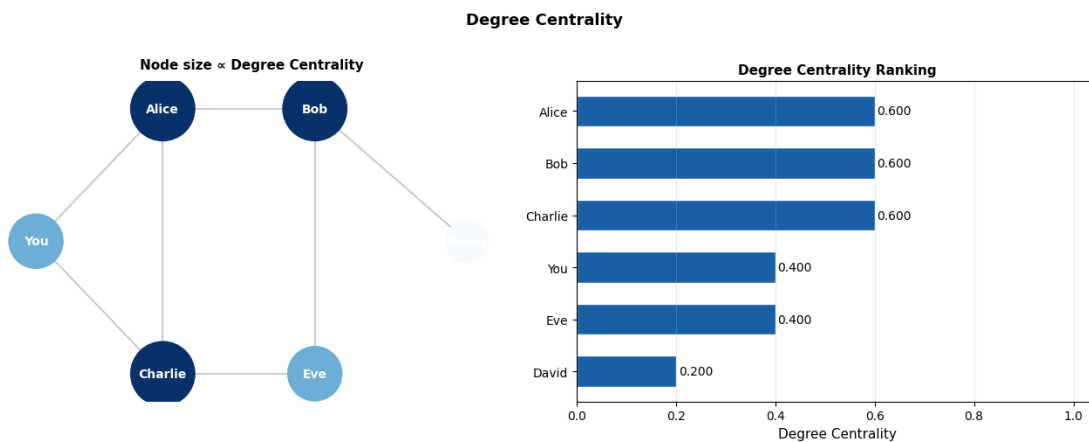
Eigenvector: {'You': 0.5211200923556694, 'Alice': 0.5211200923556694, 'Charlie': 0.41790694813836154, 'Bob': 0.41790694813836154, 'David': 0.23192160753344954, 'Eve': 0.23192160753344954}

## 176 Some Example Codes (In Python):

=== Degree Centrality ===

Formula:  $C_D(v) = \text{degree}(v) / (N - 1) = \text{degree}(v) / 5$

Alice	0.600	[degree=3]
Bob	0.600	[degree=3]
Charlie	0.600	[degree=3]
You	0.400	[degree=2]
Eve	0.400	[degree=2]
David	0.200	[degree=1]



=== Shortest paths from Alice to all others ===

Alice → You	: 1 hop(s)
Alice → Bob	: 1 hop(s)
Alice → Charlie	: 1 hop(s)
Alice → David	: 2 hop(s)
Alice → Eve	: 2 hop(s)

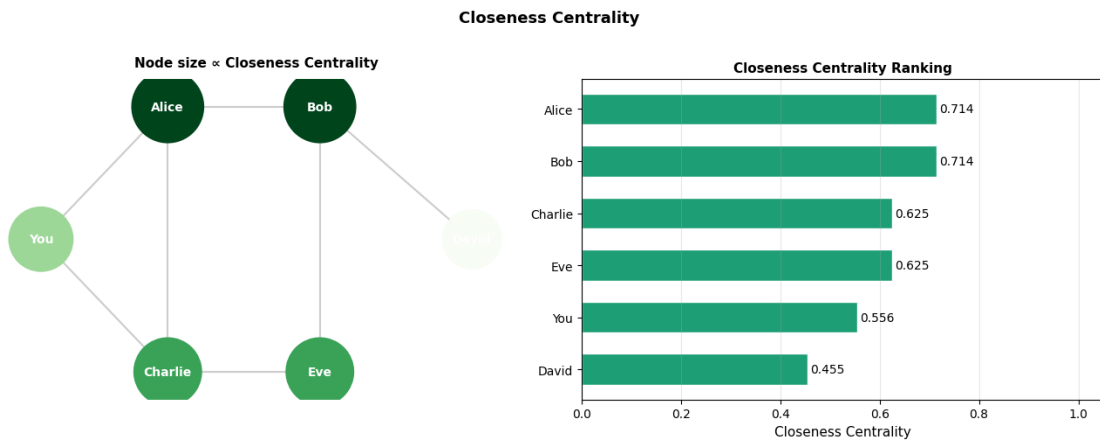
Sum of distances = 7

Closeness(Alice) =  $(N-1) / \text{sum} = 5/7 = 0.714$

=== All Closeness Centralities ===

Alice	0.714
Bob	0.714
Charlie	0.625
Eve	0.625
You	0.556

David 0.455

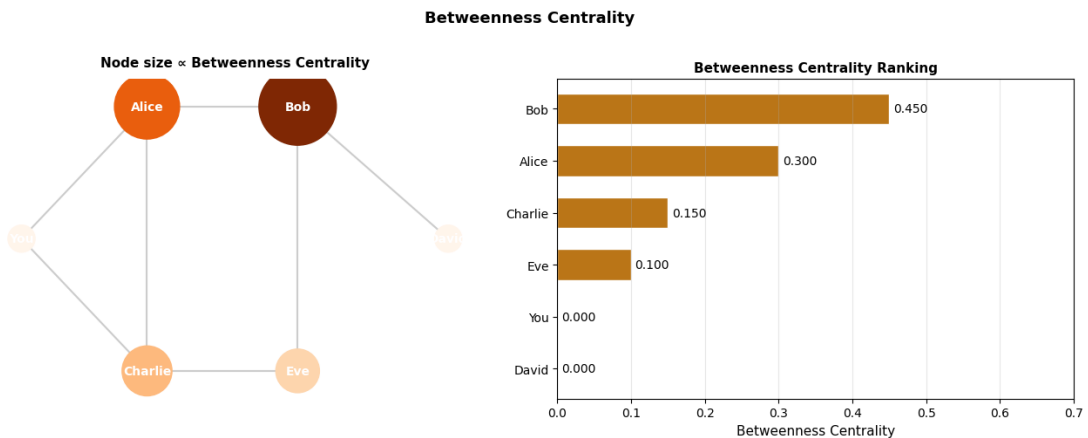


=== Paths that pass through Alice ===

You  $\rightarrow$  Alice  $\rightarrow$  Bob  
 You  $\rightarrow$  Alice  $\rightarrow$  Bob  $\rightarrow$  David  
 Bob  $\rightarrow$  Alice  $\rightarrow$  You  
 Bob  $\rightarrow$  Alice  $\rightarrow$  Charlie  
 Charlie  $\rightarrow$  Alice  $\rightarrow$  Bob  
 Charlie  $\rightarrow$  Alice  $\rightarrow$  Bob  $\rightarrow$  David  
 David  $\rightarrow$  Bob  $\rightarrow$  Alice  $\rightarrow$  You  
 David  $\rightarrow$  Bob  $\rightarrow$  Alice  $\rightarrow$  Charlie

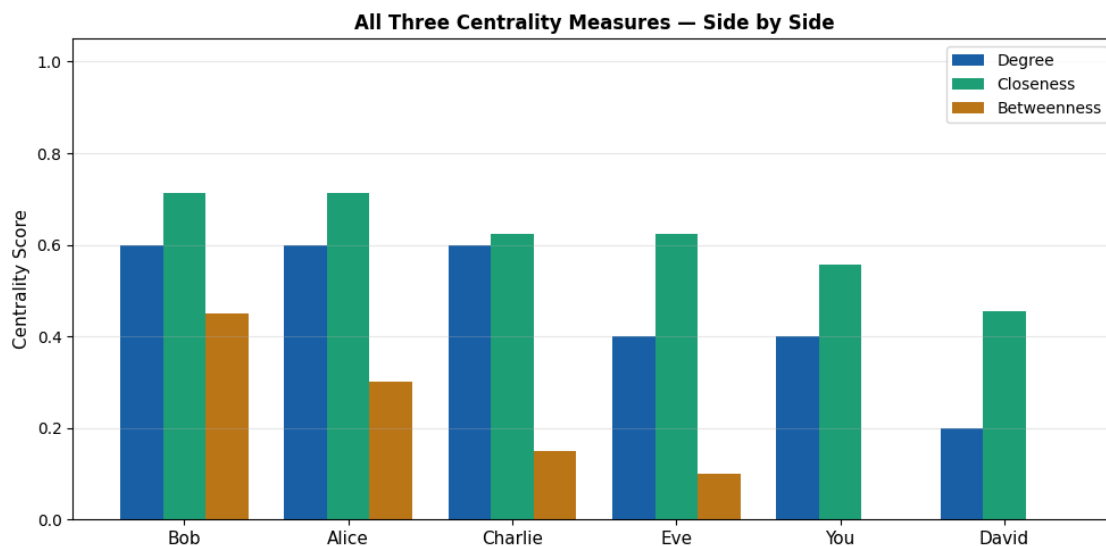
=== All Betweenness Centralities ===

Bob 0.450  
 Alice 0.300  
 Charlie 0.150  
 Eve 0.100  
 You 0.000  
 David 0.000



=== Centrality Comparison Table ===

	Degree	Closeness	Betweenness
Node			
Bob	0.6	0.714	0.45
Alice	0.6	0.714	0.30
Charlie	0.6	0.625	0.15
Eve	0.4	0.625	0.10
You	0.4	0.556	0.00
David	0.2	0.455	0.00



=== Key Takeaways ===

Most connected (Degree) : Alice  
Fastest to reach all (Close): Alice  
Biggest bridge (Between) : Bob

## 177 From Centrality to Community Detection

In the previous section, we explored **centrality measures** to understand node importance in a graph. Where we learnt:

- 
- **Degree centrality** tells us which nodes have many direct connections. These nodes are locally important because they interact with many neighbors.
  - **Closeness centrality** captures how efficiently a node can reach all other nodes. Nodes with high closeness can spread information quickly across the network.

- **Betweenness centrality** measures how often a node lies on shortest paths between other nodes. These nodes often act as **bridges** connecting different parts of the graph.
- 

### 177.1 Transition: From Important Nodes to Important Edges

So far, we focused on **important nodes**. Now, we shift our focus slightly:

Instead of asking “*Which nodes are important?*”, we ask:

**“Which connections (edges) are critical for holding the network together?”**

This is where the idea of **betweenness** becomes especially powerful.

Nodes with high betweenness sit between groups. Similarly, **edges with high betweenness connect different communities**.

### 177.2 Community Detection

Real-world graphs are rarely random; nodes cluster into **communities**: groups with dense internal connections and sparse external ones.

Finding these communities reveals friend circles in social networks, topic clusters in citation graphs, functional modules in protein interaction networks, and fraud rings in financial graphs.

#### 177.2.1 What is a “Community”?

In many real-world networks, nodes naturally organize into **groups**, called **communities**.

**Informally:** a set of nodes that are more connected to each other than to the rest of the graph.

A **community** is a set of nodes that: - are **densely connected to each other**  
- have **fewer connections to nodes outside the group**

Think of friend circles in a social network or research groups in collaboration networks; within a community, there are many internal connections, while only a few edges connect it to other communities.

*[Image omitted in PDF — see the online textbook]*

Community structure in an NCAA football network (teams grouped by conferences). Source: Stanford Network Analysis Project (SNAP)

**Why Do Communities Form?** Communities arise because: - People interact more within their group

- Systems are organized into functional units
- Networks evolve with localized connections

The ability to identify these communities through a process called community detection is arguably the most important and practical application of computer algorithms and graph theory.

### 177.2.2 Role of Connections Between Communities

While communities are densely connected internally, they are typically linked to other groups by only a **small number of edges**. These edges act as **bridges**, enabling communication across different parts of the network.

From our earlier discussion on centrality: - **Degree** captures local connectivity

- **Closeness** reflects how efficiently a node reaches others

- **Betweenness** identifies nodes or edges that lie on many shortest paths

Key Insight: Edges with **high betweenness** often connect different communities because many shortest paths must pass through them.

### 177.2.3 Why Betweenness Reveals Communities

Consider a network with two tightly connected groups and only a few connections between them.

These cross-group edges: - appear frequently in shortest paths

- carry a large portion of network flow

- function as bridges between communities

If we remove these high-betweenness edges, the network naturally separates into distinct clusters.

**Bridge Edge Example:** *[Image omitted in PDF — see the online textbook]*

A small number of edges connect two dense groups (bridge edges)

**Community Separation via Edge Removal:** *[Image omitted in PDF — see the online textbook]*

Removing high-betweenness edges gradually reveals community structure (Source: Miro Medium)

Interpretation:

**Central edges** → connect different groups; **Removing them** → graph splits into communities.

We will deep-dive into the two foundational ideas: **edge betweenness** and the **Girvan–Newman algorithm**, and the quality metric **modularity**.

### 177.3 Edge Betweenness Centrality

**Edge betweenness** measures how many shortest paths pass through a given edge:

$$C_B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}}$$

where  $\sigma_{st}$  is the total number of shortest paths from  $s$  to  $t$ , and  $\sigma_{st}(e)$  is the number of those paths that use edge  $e$ .

**Key intuition:** edges that connect two otherwise loosely connected clusters will lie on a huge number of shortest paths. They are the *bridges* between communities. These edges will have very high betweenness scores.

**Algorithm (Brandes):** 1. For each source node  $s$ , run BFS/Dijkstra to find all shortest paths. 2. Back-propagate path counts along those trees to accumulate edge scores. 3. Repeat for all  $n$  source nodes and sum.

Complexity:  $O(nm)$  for unweighted graphs ( $n$  nodes,  $m$  edges).

---

### 177.3.1 Note for This Course

For simplicity, we will use the **non-normalized version**:

$$C_B(e) = \sum_{s \neq t} \sigma_{st}(e)$$

Instead of dividing by  $\sigma_{st}$ , we simply count how many shortest paths pass through each edge.

---

## 177.4 Girvan–Newman Algorithm (Core Idea)

The Girvan–Newman algorithm is based on a simple but powerful idea:

Repeatedly **remove** the most “important” edges to reveal hidden communities.

Proposed by Michelle Girvan and Mark Newman in 2002, this algorithm detects communities by **progressively dismantling the graph**; removing the edges that most bridge communities until the graph splits apart.

**Algorithm steps:** 1. Compute **edge betweenness** for *every edge* in the graph. 2. Identify the edge(s) with the highest betweenness score. 3. Remove that edge(s) from the graph. 4. Recompute edge betweenness for all remaining edges (structure has changed). 5. Repeat until the graph breaks into the desired number of components (or until all edges are gone).

As edges are removed, the graph gradually breaks into **separate components**, which correspond to communities.

### Why does this work?

Edges that connect two communities tend to carry most of the **shortest-path traffic** between them, which gives them very high betweenness values.

In contrast, edges within a community are part of a dense structure, so there are many alternative paths and no single edge is as critical.

As a result, when we remove the edge with the highest betweenness, we are typically cutting a **bridge between communities**, allowing the graph to separate naturally into clusters.

## 177.5 Computing Edge Betweenness (Intuition)

Edge betweenness answers the question:

*Which edges are used the most when moving from one node to another?*

Consider a graph that naturally splits into two groups. To travel from a node on one side to a node on the other, most shortest paths must pass through a **small set of connecting edges**.

*[Image omitted in PDF — see the online textbook]*

Edges connecting two groups are used more frequently in shortest paths

For example, if node 1 wants to reach node 12, a shortest path might look like:

$1 \rightarrow 3 \rightarrow 7 \rightarrow 8 \rightarrow 12$

Here, the edge between nodes **7 and 8** lies on many such paths, making it highly important. Edges like this have **high betweenness** because they carry a large portion of the network's shortest-path traffic.

---

### 177.5.1 Why is Edge Betweenness of (7,8) = 49?

The edge between nodes **7 and 8** connects two large parts of the graph (left and right). Any shortest path from a node on the left to a node on the right must pass through this edge.

- Left side has **7 nodes**: {1,2,3,4,5,6,7}
- Right side has **7 nodes**: {8,9,10,11,12,13,14}

The total number of shortest paths between the two sides is:

$$7 \times 7 = 49$$

Each pair contributes one shortest path, and all of them pass through edge (7,8).

#### Key Insight:

The edge (7,8) has very high betweenness because: - it is the **only bridge** between the two halves  
- it carries **all cross-group shortest paths**

This is why it gets the highest score and is removed first in the Girvan–Newman algorithm.

---

To compute edge betweenness, we find shortest paths between all pairs of nodes (e.g., using BFS for unweighted graphs) and count how many times each edge is used. At the end, each edge has a score representing how often it is used.

The edge with the highest score (highest betweenness) is removed, and the process is repeated on the updated graph.

**Note:** If multiple edges have the same highest value, any one of them can be removed (or all of them together, depending on the implementation).

#### Complexity:

Each edge removal requires recomputing betweenness in **O(nm)** time, and repeating this many times leads to **O(m<sup>2</sup> n)** overall. Because of this, Girvan–Newman is slow for large graphs, but works well for small networks and for building intuition.



**Output:** The result of the algorithm is a **dendrogram**, a tree-like representation that shows how the graph splits over time.

*[Image omitted in PDF — see the online textbook]*

Example of a dendrogram showing hierarchical splitting of groups, Source: Jesse Port et al., Hierarchical clustering of vertebrate communities

The algorithm starts with the whole graph and gradually splits it into smaller groups. By cutting the tree at different levels, we can obtain a few large communities or many smaller ones.

## 177.6 Selecting Number of Communities

Wait! Girvan–Newman keeps removing edges. If we remove all edges, every node becomes its own community. So the key question is: **when should we stop?**

*[Image omitted in PDF — see the online textbook]*

The graph starts as one group and gradually splits into smaller communities

One simple approach is to stop when we reach a desired number of groups. This works only if we already know how many communities we expect.

A more flexible approach is to use **modularity**. After each split, we measure how good the community partition is. The partition with the **highest modularity** is considered the best because it has many connections within communities and fewer connections between communities.

## 177.7 Modularity: Measuring Partition Quality

Girvan–Newman tells us *how* to split a graph, but modularity helps decide *which split is best*. **Modularity  $Q$**  measures the quality of a community partition.

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where: -  $m$  = number of edges

-  $A_{ij}$  = 1 if edge  $(i, j)$  exists, 0 otherwise

-  $k_i$  = degree of node  $i$

-  $\frac{k_i k_j}{2m}$  = expected number of edges between  $i$  and  $j$  in a random graph

-  $\delta(c_i, c_j)$  = 1 if  $i$  and  $j$  are in the same community, 0 otherwise

### 177.7.1 Intuition

Modularity compares the number of edges **within communities** to what we would expect by chance:

$$Q \propto \sum_{\text{communities}} (\text{actual edges within group} - \text{expected edges within group})$$

Higher  $Q$  means stronger community structure (more internal connections than expected).

### 177.7.2 Interpretation

Q value	Interpretation
$< 0$	Worse than random
0	No clear community structure
0.3 – 0.7	Meaningful community structure
$> 0.7$	Very strong community structure (rare)

**Stop criterion (Girvan–Newman):** compute  $Q$  after each split and choose the partition with the highest value.

### 177.7.3 Small Modularity Calculation Example

Consider this graph: A — B C — D

Edges: (A–B), (C–D)

So:

- Number of edges:  $m = 2$
- Communities:
  - Community 1: {A, B}
  - Community 2: {C, D}

We use:

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

#### Step 1: Node Degrees

- $k_A = 1, k_B = 1, k_C = 1, k_D = 1$

#### Step 2: Compute for pairs in same community

##### Community 1: (A, B)

- $A_{AB} = 1$
- Expected:  $\frac{k_A k_B}{2m} = \frac{1 \cdot 1}{4} = 0.25$

Contribution:

$$1 - 0.25 = 0.75$$

##### Community 2: (C, D) Same:

$$1 - 0.25 = 0.75$$

#### 177.7.4 Step 3: Final Q

Sum =  $0.75 + 0.75 = 1.5$

$$Q = \frac{1}{2m} \times 1.5 = \frac{1}{4} \times 1.5 = 0.375$$

$Q = 0.375$  indicates a **meaningful community structure**  
(since internal edges are higher than expected by chance)

## 178 Summary of the Chapter

Concept	Key idea
<b>Graph <math>G = (V, E)</math></b>	Nodes + edges encode entities and relationships
<b>3 representations</b>	Matrix (fast lookup), List (memory efficient), Dictionary (flexible, weighted)
<b>Featurization</b>	Convert structure into numeric vectors for ML
<b>Degree centrality</b>	Who has the most direct connections?
<b>Closeness centrality</b>	Who spreads information fastest?
<b>Betweenness centrality</b>	Who is the gatekeeper / bridge?
<b>Community</b>	Groups of nodes densely connected internally
<b>Girvan–Newman</b>	Detect communities by removing high-betweenness edges
<b>Modularity (Q)</b>	Measures how strong a community partition is
<b>BFS</b>	Level-by-level traversal; shortest paths in unweighted graphs
<b>DFS</b>	Deep-first traversal; cycle detection, topological sort
<b>PageRank</b>	Importance via incoming link quality — powers Google Search

---

The same mathematical object; a graph; routes your commute, serves your Netflix recommendations, and helps researchers discover new medicines. The abstraction is the power.

### 178.1 BONUS: Graph Processing

Graph processing means running algorithms over the graph structure to extract insights or support decisions.

#### 178.1.1 Core processing tasks

Task	Goal	Algorithm	Use case
<b>Traversal</b>	Visit all reachable nodes	BFS, DFS	Component finding, web crawling
<b>Shortest path</b>	Minimum hops or cost	Dijkstra, A*, BFS	Navigation, network routing
<b>Minimum spanning tree</b>	Connect all nodes at minimum cost	Kruskal, Prim	Laying cables, circuit design
<b>Community detection</b>	Find dense subgroups	Louvain, Label Propagation	Social clusters, fraud rings
<b>Node ranking</b>	Score nodes by importance	PageRank, HITS	Web search, citation analysis
<b>Link prediction</b>	Predict missing edges	Common neighbors, GNN	Friend suggestions
<b>Graph classification</b>	Label an entire graph	GCN, GraphSAGE	Molecule toxicity, code analysis

## 178.2 BONUS: Graph neural networks (GNNs)

Classical graph algorithms are hand-crafted rules. Graph Neural Networks learn representations directly from graph structure and node/edge features, enabling tasks like:

- **Node classification:** predict a node's category (is this account a bot?)
- **Link prediction:** will these two nodes form an edge? (friend suggestion, drug-target interaction)
- **Graph classification:** classify an entire graph (is this molecule toxic?)
  - Traditional ML → manual feature engineering
  - Graph Neural Networks → automatic feature learning  
Key Idea:  $\text{Feature}(\text{node}) = \text{function}(\text{node} + \text{neighbors})$

The core operation is message passing: each node aggregates feature vectors from its neighbors, transforms them, and updates its own representation. After  $k$  rounds, a node's embedding captures its  $k$ -hop neighborhood. This is the basis of models like GCN, GAT, and GraphSAGE, which power modern recommendation systems at Pinterest, Uber, and Alibaba.

In this book, we are not going to cover GNN in details but here is an interesting article to read: <https://distill.pub/2021/gnn-intro/>.

## 179 Knowledge Check

### 179.1 interactive topology

## 180 Interactive Topologies

*[Interactive content omitted in PDF]*

## 181 Chapter 22 - Recommender Systems

### 182 How Netflix Knows What You Want?

It's Friday night. You open Netflix. You don't search. You don't browse much. Yet somehow... the perfect movie is already there. **How?**

This is the power of Recommender Systems. The Core Idea is that a recommender system tries to answer one simple question:

“What should we show this user next?”

It learns from:

- What you watched
- What others watched
- Patterns across millions of users

### 183 What are Recommender Systems?

Recommender systems are algorithms that predict what a user might like based on data; what they've watched, bought, rated, or clicked.

Recommender systems are design to suggest relevant items (e.g., movies, products, songs) to users based on their preferences and behavior.

Netflix, Spotify, and Amazon all use them. A **recommender system** predicts what a user might like, based on available data. You encounter them daily: - Netflix → “*Because you watched Inception...*” - Amazon → “*Customers who bought this also bought...*” - Spotify → “*Your Daily Mix*”

The core challenge is the “**cold start**” problem: what do you recommend when you know nothing about a new user or a new item?

#### 183.1 Types of Recommender System

There are two main families of techniques: - **content-based filtering** (focus on item attributes) and - **collaborative filtering** (focus on user behavior patterns).

Most production systems combine both.

*[Image omitted in PDF — see the online textbook]*

Figure: Overview of a Recommender System. Source: [media.geeksforgeeks.org](http://media.geeksforgeeks.org).

Approach	Key idea	Data needed	Key Comparison
<b>1. Content-Based</b>	Recommend items similar to what you liked before	Item features (genre, cast, etc.)	<b>User profile vs Item</b>
<b>2. User-Based CF</b>	Find users like you, recommend what they liked	Rating history of all users	<b>User vs User</b>

Approach	Key idea	Data needed	Key Comparison
<b>3. Item-Based CF</b>	Find items similar to ones you rated highly	Rating history of all users	<b>Item vs Item</b>

\*\* CF = Collaborative Filtering

## 184 1. Content-Based Filtering Recommender System

Content-based filtering **builds a profile** of each item (e.g. a movie's genre, director, runtime) and a **profile of each user** based on the items they have liked.

It then recommends items whose profiles are **\*\*most similar** to the user's profile, typically using cosine similarity.

Content-based filtering builds a profile of each item (e.g., genre, keywords, director) and a profile of each user based on the items they have liked.

It then recommends items whose profiles are most similar to the user's profile, typically using cosine similarity.

### 184.0.1 The key idea:

*"Show me more of what I've already shown I like."* > Recommend items *similar* to what the *user liked before*.

—  
—

[Image omitted in PDF — see the online textbook]

|

[Image omitted in PDF — see the online textbook]

|

Figure: Two visual representations of Content-Based Filtering. The system builds a user profile from previously liked items and recommends new items with similar features. (Sources: shaped.ai, spotintelligence.com)

**Algorithm:** 1. Each item has features (e.g., genre, keywords). 2. Build a **user profile**; a weighted average of the feature vectors of movies they've rated (based on past preferences) 3. For each unseen movie, compute the **cosine similarity** between its feature vector and the user profile 4. Recommend the unseen movie with the highest similarity (Recommend items with similar features)

User likes Action, Sci-Fi movies. Then **recommend more Action + Sci-Fi movies**.

### Example (User Profile)

Suppose features = [Action, Romance, Sci-Fi]

If a user liked:

- Movie A  $\rightarrow [1, 0, 1]$
- Movie B  $\rightarrow [1, 0, 0]$

Then the **user profile** can be **computed as the average**:  $[1, 0, 0.5]$

This means the user: - strongly prefers **Action** - somewhat likes **Sci-Fi** - does not prefer **Romance**

The system then compares this user profile with new items and recommends those with the highest similarity (typically using cosine similarity).

**Key summary:** Content-based Filtering recommends items whose features are most similar to the **user profile**, where the **user profile is a vector** that **summarizes** the types of items the user has liked in the past.

---

**Important Clarification:** In content-based filtering, we **do NOT** compare users with other users. We **compare** user profile (preferences) with item features.

---

### 184.0.2 Advantages and Disadvantages:

The strength is that it needs **only that user's history; no data from others**. But the weakness is a **“filter bubble”**: users only ever see more of what they already like.

Advantages: - Uses only the user's own history (no need for other users) - Provides personalized recommendations - Works well with smaller datasets

Disadvantages: - Limited diversity (recommends similar items repeatedly) - Filter bubble problem (hard to discover new interests) - Depends on quality of item features

---

### 184.0.3 Cosine Similarity: quick refresher

$$\text{cosine\_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Returns a value between **-1** (opposite) and **1** (identical). We want values close to 1.

---

## 185 2. Collaborative Filtering Recommender System

Collaborative filtering recommends items based on **patterns of user behavior**, such as ratings, clicks, purchases, or views.

Unlike content-based filtering, it does **not use item features** (e.g., genre, keywords). Instead, it relies on the idea:

*Users who behaved similarly in the past may like similar items in the future.*

### 185.0.1 How is Collaborative Filtering Different from Content-Based Filtering?

Method	Main Idea	Uses Item Features?	Uses Other Users?
<b>Content-Based Filtering</b>	Recommend items similar to what the user liked before	Yes	No
<b>Collaborative Filtering</b>	Recommend items based on similar users/items	No	Yes

Content-based filtering uses **item features** while on the other hand Collaborative filtering uses **user–item interaction patterns (matrix)**.

## 185.1 2.1 User-Based Collaborative Filtering (User–User CF)

User-based collaborative filtering asks:

**“Who else has similar taste as me?”**

It finds the **k most similar users (neighbors)** and recommends items those users liked that the target user has not seen.

*“Find people who think like you, then recommend what they loved.”*

[Image omitted in PDF — see the online textbook]

Figure: Illustration of a User-Based Collaborative Filtering recommender system. Source: Medium.

**Key Observation::** We do **not use item features** (like genre). We only use **rating patterns**.

### 185.1.1 Similarity Measures

Similarity between users is computed using:

#### Pearson Correlation

$$\text{Pearson}(A, B) = \frac{\sum_i (A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum_i (A_i - \bar{A})^2} \sqrt{\sum_i (B_i - \bar{B})^2}}$$

### 185.1.2 Intuition

- Measures **linear relationship**
- Removes **rating bias** (mean-centering)
- Focuses on **relative preferences**



### 185.1.3 Why Pearson over Cosine?

Pearson handles **user rating bias**. Example: - Alice rates everything **4–5** - Bob rates everything **1–2** - But both like the *same movies*

**Pearson** → identifies them as similar

**Cosine** → may treat them as different

Example:

Cosine Similarity: Cosine compares the **raw vectors** (no mean adjustment).

- Alice = [5, 5, 4]
- Bob = [2, 2, 1]

Even though the pattern is similar, the **magnitudes are different**, so cosine may not fully capture their similarity.

Pearson Correlation: Pearson first **subtracts the mean**:

- Alice mean = 4.67 → [0.33, 0.33, -0.67]
- Bob mean = 1.67 → [0.33, 0.33, -0.67]

Now they are **identical after normalization**

Pearson → similarity 1 (very similar)

So we can say: - **Cosine similarity** compares **absolute values**

- **Pearson correlation** compares **relative preferences (after removing bias)**

Use **cosine similarity** → item features / embeddings

Use **Pearson correlation** → user ratings

---

### 185.1.4 Algorithm

1. Construct a **user–item rating matrix**
2. Compute similarity between users (on commonly rated items)
3. For a target user, find **k nearest neighbors**
4. Predict ratings using a **weighted average** of neighbors' ratings
5. Recommend items with the highest predicted ratings

### 185.1.5 Example

User	Movie A	Movie B	Movie C
User 1	5	4	?
User 2	5	4	5

User	Movie A	Movie B	Movie C
User 3	1	2	1

User 1   User 2  $\rightarrow$  similar preferences

Recommend **Movie C** to User 1

---

**Important Clarification:** In user-based collaborative filtering, we compare:

- **one user's rating pattern**  
with
- **other users' rating patterns**

We do **not** use item features.

---

#### 185.1.6 Advantages

The intuition is powerful; it enables serendipitous discovery beyond your existing taste profile.

- No need for item features
- Can recommend **new and diverse items**
- Captures **collective intelligence** of users

But it's expensive to compute, since comparing every user pair scales as  $O(N^2)$ .

#### 185.1.7 Disadvantages

- Requires sufficient user interaction data
- **Cold start problem** (new users/items)
- Computationally expensive (can scale as  $O(N^2)$ )
- Similarity unreliable with sparse ratings

**Key Insight:** User-based collaborative filtering works best when we have:

- Many users
- Sufficient rating history

It answers: *“What did similar users like that this user hasn't seen yet?”*

## 185.2 2.2 Item-Based Collaborative Filtering (Item–Item CF)

*“Find movies that are always rated the same way by the same people, then use that.”*

Item-based collaborative filtering **flips the perspective**: instead of finding similar users, it finds **similar items** based on user interaction patterns.

At recommendation time, it looks at what a user has already rated, finds items most similar to those, and recommends them.

**The Key Idea:**

**“Which items are similar to the ones this user liked?”**

*“If you liked this item, you may also like similar items.”*

*[Image omitted in PDF — see the online textbook]*

Figure: Illustration of a Item-Based Collaborative Filtering recommender system. Source: Medium.

This is what Amazon uses (“customers who bought X also bought Y”).

**Key Observation:** We do not use item features; instead, we compute item similarity using user behavior (ratings, clicks, purchases) via **cosine similarity or Pearson correlation**.

**Intuition:** Two items are similar if **many users interacted with both items in a similar way**

### 185.2.1 Algorithm

1. Construct a **user–item rating matrix**
2. Compute **item–item similarity** (compare columns using cosine or Pearson similarity)
3. For a target user and an unseen item, find items **most similar to that item** that the user has already rated
4. Predict the rating using a **weighted average of the user’s ratings on similar items**
5. Recommend items with the **highest predicted scores**

Predict how much a user will like an item based on how similar it is to items they already liked.

### 185.2.2 Example

User	Movie A	Movie B	Movie C
User 1	5	4	?
User 2	5	4	5
User 3	1	2	1

Movie A and Movie B have similar rating patterns across users

- If a user liked **Movie A**, recommend **Movie B**

---

**Important Clarification:** In item-based collaborative filtering, we compare:

- **item rating patterns (columns)**  
with
- **other item rating patterns**

We are **not comparing users directly**. \*\*\*

### 185.2.3 Advantages

- More **scalable** than user-based CF
- Item similarities are more **stable over time**
- Can be **precomputed** for faster recommendations
- Works well in real-world systems (e.g., e-commerce)

### 185.2.4 Disadvantages

- Requires sufficient interaction data
- **Cold start problem for new items**  
> > Cold start = *no data* → *no recommendation*
- Cannot recommend items with no prior interactions

---

**Key advantage over User-Based:** Item similarity is **stable** (Inception vs Interstellar don't change relationship over time), so we can **pre-compute it once** and look it up cheaply. This is why Amazon and Netflix prefer item-based CF.

---

**Key Insight:** Item-based collaborative filtering works best when:

- Items are relatively stable
- There is sufficient user interaction data

It answers: “*What items are similar to the ones this user already liked?*”

---

## 185.3 Cold Start Problem in Collaborative filtering

The **cold start problem** occurs when a recommender system has **insufficient data** to make reliable recommendations.

### 185.3.1 Types of Cold Start

#### 1. New User Problem

- A new user joins the system
- No history (ratings, clicks, purchases)  
System doesn't know their preferences

#### 2. New Item Problem

- A new item is added
- No user interactions yet  
System cannot recommend it

#### 3. New System Problem

- Entire system is new
- Very little interaction data overall

### 185.3.2 Why it happens

Collaborative filtering relies on: - user-item interactions  
- similarity between users/items

Without data → **no similarity can be computed**

**Example:** A new user signs up on Netflix and has not watched anything yet. System cannot recommend movies effectively

Some Common Solutions:

- Ask users for **initial preferences** (onboarding questionnaire)
  - Use **content-based filtering** initially
  - Recommend **popular/trending items**
  - Use **hybrid systems**
- 

## 186 Example: Recommender Systems — Complete Walkthrough

### 186.0.1 Content-Based · User-Based CF · Item-Based CF

**What you'll build:** Three recommender systems from scratch, with every calculation shown as a printed table you can trace step by step.

---

**Dataset:** 5 users (Alice, Bob, Carol, Dave, Eve) rate 5 movies on a scale of 1–5.  
A ? means they haven't seen it yet; that's what we want to predict.

	Inception	Interstellar	The Notebook	Alien	Titanic
<b>Alice</b>	5	4	2	?	?
<b>Bob</b>	5	4	1	5	2
<b>Carol</b>	1	2	5	2	5
<b>Dave</b>	4	5	2	4	1
<b>Eve</b>	2	1	5	3	4

## 186.1 1. Content Based Recommender Systems

**Step 1:** The Rating Matrix (Alice's History)

User	Inception ( <i>Action, Sci-fi</i> )	Interstellar ( <i>Sci-fi</i> )	The Notebook ( <i>Romance</i> )	Alien ( <i>Action, Sci-fi</i> )	Titanic ( <i>Romance</i> )
<b>Alice</b>	<b>5</b>	<b>4</b>	<b>2</b>	<b>?</b>	<b>?</b>
Bob	3	5	1	4	2
Carol	1	2	5	1	4

Notice that, Alice rated Inception (5) and Interstellar (4) highly (both are Sci-fi genre). She rated The Notebook (2) low, the Romance genre. We need to **predict her ratings for Alien and Titanic**.

**Step 2:** Build Alice's Genre Preference Profile (**User Profile**)

Average rating per genre across movies she has seen:

Genre	Movies seen	Avg rating
Sci-fi	Inception (5), Interstellar (4)	$(5 + 4) / 2 = 4.5$
Action	Inception (5)	$5 / 1 = 5.0$
Romance	The Notebook (2)	$2 / 1 = 2.0$

Detailed Calculation: - **Sci-fi:** Alice watched Inception and Interstellar =  $(5 + 4) / 2 = 4.5$  - **Action:** Alice watched Inception =  $5 / 1 = 5.0$  - **Romance:** Alice watched The Notebook =  $2 / 1 = 2.0$

Alice's profile: Likes Sci-fi (4.5) and Action (5.0) Dislikes Romance (2.0)

**Step 3:** Score Unseen Movies

General Formula: Predicted score for a movie:

$$\text{Score(movie)} = \frac{\sum(\text{user's average rating for each genre in the movie})}{\text{number of genres in the movie}}$$

Movie	Genres	Predicted score	Calculation	Recommend?
Alien	Action, Sci-fi	<b>4.75</b>	$(5.0 + 4.5) / 2$	Yes

Movie	Genres	Predicted score	Calculation	Recommend?
Titanic	Romance	<b>2.0</b>	(2.0)	No

Step-by-Step Calculation: - Alien (the movie has 2 genre: Action + Sci-fi) = Score(Alien)=(5.0 + 4.5) / 2 = 4.75 - Titanic (the movie has 1 genre: Romance) = Score(Titanic)= 2.0/ 1 =2.0

**Final Recommendation:** Recommend “**Alien**” because its genres closely match Alice’s demonstrated preferences.

## 186.2 2. User-Based Collaborative Filtering Recommender Systems

**Step 1:** User-Item Rating Matrix where (? = unseen)

User/Item (Movie)	Inception	Interstellar	The Notebook	Alien	Titanic
<b>Alice</b>	<b>5</b>	<b>4</b>	<b>2</b>	<b>?</b>	<b>?</b>
Bob	5	4	1	5	2
Carol	1	2	5	2	5
Dave	4	5	2	4	1
Eve	2	1	5	3	4

We use the 3 movies that Alice and other users have both rated (**Inception, Interstellar, and The Notebook**) to compute similarity. Then we use ratings from **the most similar users** to predict Alice’s missing ratings.

**Step 2:** Compute Similarity Between Alice and Each User

We compute similarity using only the movies that both Alice and other users have rated (**overlapping / co-rated items**): *Inception, Interstellar, The Notebook*.

- Similarity is computed using **cosine similarity** or **Pearson correlation**
- Then, we select the **top-k most similar users** to predict Alice’s missing ratings.

Alice’s rating vector:

$$Alice = [5, 4, 2]$$

**Option A: Cosine Similarity**

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

**Example: Alice vs Bob**

$$\frac{(5 \times 5 + 4 \times 4 + 2 \times 1)}{\sqrt{5^2 + 4^2 + 2^2} \cdot \sqrt{5^2 + 4^2 + 1^2}} = \frac{43}{\sqrt{45} \cdot \sqrt{42}} \approx 0.989$$

### Cosine Similarity Results:

User	Their ratings (Inception / Interstellar / Notebook)	Alice's ratings	Similarity
<b>Bob</b>	5 / 4 / 1	5 / 4 / 2	<b>0.99</b> (very similar)
<b>Dave</b>	4 / 5 / 2	5 / 4 / 2	<b>0.98</b> (similar)
Carol	1 / 2 / 5	5 / 4 / 2	0.63
Eve	2 / 1 / 5	5 / 4 / 2	0.65

Bob and Dave are Alice's nearest neighbors.

### Option B: Pearson Correlation

$$\text{Pearson}(A, B) = \frac{\sum(A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum(A_i - \bar{A})^2} \sqrt{\sum(B_i - \bar{B})^2}}$$

Alice's mean:

$$\bar{A} = \frac{5 + 4 + 2}{3} = 3.67$$

#### 186.2.1 Example: Alice vs Bob

Bob's mean:

$$\bar{B} = \frac{5 + 4 + 1}{3} = 3.33$$

Centered vectors:

$$Alice = [1.33, 0.33, -1.67]$$

$$Bob = [1.67, 0.67, -2.33]$$

$$\text{Pearson}(Alice, Bob) \approx 0.98$$

#### 186.2.2 Pearson Similarity Results

User	Ratings (I / IS / N)	Similarity
<b>Bob</b>	5 / 4 / 1	<b>0.99</b> (very similar)
<b>Dave</b>	4 / 5 / 2	<b>0.79</b> (similar)



User	Ratings (I / IS / N)	Similarity
Carol	1 / 2 / 5	-1.00 (very different)
Eve	2 / 1 / 5	-0.84 (very different)

Key Insight:

- **Cosine similarity** → measures angle (raw rating patterns)
- **Pearson correlation** → measures similarity after removing user bias (mean-centered)

For recommendation systems, **Pearson is often preferred** because it handles users with different rating scales.

### Step 3: Choose **Top-k** Neighbors

We compute similarity using the movies Alice and the other users have both rated. Then we use the top-k most similar users to predict Alice's missing ratings.

Since we use:

$$k = 2$$

we select the top 2 most similar users:

Nearest neighbors = {*Bob*, *Dave*}

Neighbor	Similarity
Bob	0.99
Dave	0.98

### Step 4: Predict Alice's Missing Ratings

We use the weighted average formula:

$$\text{Predicted rating} = \frac{\sum(\text{similarity} \times \text{neighbor rating})}{\sum(\text{similarity})}$$

Unseen Movie	Similar Users Used	Prediction Formula	Score
<b>Alien</b>	Bob (sim 0.99, rated 5) Dave (sim 0.98, rated 4)	$(0.99 \times 5 + 0.98 \times 4) / (0.99 + 0.98)$	<b>4.50</b>

Unseen Movie	Similar Users Used	Prediction Formula	Score
<b>Titanic</b>	Bob (sim 0.99, rated 2) Dave (sim 0.98, rated 1)	$(0.99 \times 2 + 0.98 \times 1) / (0.99 + 0.98)$	<b>1.50</b>

### 186.2.3 Step-by-Step Calculation

**Alien: Predict Alice's Rating for Alien** Bob rated Alien = 5  
Dave rated Alien = 4

$$\text{Predicted rating for Alien} = \frac{(0.99 \times 5) + (0.98 \times 4)}{0.99 + 0.98} = \frac{4.95 + 3.92}{1.97} = 4.50$$

**Titanic: Predict Alice's Rating for Titanic** Bob rated Titanic = 2  
Dave rated Titanic = 1

$$\text{Predicted rating for Titanic} = \frac{(0.99 \times 2) + (0.98 \times 1)}{0.99 + 0.98} = \frac{1.98 + 0.98}{1.97} = 1.50$$

### 186.2.4 Final Recommendation

Movie	Predicted Rating	Recommend?
<b>Alien</b>	<b>4.50</b>	<b>Yes</b>
Titanic	1.50	No

**Recommendation:** Recommend **Alien (predicted 4.50)**, because Alice's nearest neighbors, Bob and Dave, both rated Alien highly.

**Remember:** In user-based collaborative filtering, we rely entirely on similar users' behavior; we do not use genre or content information.

## 186.3 3. Item-Based Collaborative Filtering Recommender Systems

Item-based CF flips the question: instead of asking "who has similar taste to Alice?", it asks "which movies tend to get rated similarly by the same people?"

**Idea:** Instead of finding **similar users**, we find **similar items (movies)**.

We predict Alice's ratings by finding **movies similar to those she already rated**, and using her past ratings to estimate new ones.

- If two movies are **rated similarly by many users** → they are similar
- We predict a user's rating based on **movies they already liked**

### Step 1: Represent Movies as Vectors

Instead of comparing users, we compare how movies are rated across users.

Movie(Item)/User	Alice	Bob	Carol	Dave	Eve
Inception	5	5	1	4	2
Interstellar	4	4	2	5	1
The Notebook	2	1	5	2	5
Alien	?	5	2	4	3
Titanic	?	2	5	1	4

Here, we have adjusted the the matrix: rows are now movies, columns are users. **Why?** Instead of comparing users, we **compare how movies were rated** across **all users**.

Here, as you can see, **each movie** is **represented** using **ratings from all users**:

- Inception = [5, 5, 1, 4, 2]
- Interstellar = [4, 4, 2, 5, 1]
- The Notebook = [2, 1, 5, 2, 5]
- Alien = [?, 5, 2, 4, 3] (Here, Alice has not rated Alien) → ignore Alice → [5, 2, 4, 3]
- Titanic = [?, 2, 5, 1, 4] → ignore Alice → [2, 5, 1, 4]

We ignore Alice's missing values (?) when computing similarity.

---

#### 186.3.1 Why Do We Ignore “?” (Missing Ratings) above to represent vector?

When computing similarity between two movies, we must use **only users who have rated both movies**.

Missing values (“?”) mean the user has **not seen or rated the movie**, so we do not know their preference. Including them would introduce incorrect or undefined values in the calculation.

**Key Idea:** Similarity is computed only on **co-rated users** (users who rated both items). You can't compare two movies based on a user who **hasn't watched one of them**.

#### 186.3.2 Example

We want to compute similarity between **Inception** and **Alien**.

Original vectors:

- Inception = [5, 5, 1, 4, 2]
- Alien = [?, 5, 2, 4, 3]

Here, Alice has not rated Alien. Problem: - “?” is unknown → cannot multiply or compute distance  
- Leads to invalid similarity

So, Remove Alice and use only:

- Inception = [5, 1, 4, 2]
- Alien = [5, 2, 4, 3]

(users: Bob, Carol, Dave, Eve)

Now both vectors have: - Same length

- Only known values
- Valid comparison

### 186.3.3 Summary

- Ignore missing values (“?”)
- Use **only overlapping** users
- Ensures fair and correct similarity computation

---

### Step 2: Compute Item-Item Similarity

So, we compute similarity between movies using only **co-rated users** (ignore “?”).

From Step 1, we have found: Movie Vectors (Example: Inception vs Alien)

Using users: Bob, Carol, Dave, Eve

- Inception = [5, 1, 4, 2]
- Alien = [5, 2, 4, 3]

So, we compare each movie’s rating vector using **only users** who **rated both movies** (Bob, Carol, Dave, Eve).

---

### 186.3.4 Similarity Table

Movie Pair	Raters	Cosine Similarity	Pearson Similarity	Interpretation
Inception vs Alien	Bob (5,5), Carol (1,2), Dave (4,4), Eve (2,3)	<b>0.983</b>	<b>0.99</b>	Very similar audiences
Interstellar vs Alien	Bob (4,5), Carol (2,2), Dave (5,4), Eve (1,3)	<b>0.943</b>	<b>0.71</b>	Similar audiences

Movie Pair	Raters	Cosine Similarity	Pearson Similarity	Interpretation
Inception vs Titanic	Bob (5,2), Carol (1,5), Dave (4,1), Eve (2,4)	<b>0.587</b>	<b>-0.90</b>	Opposite preferences by Pearson
The Notebook vs Titanic	Bob (1,2), Carol (5,5), Dave (2,1), Eve (5,4)	<b>0.974</b>	<b>0.89</b>	Very similar audiences

### 186.3.5 Step-by-Step Example Calculation (Inception vs Alien)

#### Option A: Cosine Similarity

$$\text{Cosine}(i, j) = \frac{i \cdot j}{||i|| \times ||j||}$$

#### Calculation

$$\begin{aligned} & \frac{(5 \times 5 + 1 \times 2 + 4 \times 4 + 2 \times 3)}{\sqrt{5^2 + 1^2 + 4^2 + 2^2} \cdot \sqrt{5^2 + 2^2 + 4^2 + 3^2}} \\ &= \frac{25 + 2 + 16 + 6}{\sqrt{46} \cdot \sqrt{54}} = \frac{49}{\sqrt{46} \cdot \sqrt{54}} \approx 0.98 \end{aligned}$$

#### Option B: Pearson Correlation

$$\text{Pearson}(i, j) = \frac{\sum (i_k - \bar{i})(j_k - \bar{j})}{\sqrt{\sum (i_k - \bar{i})^2} \cdot \sqrt{\sum (j_k - \bar{j})^2}}$$

#### Step 1: Compute Means

$$\bar{i} = \frac{5 + 1 + 4 + 2}{4} = 3$$

$$\bar{j} = \frac{5 + 2 + 4 + 3}{4} = 3.5$$

#### Step 2: Centered Vectors

$$i - \bar{i} = [2, -2, 1, -1]$$

$$j - \bar{j} = [1.5, -1.5, 0.5, -0.5]$$

#### Step 3: Compute Pearson

$$\frac{(2 \times 1.5) + (-2 \times -1.5) + (1 \times 0.5) + (-1 \times -0.5)}{\sqrt{(2^2 + (-2)^2 + 1^2 + (-1)^2)} \cdot \sqrt{(1.5^2 + (-1.5)^2 + 0.5^2 + (-0.5)^2)}} \\ = \frac{3 + 3 + 0.5 + 0.5}{\sqrt{10} \cdot \sqrt{5}} = \frac{7}{\sqrt{10} \cdot \sqrt{5}} \approx 0.99$$

### 186.3.6 Method Comparison

Method	Similarity
Cosine	<b>0.98</b>
Pearson	<b>0.99</b>

## 186.4 Key Insight

- **Cosine similarity** → compares raw rating patterns
- **Pearson correlation** → compares rating patterns after removing bias (mean-centered)

Pearson is often better when users have **different rating scales**.

---

**Step 3:** Predict Alice's Ratings (Item-Based):

We use movies Alice has already rated as anchors: - Inception (5) - Interstellar (4) - The Notebook (2)

**Top-N Selection (i.e N=2)** For each target movie, we select the **top-2 most similar movies** among the movies Alice has already rated.

Here, we use:

$$N = 2$$

Using **cosine similarity**, the selected neighbors are:

- For **Alien** → **Inception (0.98)**, **Interstellar (0.94)**
- For **Titanic** → **The Notebook (0.97)**, **Inception (0.59)**

*(Neighbors are selected using cosine similarity for prediction. Pearson correlation can also be used; however, negative similarities should be ignored. An example is provided below.)*

### 186.4.1 General Formula

$$\text{Score}(i) = \frac{\sum (\text{sim}(i, j) \times r_{\text{Alice}, j})}{\sum \text{sim}(i, j)}$$

### 186.4.2 Predict Alien

$$\frac{(0.98 \times 5) + (0.94 \times 4)}{0.98 + 0.94} = \frac{4.90 + 3.76}{1.92} = \frac{8.66}{1.92} \approx 4.51$$

### 186.4.3 Predict Titanic

$$\frac{(0.97 \times 2) + (0.59 \times 5)}{0.97 + 0.59} = \frac{1.94 + 2.95}{1.56} = \frac{4.89}{1.56} \approx 3.14$$

### 186.4.4 Final Results

---

Movie	Predicted Score	Recommend?
<b>Alien</b>	<b>4.51</b>	Yes
Titanic	3.14	Maybe / weaker

---

### 186.4.5 (OPTION B) Using Pearson Prediction by Ignoring Negative Similarities

When using Pearson correlation, negative similarity means opposite preference. So for prediction, we ignore negative similarities and use only positive similarities.

---

### 186.4.6 Predict Alien using Pearson

Positive similarities:

- $\text{sim}(\text{Alien}, \text{Inception}) = 0.99$
- $\text{sim}(\text{Alien}, \text{Interstellar}) = 0.71$

$$\frac{(0.99 \times 5) + (0.71 \times 4)}{0.99 + 0.71} = \frac{4.95 + 2.84}{1.70} = \frac{7.79}{1.70} \approx 4.58$$

---

### 186.4.7 Predict Titanic using Pearson

Positive similarity:

- $\text{sim}(\text{Titanic}, \text{The Notebook}) = 0.89$
- $\text{sim}(\text{Titanic}, \text{Inception}) = -0.90 \rightarrow \text{ignored}$

$$\frac{0.89 \times 2}{0.89} = 2.00$$

---

### 186.4.8 Pearson Results After Ignoring Negative Similarities

---

Movie	Pearson Score	Recommend?
<b>Alien</b>	<b>4.58</b>	Yes
Titanic	2.00	No

---

### 186.4.9 Interpretation

Using Pearson, Titanic receives a low predicted score because its strongest positive match is **The Notebook**, which Alice rated low.

---

### 186.4.10 Final Recommendation

**Recommendation:** Recommend **Alien** (predicted 4.51) because it is highly similar to movies Alice already rated highly.

Intuition: If Alice liked movies similar to Alien, she will likely like Alien as well.

**Remember:** Item-based uses **movie similarity**, not users. Since movies don't change, their similarity stays consistent. This allows us to compute similarities once and reuse them, making the system efficient.

- We use **Top-N similar items** ( $N = 2$ )
- Predictions are based on **weighted averages of similar movies**
- Item-based filtering leverages **movie similarity, not user similarity**

Here's the **core intuition** from all three methods: All three methods predicted **Alice should watch Alien**, but for completely different reasons.

- Content-based said "Alien is Sci-fi, and Alice likes Sci-fi."
- User-based said "Bob and Dave love Alien, and they think just like Alice."
- Item-based said "Alien gets rated by the same people who rated Inception highly."

The practical reason why real systems like Amazon and Netflix prefer item-based CF is scalability. With 100 million users, you'd need to compare every user pair (that's 10 quadrillion comparisons). But movies are fewer, their similarity scores are stable, and you can pre-compute them once overnight and reuse them for every user lookup in milliseconds.

## 187 Association Rule Mining

Imagine you own a small grocery store. Every day, hundreds of customers walk through the aisles, picking up different combinations of products.

Some buy:

- Bread and Butter
- Milk and Cereal
- Chips and Soda
- Diapers and Baby Wipes

After months of sales, you start wondering:

"Are there certain products customers frequently buy together?"



This is exactly the type of problem solved by **Association Rule Mining**.

Association Rule Mining is a data mining technique used to discover relationships between items in large datasets. Instead of manually checking thousands of receipts, we allow algorithms to automatically discover patterns such as:

- Customers who buy bread often buy butter.
- Customers who buy pasta may also buy tomato sauce.
- Customers who buy diapers may also buy baby wipes.

These discovered relationships are called **association rules**.

### 187.1 What is “Association Rules”?

Association Rule Mining is a technique used to find interesting relationships or patterns among items in large datasets. It is commonly used in market basket analysis, where we want to understand which items are frequently purchased together. For example:

If many customers who buy **bread** also buy **butter**, we may discover the rule: **Bread**  
→ **Butter**

This means: If a customer buys bread, they are likely to buy butter.

Retail stores, online shopping websites, streaming platforms, and recommendation systems use these ideas every day.

### 187.2 Understanding Itemsets

Before finding patterns, we first need to understand something called an **itemset**.

**An itemset:** is simply a **group of items purchased together**.

Examples:

- {Bread}
- {Bread, Butter}
- {Milk, Bread, Eggs}

Now imagine looking through thousands of shopping transactions.

- Some combinations appear only once.
- Others appear again and again.

The combinations that appear frequently are called **frequent itemsets**.

### 187.3 Support Count vs Support

Suppose the itemset: **X = {Milk, Bread, Diaper}**

appears in 2 transactions out of a total of 5 transactions.

## 187.4 Support Count

The **support count** (also called frequency) is the number of transactions that contain the itemset.

$$\sigma(X) = 2$$

This means the itemset {Milk, Bread, Diaper} appears 2 times in the dataset.

## 187.5 Support

The **support** of an itemset measures the **fraction** (or percentage) of transactions that contain the itemset. The formula for support is:

$$s(X) = \frac{\sigma(X)}{|T|}$$

where:

- $\sigma(X)$  = support count of itemset (X)
- $|T|$  = total number of transactions

For this example:  $s(\{\text{Milk, Bread, Diaper}\}) = \frac{2}{5} = 0.4 = 40\%$

This means 40% of all transactions contain the itemset {Milk, Bread, Diaper}.

## 187.6 Frequent Itemset

Before discovering association rules, we first need to identify which groups of items appear frequently in the dataset.

An itemset is considered a **frequent itemset** if its support is greater than or equal to a minimum support threshold.

If the minimum support threshold is 0.3, then {Bread, Butter} is considered a frequent itemset because:  $0.4 > 0.3$

Now let's look at a small grocery store example. Suppose the store has the following transactions:

Transaction ID	Items Purchased
T1	Bread, Milk
T2	Bread, Butter
T3	Bread, Milk, Butter
T4	Milk, Butter
T5	Bread, Milk, Butter

As the store owner, you begin counting how often each item appears.

Item	Count
Bread	4

Item	Count
Milk	4
Butter	4

If we decide that an item must appear at least 3 times to be considered important, then all three individual items are frequent itemsets.

Next, we check pairs of items.

Itemset	Count
Bread, Milk	3
Bread, Butter	3
Milk, Butter	3

These pairs are also frequent because each appears at least 3 times.

Finally, we check the larger 3-itemset:

Itemset	Count
Bread, Milk, Butter	2

This combination appears only twice, so it is **not** considered a frequent itemset if the minimum support count is 3.

The main goal of frequent itemset mining is to identify the combinations of items that occur together often enough to reveal meaningful patterns in the data.

## 188 Mining Association Rules: Two Steps

Association rule mining usually has two main steps:

### 188.1 Step 1: Frequent Itemset Generation

First, we find all itemsets that appear frequently (satisfy the minimum support threshold) in the dataset.

Example frequent itemsets:

- {Bread}
- {Milk}
- {Bread, Butter}
- {Milk, Bread}

## 188.2 Step 2: Rule Generation

After finding frequent itemsets, we generate association rules from them.

For example, from the frequent itemset:

{Bread, Butter}

we can generate rules such as:

- Bread  $\rightarrow$  Butter
- Butter  $\rightarrow$  Bread

Then we evaluate these rules using measures such as confidence and lift.

As datasets grow larger, the number of possible item combinations increases very quickly. Checking every possible itemset would be computationally expensive.

To make frequent itemset generation more efficient, we use the **Apriori Principle**, which helps eliminate impossible candidate itemsets early.

## 188.3 Apriori Principle

The **Apriori principle** helps reduce the number of itemsets we need to check.

It says:

If an itemset is *frequent*, then all of its **subsets** must also be *frequent*.

It also means:

If an itemset is not frequent, then any larger itemset containing it cannot be frequent.

*[Image omitted in PDF — see the online textbook]*

**Figure:** Apriori Principle: If an itemset is not frequent, then any larger itemset containing it cannot be frequent

Example:

If {Bread, Butter} is not frequent, then {Bread, Butter, Milk} cannot be frequent either.

This saves time because we do not need to check larger itemsets that contain infrequent smaller itemsets.

### 188.3.1 Apriori Algorithm

The Apriori algorithm is a classic algorithm used for **frequent itemset generation**.

It works level by level:

1. **Find** all frequent 1-itemsets by checking their support count.
2. Generate **candidate 2-itemsets** from the frequent 1-itemsets
3. **Count** the support count of each candidate itemset
4. **Remove infrequent itemsets** whose support count is below the minimum threshold using the Apriori principle

5. Repeat the process to generate larger frequent itemsets **until no more** frequent itemsets remain

At each step, Apriori prunes itemsets that cannot possibly be frequent, which greatly reduces computation.

*[Image omitted in PDF — see the online textbook]*

**Figure:** Frequent itemset generation using the Apriori algorithm with minimum support count = 2. Itemsets marked with \* are considered frequent because their support count satisfies the minimum threshold.

After discovering frequent itemsets, the next step is to generate useful **association rules** from them.

## 188.4 Rule Generation

Once we find frequent itemsets, we generate association rules.

For example, if {Bread, Butter} is frequent, possible rules include:

- Bread  $\rightarrow$  Butter
- Butter  $\rightarrow$  Bread

To evaluate rules, we commonly use:

### 188.4.1 (1) Support

Support tells us how often the itemset appears in the dataset.

$$Support(A \rightarrow B) = Support(A \cup B)$$

### 188.4.2 (2) Confidence

Confidence measures how often item B appears in transactions that already contain item A.

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

Example:

If 4 transactions contain Bread, and 3 of those also contain Butter:

$$Confidence(Bread \rightarrow Butter) = \frac{3}{4} = 0.75$$

This means 75% of customers who bought Bread also bought Butter.

If confidence is close to 1 (or 100%), we say the rule has **strong confidence** because item B almost always appears whenever item A appears. \*\*\*

### 188.5 (BONUS): Lift

Lift measures whether two items occur together more often than expected by chance.

$$Lift(A \rightarrow B) = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Interpretation:

- **Lift** > 1: A and B occur together more often than expected, so they are positively associated.
- **Lift** = 1: A and B are independent.
- **Lift** < 1: A and B occur together less often than expected, so they are negatively associated.

For example, a high lift value means customers who buy item A are much more likely to also buy item B compared to a random customer.

---

### 188.6 Simple Example

Suppose we have these transactions:

Transaction	Items
T1	Milk, Bread
T2	Milk, Diaper, Beer
T3	Bread, Diaper, Beer
T4	Milk, Bread, Diaper, Beer
T5	Milk, Bread, Diaper

A possible association rule is:

$$Diaper \rightarrow Beer$$

This means customers who buy diapers may also buy beer.

Suppose:

- Diaper appears in 4 transactions
- Diaper and Beer together appear in 3 transactions

Then:

$$Confidence(Diaper \rightarrow Beer) = \frac{3}{4} = 0.75$$

This means 75% of customers who bought diapers also bought beer. If many transactions containing diapers also contain beer, then this rule has high confidence.

## 189 Summary of the Chapter

This chapter introduced three recommender system approaches:

- **Content-Based** uses item features (e.g., genres) to recommend similar items
- **User-Based CF** finds similar users and uses their ratings
- **Item-Based CF** finds similar items and uses a user's past ratings

Similarity is computed using **overlapping (co-rated) items**, typically with **cosine similarity** or **Pearson correlation**. Predictions are made using a **weighted average** of the most similar users (top-k) or items (top-N).

All three methods recommended **Alien**, but for different reasons: matching genres, similar users, and similar items.

In practice, **item-based CF is preferred** because item relationships are more stable and can be precomputed efficiently.

We also introduced **Association Rule Mining**, which discovers relationships between items in transaction data.

The process involves:

1. Frequent Itemset Generation
2. Rule Generation

We discussed the **Apriori Principle**, which reduces computation by eliminating impossible item combinations early.

Concept	Meaning
Association Rule	A relationship such as $(A \rightarrow B)$
Frequent Itemset	An itemset that appears frequently enough
Support	How often an itemset appears
Confidence	How often B appears when A appears
Lift	Measures association strength beyond chance
Apriori Principle	Larger itemsets cannot be frequent if subsets are infrequent

Association rule mining is widely used in recommendation systems, retail analysis, and customer behavior analysis.

## 190 Knowledge Check

*[Interactive content omitted in PDF]*